

—— // THE CLI FLEET // ——

The CLI Fleet

First Edition

THE CLI FLEET

the tools · the ports · the compendium



MENKE
TECHNOLOGIES

Copyright and Attribution

The CLI Fleet — the shipped Rust command-line tools, in one place

Copyright © 2026 MenkeTechnologies. All rights reserved.

No part of this book may be reproduced, distributed, or transmitted in any form or by any means without the prior written permission of the publisher, except for brief quotations in reviews and for the source excerpts identified below, which carry their own upstream licenses.

Published by MenkeTechnologies.

The trademarks and product names referenced in this book are the property of their respective owners. This book is an independent work: it is not affiliated with, authorized by, sponsored by, or otherwise endorsed by the `lsof`, `Nmap`, `iftop`, `htop`, or `grc` projects, their maintainers, the Rust Foundation, or any other project named in these pages.

The software this book describes

The eight tools covered here — `lsofrs`, `nmaprs`, `iftoprs`, `htoprs`, `storageshower`, `zcolorizer`, `grcrs`, and `temprs` — are each released under the MIT License, Copyright © 2026 MenkeTechnologies. Each has its own repository under github.com/MenkeTechnologies.

The upstreams these ports were written against

Five of the eight rebuild a specific, named Unix program: `lsofrs` rebuilds `lsof` (by way of `lsofng`, the modernized fork of Vic Abell's original), `nmaprs` rebuilds Nmap's scan engine and CLI dialect, `iftoprs` rebuilds `iftop`, `htoprs` rebuilds `htop`, and `grcrs` rebuilds `grc`, the Generic Colouriser. For each of those there is an upstream whose behaviour is the specification of record. Those programs are the work of their own authors and contributors and are distributed under their own license terms; naming one here credits it as the specification, not as a component of the port.

One upstream tree is vendored: `grcrs` carries a read-only copy of `grc` under `vendor/grc/`, whose `COPYING` records that `grc` was “written and maintained by Radovan Garabik” and “is released under GPL v2 or any later version”.

Short excerpts of these upstreams are quoted in this book for the purpose of commentary, criticism, and technical explanation — specifically, to show what a ported Rust function was validated against. Every such excerpt is cited to the file it came from.



MT PUBLISHING



MT OPEN SOURCE

Contents

Copyright and Attribution	1
Before You Begin	5
A Fleet Primer	7
The shape of the fleet — shared architecture and the port method	12
The covered fleet — tools with their own books	18
Isofrs — a Rust reimplementation of Isof	24
nmaprs — a Rust port of nmap’s scan engine and CLI dialect	31
temprs — a stack-structured temporary-file manager for the shell	38
iftoprs — iftop rebuilt on ratatui and pcap	45
htoprs — the interactive process viewer, ported function-for-function into Rust	53
storageshower — a real-time disk-usage TUI in Rust	60
zcolorizer — the log stream, repainted in real time	67
grcrs — a two-binary Rust port of the Generic Colouriser	74
zpwrrchrome-host and zwire-host — the browser-adjacent command line	81
Matching the upstream — the parser-compatibility layer, tool by tool	87
The output contract — TTY-awareness, ANSI, and the cyberpunk chrome	91
The theme engine — the 31-palette lineage and shift_lighter	95
Talking to the kernel — procfs, libproc, pcap, raw packets, sysctl	99
The concurrency map — threads, channels, locks, and the process boundary	104
Failure, degradation, and the discipline of saying no	107
The cost model — where the time goes, and why the shapes differ	110
Measuring the fleet — benchmark and test methodology, honestly	113
Anatomy of a port — from vendored C to a shipped binary	117
Packaging the fleet — binaries, completions, man pages, release	121
The security posture of tools that read the whole machine	125

Bytes, characters, and columns — three coordinate systems and the bugs between them	129
Signals, terminal state, and the obligation to hand the terminal back	132
Namespaces, containers, and the boundaries of a view	135
Extending the fleet without breaking the port	138
Diagnosing the fleet — a symptom index	141
The environment contract — variables, precedence, and the state a tool keeps	145
Identity — the keys these tools use, and the ones that get reused	148
Sampling — what a polled view can and cannot see	151
Clocks — monotonic time, wall time, and the places they are mixed	154
Determinism — what these tools promise to produce twice	156
The unsafe surface — every departure from safe Rust, and what pays for it	159
Byte order, ABI, and the architecture a port is actually built for	162
Names — turning a number into a label, and the three ways it goes wrong	165
The fleet at the prompt — seven worked sessions	168
The shell-side fleet — four tools that ship no compiled code	173
The dependency ledger — fifty-eight direct crates and what the lockfiles make of them	177
The umbrella hygiene gate — twenty-one identical scripts and what they refuse	180
The flag surface, measured — what the ports advertise against what the classics do	184
From inventory to result — the fleet’s suites, actually run	188
What ships — release profiles, binary size, and the linkage the ports do not have	193
The repository shape — what a fleet repository contains besides its source	198
The TUI layer, compared — four full-screen programs and the widget library none of them use	202
What the tools remember — six persistence conventions and the files they leave behind	206
The completion sets, measured — six zsh files against six binaries	209
The man pages against the binaries	213
The theme module’s real API — one file, three copies, and the fields nobody reads	218
Process attribution — the backends that turn a descriptor into a name	222
Headless — NDJSON, JSON, and the fleet without a terminal	225
The suite that would not finish, finished	229
Appendix A — the complete command & flag reference	233

Appendix B — the palette and theme catalog	239
Appendix C — the provisional-numbers ledger	241
Colophon	245
Reproducing the Book’s Claims	246
Also from MenkeTechnologies	247

Before You Begin

Provenance. Every figure in this book was re-derived on 2026-08-10 from the working checkouts under the MenkeTechnologies meta repository, whose own HEAD was `5667d050bb`. The per-tool crates were read at the commits checked out there — `iftoprs f194d91dfa`, `htoprs 47cd4b1cb9`, `temprs 49a58ac9c0`, `storageshower 7f8577061f` — which for several repositories is ahead of the submodule pointer that meta commit records. Where a chapter states a number that a later reading changed, it says so at the site rather than silently replacing it.

This is the book about the command-line tools — the whole fleet of them, in one place. The MenkeTechnologies stack has grown a large surface of terminal programs: a compiled shell, four language runtimes that share one bytecode VM, a handful of from-source Unix ports, a set of colourising filters, and a pair of browser-adjacent native hosts. Several of those already have books of their own. Many do not. This compendium is the map to all of them and the full text for the ones that were, until now, documented only by their man pages and their source.

It is deliberately the CLI-side twin of *Rewriting the Desktop in Rust*, the companion that covers the graphical fleet — the ported Tauri applications and their embeddable engines. Where that book follows the desktop programs, this one follows the things you run in a terminal and pipe into each other. Together the two books account for the shipped tools end to end: the ones with a window and the ones with a prompt.

The structure follows from that goal. After this preface comes a short chapter on the shape of the fleet — the architecture, dependencies, and porting method the tools share, pulled out once so the per-tool chapters can stay focused. Then an index: the seven CLI tools that already carry a dedicated reference manual or companion volume — `zshrs`, `strykelang`, `awkrs`, `vimlrs`, `elisprs`, `powerliners`, and `ztmux` — are situated in the fleet and pointed at their own books, not re-documented here. Everything after that is a full chapter for a tool that had no book: the Unix reimplementations (`lsofrs`, `nmaprs`, `iftoprs`, `htoprs`), the shell temporary-file stack manager (`temprs`), the disk-usage explorer (`storageshower`), and the two colourisers (`zcolorizer` and `grcrs`) — closing with `zpwchrome-host` and `zwire-host`, the two native-messaging host binaries that finish the catalog's CLI-tools shelf. Each chapter is written to be read front to back and grounded in the program's own source — the README, the crate manifest, the binary entry points — rather than in recollection.

This book is honest about foundations, because most of the fleet is a port and the honesty is the point. When a tool reimplements an existing Unix program — `lsof`, `nmap`, `iftop`, `htop`, `grc` — the chapter names the upstream plainly. The original work in those cases is the *engineering* of the Rust port: the architecture, the binary suite, the parity against the original, the drop-in compatibility with the world the upstream tool lived in. Where a tool is not a port — the log colouriser with its own theme grammar, the disk explorer, the two native hosts — the chapter says that too, and spends its pages on the design rather than on a comparison. No tool in this book is dressed up as more than it is; each is documented as what it is.

Where a number is provisional — a version string, a test count, a benchmark — the text marks it rather than rounding it into a claim. Versions move, suites grow, and a book that pretends otherwise goes stale the day it prints. The counts here are the counts the sources reported at the time of writing.

THE CLI FLEET – what this book contains

Front matter

The shape of the fleet shared architecture + the port method

Part I – the covered fleet (index only; see each tool's own book)

zshrs · strykelang · awkrs · vimlrs · elisprs · powerliners · ztmux

Part II – the uncovered fleet (full chapters)

lsofrs	list open files	(ports lsof)
nmaprs	network scanner	(ports nmap)
temprs	shell temporary-file stack manager	
iftoprs	per-connection bandwidth	(ports iftop)
htoprs	process monitor	(ports htop)
storageshower	disk-usage explorer	
zcolorizer	real-time log colouriser	(own theme grammar)
grcrs	generic colouriser	(ports grc)
zpwrchrome-host	browserpass-native port + extension actions	
zwire-host	universal local host	(sysinfo/fs/exec/pty/kv)

Part III – reading across the fleet (synthesis + reference)

Matching the upstream	the parser-compatibility layer, tool by tool
The output contract	TTY-awareness, ANSI, and the cyberpunk chrome
The theme engine	the 31-palette lineage and shift_lighter
Talking to the kernel	procfs, libproc, pcap, raw packets, sysctl
The concurrency map	threads, channels, locks, process boundaries
Failure and degradation	what each tool does when the answer is no
The cost model	where the time goes, and why shapes differ
Measuring the fleet	benchmark and test methodology, honestly
Anatomy of a port	vendored C to shipped binary, as a method
Packaging the fleet	binaries, completions, man pages, release
Identity	the keys, and the ones that get reused
Sampling	what a polled view can and cannot see
Clocks	monotonic, wall, and where they are mixed
Determinism	what the tools promise to produce twice
The unsafe surface	every departure from safe Rust
Byte order and ABI	the architecture a port is built for
Names	numbers to labels, and how that fails
The fleet at the prompt	seven worked operational sessions
Appendix A	the complete command & flag reference
Appendix B	the palette and theme catalog
Appendix C	the provisional-numbers ledger

The tools in Part II are what most of this book is about. Read the shape-of-the- fleet chapter for the architecture they share, Part I as the index that keeps the fleet whole, Part II as the coverage that was missing, and Part III as the cross-grain synthesis — the shared parser, output, theme, kernel, concurrency, failure, cost, benchmark, port-method, and packaging machinery pulled together, then the properties that cut across all of it (which keys the tools identify things by, what a sampled view can see, which clock each deadline is measured against, what output is reproducible, where the code leaves safe Rust, what depends on the target architecture, and how a number becomes a name), then a chapter of worked sessions that puts several tools in one pipeline — closing with three reference appendices that consolidate every command, palette, and provisional number the per-tool chapters state in passing.

Before any of that, this edition opens with a short primer for readers meeting the fleet for the first time: what the collection is, the method every tool in it follows, what a per-tool chapter contains, and how to try one of these binaries without committing to it. Readers who already run these tools daily can skip straight to the shape-of-the-fleet chapter.

A Fleet Primer

Everything after this section assumes you are at home at a Unix prompt and have used at least one of the programs these tools reimplement. If you have, skip ahead — nothing here is new to you. If you opened this book to find out what “the fleet” *is* before committing to the chapters, this is the twenty minutes that makes the rest legible: what the collection is and is not, the method every tool in it follows, what a per-tool chapter contains, which tools have books of their own, and how to try one of these binaries without rearranging your machine. It is deliberately short and not a reference: the chapter immediately after this one — “The shape of the fleet — shared architecture and the port method” — is the same ground covered properly and cited, Appendix A is the complete command and flag reference, and Appendix C the ledger of every provisional number the book prints.

Every transcript below is real output. Every command was run on one macOS 26.5.2 arm64 machine against the binaries installed on it: `lsotrs` 4.9.3 (also installed as `lsf`), `nmaprs` 0.1.10, and `temprs` 2.9.13 (also `tp`), beside the programs two of them reimplement — Apple’s `lsot` revision 4.91 and Nmap 7.991 — plus `head`, `wc`, `grep`, `awk`, and `brew`. Every count below was re-derived at the prompt from the book’s own text, and where a re-derived figure disagrees with what the book printed, the disagreement is shown rather than quietly corrected.

```
$ lsotrs --version
lsotrs 4.9.3
$ nmaprs --version
nmaprs 0.1.10
$ temprs --version
temprs 2.9.13
```

What the fleet is, and what it is not

The fleet is a shelf of separately shipped command-line binaries. Each is its own Rust crate, its own repository, its own version number, and its own install; there is no umbrella binary, no plugin host, no shared runtime to install first, and no package that pulls the rest in. They are ordinary `argv/stdin/stdout` Unix programs, and what makes them a *fleet* rather than a pile is one set of conventions. The shape-of-the-fleet chapter puts it plainly: what ties them together “is not a framework — it is the agreement that a shipped CLI tool should be terse on `stderr` in the `<tool>`: `<thing>`: `<reason>` shape, quiet on a pipe, structured when asked, and drop-in against whatever it reimplements.”

So it is not a busybox — nothing here is a subcommand of anything else — and not a shell framework, since none of these tools reaches into a shell’s internals or is a plugin. You can install exactly one and never notice the rest exist. Because they ship separately they also drift separately, which is the first thing to know before trusting any number in this book:

```
$ brew list --versions lsofrs nmaprs temprs iftoprs htoprs storageshower zcolorizer grcrs
grcrs 1.13.1
htoprs 0.5.12
iftoprs 2.22.10
lsofrs 4.9.3
nmaprs 0.1.10
storageshower 0.28.9
temprs 2.9.13
zcolorizer 0.2.3
```

All eight moved since Appendix C recorded them — `lsofrs` 4.9.1 → 4.9.3, `nmaprs` 0.1.9 → 0.1.10, `temprs` 2.9.12 → 2.9.13, `iftoprs` 2.22.9 → 2.22.10, `htoprs` 0.5.7 → 0.5.12, `storageshower` 0.28.8 → 0.28.9, `zcolorizer` 0.2.2 → 0.2.3, `grcrs` 1.13.0 → 1.13.1. Not one version number survived contact with a package manager: read Appendix C as a dated snapshot, not a fact about your machine. Of the two native hosts, which install by `cargo install` rather than the tap, only `zwire-host` can be checked this way — it reports 0.3.15 against the appendix’s 0.3.14, while `zpwchrome-host --version` prints `Browserpass host app version: 3.1.2`, the upstream host-protocol version rather than a crate version.

The method every tool follows

The fleet divides once, and the division decides what “finished” means. Five of the eight tools the shape-of-the-fleet chapter measures reimplement a specific, named Unix program — `lsof`, `Nmap`, `iftop`, `htop`, `grc` — and for those the upstream’s behaviour is the specification of record and the question is fidelity. The other three, `storageshower`, `zcolorizer`, and `temprs`, are originals with no single upstream, and their chapters spend their pages on design instead.

For the ports the house rule is what the book calls **port-honest**, and it means three mechanical obligations rather than a tone. Name the upstream, which every port’s manifest or README does. State where the port stops: `nmaprs` does not embed the NSE Lua runtime and says so in a capability matrix; `htoprs` deliberately does not port `htop`’s hand-rolled allocator families, since reimplementing them faithfully would mean writing an allocator in `unsafe` — the exact bug class the port exists to remove — so `Vec` and `HashMap` stand in and the omission is documented. And accept-and-ignore: `lsofrs` parses and silently discards upstream flags that mean nothing in Rust, so scripts passing them keep running. The book’s phrase is “honored syntactically, not behaviorally.”

The dependency floor is deliberately boring. Across the eight the recurring crates are `serde` in six, `clap` in six, `regex` in five, `libc` in five, and `ratatui` plus `crossterm` in the four that draw a full-screen interface, the stated preference being crates that still build years from now over fashionable ones. Nothing in the set memory-maps its input.

Parity is not one shared harness, and the book refuses to imply it is. Exactly one tool, `htoprs`, enforces parity at build time, with a gate that fails the build if a function appears under its ported tree with no counterpart in `htop`’s C. `grcrs` has a byte-for-byte oracle without a build gate. The remaining four have no automated parity oracle at all, and the chapter says so in those words. That distinction is the first thing to look for in a chapter’s Status section.

Names are a contract. A port of a named program is that program’s name with `rs` appended, and frequently-typed tools also build a short alias binary from the same source — `lsf`, `nms`, and `tp` for `lsofrs`, `nmaprs`, and `temprs`, byte-identical to the long form. `grcrs` is the exception and the one thing to notice before installing anything: it builds two *functionally distinct* binaries under exactly the upstream names, mirroring the split real `grc` uses, so installing it replaces `grc`:

```
$ command -v grc lsf
/opt/homebrew/bin/grc
/opt/homebrew/bin/lsf
```

What a per-tool entry contains

The per-tool chapters follow a recognisable shape. Here is one, read out of the book itself:

```
$ awk '/^# lsofrs/{p=1} /^# nmaprs/{p=0} p && /^## /' docs/book.md
## What it ports, and how honestly
## Architecture
## The CLI surface
## The TUI layer
## Repeat, delta, and the live loop
## Integration into the stack
## Parity and its limits
## Status
```

In order: what the tool takes from its upstream and where it stops taking; how the crate is laid out; the flags and their grammar; the full-screen interface if it has one; how it fits the rest of the stack; how correctness is checked and how far the check reaches; then version, tests, licence, and packaging. Tools with no terminal interface drop the TUI sections, and the originals replace “what it ports” with “what it is”. Status is the section to read sceptically: a test count here is a `grep` count of `#[test]` attributes, not a `test result:` line, and the book says so at each site.

Which tools have their own books, and which are only here

Some of the fleet is documented at book length elsewhere, and this compendium indexes those rather than repeating them:

```
$ awk '/^# The covered fleet/,/^# lsofrs/' docs/book.md | grep -c '^## .* - '
7
```

Seven: `zshrs`, `strykelang`, `awkrs`, `vimlrs`, `elisprs`, `powerliners`, and `ztmux`. The chapter “The covered fleet — tools with their own books” situates each and names the volume that carries it in full; five of the seven run on one shared bytecode VM with a Cranelift JIT, which is why none of that machinery is re-explained here. Everything else gets its full treatment in this book:

```
$ grep -c '^# [a-z].* - ' docs/book.md
9
$ awk '/^ Part II/,/^ Part III/' docs/book.md | grep -cE '^ [a-z]'
10
```

Nine per-tool chapters covering ten tools — the last takes `zpwchrome-host` and `zwire-host` together, which is why the counts differ. The shape-of-the-fleet chapter opens by saying “Eight command-line tools get full chapters in this book”; re-derived, that eight is the set *it* measures, the eight compiled tools whose manifests it compares, and it excludes the two browser-adjacent native hosts. Both statements are true of different sets; only one counts chapters. One further chapter adds four tools that ship no compiled code at all:

```
$ awk '/^# The shell-side fleet/{print substr($0,3)}' docs/book.md
The shell-side fleet - four tools that ship no compiled code
```

Two ports and an original, at the prompt

Nothing below needs privileges, touches configuration, or sends a packet anywhere; it all runs in a scratch directory holding one four-line file.

`lsofrs`, beside `lsOf`. A background `tail -f` holds that file open on descriptor 3, and both programs are

asked, in `lsf`'s own field-output grammar, for the name of that descriptor:

```
$ lsof -a -c tail -d 3 -Fn
p77669
f3
n/private/tmp/fleet-demo/staff.txt
$ lsf -a -c tail -d 3 -Fn
n/private/tmp/fleet-demo/staff.txt
```

Same selectors, same answer: the grammar — `-a` to AND the selections, `-c` for command name, `-d` for descriptor set, `-F` for field output — is `lsf`'s, unchanged. The difference is real and worth seeing early: `lsf`'s field format always emits its process (`p`) and descriptor (`f`) record lines whether or not you selected those fields, and this build of `lsf` prints only what you selected. That is the kind of gap a “Parity and its limits” section enumerates, and `lsf` is one of the four with no oracle diffing it against a running `lsf`.

`nmaprs`, beside `nmap`. A list scan expands targets and prints them without scanning — the `nmaprs` chapter says exactly that of `--sL` — and `-n` suppresses name resolution, so nothing was contacted. The addresses come from the same documentation-reserved `192.0.2.0` block that chapter uses for its own examples:

```
$ nmap -sL -n 192.0.2.5-7
Starting Nmap 7.991 ( https://nmap.org ) at 2026-08-16 22:09 -0400
Nmap scan report for 192.0.2.5
Nmap scan report for 192.0.2.6
Nmap scan report for 192.0.2.7
Nmap done: 3 IP addresses (0 hosts up) scanned in 0.00 seconds
$ nmaprs -sL -n 192.0.2.5-7
192.0.2.5
192.0.2.6
192.0.2.7
```

Both flags were typed in Nmap's glued short-option form and both programs accepted them. The target expansion agrees exactly, the framing does not, and on CIDR input the two also disagree about endpoints — `nmaprs -sL -n 192.0.2.0/30` prints `192.0.2.1` and `192.0.2.2`, dropping network and broadcast, where `nmap` prints all four. Neither is a bug report; both are why the book documents parity as a capability matrix rather than as a claim.

`temprs`, which ports nothing. It is a stack of temporary files addressed by index or tag instead of by path. Piping into it pushes; its read verbs reproduce what `head`, `wc`, and `grep` do, against the stack rather than a path:

```
$ export TEMPRS_DIR=/private/tmp/fleet-demo/temprs-demo
$ tp -w staff < staff.txt
$ tp --head staff 2
alice 30 engineering
bob 25 sales
$ head -2 staff.txt
alice 30 engineering
bob 25 sales
$ tp --wc staff
4
$ wc -l < staff.txt
4
```

`tp --head staff 2` and `head -2 staff.txt` produce identical bytes; `tp --wc` and `wc -l` do not, because `wc` pads its count and `tp` does not. That is the character of the tool: the *observable behaviour* of the coreutils verbs, reached by tag or index, with the engineering spent on the locked, null-delimited master record underneath rather than on reinventing `head`. `TEMPRS_DIR` relocates the entire stack, which is how this was demonstrated without touching the default one.

Trying one without committing to it

Every tool is published to crates.io and carries a Homebrew tap path, so `cargo install <tool>` or `brew install <tool>` gets you one binary and nothing else, in any order you like. Nothing shadows an existing command except `grcs`, which installs `grc` and `grcat` under exactly those names; every other tool answers to a name upstream does not own, so `lsofrs` and `lsof`, `htoprs` and `htop`, `iftoprs` and `iftop` coexist on one `PATH` and can be run side by side — the only honest way to evaluate a port.

What a tool leaves behind is small and per-tool. The three interactive tools that keep preferences write a TOML dotfile in `$HOME` (`~/.lsofrs.conf`, `~/.iftoprs.conf`, `~/.storageshower.conf`); `htoprs` writes JSON under `~/.config/htoprs/` and pointedly does *not* touch `htop`'s own `htoprc`, so installing it cannot disturb an `htop` you already configured; `zcolorizer` uses `~/.config/zcolorizer/config.toml`; `temprs` keeps no config at all, only files under `$TMPDIR/temprs`; and `nmaprs` and `grcs` persist nothing per-user. Backing out is uninstalling the binary and, if you care, deleting the one file named after it. “What the tools remember — six persistence conventions and the files they leave behind” is the complete table.

What is different about this fleet specifically

Most of it is a port, and the book leads with that rather than burying it. When a tool reimplements `lsof`, `nmap`, `iftop`, `htop`, or `grc`, the chapter names the upstream in its first paragraph and treats that program's behaviour as the specification. What is claimed as original is the Rust architecture, the kernel-interface work, the drop-in compatibility, and the measured parity — not the invention of the tool.

The numbers are marked provisional and they mean it. A test count is a `grep` inventory of `#[test]` attributes, a benchmark is the author's own measurement on one machine, a version is read from a manifest. The comparison at the top of this primer is what happens when you check one class of those numbers later: all eight had moved.

Coverage is uneven on purpose. Seven tools are indexed and pointed at their own books; ten get full chapters because nothing else documents them; four shell scripts get a chapter because leaving them out would misdescribe the fleet. This is a survey, and one that gave every tool equal pages would be a worse map.

Where to go from here

You now know enough to pick a chapter. “The shape of the fleet — shared architecture and the port method” is everything sketched here, done properly and cited, and any per-tool chapter is readable on its own — start with the tool whose upstream you already use. For the method rather than the tools, “Anatomy of a port — from vendored C to a shipped binary” generalises what the ports do, and “Measuring the fleet — benchmark and test methodology, honestly” explains why every number in the book carries a hedge. “The fleet at the prompt — seven worked sessions” puts several of these binaries in one pipeline, and is the best next thing to read if you want them used rather than described.

The shape of the fleet — shared architecture and the port method

Eight command-line tools get full chapters in this book, and before any of them is opened on its own terms it is worth reading them sideways — across the grain — for what they have in common. They were built by the same organization, on the same durable-dependency rules, with the same house conventions for how a shipped Rust CLI should behave, and they divide cleanly into two kinds: reimplementations of a named Unix program, and originals that answer a problem the author first solved somewhere else. This chapter pulls the shared machinery out once — the substrate split, the dependency floor, the terminal stack, the argument parsers, the kernel interfaces, the licences, and the parity discipline — so that each per-tool chapter can spend its pages on what makes that tool specific rather than re-deriving the parts they all agree on. Every claim here is grounded in the crate manifests and source of the eight tools it covers: `lsofrs`, `nmaprs`, `iftoprs`, `htoprs`, `storageshower`, `zcolorizer`, `grcrs`, and `temprs`. Where a pattern is only true for some of them, the text says which; a shared convention that four tools follow and four do not is not a fleet-wide law, and the value of reading across the grain is lost the moment a convenient generalization papers over the exceptions.

The substrate split: ports and originals

The first cut divides the eight into reimplementations and originals, and the division is not cosmetic — it changes what “done” means for the tool. Five of them reimplement a specific, named Unix program: `lsofrs` rebuilds `lsof` (by way of `lsofng`, the modernized fork of Vic Abell’s original), `nmaprs` rebuilds Nmap’s scan engine and CLI dialect, `iftoprs` rebuilds `iftop`, `htoprs` rebuilds `htop`, and `grcrs` rebuilds `grc`, the Generic Colouriser. For each of these there exists an upstream whose behaviour is the specification of record, and the engineering question is fidelity: does the port produce the output the original would have produced, accept the flags the original accepted, consume the same data files, and interoperate with the same scripts. The design of the tool is not up for invention. The option grammar, the columnar layout, the network selector, the colourfile format — these belong to the upstream, and the Rust work is the faithful translation plus whatever memory-safety, cross-architecture, and vendorability the rewrite buys.

The other three are originals, and they are honest about it. `storageshower` is a disk-usage TUI that is not a port of `ncdu`, `du`, or `df` — its own repository carries the bash predecessor it descends from under `old_scripts/`, a 2018-vintage `storageShower.sh` dispatching to a ~31 KB `storageshowerCyberpunk.sh`; the Rust crate is the reimplementations of the author’s own concept as a compiled, threaded, tested binary, and it arrives at whatever it shares with `ncdu` independently rather than by porting anyone’s code. `zcolorizer` is subtler: its manifest calls it a “ccze/pygments port” and it does carry a full copy of ccze’s C source under `vendor/ccze/` as a build-time reference, faithfully reproducing ccze’s plugin-then-word-colorizer architecture — but its 47-module format library, its themeable colour model, and its three-layer rules/themes/engine decomposition are original work extending a 2003 tool into formats (systemd, JSON, Kubernetes, Postgres) that did not exist when ccze was written. `temprs`, the shell temporary-file stack, is the cleanest original: there is no `temp(1)` it descends from. What it ports is the *observable behaviour* of several coreutils — its `--head`, `--tail`, `--wc`, `--grep`, `--diff`, `--cat` reproduce what `head`, `tail`, `wc`, `grep`, `diff`, and `cat` do — but addressed by stack index or tag name instead of by path, and the engineering that matters is the atomic, locked, null-delimited master record underneath, not the reinvention of a diff.

Port-honest, mechanically – what the word means here

“Port-honest” is the house rule that governs how the reimplementations describe themselves, and it has a mechanical meaning rather than a rhetorical one. Naming the upstream is the first obligation: `lsotrs` names `lsot`, `nmaprs` names Nmap in its own `Cargo.toml` description and points anyone needing “authoritative Nmap NSE/OS DB behavior” back at Nmap, `iftotrs`’s manifest calls it an “iftop clone in Rust,” `htotrs`’s README leads with “the C source is the spec,” and `grctrs` names `glibc` and its upstream version. The second obligation is to state where the port stops. `nmaprs` does not embed the NSE Lua runtime and says so; its OS detection is partial and its version detection skips Perl-only regex constructs, all documented in a capability matrix rather than glossed over. `htotrs` deliberately does not port the hand-rolled allocator families (`xmalloc`, `Vector.c`’s raw `Object**` machinery, `Hashtable.c`’s open-addressing table) because reimplementing them byte-for-byte would mean writing an allocator in `unsafe` — precisely the bug class the port exists to eliminate — so `Vec` and `HashMap` stand in, and the omission is documented, not hidden. The third obligation is the accept-and-ignore contract: `lsotrs` parses and silently discards a whole set of upstream flags (`-b`, `-O`, `-D`, `-K`, and their `+` counterparts) that have no meaning in the Rust implementation, so that scripts passing them keep running rather than erroring, even though the behaviour behind them is absent. That is the correct trade for drop-in substitution, and calling it “honored syntactically, not behaviorally” is the port-honest way to describe it.

The dependency floor

The manifests share a deliberately foundational dependency set — the meta rule is to prefer crates that will still build cleanly years from now over fashionable ones, and it shows. Across the eight, the recurring names are `serde` (six of eight), `clap` (six), `regex` (five, plus `fancy-regex` in the one case that needs backtracking), `libc` (five), `dirs` for config-path resolution (four), `toml` (four), and `crossterm/ratatui` for the four with a terminal UI. The extremes bracket the range: `grctrs` depends on exactly two crates — `fancy-regex 0.14` and `libc 0.2` — while `nmaprs`, which has to build raw packets and speak TLS and SOCKS, pulls the widest set (`tokio` with `full`, `pnet/pnet_sys` for raw packet construction, `rustls` on the `ring` backend, `hickory-resolver` for DNS, `tokio-socks` for proxies, `crc32c/crc32fast` for SCTP checksums, and `cc` to compile one C++ FFI object). `temprs` sits near the minimal end with `clap` plus `fs2` for advisory file locking. None of the eight depends on a memory-mapping crate — there is no `memmap/memmap2` anywhere in the set — which is worth stating plainly because it is easy to assume a systems tool memory-maps its input. These tools stream: `grctrs`, `zcolorizer`, and `temprs` read stdin line-buffered, and the rest gather their data from kernel APIs and pseudo-filesystems, never from an `mmap`’d file.

The release profiles rhyme. Six of the eight tune for a lean shipped binary with `lto = true` and `strip = true`, most adding `opt-level = 3` and `codegen-units = 1`, and three (`nmaprs`, `temprs`) also set `panic = "abort"`. Editions split evenly: `lsotrs`, `iftotrs`, `storageshower`, and `temprs` are on the 2024 edition; `nmaprs`, `htotrs`, `zcolorizer`, and `grctrs` are on 2021. Per the meta rule, local development builds with a plain `cargo build`; the tuned profile is what the installed binary carries, not what a developer waits on.

The TUI stack: ratatui and crossterm, and one deliberate refusal

Four of the eight are interactive terminal applications, and all four reach for the same pair: `ratatui` (0.30 across the board) for widget layout and `crossterm` (0.29) for the terminal backend. `lsotrs`, `iftotrs`, `htotrs`, and `storageshower` all carry both in their manifests. The event-loop shape is consistent enough to describe once. A single-threaded render/input loop owns the terminal: it draws one frame with `ratatui`, then polls `crossterm` for input within a bounded tick budget, and it never does expensive work inline. The data collection runs elsewhere — on background OS threads that write into `Arc<Mutex<...>>` shared state the UI reads from. `storageshower` spawns a `spawn_bg_collector` thread that refreshes disks and system stats on a three-second cadence independent of the user-facing refresh rate; `iftotrs` runs a dedicated

`packet-capture` thread and a `proc-lookup` thread feeding a poison-tolerant `FlowTracker`; `lsofrs` gives every live mode the same keybindings, alternate-screen handling, and atomic frame rendering through a shared `TuiMode` trait. The loop cadence is explicit where measured — `iftoprs` runs a 33 ms tick (roughly 30 fps), `storageshower` polls `crossterm` with a 200 ms timeout — but the structure is the same: drain events, maybe rotate, snapshot, draw, poll.

`htoprs` is the deliberate refusal, and it is instructive. It carries `ratatui` in its manifest too, but the ported htop core does not draw through it — the constraint on `htoprs` is that its output be byte-for-byte identical to htop, whose C source renders through ncurses, so the ported draw code drives `crossterm` directly and transcribes `CRT.c`'s `ColorElements` enum and every `CRT_colorSchemes` entry verbatim to match htop's colours glyph for glyph. `ratatui` appears only in the gate-exempt `src/extensions/` tree — `finder.rs`, `panels.rs`, `overlay.rs` — where `htoprs` adds capability htop never had. So even the one shared library is used two different ways within a single crate, and reading the manifest without reading the source would mislead you into calling `htoprs` a `ratatui` app in the same sense the other three are.

The four TUI tools also share a themeing lineage that is worth tracing because it is real rather than coincidental. The 256-colour palette set — thirty-odd named schemes running from `NeonSprawl` through `Zaibatsu` — originates in `storageshower` (30 `ColorMode` variants), was carried into `iftoprs` (31, adding the parity palette `Iftopcolor`), and into `lsofrs` (31, asserted by an enum-length check) and `htoprs` (31 six-colour palettes). `zcolorizer` inherits the same set explicitly: its 31 generated palettes are, per its own source, “ported verbatim from the shared MenkeTechnologies theme set (`iftoprs`, itself derived from `storageshower`),” plus a 32nd hand-written `ccze-classic` for parity with the tool it colours after. A six-index cube-arithmetic helper, `shift_lighter`, moves with the palettes from `iftoprs` into `zcolorizer`. This is a genuine shared component, not merely a shared aesthetic.

Argument parsing: clap, and the two tools that refuse it

Six of the eight parse their command line with `clap` 4 — `lsofrs`, `nmaprs`, `iftoprs`, `storageshower`, `temprs`, and `zcolorizer` (behind a `cli` feature so the library can be embedded without pulling clap in). But two of the reimplementations refuse clap on purpose, and the reason is the same in both cases: fidelity to a parser that predates clap's grammar.

`htoprs` does not use clap; it calls the real `libc::getopt_long` symbol — the same one htop's own `getopt_long` fills — so that its option-parsing behaviour is not merely similar to htop's but produced by the identical library routine. `grcrs` does not use clap either; its `parse_opts` hand-mirrors `getopt`, including the two subtleties a clap parser would get wrong. First, abbreviated long options: `resolve_long` reproduces getopt's rule that an exact name wins, a *unique* prefix resolves, and an ambiguous prefix is rejected — so `--conf=name` resolves to `--config`, `--col=off` to `--colour`, but `--std` is refused because it could be `--stdout` or `--stderr`. Second, option-scanning that stops at the first non-option token and treats everything after -- as the command even if it looks like a flag. Two other tools do their own pre-parsing even while keeping clap downstream: `lsofrs` runs a hand-written `normalize_lsof_args` pass *before* clap sees anything, because lsof's grammar is not getopt-clean (the `+L/+d/+D` prefix family, attached arguments like `-o9` and `-sTCP:LISTEN`), and `nmaprs` runs an `argv_expand` pass that splits Nmap's glued tokens (`-sS`, `-Pn`, `-PS80`, `-T4`) apart before clap's derive parser, which expects separated options. The pattern here is not “everyone uses clap” — it is “everyone matches the upstream parser, using clap where clap can express the grammar and refusing or pre-processing it where it cannot.”

Talking to the kernel: procfs, libproc, pcap, and raw packets

The system tools gather their data straight from kernel interfaces, and none of them shells out to another program to do it — the anti-subprocess discipline is a shared value. The Linux path is hand-parsed procfs everywhere it appears: `lsofrs` walks `/proc/<pid>/fd` and reads the `cwd/root/exe`

symlinks; `iftoprs` parses `/proc/net/tcp`, `/proc/net/tcp6`, `/proc/net/udp`, `/proc/net/udp6`, `/proc/[pid]/fd/`, and `/proc/[pid]/comm` to attribute flows to processes; `storageshower` reads `/proc/mounts` and `/proc/diskstats`. Notably, none of them uses the `procfs` crate — the parsing is done by hand against the files. The macOS path is direct libproc: `lsofrs` declares `proc_listpids` and `proc_pidinfo` as raw `extern "C"` and calls them against `repr(C)` structs matched to the kernel headers; `iftoprs` uses the `libproc` crate to read each process's open sockets via `pidfdinfo/SocketFDInfo` (its manifest comment notes this replaces a runtime `lsof`); `storageshower` calls `getmntinfo` and reads IOKit block-storage byte counters through `libc`. `htoprs` carries the full per-OS backend tree (`darwin/`, `linux/`, `freebsd/`, `openbsd/`, `netbsd/`, `dragonflybsd/`, `solaris/`) that htop itself carries, and on Linux pulls three pure-Rust, no-FFI helper crates — `libmedium` for hwmon sensors, `neli` speaking generic-netlink for the TASKSTATS delay-accounting path, and `zbus` speaking D-Bus directly for the systemd meter — deliberately keeping C libraries out of the build.

Two tools stand apart on the wire. `iftoprs` is the only one that uses `pcap` (2.4) — it opens the interface with a 256-byte snaplen (enough for L2–L4 headers, never payload), decodes Ethernet, BSD loopback, Linux cooked capture, and raw IP link types, and pushes decoded packets across a `tokio::sync::mpsc` channel off the capture thread. `nmaprs` is the only one that constructs raw packets, via `pnet/pnet_sys`, and it is also the only one with a genuinely async core (`tokio` with `full`); interestingly, its default TCP-connect scan does *not* use async sockets but a worker pool of OS threads running blocking `connect_timeout`, falling back to tokio tasks only when a SOCKS/HTTP-CONNECT proxy state machine is needed. Raw capture and raw scanning both need privilege, and both tools degrade gracefully: `nmaprs` detects `geteuid()` and silently falls back from SYN to connect scan when unprivileged, while `iftoprs`'s capture loop treats device failures as transient and retries with exponential backoff.

Only two of the eight carry a `build.rs`, and for opposite reasons. `nmaprs` uses it to compile one C++ object (`c/expr_match.cpp`) via the `cc` crate for its OS-fingerprint expression matcher. `htoprs` uses it as an anti-drift gate — described below — that runs on every build.

Licensing

Every tool in the fleet is licensed MIT, and the Rust in each is original code. `lsofrs`, `nmaprs`, `iftoprs`, `htoprs`, `storageshower`, `zcolorizer`, `grcrs`, and `temprs` each declare `license = "MIT"` in their manifests, ship a single `LICENSE` file with the MIT text, and carry the same MIT badge and MenkeTechnologies copyright. Where a tool reimplements a Unix program — `lsof`, `nmap`, `iftop`, `htop`, `grc` — it studies the upstream's *behaviour* to reach parity, and some keep the upstream source under a `vendor/` tree purely as a read-only reference for that work; the binary is built from the tool's own `src/`, never from `vendor/`. There is no copyleft anywhere in the CLI fleet and no per-tool licence divergence to track — the whole shelf is uniformly, permissively MIT.

Parity methodology, tool by tool

The most important thing to *not* overstate is the parity discipline, because it varies enormously across the eight and only one of them enforces it at build time. It is tempting to describe “the fleet's parity harness” as a shared thing; it is not.

`htoprs` has the strictest apparatus, and it is a build gate rather than an aspiration. Its source splits into `src/ported/<file>.rs` (one Rust module per htop C file, every free `fn` carrying a `/// Port of` doc comment — 1,937 such citations) and `src/extensions/<name>.rs` (htoprs-original, gate-exempt). The gate in `build.rs` runs on every `cargo build`, `test`, and `check`, extracts every free `fn` name under `src/ported/`, and checks it against a snapshot of htop's real C function names at `tests/data/htop_c_fn_names.txt`; a function with no upstream counterpart *fails the build*, and it fires even under `cargo test --test X` so a reimplementation cannot be routed around it. On top of that sits a byte-diff oracle: the `tests/parity/` suite runs the same inputs through the reference C `htop` and through `htoprs` and compares output byte-for-byte

(skipping gracefully when no htop 3.5.x is installed, so CI stays green), and `scripts/gen_port_report.py` derives the coverage figure from source at run time, counting a `todo!()` body as stubbed, never ported.

```
cargo build          # runs the port-purity fn-name gate
cargo test --test parity # diff htoprs vs reference htop 3.5.x
python3 scripts/gen_port_report.py # regenerate port_report.html
```

`grcrs` has the next-strongest discipline: a byte-for-byte oracle without a build gate. Its 68 tests include 27 end-to-end tests in `tests/cli.rs` that spawn the built binaries and assert on the exact coloured bytes, with expectations cross-checked against the reference Python `grcat` — covering `count=stop/once/more/block`, `unchanged/previous` colours, `replace-then-colour`, `</>` literals, CRLF stripping, and the invalid-UTF-8 round-trips that pin its `surrogateescape` reproduction. `nmaprs` documents parity as a capability matrix in its README (which port-state mappings each scan technique produces) backed by contract tests over the port database, scanflags, OS-DB parsing, and target expansion, plus repo-hygiene guards. `zcolorizer` keeps ccze's C source vendored as the reference and pins its precedence model (`earlier_rule_wins_overlap`) and input-sanitization behaviour with engine tests, but it colours new formats ccze never saw, so full byte-parity is not even the goal for most of its surface.

The remaining four have no automated parity oracle at all, and it would be wrong to imply otherwise. `lsofrs` reproduces lsof's option grammar, type vocabulary (`FileType`, `TcpState`), and output formats (`-F`, `-t`, `+L`, `-o`, `-s`) and asserts them with its own tests, but it does not diff against a running `lsof`. `iftoprs` ships a parity palette (`Iftopcolor`) and mirrors iftop's keybinds and the flow-table/rolling-window model, but its 2s/10s/40s columns are *summed byte counts over trailing one-second slots scaled to bits*, not slot-averaged bytes/sec — a difference the chapter states precisely rather than calling it “identical to iftop.” `storageshower` and `temprs`, being originals, have nothing upstream to be parity-tested against; their large test suites pin *their own* invariants (symlink-cycle termination in `storageshower`, master-record tag alignment and round-trips in `temprs`), which is a different kind of discipline entirely. The counts each project reports are large — on the order of a thousand or more `#[test]` annotations in several of them — but every per-tool chapter marks those as provisional grep counts of test *attributes*, not results of a `cargo test` run, and this chapter does the same: treat a test count as a count of intentions until a `test result:` line confirms it.

The shared surface: binaries, completions, man pages, config

Below the architecture, the eight share a set of packaging conventions, again with honest exceptions. The short-alias binary convention gives frequently-typed tools a two- or three-character name from the same source: `lsofrs` also builds `lsf`, `nmaprs` also builds `nms`, `temprs` also builds `tp`, all byte-identical to their long form. `grcrs` builds two *functionally distinct* binaries rather than an alias — `grc` the launcher and `grcat` the colouriser — mirroring the same split upstream `grc` uses so the colouring logic stays a standalone filter you can drive from a pipeline. The remaining four (`iftoprs`, `htoprs`, `storageshower`, `zcolorizer`) ship a single binary.

Six of the eight ship a checked-in zsh completion under `completions/_<tool>` with a `#compdef` header — `lsofrs`, `nmaprs`, `iftoprs`, `htoprs`, `storageshower`, and `temprs` — several of them kept in lockstep with the parser (`htoprs`'s tracks the getopt table in `commandline.rs`; `temprs`'s is *live*, reading the running stack's master file so tab-completion offers the actual indices and tags present at the moment you press Tab). `zcolorizer` and `grcrs` ship no completion in-tree — `grcrs` relies on the upstream `grc.zsh/grc.sh/grc.fish` shims instead. The same six also ship the paired man-page convention: a short `<tool>.1` and a comprehensive `<tool>all.1` modelled on `zshall(1)`, folding invocation, keybinds, themes, and architecture into one grep-able document. `grcrs` ships per-binary `grc.1` and `grcat.1` instead of an “all” page, and `zcolorizer` ships no `man/*.1` in the tree searched here.

Persisted configuration splits by whether the tool is interactive. The three TUI tools that keep user preferences use a TOML dotfile in `$HOME` — `~/.lsofrs.conf`, `~/.iftoprs.conf`, `~/.storageshower.conf` — auto-saved on every in-app toggle with no explicit save step. `htoprs` breaks the pattern by storing JSON

under `~/.config/htoprs/prefs.json`, and `zcolorizer` uses TOML but under XDG at `$XDG_CONFIG_HOME/zcolorizer/config.toml`. The three non-interactive tools (`nmaprs`, `grcrs`, `temprs`) keep no preference file at all — `grcrs` reads the upstream `grc.conf` data files, but those are shared config, not per-user state, and `temprs` keeps its stack on disk rather than a settings file.

How they sit in the shell

None of these tools is a shell plugin, and none reaches into a shell's internals — they are ordinary `argv/stdin/stdout` Unix filters, which is exactly why they compose under any POSIX shell including `zshrs`, the compiled shell at the centre of the wider fleet. What makes them compose *cleanly* is a shared TTY-awareness that every one of them implements: colour and interactive chrome are gated on whether the relevant descriptor is a terminal, and disabled the moment output is piped. `lsOfrs` drops its colours and cyberpunk headers and prints plain script-safe text when stdout is not a TTY, and its live modes never enter the alternate screen non-interactively — they print one snapshot and exit. `grcrs` initialises its colour default from `libc::isatty(1)`. `zcolorizer` auto-disables colour off a TTY unless `-C` forces it. `nmaprs`'s styled help suppresses escapes on a non-TTY or under `NO_COLOR`. `temprs` branches on `stdin().is_terminal()` so `CMD | tp` reads the pipe while a bare `tp` waits for typed input. Two tools go further and emit structured data for downstream tooling: `iftoprs --json` streams NDJSON — one object per refresh — that a `jq` filter or a dashboard can consume, and `nmaprs` writes Nmap-compatible XML (`-oX`) and grepable (`-oG`) formats. In practice that turns the fleet into pipeline stages:

```
ps aux | tp -w procs          # push tagged onto the temprs stack
tp -g python procs && echo hit # grep the tagged entry, compose in &&
journalctl -f | zcolorizer    # repaint a live log stream
sudo iftoprs --json | jq -c '.flows[] | {dst, sent_2s}'
kill "${lsf -t -c nginx}"     # bare PIDs feed another command
```

The convention that ties them together is not a framework — it is the agreement that a shipped CLI tool should be terse on stderr in the `<tool>: <thing>: <reason>` shape, quiet on a pipe, structured when asked, and drop-in against whatever it reimplements. Under `zshrs` they are dispatched the same way any external command is; the shell provides the completions, the history, and the pipeline plumbing, and the tools provide clean, TTY-aware, script-safe behaviour on the other end of it. That is the whole shared contract, and it is why the per-tool chapters can now assume it and move on to what each tool does that no other does.

The covered fleet — tools with their own books

This compendium is the single index to every command-line tool in the MenkeTechnologies stack, and it hands a full chapter to each tool that lacks a book of its own. Seven tools do not need that treatment here, because each already has a dedicated reference manual or companion volume in MenkeTechnologiesPublications. This chapter situates those seven inside the fleet — what each one is, what it ports, where it sits in the runtime picture, and exactly which book carries it in full — so nothing is duplicated and nothing is lost. Read this as the map; read the named books for the territory.

Six of the seven share one substrate worth naming up front: `fusevm`, a bytecode virtual machine with a three-tier Cranelift JIT. `zshrs`, `strykelang`, `awkrs`, `vimlrs`, and `elisprs` all lower their source to `fusevm` bytecode and execute on the same engine rather than each carrying a bespoke VM. `ztmux` is the outlier — a from-source terminal-multiplexer port with no language-runtime role. `powerliners` is the other outlier — a statusline renderer that both consumes the fleet (it ships segments that read the rkyv and JIT caches of `zshrs`, `strykelang`, and `awkrs`) and stands entirely on its own Rust binary suite.

```
FLEET INDEX — tools documented in their own volumes
tool          port / origin          dedicated book
----          -
zshrs         zsh (Bourne lineage)    zshrs — zsh in Rust (ref.)
strykelang    Perl 5 superset         strykelang — the Stryke lang (ref.)
awkrs        AWK (POSIX/gawk/mawk)   awkrs — AWK in Rust (ref.)
vimlrs       Vimscript (nvim eval)   vimlrs — Vimscript in Rust (ref.)
elisprs      Emacs Lisp              elisprs — Emacs Lisp/fusevm (ref.)
powerliners  powerline-status (Py)   powerliners — powerline port (comp.)
ztmux        tmux (C)                ztmux — tmux port (companion)
```

zshrs — the compiled shell at the center of the fleet

`zshrs` is a drop-in `zsh` replacement written in Rust, and it is the tool everything else in the stack is built to run inside. Its README states the codebase at 658k+ lines across 490 source files in a two-crate workspace (`zshrs` runtime plus `zshrs-daemon`), with the runtime split mechanically into a strict 1:1 port directory (`src/ported/` — 106 files including its `builtins/`, `zle/`, and `modules/` subtrees, each function mapping to a real `Src/<x>.c` `zsh` function and carrying a `/// Port of <cname>() from Src/<file>.c:NNNN` marker, enforced by `tests/port_purity.rs`) and a non-port extensions directory (`src/extensions/`) for features `zsh`'s C does not have. The design premise is that since the Bourne shell in 1970 every Unix shell has been an interpreter; `zshrs` compiles every command to `fusevm` bytecode and executes it on the VM. Its lexer and parser are direct ports of `Src/lex.c` and `Src/parse.c`, and the layered `ZshProgram` → `ZshList` → `ZshSublist` → `ZshPipe` → `ZshCommand` AST is preserved verbatim; only the bytecode compiler is original Rust, and it targets the same `Op` enum that `strykelang` emits, so the two frontends share fused superinstructions and the Cranelift JIT path.

Execution runs through the four tiers the README lays out. A warm script whose source is unchanged hits its rkyv image and skips `lex`, `parse`, and `compile` entirely — it mmaps a `fusevm::Chunk` and runs it; loops and branches take the block-JIT path to native code through Cranelift; straight-line arithmetic takes the linear-JIT path; and everything else — builtins, I/O, string work — falls to the jump-table interpreter with fused superinstructions. The README's figure for that spread is a 717 ms cold run against a 7 ms warm run on

the same script. Around that core the extensions layer supplies the pieces zsh forks for: a persistent worker pool of [2–18] threads (sized in `~/.zshrs/zshrs.toml`) that replaces `fork + exec`; 23 coreutils commands (`cat`, `head`, `tail`, `wc`, `sort`, `find`, `date`, `mktemp`, `seq`, `tr`, ...) run in-process as zero-fork builtins; parallel primitives (`async/await`, `pmap`, `pgrep`, `peach`, `barrier`) that compile their body to a `fusevm::Chunk` once and substitute a `{}` placeholder per iteration rather than forking N times, drawing a VM from a pool that preserves allocations across iterations; AOP `intercept` before/after/around advice; and 193 registered ZLE widgets. Bytecode images live under `~/.zshrs/images/` as an `index.rkyv` plus mmap-ready shards, with a read-only SQLite mirror (`catalog.db`, `history.db`) beside them for `dbview` and ad-hoc SQL inspection only — never consulted for cache hit/miss or execution. The binary also carries an LSP server, a DAP debug adapter, and a JetBrains plugin that drives both. The README's test-corpus table totals 869 tests across seven suites — 392 construct-corpus, 160 no-tree-walker behavioral pins, 87 parser-shape probes, 70 upstream `.ztst` files, and three smaller suites (provisional, as reported in the README, not independently run here). Its role in the fleet is the host: the runtime the entire CLI environment dispatches through.

```
zshrs --lsp                # language server over stdio
zshrs --dap HOST:PORT      # DAP debugger, TCP connect-back
pmap 'gzip {}' *.log        # parallel map, VM-run, zero fork
intercept before git \
{ echo "git $INTERCEPT_ARGS" >> ~/git.log }
dbview autoloads _git       # read-only browse of the mirror
```

Covered in full by its own book: **zshrs — zsh in Rust (reference manual)**.

strykelang — the Perl-compatible parallel language

strykelang (binary `stryke`, also installed as `s`) is a Perl 5 compatible interpreter in Rust with native parallel primitives. Per its README it uses NaN-boxed runtime values (`StrykeValue` is a `u64` carrying immediates — `undef`, an `i32`, or raw `f64` bits — or tagged `Arc<HeapObject>` pointers for big ints, strings, arrays, hashes, refs, regexes, and channels), a three-tier regex stack (Rust `regex` → `fancy-regex` for backreferences → `pcre2` for PCRE-only verbs), a match-dispatch bytecode VM (`vm.rs`) with a Cranelift block-and-linear-sub JIT (`jit.rs`), and rayon work-stealing across all cores for every parallel builtin. It parses and executes Perl 5 scripts directly — `-e` is optional when the first argument is not a file on disk and looks like code — through a context-sensitive lexer (regex-vs-division, heredocs, interpolation) and a recursive-descent-plus-Pratt parser, then lowers everything to bytecode; the project's stated policy is to add new VM opcodes rather than new AST node kinds when extending the language.

Where it meets the rest of the fleet is the `fusevm` bridge (`fusevm_bridge.rs`). **strykelang**'s own block JIT handles pure-integer segments, while string-bearing segments — comparisons, concat, `length/index`, `sprintf` with a literal format, mixed chains such as `length($s) >= $min` — are lowered to `fusevm`'s Cranelift block JIT through an abstract-stack analyzer with per-slot kind inference. A per-sub cache (`sub_fusevm_meta`) runs the eligibility detectors once for a hot sub and afterward hits an `OnceCell<Arc<fusevm::Chunk>>` for O(1) dispatch. On top of Perl it adds a wide surface: pipe-forward `|>` and thread-macro `->` syntax, `pmap/pgrep/psort/preduce` plus streaming variants (`pmaps/pgreps` over lazy, bounded-memory iterators), `varsync` shared state backed by `Arc<Mutex>` with atomic compound ops, `fan/fan_cap`, channels with a Go-style `pselect`, structs/enums/classes/traits, an `s_web` framework, distributed `pmap_on` over an SSH worker pool (`cluster`), and AI primitives. The standard library is spread across a large family of `builtins_*` and `math_wolfram_*` modules in the source tree; the README reports 10,487 primaries in its `%b` namespace and a 44 MB single static binary (both provisional as stated). It shares the `fusevm` opcode set and JIT path with **zshrs** — the two frontends target the same `Op` enum. In the stack **stryke** is the scripting layer — the one-liner and data-processing language that embeds inside **zshrs** — and it is the origin of the unit-test assertion framework (`zassert_*`) that **zshrs** ports as builtins.

```

val @doubled = @data |> pmap $_ * 2, progress => 1 # parallel map
val @evens   = @data |> pgrep $_ % 2 == 0         # parallel filter
varsync $counter = 0
fan 10000 { $counter++ }                         # always 10000
range(0, 1e9) |> pmaps { expensive(_) } |> take 10 # streaming

```

Covered in full by its own book: **strykelang — the Stryke language (reference manual)**.

awkrs — AWK as a bytecode engine

awkrs runs pattern → action programs over input records like POSIX **awk**, GNU **gawk**, and **mawk**, with a CLI that accepts the union of those three option sets. Its README positions it as POSIX **awk** plus the **gawk** extensions that appear in real scripts — **BEGINFILE/ENDFILE**, **coprocess** **|&**, **CSV** mode (**-k/--csv**), **PROCINFO/SYMTAB/FUNCTAB**, **@include/@load/@namespace**, **/inet/tcp|udp** client sockets, and **MPFR** big numbers via **-M** (backed by the **rug/gmp-mpfr-sys** crates, which is why the build wants a C compiler and **make**). The engine is a fused-superinstruction stack bytecode VM with peephole fusion — **print \$N** collapses to **PrintFieldStdout**, **i++/++i/i = i + 1** to **IncrSlot**, **s += \$N** to **AddFieldToSlot**, and **\$1 ", " \$2** to **ConcatPoolStr** — and scalars get **u16** slot indices so reads and writes are flat-array indexing instead of **HashMap** lookups. Fields are stored as **(u32, u32)** byte ranges into the record with owned strings only on assignment, and **print \$N** under the default **FS** scans raw bytes straight out of the mmap'd file to find the *N*th whitespace field.

A default-on **fusevm** offload path (the crate pins **fusevm** 0.14.5 with the **jit**, **jit-disk-cache**, and **aot** features) lowers eligible numeric chunks — arithmetic, comparisons, compound assignments, **int**, the transcendentals, the bitwise builtins, and trapping division/modulo — to **fusevm::Ops** and block-JIT-compile hot loops on their first run, persisting native code to **~/.cache/fusevm-jit**, the same on-disk JIT cache **zshrs** uses; the README measures 14–110× over the bytecode interpreter on pure-numeric accumulator loops and reports it bit-for-bit identical to **AWKRS_FUSEVM=0**. Separately, **-f script.awk** runs memoize the compiled program to **~/.awkrs/scripts.rkyv** as an rkyv shard (rkyv outer envelope, bincode inner blob — the same layout as **zshrs** and **stryke**) with zero-copy mmap reads, invalidated on source or binary mtime change and disabled by **AWKRS_CACHE=0**; the README's survey of **BWK** **awk**, **gawk**, **mawk**, **goawk**, **fawk**, and **zawk** concludes **awkrs** is the first **awk** to pair a bytecode VM with a persistent on-disk bytecode cache (**fawk** has the JIT but recompiles each run; **gawk**'s **pm-gawk** persists variables, not compiled bytecode). Record processing parallelizes via **rayon** with deterministic output reordering when a static parallel-safety check passes (**-j/--threads**, default 1; regular files are memory-mapped, stdin chunks up to **--read-ahead** lines). Its **examples/** directory carries dozens of standalone **awk** programs — a recursive-descent calculator, a Brainfuck interpreter, Dijkstra, Aho-Corasick, an LZW codec — each byte-for-byte verified against **gawk** in CI, so they double as **gawk**-extension regression tests. Reported benchmarks against BSD **awk**, **gawk** 5.4.0, and **mawk** 1.3.4 (hyperfine, 1 M lines, Apple silicon) are provisional as printed. In the fleet **awkrs** absorbs the **AWK** text-processing role and shares the **fusevm** numeric-op set with the language siblings.

```

awkrs -f script.awk data.txt           # cached: skips recompile
awkrs -j 8 '{ print $1 }' big.txt       # parallel, order preserved
awkrs -k '{ print $2 }' data.csv        # gawk-style CSV mode
AWKRS_FUSEVM=0 awkrs -f numeric.awk    # force the interpreter

```

Covered in full by its own book: **awkrs — AWK in Rust (reference manual)**.

vimlrs — Vimscript outside the editor

vimlrs (binary **viml**) runs **.vim** scripts as ordinary standalone programs. Its README describes it as the first compiled standalone VimL interpreter — Vimscript has only ever run embedded inside Vim or Neovim — and a faithful port of Neovim's C **eval/*** tree, with the C source treated as the spec rather than reinvented. The ported subtree under **src/ported/** mirrors that C in 54 Rust files (**eval**, **typval**, **list**, **dict**, **blob**,

`charset`, `mbyte`, `ex_eval`, `mark`, and the rest), and `scripts/gen_port_stubs.sh` emits one cited stub per not-yet-ported C function so the remaining surface is enumerated and the drift gate covers it. It is the fourth language hosted on `fusevm`: it lexes and parses VimL through the `eval1...eval7` grammar to an AST, lowers that to `fusevm` bytecode, and lets the shared engine's three-tier Cranelift JIT run it, carrying no VM or JIT of its own. The value layer — `typval`, `list`, insertion-ordered `dict`, `blob`, with the typed `tv_dict_*` accessors, coercions, `typval_compare`, and `string():echo` rendering — is ported and marked working.

The JIT story the README documents is that provably-Number locals become `Op::GetSlot/SetSlot` and a function's numeric `while` loop rotates so the condition is the backedge and trace-JIT-compiles to native code, at both function and script (top-level) scope; float accumulators, short-circuit compound conditions, native `%`, native bitwise builtins, and numeric ternaries all stay trace-eligible, and `VIMLR5_JIT_STATS=1 viml script.vim` reports the traces it compiled while `VIMLR5_NO_JIT=1` forces the interpreter baseline — the README's figure is ~15–100× on a 20M-iteration loop (provisional as stated). The builtin surface is partial: roughly 113 functions from `funcs.c` (string/list/dict, char-indexed string ops, `json_encode/json_decode`, `system`, `sha256`, `rand/srand` bit-exact against Neovim's xoshiro128** generator, date/time, transcendental math), with the full table pending. A large share of the work is in the regex dialect — Vim magic, ASCII-only `\a/\w/\d` character classes per `:help /\a`, option-derived atoms, POSIX bracket classes, and `\c/\C` case-fold rules — and every `examples/*.vim` is written as a self-test run in CI against real vim/nvim, so the regex and builtin behavior is pinned to the editors it imitates. It ships the standalone `viml` binary with `-e/-c/file/--repl`, an rkyv script cache at `~/.cache/vimlrs`, an AOT `--build`, a `--disasm` disassembler, an `--lsp` server, and a `--dap` debugger, plus a vim9script foundation (`:def` with typed parameters, automatic line continuation). Its fleet role is the Vimscript execution engine, paired with the vim-editor work elsewhere in the stack.

```
viml -e 'echo range(5)->map({_, v -> v * v})' " [0, 1, 4, 9, 16]
viml script.vim                             " run a .vim standalone
VIMLR5_JIT_STATS=1 viml hotloop.vim          " report traces compiled
viml --lsp                                   " diagnostics, hover, ...
```

Covered in full by its own book: **vimlrs — Vimscript in Rust (reference manual)**.

elisprs — Emacs Lisp on the shared VM

`elisprs` (binary `elisp`) runs Emacs Lisp `.el` files as standalone programs from the command line, with a REPL and no Emacs process required. Its README frames it as the fifth language hosted on `fusevm`. The reason it exists as a distinct effort is that Emacs Lisp is a Lisp-2 — every symbol carries a separate value cell and function cell — and supports both lexical and dynamic scoping; those facts are the personality of the language, so `elisprs` owns the value model and semantics and leans on `fusevm` only for execution. The source splits along that line: `reader.rs` is an elisp-correct S-expression reader (`1+/1-`, `#'foo`, `?c`, `:kw`, backquote, dotted pairs); `host.rs` holds the `ElispHost` object heap — interned symbols, real dotted cons cells, vectors, hash tables, and closures — the Lisp-2 obarray, the dynamic binding stack, and the `fusevm` extension handler that runs elisp ops, with every heap object reaching the VM as a `Value::Obj(u32)` handle; `compiler.rs` lowers each macro-expanded top-level form to a `fusevm::Chunk` with lambda bodies as sub-chunks; and `builtins.rs` plus `prelude.rs` carry the standard library, part native and part written in elisp on top of the primitives. An earlier bootstrap on the `rust_lisp` crate was replaced by this own value model, and `rust_lisp` is no longer a dependency.

The README enumerates 21 special forms (`quote`, `lambda`, `if`, `cond`, `let/let*`, `defun`, `defmacro`, `defvar`, `condition-case`, `unwind-protect`, ...) and roughly 90 subrs across arithmetic, lists, the `c*r` combinators, vectors, hash tables, predicates, strings, regexp, markers, and text properties, plus `pcase` (including backquote patterns), generalized `setf`, and both lexical (`lexical-binding: t`) and dynamic binding. It goes far enough into the editor layer to carry a registry of named live buffers with point, mark, and narrowing, first-class markers that auto-adjust across inserts and deletes, and string and buffer text-property intervals — enough, the README claims, that `tabulated-list-print` produces byte-identical propertyized output to GNU Emacs. It AOT-compiles to standalone native binaries (`--aot-exe`, through `fusevm::aot::compile_object`), caches lowered bytecode in an rkyv shard at `~/.elisprs`, and ships `--lsp` and `--dap` servers. It is explicit that this

is a useful elisp core, not the ~1000-subr GNU Emacs surface — overlays, a real interval tree, and redisplay are named out of scope rather than faked. In the fleet it is the elisp execution engine that also underpins the embedded-elisp editor work in **zmax**.

```
elisp -e '(mapcar (lambda (x) (* x x)) (list 1 2 3 4))'
; => (1 4 9 16)
elisp examples/demo.el           ; run a self-testing ERT file
elisp --aot-exe prog.el -o a.out  ; standalone native binary
elisp --lsp                       ; completion / hover / diagnostics
```

Covered in full by its own book: **elisprs — Emacs Lisp on fusevm (reference manual)**.

powerliners — the statusline without the Python tax

powerliners is a Rust port of **powerline-status**, the Python-driven statusline and prompt renderer used in tmux, zsh, bash, vim, and the Python REPL. Its README's rationale is startup cost: the Python implementation pays a ~50–150 ms interpreter-startup tax on every render, which compounds because tmux refreshes the statusline per interval and per window and zsh redraws its prompt after keystroke-triggered **precmd** hooks. **powerliners** is a single static binary with sub-millisecond render, and it is drop-in compatible with the existing **powerline/config** JSON theme and segment files, so themes stay unchanged. It ships a five-binary suite mirroring the upstream layout — **powerline** (the native client, forwarding **argv + cwd + env** to the daemon and falling back to a **powerline-render** exec if the daemon is unreachable), **powerline-config**, **powerline-lint**, **powerline-render**, and **powerline-daemon** — and the daemon speaks the identical **argc\0arg\0...\0cwd\0KEY=VAL\0...\0\0** wire format with an **E0F\0\0** shutdown sentinel, so the pip-installed Python C client talks to a running Rust daemon unchanged over the same socket path (**/tmp/powerline-ipc- $\$$ UID** on macOS, an abstract socket on Linux).

The render path is a function-level port of the Python pipeline: config-cascade load and **mergedicts**, colorscheme alias chasing and cterm resolution, **gen_segment_getter**, **process_segment**, the **do_render/render_segments** loop with hard/soft divider insertion and outer padding, and **TmuxRenderer #[fg=...,bg=...]** markup emission. It wires 54 segment adapters (55 **ADAPTERS** keys with the bare **exec** alias) and adds net-new ones — a p10k-style single-fork **git_status**, plus Docker/Kubernetes/AWS/GCP/GitHub-CI context segments — and, most relevant to this book, four that read the fleet's own caches: **powerliners.zshrs.rkyv_cache**, **powerliners.stryke.rkyv_cache**, **powerliners.awkrs.rkyv_cache**, and **powerliners.fusevm.jit_cache**, each a pure filesystem probe over a documented resolution chain, plus version segments for **zshrs/stryke/awkrs**. A separate extension (**src/extensions/watch.rs**) turns the normally pull-only prompt into a push: the warm daemon watches each client's **cwd**, **.git/HEAD**, **.git/index**, and active branch ref through the **notify** crate (kqueue/FSEvents on macOS, inotify on Linux) and writes a single wake byte to a per-client FIFO that zsh's ZLE reads via **zle -F**, redrawing the prompt in place the instant you check out a branch in another pane. The README reports 134/137 upstream **.py** files at its DONE tier (97.8% — the remaining three are class-only shells with no function bodies left to port), 2436 passing library tests, and 462 parity tests run against the live upstream Python (all provisional as printed). That makes **powerliners** the fleet's prompt-and-statusline surface, and the one tool here whose output visibly reflects the state of the rest of the stack.

```
brew install powerliners           # installs the 5-binary suite
powerline-daemon -q                # double-fork, detach, bind the socket
POWERLINE_CONFIG_PATHS=~/config/powerline \
  powerline-daemon --foreground --socket /tmp/powerliners
powerline-render tmux right -p ~/config/powerline # no daemon
```

Covered in full by its own book: **powerliners — powerline port (companion)**.

ztmux — tmux, reimplemented in Rust

ztmux is a from-source port of **tmux** to Rust — the whole program: server, client, grid/screen model, input parser, layouts, the command language, formats, and the terminal back end. Its README is emphatic that it is not a wrapper around the **tmux** binary and not control mode (**tmux -CC**); it is **tmux** itself, reimplemented. The port stands on the upstream **tmux** C sources, vendored read-only and SHA-pinned under **vendor/tmux/**, with **Cargo.toml** declaring its own **[workspace]** that excludes the reference so Cargo never walks into it, and every ported function carries a back-link comment to its C origin (e.g. `/// C `vendor/tmux/grid.c:374`:
`struct grid *grid_create(u_int sx, u_int sy, u_int hlimit)``). Where **tmux** links C libraries, **ztmux** does not: the event loop is Rust under **src/extensions/event_loop/** and terminfo is read by the **terminfo-lean** crate, so there is nothing to **pkg-config** and no **-dev** package to install — **build.rs** is left generating only the **lalrpop** command grammar and the REPL's completion tables. It opens its own socket namespace (**ztmux-<uid>**) so it never collides with a running **tmux**.

Correctness is measured rather than asserted: a parity suite runs identical inputs through the real **tmux** and **ztmux** and diffs them byte-for-byte, reported at 1188/1188 cases passing against the vendored **tmux**, and it has earned that by root-causing real divergences — a **{l:...}** server crash traced to a dropped pointer increment in **format_unescape**, even-horizontal layout rounding, **{pane_current_command}** on macOS. An anti-drift gate (**tests/ported_fn_names_match_c.rs**) fails the build if a free **fn** is added to **src/** whose name has no counterpart in the **tmux** C source, with pre-existing wrappers frozen in an allowlist to burn down. The server pins **PROTOCOL_VERSION = 8** (verified in **src/ported/tmux_protocol_h.rs**) to match the separate **ztmux-core** client engine, so a GUI host can drive it over the same wire protocol without upstream **tmux**'s release cadence being able to break the contract. On top of the port it ships 130 extension modules under **src/extensions/**, carrying 113 named **ztmux <name>** subcommands — JSON-emitting inspectors over the live server (**ps**, **pstree**, **git**, **net**, ...), live **ratatui** TUIs (**dashboard**, **switcher**, **watch**), and opt-in zellij-style modes (framed/inset panes, a pane stack, a tab bar, floating panes, modal keybindings, and **resurrect** session persistence) — all exempt from the anti-drift gate precisely because they are not **tmux**, and all gated behind **set -g** toggles so **@ztmux-ratatui off** restores the byte-for-byte plain-**tmux** draw path. In the fleet **ztmux** is the multiplexer the shell lives inside; unlike the five language tools it does not touch **fusevm**.

```
cargo run --release -- new-session      # start server + session
bash parity/run_parity.sh --summary     # ztmux vs tmux, every case
ztmux ps --json                        # original inspector, JSON
ztmux dashboard                        # full-screen server monitor
```

Covered in full by its own book: **ztmux — tmux port (companion)**.

How to read the rest of this compendium

Those seven are the tools with dedicated volumes; everything after this chapter is a tool that has no book elsewhere and therefore gets its full chapter here. When a later chapter references a shell primitive, a parallel builtin, an AWK cache detail, or a statusline segment, the authoritative treatment lives in the volume named above — this index is deliberately thin on internals so those books stay the single source. Two structural facts carry across the fleet and are worth holding while you read on: five of these tools converge on one bytecode VM and JIT (**fusevm**), which is why their performance stories rhyme; and each porting effort treats the upstream source as the specification, with the original engineering being the Rust architecture, the binary suite, the drop-in compatibility, and the measured parity — not the invention of the language or format being ported.

lssofrs — a Rust reimplementation of lsof

`lssofrs` lists the open files of every process on the system: regular files, directories, character and block devices, FIFOs, pipes, kqueues, and — the part most people actually reach for it for — sockets. It is a Rust rewrite of `lsofng`, which is itself a modernized fork of Vic Abell's original `lsof`(8). The lineage matters, because the design of the tool is not an invention: the option grammar, the columnar output, the `-i` network selector, the field-output mode, the `+L` link-count filter, and the terse `-t` form are all lifted from `lsof` and reproduced faithfully. The original engineering here is the port — the FFI against the Darwin kernel, the `/proc` walker on Linux, the `sysctl` path on FreeBSD, the parallel gathering, and a dense set of interactive TUI modes layered on top of the classic single-shot behavior. The binary is published on crates.io as `lssofrs` (`Cargo.toml:2`), version 4.9.1 (`Cargo.toml:3`), under MIT, and ships two identical binaries — `lssofrs` and the shorter `lsf` — both built from `src/main.rs` (`Cargo.toml:17-23`, `default-run = "lssofrs"`). The dependency set is small and durable: `clap` for parsing, `serde/serde_json` for interchange, `ratatui` + `crossterm` for the TUI, `nix` and `libc` for the syscall surface, `users` for name resolution, `regex` for command matching, `chrono` for timestamps, `rayon` for the parallel gather, `toml` + `dirs` for config (`Cargo.toml:25-38`). There is no build script, no network fetch at build time, and nothing that needs a C toolchain beyond what `libc` already links.

What it ports, and how honestly

The compatibility surface is deliberate. `src/cli.rs` does not just define a clap parser and stop; it carries a hand-written `normalize_lsof_args` pass (`cli.rs:368`) that runs *before* clap sees anything. `lsof`'s option grammar is not getopt-clean — it has a `+` prefix family (`+L`, `+d`, `+D`, `+c`, `+r`), options with attached arguments (`-o9`, `-sTCP:LISTEN`, `+L1`), and a long tail of platform-only flags that scripts pass but which mean nothing on a given host. The normalizer splits `--clusters` getopt-style through `expand_dash_cluster` (`cli.rs:423`), expands `+` tokens through `expand_plus_token` (`cli.rs:517`), maps each `lsof` option to its clap equivalent (a short flag, a `--long`, or nothing at all), and passes everything after `--` through untouched (`cli.rs:388-415`). The program name in `argv[0]` is copied verbatim so `lsf` and `lssofrs` both parse identically.

Two categories are worth calling out because they are where a naive rewrite would break real scripts. First, the accept-and-ignore set: single-char options like `-b` `-O` `-C` `-M` `-X` `-E` `-L` are consumed inside a cluster and dropped (`cli.rs:452`), `-z` `-Z` `-S` `-f` drop any attached argument and end the cluster (`cli.rs:456`), and `-A` `-k` `-m` `-D` `-e` are treated as taking a required argument, so the following `argv` token is swallowed if none is attached (`cli.rs:460-463`). A script that invokes `lsof` with, say, `-b` (avoid kernel blocking) or `-O` (bypass overhead) runs under `lssofrs` without error even though those flags have no meaning in this implementation. The `+` side mirrors this: `+w` `+f` `+E` `+M` `+X` `+m` are silent toggles and `+e` consumes a separate argument (`cli.rs:564-569`).

Second, the semantically overloaded flags are disambiguated exactly the way `lsof` does. `-s` with no argument means “always show size,” `-s` with a colon spec (`-sTCP:LISTEN`) means “filter sockets by state,” and a `-s` followed by a non-colon rest is still treated as size-always with the cluster continuing (`cli.rs:479-491`). `-o` optionally takes a leading digit count for offset precision (`-o9` = nine digits), which the normalizer peels off character by character and re-emits as `--offset-always` plus `--offset-digits` (`cli.rs:465-476`). `+L` alone lists the NLINK column; `+L1` additionally selects only files whose link count is below 1 — i.e. unlinked-but-open files, the classic deleted-file-holding-a-disk-block case (`cli.rs:521-531`). A handful of `lsof`-only flags are mapped to real behavior rather than ignored: `-T` becomes `--tcp-info`, `-x` becomes `--cross-over`

(follow symlinks/mounts while walking `+d/+D`), and `-K` becomes `--list-tasks` (`cli.rs:494-505`). `+c` sets the command-column width with `+c0` meaning unlimited, and `+r` sets repeat-until-empty with an optional attached interval (`cli.rs:546-561`).

The point of all this is drop-in substitution. You can alias `lsyf` to `lsf` and expect existing muscle memory and existing scripts to keep working, including the flags that do nothing here. The whole bridge is one file and one function pair, which keeps the compatibility contract auditable in one place rather than smeared across the parser.

Architecture

The crate is split cleanly between platform data-gathering, filtering, and output, with a large TUI subsystem bolted on — 29 modules under `src/`. `src/main.rs` is the dispatcher: it parses args, prints help or version and returns early, resolves the effective theme and refresh interval (CLI flags override the saved `~/.lsyfrs.conf` preferences, `main.rs:64-79`), then branches into exactly one mode. The dispatch order is explicit and matters, because several flags can coexist on the command line; the first match wins. It runs: unified TUI, watch, follow, top, monitor, leak-detect, the socket-stall classifier, live-summary (`--summary` with `-r`), generic repeat, and only then the single-shot modes — stale, ports, pipe-chain, csv, net-map, tree, summary, json, terse, field-output, and finally the default columnar printer (`main.rs:82-191`).

Platform enumeration is `#[cfg]`-gated behind three modules that all produce the same `Vec<types::Process>`:

- `src/darwin.rs` — macOS via `libproc` FFI. It declares `proc_listpids` and `proc_pidinfo` as raw `extern "C"` and calls them against `repr(C)` structs matched to the kernel headers, first to size the PID buffer, then to fill it. No shelling out, no parsing of another tool's output. Because macOS exposes socket send/rcv buffer *limits* alongside occupancy, the Darwin path fills in the `send_buf_size/rcv_buf_size` fields that the stall classifier later needs.
- `src/linux.rs` — Linux via the `/proc` filesystem. Per PID it reads `cwd`, `root`, and `exe` symlinks into the `cwd/rtd/txt` pseudo-descriptors Isof users expect, then walks `/proc/<pid>/fd` for the numbered descriptors. Sockets are correlated through a prebuilt socket map keyed by inode. Queue occupancies come from `/proc/net/tcp`, which does not expose the buffer limits, so `send_buf_size/rcv_buf_size` stay `None` on Linux.
- `src/freebsd.rs` — FreeBSD via `sysctl` plus `procfs`.

`gather_processes()` in `main.rs:193-211` selects the right backend at compile time and prints an “unsupported platform” line to `stderr` on anything else. Gathering is parallelized with `rayon` across PIDs, and username resolution is memoized per `Process` through a `OnceCell` so the `getpwuid` lookup runs at most once regardless of how many files that process holds (`types.rs:416`, `types.rs:466-472`). `Process` even hand-implements `Clone` so that a populated username cache is carried into the clone rather than recomputed (`types.rs:419-440`).

The core data model lives in `src/types.rs`. `FileType` enumerates the full Isof type vocabulary — `REG`, `DIR`, `CHR`, `BLK`, `FIFO`, `sock`, `LINK`, `PIPE`, `KQUE`, `unix`, `IPv4`, `IPv6`, `sysm`, `PSEM`, `Pshm`, `ATALK`, `FSEV`, plus an `Unknown(String)` fallback for raw type codes (`types.rs:41-104`). `TcpState` maps the eleven kernel TCP states to Isof's spelling (`ESTABLISHED`, `TIME_WAIT`, `FIN_WAIT_1`, and so on) with an `Unknown(i32)` catch-all, and `from_raw` decodes the numeric kernel codes 0–10 (`types.rs:174-209`). `FdName` is the descriptor identity — the special forms `Cwd`, `Rtd`, `Txt`, `Mem`, `Err` render as `cwd/rtd/txt/mem/err`, numbered descriptors carry an access suffix (`3r`, `3w`, `3u`) through `with_access`, and `Other(String)` handles anything else (`types.rs:243-288`). `OpenFile` carries everything a column can display — fd, access mode, lock char, type, device, size, offset, inode, nlink, name, an optional `name_append` (this is where `(deleted)` lands), plus platform extras `rdev`, `file_flags`, and `file_struct_addr` (`types.rs:291-327`). Its `SocketInfo` is richer than the columns show: local and foreign endpoints, protocol, TCP state, `recv_queue/send_queue` occupancy, `recv_buf_size/send_buf_size` limits, and raw `socket_options/socket_state` words (`types.rs:218-240`) — the queue and buffer fields exist specifically to feed the backpressure classifier. The selection-flag constants (`SEL_CMD`, `SEL_FD`, `SEL_NET`, `SEL_PID`, `SEL_UID`, and the composite `SEL_PROC/SEL_FILE/SEL_`

NW unions) are the C implementation's bitmask scheme reproduced verbatim, and three tests pin each union to its documented bit-OR ([types.rs:9-37](#), [types.rs:519-531](#)).

Filtering is centralized in [src/filter.rs](#). `Filter::from_args` parses the comma-separated, `^-negatable` PID/user/command/fd selectors into separate include and exclude vectors, compiles `/regex/` command specs into `regex::Regex`, and precomputes prefix strings for file and directory matching so the hot path avoids re-allocating. `-s` protocol:state specs are parsed into `StateFilter` structs with an uppercased protocol and a state list where empty means “any state for this protocol.” The `-a` flag flips the whole thing from OR to AND semantics, matching Isof’s selection algebra. `gather_and_filter` in [main.rs:281-297](#) applies the process predicate, prunes each surviving process’s file list, drops processes left with no files (unless `-t` terse is set, which keeps bare PIDs), and finally sorts by PID for stable output.

Output splits across [src/output.rs](#) (columnar and field formats, ANSI theming), [src/json.rs](#) (serde-derived JSON), and [src/csv_out.rs](#) (RFC 4180 CSV). The remaining modules are the specialized modes — `summary`, `tree`, `stale`, `ports`, `pipe_chain`, `net_map`, `watch`, `follow`, `leak`, `delta`, `monitor`, `stall`, `top` — each a self-contained view over the same gathered `Vec<Process>`. The live TUI framework is [src/tui_app.rs](#) (a `TuiMode` trait giving every live mode the same keybindings, alternate-screen handling, and atomic frame rendering) with [src/tui_tabs.rs](#) implementing the unified tabbed dashboard on `ratatui` and `crossterm`.

The CLI surface

The classic invocations are unchanged from Isof:

```
lsf                # every open file on the system
lsf -p 1234         # files held by PID 1234
lsf -c nginx        # command starting with "nginx"
lsf -c '/nginx|apache/' # command matched by regex
lsf -u root,www-data # files for either user
lsf -u ^root        # every user except root
lsf -a -p 1234 -i    # AND: PID 1234's network files
lsf -d 0-10         # only descriptors 0 through 10
lsf /var/log/syslog # who has this path open
lsf -t -c nginx     # bare PIDs, for kill $(lsf -t ...)
```

The network selector follows Isof’s `-i [4|6][proto][@host][:port]` grammar. `NetworkFilter` in [types.rs:476-492](#) holds an optional protocol, address family, address, host, precomputed `parsed_host` (the host string parsed once as an `IpAddr` for O(1) endpoint comparison rather than re-parsing per file), and a port range:

```
lsf -i              # all internet connections
lsf -i 4            # IPv4 only
lsf -i 6            # IPv6 only
lsf -i TCP          # TCP only
lsf -i :443         # port 443
lsf -i TCP:443      # TCP on port 443
lsf -i 4TCP:22      # IPv4 TCP on port 22 (one token)
lsf -s TCP:LISTEN   # TCP sockets in LISTEN state
```

Output formats cover scripting and interchange. `-t` is terse (PIDs only); `-F` is Isof field output where each requested field is emitted on its own line prefixed by a type letter — `p` pid, `c` command, `g` pgid, `R` ppid, `u` uid, `L` login name, `f` fd, `a` access, `t` type, `D` device (hex), `s` size, `o` offset (as `0t<n>`), `i` inode, `n` name, `P` protocol, `T` TCP info (emitted as `TST=<state>`) ([output.rs:426-499](#)), with `p f n` as the default field set when the argument is empty ([output.rs:417-418](#)). `-0` swaps the field terminator from newline to NUL for safe parsing of paths with embedded whitespace. `-J/--json` emits a JSON array via `serde`; `--csv` emits RFC 4180 CSV. When stdout is not a TTY, columnar output drops its colors and cyberpunk headers and prints plain, script-safe text ([main.rs:57-62](#)); the `--color` flag forces `always/never/auto`. A version query is verbatim Isof-shaped:

```
$ lsf -v
lsufrs 4.9.1
```

Terse mode is the pipeline workhorse — one PID per line, deduplicated by the sort-and-retain in `gather_and_filter`:

```
$ lsf -t -c zsh
13855
13860
13925
13927
```

Beyond the Isof-compatible core, `lsufrs` adds a set of derived views that are all single-shot when piped and interactive when attached to a terminal:

```
lsf --tree           # process tree + per-node FD breakdown
lsf --summary        # aggregate FD stats, bars, top procs
lsf --ports          # listening TCP/UDP ports (cross-platform)
lsf --net-map        # connections grouped by remote host
lsf --pipe-chain     # pipe / unix-socket IPC pairs
lsf --stale          # FDs pointing at deleted files (+L1)
lsf --top 10         # live top-N processes by FD count
lsf --watch /tmp/x.sock # +OPEN / -CLOSE events for one path
lsf --follow 1234    # one process's FDs in real time
lsf --leak-detect=10,5 # flag FD counts rising 5 times running
lsf --stall          # per-FD socket backpressure classifier
lsf --monitor        # full-screen top(1)-style refresh
lsf --delta -r 2     # repeat every 2s, color new/gone FDs
lsf --tui            # unified tabbed dashboard for the above
```

Every one of these accepts the same selection filters, so `lsf --tree -u root`, `lsf --stale -u www-data --json`, or `lsf --ports -u root` all compose. The leak detector defaults to a 5-second interval and a 3-consecutive-increase threshold, clamps the threshold to a minimum of 2, and parses an optional `interval,threshold` spec (`cli.rs:346-357`). The `-V` flag reproduces Isov's "report search items that matched nothing" behavior: after filtering, `unfound_specs` walks the positive `-p/-c/-u/file` specs and prints a `no PID found/no command found/no user found/no file found` line to stderr for each that matched no process, skipping exclusions (^) and command regexes (`main.rs:216-279`). Directory arguments count as matched when any held file lives under them, so `-V /tmp` is satisfied by `/tmp/anything`.

--ports

`--ports` is the `ss -tlnp` analogue and its selection rule is exact (`ports.rs:22-54`): a file qualifies only if its `SocketInfo` protocol is TCP or UDP; TCP additionally must be in `LISTEN` state; UDP qualifies when it has a bound local port greater than zero. SCTP and other protocols are excluded, established/`TIME_WAIT`/`CLOSE_WAIT` TCP sockets are excluded, and protocol matching is case-insensitive. Rows are grouped into a `BTreeMap<u16, ...>` keyed by port, which sorts the output ascending by port number. A run on a developer laptop:

```
== Listening Ports ==

PROTO  LOCAL ADDR      PORT    PID  USER  COMMAND
TCP    0.0.0.0         6986   13890 user  WavesLocalServer
TCP    ::             6986   13890 user  WavesLocalServer
UDP    0.0.0.0         7579   13917 user  NIHostIntegration
UDP    ::             55135  13752 user  replicatord
TCP    0.0.0.0         56230  13676 user  rapportd
UDP    203.0.113.3    59961  86624 user  vpn Helper
```

(Login name and remote addresses above are sanitized for print; the tool shows the real values.) The column widths are computed from the data with sane minimums, alternating rows get a subtle background, and a `N`

`listening port(s) found` footer closes the block. `--net-map` uses the mirror-image rule: it keeps only IPv4/IPv6 files with a *foreign* endpoint, skips wildcard/unconnected sockets, and groups by remote host into a `HashMap`, accumulating per-host protocol sets, port sets, and the distinct processes talking to that host (`net_map.rs:26-90`).

--summary

`--summary` aggregates the whole scan into a type histogram and a top-N-by-FD table. It is one of the modes that goes live when combined with `-r` (`main.rs:125-128` routes `--summary -r` to `run_summary_live`). A single-shot run:

```
=== lsof.rs summary ===

Processes: 431          Open files: 4,032

-- File type breakdown --

REG      1,739    43.1% ██████████
CHR      1,326    32.9% ██████████
DIR        422    10.5% ████████
PIPE       196     4.9% ██████
unix       186     4.6% ██████
IPv4        55     1.4% █████
FIFO        45     1.1% █████

-- Top 10 processes by FD count --

  PID  COMMAND      USER      FDs
  ---  -
13649  UserEventAgent  user       247
86359  vpng             user       246
13711  corespotlightd  user       136
```

--stall

The socket backpressure classifier is the newest live mode and has no lsof antecedent (`src/stall.rs`). It samples every socket FD's kernel send/rcv buffer occupancy across successive scans (`--stall` is meant to be paired with `-r` to set the interval) and issues a directional verdict per `(pid, fd)`: `TX-STALLED` when the send buffer stays persistently near capacity and is not draining — the local process keeps writing but the peer or the network is not reading; `RX-STARVED` when the receive buffer stays persistently high and grows — bytes have arrived but the process is not `read()`-ing them fast enough; and `HEALTHY` otherwise. `StallDetector` keeps a bounded per-FD history (`HISTORY_SIZE = 16`), requires at least three consecutive samples before it will commit to a verdict (`MIN_SAMPLES = 3`), and treats an occupancy at or above 80% of the buffer limit as “near capacity” (`HIGH_RATIO = 0.80`) (`stall.rs:27-34`). Classification adapts to platform: on macOS all four inputs are available so it uses a fill-ratio rule (`tx_severity/rx_severity` compare queue against buffer size); on Linux, where `/proc/net/tcp` gives occupancy but not the limit, it falls back to a trend rule — persistently non-zero and non-draining for TX, persistently non-zero and strictly growing for RX (`stall.rs:265-324`). When both directions are backed up, the higher-severity side wins (`stall.rs:328-347`). PID reuse is handled: if the command name on a `(pid, fd)` changes, the history is cleared and the verdict resets. Only currently-seen stalled sockets are printed, so a quiet system reports `No stalled sockets detected`.

The TUI layer

The interactive modes are where `lsOfrs` departs most from anything in the `lsOf` tradition. `--tui` opens a full-screen tabbed dashboard — TOP, SUMMARY, PORTS, TREE, NET-MAP, PIPES, STALE — navigable by clicking, `Tab`/arrow, or number keys 1–7. There are 31 built-in color themes, confirmed by both the enum-length assertion in `theme.rs` (`ThemeName::ALL.len() == 31`, `theme.rs:860`) and the documented list in `lsOfrs.default.conf` (NeonSprawl through Zaibatsu). A theme chooser (`c`) previews them live with color swatches; a theme editor (`C`) builds custom six-color palettes and writes them to `~/.lsOfrs.conf`. Config persistence is TOML via `src/config.rs`, and the `Prefs` struct stores more than the visible knobs suggest: the selected theme, refresh rate, border toggle, the last active tab, a named map of custom themes, the active custom theme, a list of pinned PIDs, and the `sort_frozen`, `compact_view`, and `hover_tooltips` toggles (`config.rs:24-56`). Everything is `#[serde(default)]`-guarded so an old or partial config never fails to load — a missing field falls back to its default rather than erroring, which is the forward-compat behavior you want for a file the tool rewrites itself. `load()` reads `~/.lsOfrs.conf` and silently returns defaults on any parse error (`config.rs:88-99`); `save()` is compiled out under `#[cfg(test)]` so the test suite never touches the user's real config (`config.rs:102-114`). All live modes share a common keymap through the `TuiMode` trait — digit keys set the refresh interval, `p` pauses collection, `/` opens a regex filter popup, `y` copies the selected row, `e` exports the tab, `q/Esc/Ctrl-C` quit. Non-TTY output never enters the TUI; it prints one snapshot and exits, which keeps every mode pipeline-safe.

Repeat, delta, and the live loop

The plain repeat path (`-r N`, no dedicated mode) lives in `run_repeat` (`main.rs:299-365`). It optionally wires a `delta::DeltaTracker`: each iteration calls `begin_iteration`, gathers and filters, records every current `(pid, fd, name)`, and after printing counts what went away with `count_gone`. When `--delta` is set the columnar printer receives a per-row classifier closure that returns `New/Gone/Unchanged`, so appearing FDs are colored and vanished ones are listed after the table, followed by a one-line change summary and a `=====` separator between frames. JSON, terse, and summary outputs are honored inside the loop too, so `lsf --summary -r 2` and `lsf -J -r 2` both stream. Leak detection (`run_leak_detect`) and the stall classifier (`run_stall`) follow the same shape — construct a stateful detector once, then `update/report` on each interval — which keeps all three live analyzers structurally identical and easy to reason about (`main.rs:367-389`).

Integration into the stack

`lsOfrs` installs as two binaries on `PATH` (`lsOfrs` and `lsf`), a zsh completion, and two man pages. The completion at `completions/_lsOfrs` is a real `#compdef lsOfrs lsf` function covering both command names, built on `_arguments -C -s -S` with dedicated state handlers — `->filedes` for `-d`, `->addresses` for `-i`, `->states` for `-s`, `->fields` for `-F` — and it reuses the shell's own `_pids`, `_users`, and `_pgids` completers with comma-sequencing (`_sequence -s ,`). It drops into any `fpath` site-functions directory. The man pages split by depth — `lsOfrs.1` is the short reference (dated 2026-07-02, versioned `lsOfrs 4.9.1`) and `lsOfrsall.1` documents every flag, mode, theme, and architectural note. `lsOfrs.1` defines a real exit-status contract (0 success, 1 on error or no match), a `SEE ALSO` block pointing at `lsOf(8)`, `ss(8)`, `fstat(1)`, `netstat(1)`, `pstree(1)`, `top(1)`, and `proc(5)`, and a full OPTIONS section that mirrors the `-h` help screen. On a machine that already has `lsOf`, aliasing `lsOf=lsf` is the intended path: the accept-and-ignore flag handling exists specifically so that existing tooling — anything that shells out to `lsOf` with platform-specific options — continues to run unmodified.

Parity and its limits

The parser reproduces the `lsOf` grammar; the data model reproduces `lsOf`'s type and state vocabularies; the output formats (`-F`, `-t`, columnar, `-o`, `-s`, `+L`) match `lsOf`'s spelling, down to details like the `0t<offset>`

encoding and the `TST=<state>` field prefix. Where `lsotrs` diverges is in the flags it accepts-and-ignores rather than implements — the kernel-tuning and mode flags like `-b`, `-O`, `-D`, `-K` (the last mapped to `--list-tasks` rather than dropped) are honored syntactically, but the kernel-blocking, overhead-bypass, and device-cache behaviors behind `-b/-O/-D` are absent, not replicated. That is the correct tradeoff for drop-in script compatibility, but it means the behaviors are syntactic no-ops. Platform coverage is macOS, Linux, and FreeBSD; anything else prints an unsupported-platform notice to stderr and returns nothing (`main.rs:206-210`). The socket buffer-limit inputs the stall classifier prefers are macOS-only; on Linux it degrades to the trend heuristic rather than fabricating a ratio.

Status

Version 4.9.1 per `Cargo.toml:3`, MIT-licensed, edition 2024, published to crates.io with a Homebrew tap and a docs site. The test surface is large: grepping `#[test]` returns 1,029 functions inline across `src/` and 710 more across the integration files in `tests/`, plus 21 shell-based documentation and formatting checks (the `tests/*.sh` set that gates man-page synopsis structure, final newlines, README badges, and workflow hygiene). Those counts come from `grep`, not a `cargo test` run, so treat them as inventory rather than a green-suite claim. The tests are behavioral, not decorative — `cli.rs` alone carries well over a hundred parse-combination cases pinning flag interactions, and `stall.rs`, `ports.rs`, and `types.rs` each verify their classification rules against hand-built fixtures. The release profile is tuned for size and speed (`opt-level = 3`, `lto = true`, `codegen-units = 1`, `strip = true`, `Cargo.toml:43-47`). The README publishes hyperfine benchmarks claiming a 5–21× advantage over `lsot 4.91` and `lsotng` on macOS; those numbers are the project's own measurements on one machine (roughly 470 processes and 5,000 open files — consistent with the 431/4,032 snapshot captured above), and they were not reproduced here, so they should be read as the author's reported figures rather than independently verified results.

nmaprs — a Rust port of nmap's scan engine and CLI dialect

What it is and what it ports

`nmaprs` is a network scanner written in Rust that reimplements the command-line surface and the core scanning techniques of **Nmap**, the network-mapping tool that has been the reference port scanner since 1997. The upstream is named plainly in the project's own `Cargo.toml` description — “High-performance parallel network scanner with nmap-compatible CLI surface” — and throughout the README, which points anyone needing “authoritative Nmap NSE/OS DB behavior” back to Nmap itself. This honesty is the correct framing for the tool: the scan *techniques* (SYN half-open, NULL/FIN/Xmas, ACK/Window/Maimon, UDP, SCTP INIT/COOKIE-ECHO, idle scan, IP-protocol scan, FTP bounce) are Nmap's inventions, the file formats it consumes (`nmap-service-probes`, `nmap-os-db`, `nmap-protocols`) are Nmap's data files, and the flag vocabulary (`-sS`, `-sU`, `-Pn`, `-oX`, `-T4`, and the long-option set) is Nmap's. What `nmaprs` contributes is the engineering of the port: a parallel Rust implementation of those techniques, a `clap`-driven CLI that accepts Nmap's glued-token syntax, and a worker-pool scan engine that beats Nmap on wall-clock for the common small-scan cases while consuming the same databases.

The tool is explicit about where the port stops. The full **NSE Lua runtime is not embedded** — `--script` and `-sC` provide a small set of built-in Rust “scripts” (banner grab, a `default` set) rather than the thousands of Lua scripts Nmap ships. OS detection (`-O`) is partial: IPv4 has a raw probe engine that scores against `nmap-os-db`, but IPv6 remains a TTL-heuristic guess. Version detection (`-sV`) parses `nmap-service-probes` and runs the probes but skips Perl-only regex constructs that Rust's `regex` crate cannot express. These boundaries are documented in a capability matrix in the README rather than glossed over, which is the right posture for a port that is genuinely useful without claiming byte-for-byte fidelity.

The crate is at version **0.1.9** (from `Cargo.toml`), licensed MIT, published to crates.io with a Homebrew tap for installation, and attributed to the organization MenkeTechnologies. The man pages carry a “May 2026” date and stamp `nmaprs 0.1.9`, so the documentation is version-synchronized with the crate.

The binary suite

`Cargo.toml` declares one library (`src/lib.rs`) and two binaries. The primary binary is `nmaprs` (`src/main.rs`, seven lines — its entire `main` calls `nmaprs::run_from_cli_env()`); the second is `nms` (`src/bin/nms.rs`), a short alias whose body is the same call. Both binaries share identical CLI parsing and behavior — `nms` exists purely to save keystrokes at the prompt, the same convention MenkeTechnologies uses across the fleet. A `scan` Criterion benchmark harness is registered as well.

`run_from_cli_env` in `lib.rs` is the shared entry: it initializes `tracing_subscriber` from the `RUST_LOG` env filter, defaulting to `warn` when none is set, then parses argv, builds a multi-threaded `tokio` runtime with `enable_all()`, and blocks on the async `run(args)` before exiting with its return code. The `warn` default keeps normal runs quiet — informational chatter lives behind `info!/debug!` and only surfaces when the operator raises the log level.

The release profile is tuned for a shipped CLI tool: `lto = true`, `codegen-units = 1`, `panic = "abort"`, and `strip = true`. The benchmark profile inherits from release so microbenchmark numbers reflect the same optimization level as the installed binary.

The dependency set is conventional and durable rather than fashionable. Async runtime is `tokio` with the `full` feature; raw packet construction uses `pnet` and `pnet_sys` (0.35); TLS for `-sV` SSL probes is `rustls` 0.23 with the `ring` backend, `tls12`, and `webpki-roots`; DNS resolution for `--dns-servers` is `hickory-resolver` 0.25; proxy support is `tokio-socks`; concurrent shared maps are `dashmap` 6.1; CLI parsing is `clap` 4.5 with `derive`, `env`, and `string` features; error handling splits between `anyhow` at the boundaries and `thiserror` 2.0 for typed errors. CRC32 for SCTP checksums comes from `crc32c` and `crc32fast`. There is one piece of C++ FFI: `c/expr_match.cpp`, compiled by `build.rs` via the `cc` crate at `opt_level(3)` as `nmap_expr_match`, providing the OS-fingerprint expression matcher. The dev-dependencies (`assert_cmd`, `predicates`, `criterion`, `tempfile`) drive the integration and benchmark suites.

Source layout

The source is roughly 24,000 lines across 35 files under `src/`. The largest and most load-bearing modules map directly to scan techniques and pipeline stages:

- `src/config.rs` (2,314 lines) — the `ScanPlan` and configuration model, the single largest module.
- `src/os_scan.rs` (1,860) — the raw IPv4 OS-fingerprint probe engine (SEQ×6, ECN, T2–T7, U1, IE×2 probes).
- `src/syn.rs` (1,760) — raw half-open TCP: SYN, NULL, FIN, Xmas, ACK, Window, Maimon, plus the sharded recv/send pipeline.
- `src/lib.rs` (1,552) — the top-level orchestration, target expansion (`expand_specs_ordered`), and host batching (`host_batches`).
- `src/vscan.rs` (1,499) — version scanning against `nmap-service-probes`.
- `src/discovery.rs` (1,437) — host discovery (the `-P*` family).
- `src/sctp.rs` (1,182) and `src/ip_proto.rs` (1,137) — SCTP and IP-protocol scans.
- `src/scan.rs` (1,171) — the TCP-connect worker pool and scan-loop core.

Smaller modules cover the surrounding machinery: `argv_expand.rs` (glued-token normalization before clap), `scanflags.rs` (custom TCP flag sets), `idle.rs` (zombie idle scan), `ftp_bounce.rs`, `icmp_listen.rs` and `icmp_ping.rs` (raw ICMP), `ipv6_l4.rs`, `os_db.rs` / `os_fp_db.rs` / `fp_match.rs` / `os_detect.rs` (OS DB parsing and matching), `output.rs` and `skiddie.rs` (output formats), `resume.rs`, `trace.rs`, `nse.rs`, `ports.rs`, `target.rs`, `net_util.rs`, and `help_tp.rs` (the styled help screen). All are re-exported as public modules from `lib.rs`, so the crate doubles as a library — the `docs.rs` documentation link in `Cargo.toml` reflects that the internal API is part of the shipped surface, not just the binary.

Argv expansion: the glued-token front end

Before `clap` ever runs, `argv_expand::expand_nmap_style_argv` rewrites Nmap's compressed short-option grammar into the separated long-option form that clap's derive parser expects. This is the seam that lets `nmap.rs` accept a real Nmap command line verbatim. The rewriter walks argv token by token and applies a fixed set of rules: `-sL/-sn/-s0` map to their `--sL/--sn/--s0` long forms; `-sI` consumes a following zombie argument; `-iL/-iR` become `--iL/--iR`; three-character `-oN/-oX/-oS/-oG/-oA/-oM/-oH` become `--oN...`; a two-letter `-sX` splits into `--scan-type X` unless it is the overloaded `-sV` ($\rightarrow$ `--version-scan`) or `-sC` ($\rightarrow$ `--script-default`); `-Pn` becomes `--no-ping`; the ping family `-PS/-PA/-PU/-PY` splits off any glued port list (`-PS80,443` $\rightarrow$ `--ping-S 80,443`); `-PO` becomes `--ping-ip-proto` with Nmap's default `1,2,4` protocol list when given bare; `-PE/-PP/-PM` map to `--ping-E/-P/-M`; `-T0...-T5` become `--timing N`; and stacked `-v/-vv/-d/-dd` collapse to `--verbosity=N/--debug=N` by counting the repeated letter. The ordering of the match arms matters and is commented as such: `-s0` is handled before the generic `-sX` arm so it is not mistaken for a scan-type char, and `-PO` is handled before the generic `-P*` arm so it is not read as `--ping-0`.

The net effect is visible when tracing a command through the front end:

```
$ nmaprs -sS -Pn -PS80,443 -T4 -oX out.xml 192.0.2.0/30
# after expand_nmap_style_argv, clap receives:
# --scan-type S --no-ping --ping-S 80,443
# --timing 4 --oX out.xml 192.0.2.0/30
```

`Args::parse_from_env` performs this expansion, then re-derives the binary basename from `argv[0]` (so `nms` shows `nms` in help), builds the clap command with that name, and intercepts clap's help/version errors to route them into `help_tp::print_help` / `print_version` rather than clap's default renderer. That is why the styled help appears instead of clap's plain output. A handful of Nmap “hidden” aliases are folded in after parsing through `effective_*` accessors: `--vv` adds 2 to verbosity, `--ff` adds 2 to debug, `--rH` is an alias for `--randomize-hosts`, and `--fuzzy` is an alias for `--osscan-guess`.

The ScanPlan and timing model

`config.rs` turns the parsed `Args` into a `ScanPlan`, the single value that every later stage consumes. It is a large struct — resolved `ports`, `concurrency`, `connect_timeout`, the primary `scan_kind` plus an `extra_scan_kinds` vector (so `-sS -sU` runs SYN and UDP together), an optional `tcp_scan_flags` byte from `--scanflags`, all the output paths, evasion options, proxy chain, and the timing-derived rate and retry knobs. `ScanKind` is the enum of techniques: `TcpConnect`, `TcpSyn`, `TcpNull`, `TcpFin`, `TcpXmas`, `TcpAck`, `TcpWindow`, `TcpMaimon`, `Udp`, `IpProto`, `SctpInit`, `SctpCookieEcho`, and `Idle`. Each variant carries a `flag_name()` for error messages and a `tcp_port_raw_kind()` that maps the seven raw TCP variants onto `syn::TcpPortScanKind`.

The timing template is the clearest place to see the port's concrete choices. `from_args` maps `-T0...-T5` (default `-T3`) to a `(base_concurrency, base_timeout_ms)` pair:

```
-T0 → 32 workers, 10000 ms connect timeout
-T1 → 64 workers, 5000 ms
-T2 → 128 workers, 2000 ms
-T3 → 256 workers, 1500 ms (default)
-T4 → 512 workers, 750 ms
-T5 → 1024 workers, 300 ms
```

`--max-parallelism` overrides the base concurrency (capped at 65,535), `--min-parallelism` raises it, and `--min-rtt-timeout` / `--max-rtt-timeout` / `--initial-rtt-timeout` adjust the connect timeout with `parse_duration`. `effective_probe_concurrency()` layers one more rule on top: when `--min-rate` is set and `--max-parallelism` was **not** given explicitly, it raises the concurrency floor toward `min(min_rate, 65_535)`, so a Nmap-style minimum throughput request is not silently capped by the default template. `from_args` also validates the invariants Nmap would — `--max-rate` $\geq$ `--min-rate`, `--max-hostgroup` $\geq$ `--min-hostgroup`, `--privileged` and `--unprivileged` mutually exclusive, `-sn` and `-sL` not combined — and rejects violations with a terse `bail!` rather than a wrong scan.

The scan pipeline

The README documents a seven-stage pipeline, and the module map confirms it:

1. **Argv expansion** (`argv_expand.rs`) normalizes Nmap's glued tokens as described above.
2. **Plan** (`config.rs`, `scanflags.rs`) builds a `ScanPlan`, including an optional custom TCP flag byte when `--scanflags` is present.
3. **Targets** (`target.rs`, `lib.rs`) expand IPv4/IPv6 literals, CIDR blocks, Nmap-style IPv4 octet ranges, DNS names, `-il` files, and `-iR` random-host counts. Expansion runs in parallel with stable ordering (`expand_specs_ordered`), so multiple `-il` lines and CLI targets resolve concurrently but print in input order.
4. **Discovery** (`discovery.rs`, `icmp_ping.rs`) runs before the port scan unless `-Pn` is set. The default fires raw ICMP echo concurrently with raw TCP SYN to ports 443 and 80; `-PR` ARP ping auto-runs for

local-subnet IPv4 hosts. The full **-P*** family is supported: **-P5** (SYN), **-PA** (ACK), **-PU** (UDP, default port 40125), **-PY** (SCTP INIT, default port 80), **-P0** (IP-protocol ping, default protocols 1,2,4), and **-PP/-PM** (ICMP timestamp and address-mask, IPv4 Unix only). For **-PY** and **-P0** the IPv4 and IPv6 results are merged independently on partial failure, so a v6-only raw failure does not discard v4 liveness.

5. **Scan engine** (`scan.rs`, `syn.rs`, `sctp.rs`, `ip_proto.rs`, `ftp_bounce.rs`) executes the selected techniques, with optional `--min-hostgroup/--max-hostgroup` batching. `DEFAULT_HOSTGROUP_MAX` is 1024.
6. **Post-scan** (`ping.rs`, `trace.rs`, `resume.rs`, `nse.rs`, `os_detect.rs`, `vscan.rs`, `skiddie.rs`) handles traceroute, OS guessing, version detection, checkpointing, and script-kiddie output.
7. **Output** (`output.rs`) writes the requested report formats.

The performance story rests on stage 5. For the default TCP-connect scan, `tcp_connect_scan` in `scan.rs` chooses between two execution paths at runtime with `let use_blocking = plan.proxies.is_empty() && n >= 64`. When there are at least 64 probes and no proxy, it uses a **worker pool of OS threads**: a `TcpConnectCtx` (shared work vector, an `AtomicUsize next_idx`, timeout, retries, pacer, host deadlines) is wrapped in an `Arc`, results go into a `ResultSlots` array of `UnsafeCell<MaybeUninit<PortLine>>`, and `tokio::task::spawn_blocking` runs a `thread::scope` that spawns `conc` threads. Each thread loops `fetch_add(1, Relaxed)` on the shared index, breaks when it exceeds the work length, runs a blocking `std::net::TcpStream::connect_timeout`, and writes its result into the slot at its own index — exactly once per slot, so no lock is needed. The `unsafe impl Send/Sync for ResultSlots` is justified by that invariant, documented in the source: each slot is written by exactly one worker and read only after all workers join. The comment gives the rationale in syscall terms: the blocking probe is `socket + connect + close` (3 syscalls), where tokio's async path additionally sets the socket non-blocking with `ioctl(FIONBIO)` and registers it with the reactor (4+ syscalls). For small work sets (under 64 probes) or when `--proxies` are configured — where connect-through-proxy needs the SOCKS/HTTP-CONNECT state machine — the engine falls back to async tokio tasks draining the same atomic index. An optional `progress AtomicUsize` is incremented after each probe for `--stats-every` reporting.

The raw scan paths use a different structure. `syn.rs` runs a dedicated recv thread per address family plus main-thread sends, registering the reply key in a `DashMap` before each send to avoid a recv race, and shards work across up to `MAX_SYN_PARALLEL_SHARDS = 16` independent pipelines per family (each with its own recv thread and 64 KiB `RX_BUF`). `split_into_syn_chunks` divides the ordered work into balanced contiguous chunks so each shard keeps port ordering locally. Mixed IPv4+IPv6 target sets run both families concurrently under `tokio::join`, and `--max-retries` runs additional full rounds on a fresh raw socket for probes still unresolved, dropping scan-delay and rate pacing on later rounds to match TCP-connect retry behavior. The same sharded pattern is reused by SCTP (`sctp.rs`, CRC32c checksums, IPv4 Layer3 datagrams plus IPv6 raw protocol 132) and by the IP-protocol scan (`ip_proto.rs`). Rate control is centralized: `ProbeRatePacer` installs a shared `Mutex<Instant>` spacing gate only when `--max-rate` is set — a `--min-rate` floor alone adds no mutex overhead — and one pacer is shared across concurrent IPv4 and IPv6 senders so the ceiling is global.

The main loop

`run` in `lib.rs` is the async driver. It short-circuits the informational modes first — `--script-help` and `--script-updatedb` print an acknowledgement and exit 0 (there is no bundled NSE database), and `--iflist` enumerates interfaces via `if_addrs` and exits. It then builds the `Arc<ScanPlan>`, loads the OS fingerprint DB in a `spawn_blocking` task only if `-O` was requested, and calls `trace_nmap_compat`, which logs (at `debug` level, to the log only) when packet-level or resolver options are set that are accepted for parity but not fully wired. Absent any target, `-iL`, `-iR`, or `-sL`, it bails. `--sL` (list scan) expands targets and prints them without scanning. Otherwise it collects hosts, applies `--exclude/--excludefile` via `apply_exclude`, deduplicates when `--unique` is set, shuffles when `--randomize-hosts/--rH` is set, and — unless `-sn` or `-Pn` — runs `discovery::hosts_after_discovery` to drop hosts that did not answer. The `-sn` ping-only path is a self-contained branch: it opens the output set, writes headers with any `--stylesheet/--webxml` handling, pings hosts with `ping::ping_hosts`, optionally attaches an OS guess line (from the fingerprint DB's

`format_os_guess` or the TTL heuristic `os_detect::guess_from_ttl`), writes per-host records to every open format, emits a footer with up/down/total counts, and optionally runs traceroute. Traceroute itself is bounded parallel: `trace.rs` caps concurrency at `MAX_TRACEROUTE_PARALLEL = 32` and preserves scan output order.

The CLI surface

`src/cli.rs` (653 lines) defines the `Args` struct with `clap` derive. The `about` line reads “NMAPRS // GRID SCANNER — rust-native parallel port probe.” The flag inventory mirrors Nmap closely. Scan-type selectors include `-sT` (connect, default), `-sS` (SYN), `-sU` (UDP), `-sN/-sF/-sX` (NULL/FIN/Xmas), `-sA/-sW/-sM` (ACK/Window/Maimon), `--sO` (IP protocol), `--sI` with a zombie (idle), `-sY/-sZ` (SCTP), `-b` (FTP bounce), and `-sn` (ping scan only). Host-discovery probes are the `-P*` set exposed through long options like `--ping-S`, `--ping-A`, `--ping-U`, `--ping-Y`, `--ping-E`, `--ping-P`, `--ping-M`, and `--ping-ip-protocol`. Port selection uses `-p` (with `-p` shorthand for all 65536), `-F` (fast), `-r` (sequential, no randomize), `--top-ports`, `--port-ratio`, and `--exclude-ports`.

Timing and pacing controls are Nmap-shaped: `--timing/-T` (0–5 templates), `--min-parallelism`, `-M/--max-parallelism`, `--min-rtt-timeout`, `--max-rtt-timeout`, `--initial-rtt-timeout`, `--max-retries`, `--host-timeout`, `--scan-delay`, `--max-scan-delay`, `--min-rate`, `--max-rate`, `--min-hostgroup`, and `--max-hostgroup`. The evasion set is wired into raw packet construction: `-f/--mtu` (fragmentation), `-D` (decoys), `-S` (source spoof), `-e` (interface), `-g/--source-port`, `--data/--data-string/--data-length` (custom payload), `--ttl`, `--badsum`, `--ip-options`, and `--spoof-mac` (applied to ARP discovery frames). Proxying is `--proxies` with a `--proxy` visible alias; custom DNS is `--dns-servers`, with `--system-dns` to force the OS resolver. Beyond the flags in `nmap --help`, the struct also carries the extended long-option set from Nmap’s `nmap.cc` — `--resolve-all`, `--defeat-rst-ratelimit`, `--defeat-icmp-ratelimit`, `--discovery-ignore-rst`, `--max-os-tries`, `--nsock-engine`, `--release-memory`, `--versiondb`, `--servicedb`, `--stats-every`, and the hidden `--vv/--ff` aliases — several of which are accepted for parity and logged rather than acted on. The `_nmaprs` zsh completion file mirrors this surface for both `nmaprs` and `nms`, and starts with `#compdef nmaprs nms` for installation onto `fpath`.

A representative set of invocations, all grounded in the README’s example section:

```
# TCP connect - top ports from the embedded frequency table
nmaprs scanme.example.com
nms scanme.example.com           # identical via the short binary

# Ping scan only, no port scan
nmaprs -sn scanme.example.com

# UDP, top 100 ports
nmaprs -sU --top-ports 100 target

# Targets from a file, resume checkpoint, normal output
nmaprs -iL hosts.txt --resume state.json -oN out.txt

# IPv6, explicit ports, a /126 block
nmaprs -6 -p 80,443 2001:db8::/126
```

Raw scans need privilege. `nmaprs` auto-detects root/sudo via `geteuid()` and silently falls back from SYN to TCP connect when unprivileged; `--privileged` forces raw mode, `--unprivileged` forces connect mode and rejects raw-only techniques (`--sO`, `--sI`, SCTP) with an error rather than a wrong answer.

Output formats and parity

Output (`output.rs`, `skiddie.rs`) covers Nmap's report formats. `-oN` is normal text, `-oG` is greppable, `-oX` is XML, `-oA` writes all three, and `-oS` is the “script-kiddie” leet-text variant that mirrors the `-oN` lines while stdout stays normal. The XML uses an Nmap-compatible `<nmaprun>` root with `<scaninfo>`, `<host>`, `<status>`, `<address>`, `<hostnames>`, `<ports>`, `<extraports>`, `<service>`, and `<runstats>` elements, so existing XML-consuming tooling has a chance of parsing it. Two formats are partial: `-oM` writes the same greppable-style machine lines as `-oG`, and `-oH` creates a placeholder file (full hex capture is not implemented). Stylesheet handling (`--stylesheet`, `--webxml`, `--no-stylesheet`) emits the XML preamble and `xml-stylesheet` processing instruction when `-oX` is active.

A greppable line from a connect scan looks like Nmap's:

```
# nmaprs -oG
Host: 192.0.2.10 () Status: Up
Host: 192.0.2.10 () Ports: 22/open/tcp//ssh///, 80/open/tcp//http///
Ignored State: closed (998)
```

The port-state model follows Nmap's semantics per technique, which is where the port earns its keep. TCP connect and SYN map SYN/ACK to `open` and RST to `closed`. Maimon (`-sM`) sends FIN+ACK but classifies replies like SYN (RST → `closed`, SYN/ACK → `open`). ACK scan (`-sA`) maps RST to `unfiltered` and no-reply to `filtered`, the firewall-detection interpretation. Window scan (`-sW`) reads the TCP window on the RST: non-zero window is `open`, zero window is `closed`, following BSD-derived stacks. UDP treats a reply as `open`, an ICMP port-unreachable as `closed`, and other unreachable codes as `filtered`; on Unix a single `poll(2)` listener thread drains ICMP errors for both families and `closed` wins over `filtered`. SCTP maps INIT-ACK/COOKIE-ACK to `open` and ABORT to `closed`; the IP-protocol scan maps ICMP protocol-unreachable (IPv4 type 3 code 2, or ICMPv6 parameter-problem type 4 code 1) to `closed` and timeout to `filtered`; the idle scan reads the zombie's IP-ID delta, treating a delta ≥ 2 as `open`. These are not cosmetic mappings — reproducing them correctly is the substance of matching Nmap's behavior, and the README's capability matrix documents each one precisely.

OS and version detection

`-sV` version detection (`vscan.rs`) loads `nmap-service-probes` and runs both TCP and UDP `Probe` blocks, honoring `ports/sslports` filtering (with rustls TLS for SSL ports), the `rarity` vs `--version-intensity` threshold, and `match/softmatch` lines with the full template field set — `p/` product, `v/` version, `i/` info, `o/` OS, `d/` device type, and `cpe:/` URIs. Each rule is a compiled `regex::bytes::Regex`; soft matches defer to later probes until a hard match is found, matching Nmap's staged behavior. Perl-only regex features that Rust's `regex` cannot express are skipped rather than mis-parsed, and per-open-port probing runs in parallel.

`-O` OS detection (`os_scan.rs`, plus the C++ `expr_match.cpp` FFI) is the more ambitious partial. On IPv4 with privileges and a `nmap-os-db` under `--datadir`, it sends Nmap's second-generation probes across phases: `NUM_SEQ_SAMPLES = 6` SEQ probes spaced by `OS_SEQ_PROBE_DELAY_MS = 100`, then ECN, T2–T7, U1, and two IE probes. From those it builds a subject fingerprint — SEQ/OPS/WIN/ECN/T1–T7/U1/IE tests, including `GCD/ISR` sequence-rate values, `seq_relation/ack_relation` classifications (Z/I/BI/RI/RD-style IPID, timestamp classes, CI from T5–T7, II from IE echo replies, shared-IPID detection) — and scores every reference entry with `MatchPoints` weighting via the `expr_match` C++ matcher. When the raw engine or the DB is unavailable, `-O` degrades to ICMP TTL heuristics with example DB titles. IPv6 OS detection remains TTL-oriented until a v6 probe path exists. `-A` bundles `-O` + `-sV` + default `-sC` scripts + `--traceroute`, the same aggregate flag Nmap defines, minus the NSE runtime.

Databases and integration

Three data files sit under `data/` (or a `--datadir DIR`). `data/top_ports.txt` holds 1,000 TCP port numbers in frequency order (embedded via `include_str!` and loaded once through a `OnceLock`); `top_ports(n)` takes the first `n`, `default_tcp_ports()` takes 1,000, and `fast_tcp_ports()` takes 100 for `-F`. `data/nmap_ip_protocols_fast.txt` is the IP-protocol fast list derived from Nmap's `nmap-protocols`, used for `-s0 -F` to match Nmap's `[P:0-]` selection. The two large upstream databases — `nmap-service-probes` and `nmap-os-db` — are deliberately **not committed**; `scripts/fetch_nmap_data.sh` fetches them (with `scripts/fetch_top_ports.sh` regenerating the port table), and without them `-sV` logs a warning and `-O` falls back to TTL-only heuristics. Keeping the multi-megabyte Nmap DBs out of the repo while consuming them at runtime is the pragmatic choice for a port. Port parsing enforces the real bounds — `-p -` expands to all 65536 values, `65535` is the maximum accepted, and `65536` is a parse error.

Shell integration follows the fleet convention. `completions/_nmaprs` is a zsh completion covering both `nmaprs` and `nms`; it is dropped into a directory on `fpath` before `compinit`. Two man pages ship: `man/man1/nmaprs.1` (short reference) and `man/man1/nmaprsall.1` (the full parity matrix and every technique, modeled explicitly on `zshall(1)` so the whole manual is one greppable page). The help screen (`help_tp.rs`) is styled after the `temprs` CYBERPUNK template with cyan/magenta/yellow color bands, and suppresses all escapes on a non-TTY or when `NO_COLOR` is set, so piped output stays clean.

Performance and status

The README reports hyperfine comparisons against nmap 7.99 on macOS arm64, all against 127.0.0.1 with `-n -Pn --min-rtt-timeout 50ms --max-retries 0` and output to `/dev/null`. The headline range is **1.5–5.1× faster** across port counts, with the largest gains (3–5×) on small scans where Nmap's startup cost (Lua/NSE init, libpcap, database loads) dominates, narrowing to 1.5–1.6× on full 65536-port sweeps where actual probe work dominates. A Criterion microbenchmark of a three-closed-port localhost connect scan reports roughly 74–82 μs. These are the author's own measurements on one machine and configuration; treat them as directional rather than as guaranteed cross-platform figures. (*Provisional: benchmark numbers are as reported in the README and were not re-run for this chapter.*) The architectural basis for the connect-scan number is the worker pool described above — blocking `connect_timeout` cutting per-probe syscalls, and the lock-free `UnsafeCell` result array avoiding contention on write-back.

Test coverage is substantial. The `tests/` directory holds 29 files mixing Rust integration tests and shell-based doc/format guards; `tests/cli.rs` alone carries 204 test functions, with `cli_extra_flags.rs` (36) and several `contract_*` files (8, 8, 7) covering the port database, scanflags, OS DB parsing, resume edges, and target expansion. Counting inline `#[test]/#[tokio::test]` attributes across source and tests together yields roughly **1,468** test functions. (*Provisional: counts are from `grep` over the test attributes and may include helper or ignored tests; not all were executed here.*) The shell guard tests enforce repo hygiene — final newlines, man-page synopsis sections, README badges and HTTPS links, docs HTML structure, no inline handlers, no deprecated tags — alongside the Rust contract tests.

The honest summary is the one the README itself offers: `nmaprs` is a fast, parallel Rust implementation of Nmap's scan techniques and CLI dialect, faithful on the connect/SYN/UDP/SCTP/IP-protocol/idle/bounce paths and their per-technique port-state semantics, partial on OS detection and version scanning, and explicitly without the NSE Lua runtime. For everyday scanning inside the shell it stands on its own; for authoritative NSE and OS-database behavior, the upstream Nmap remains the reference.

temprs — a stack-structured temporary-file manager for the shell

The name suggests a temperature probe. It is not. `temprs` (pronounced as “temp-R-S,” and shipped as the two-character command `tp`) is a manager for **temporary files**, organized as a stack. It gives the shell a persistent, named, index-addressable scratch space that survives between commands and between separate shell invocations, and it wraps that scratch space in a set of operations borrowed from two different traditions: the list/array primitives of a programming language (`push`, `pop`, `shift`, `unshift`, `swap`, `dup`, `reverse`, `sort`) and the file-inspection primitives of Unix (`head`, `tail`, `wc`, `grep`, `diff`, `cat`). The result is a single small binary that behaves like a clipboard stack for arbitrary command output, with every entry backed by a real file on disk that any other program can read.

It is worth being precise about what `temprs` ports and what it invents, because the house rule is to name the upstream honestly. `temprs` is **not** a Rust reimplement of a single named Unix utility — there is no `temp(1)` it descends from. What it does port, in the honest sense, is the *behavior* of several coreutils, scoped to its own stack. Two of those coreutils it reproduces in process — `--head`, `--tail`, `--grep`, `--cat`, and the `wc`-flavored line count are Rust code operating on the tempfile’s contents — and two it delegates to the system binary rather than reimplement: `--diff` spawns the platform `diff -u`, and `--edit` spawns `$EDITOR`. That distinction matters for a port-honest reading. Where the earlier framing said “reimplements the useful part of `grep`,” read it precisely: `--grep` is a literal substring scan (`str::contains`), not a regex engine, and `--diff` is not a reimplemented diff at all — it is a subprocess call that inherits the exit code of whatever `diff` is on `PATH`. The engineering that matters here is the *container* — the atomic, lock-protected, null-delimited master record and the stack algebra layered on top of it — not the invention of a diff.

What it is, concretely

Every time you pipe data into `tp`, it writes that data to a fresh file inside a managed directory and records the file at the top of a stack. The directory defaults to `$TMPDIR/temprs` (falling back through `std::env::temp_dir()`), and can be overridden wholesale by exporting `TEMPRS_DIR`. Each stored file is named `tempfile<milliseconds>` — the millisecond timestamp of creation, from `util_time_ms()`, is the on-disk identity — and the ordering, along with any human-assigned tag names, lives in a single sidecar file called `temprs-stack` in the same directory. That sidecar is the master record, and most of the tool’s engineering is spent keeping it honest under concurrent access.

The mental model is a stack of clipboards. The newest thing you pushed is index `-1` (or the highest positive index); the oldest is index `1`. You can read any entry back to stdout, pipe fresh data over an existing entry, append to one, open one in `$EDITOR`, or address it by a name you gave it instead of by number. Because the backing store is real files in a real directory, the stack persists across terminal sessions — close the shell, open a new one, and `tp -l` still lists everything you left there until you clear or expire it.

```
# capture some command output onto the stack
$ curl -s https://example.com/api/users | tp -w users
# ...later, in a different shell:
$ tp -l
/tmp/temprs/tempfile1749145203871
$ tp -o users | jq '.[].email'
```

There is a self-healing property baked into startup that the earlier chapter passed over. When `TempApp::new()`

reads the master record back in, it does not trust it blindly — it zips the stored paths against their names, filters with `.filter(|(p, _)| p.exists())`, and immediately rewrites the pruned record with `util_paths_and_names_to_file`. So if you or another program deletes a `tempfile*` out from under the stack, the phantom entry is dropped on the very next `tp` invocation rather than lingering as a dangling index. The master record converges toward the real directory contents every time the tool runs.

Architecture: crates, modules, binaries

The build produces two binaries — `temprs` and `tp` — from the same `src/main.rs`, which does nothing but construct a `TempApp` and call `run()`. The two names are byte-for-byte equivalent programs; `tp` exists purely because a temporary-file tool that you reach for dozens of times an hour needs a two-keystroke name. This is declared in `Cargo.toml` as two `[[bin]]` targets pointing at the same path, alongside a `[lib]` target (`src/lib.rs`) that exposes the internals for the test suite.

The dependency list is deliberately short and foundational, which matters for a tool meant to keep building for years:

Crate	Version	Role
<code>clap</code>	4.6	argument parsing (all 38 flags)
<code>fs2</code>	0.4	<code>flock</code> -style advisory locking (<code>lock_exclusive</code>)
<code>log / env_logger / simple_logger</code>	0.4 / 0.11 / 5.2	logging; runtime uses <code>simple_logger</code> at <code>Level::Info</code>
<code>criterion</code> (dev)	0.8	the <code>benches/benchmarks.rs</code> harness

The release profile is tuned for a small, fast static-ish binary: `lto = true, panic = "abort", strip = true, codegen-units = 1, opt-level = 3`. The crate is on the 2024 edition.

Internally the code splits into three layers. `src/model/state/mod.rs` defines `TempState`, the plain data record. It carries fifteen fields: the path of the file being written (`new_temp_file`), the master record path, the managed directory, and — the two parallel vectors that are the heart of the design — `temp_file_stack: Vec<PathBuf>` and `temp_file_names: Vec<Option<String>>`. Around those sit the transient per-invocation slots the CLI fills in: `arg_file`, `insert_idx`, `input_temp_file`, `output_temp_file`, `append_temp_file`, `name`, an `output_buffer` and a `holding_buffer` that shuttle piped bytes between input and output, plus the `silent` and `verbose` flags. Position `i` in both vectors describes one stack entry: its file and its optional tag. Keeping names in a parallel `Vec<Option<String>>` rather than inside the path is what lets a rename or a reorder move the tag with the file without touching the file itself. `TempState` is a pure struct with getters, mutable getters, and a `write_master()` that funnels the two vectors to the atomic writer — several of its accessors are `#[cfg(test)]`-only, existing solely so the test suite can construct and inspect state without a real lock file.

`src/model/app.rs` is the controller — `TempApp` — holding a `TempState` and, importantly, an owned `_lock_file: File`. The lock is acquired in `TempApp::new()` and held for the entire lifetime of the process; the leading underscore signals that the field exists only for its RAII side effect (the exclusive lock releases when the file handle drops at process exit). `run()` is a three-phase pipeline: `parse_opts()` to interpret the CLI, `input()` to read from stdin or the argument file, and `output()` to emit results. `input()` branches on `stdin().is_terminal()` so that `CMD | tp` reads the pipe while a bare `tp` at a TTY waits for typed input; `output()` mirrors that with `stdout().is_terminal()`, choosing the boxed cyberpunk renderer for a terminal and a plain byte-faithful `print!` for a pipe.

`src/model/opts.rs` holds the `clap` command definition and — this is the bulk of the source at roughly 47,000 lines — an enormous generated body of flag-recognition tests. `src/util/utils.rs` (~2,200 lines) is the free-function toolkit: path-to-string conversion, the millisecond clock, the atomic master-record writer, index transformation, the terminate-on-error helper, and the whole family of `cyber_*` terminal-output routines. `src/util/consts.rs` centralizes every string constant, from the directory name to each error message to all 38 CLI flag identifiers, so that nothing is stringly-typed at a call site.

The master record: null-delimited, atomic, locked

The single most load-bearing piece of engineering in `temprs` is how it persists the stack. Three problems have to be solved at once: filenames can contain any byte except NUL and `/`; the file can be written by several shells at the same moment; and a crash mid-write must never leave a half-written record.

The format solves the first problem with NUL delimiters. Within a record, the file path and its optional tag name are separated by a single `\0` (`MASTER_FIELD_DELIM`); records are separated by `\0\0` (`MASTER_RECORD_DELIM`). Because NUL is the one byte that cannot appear in a Unix path, this framing is unambiguous even for filenames containing newlines, tabs, or spaces. The reader, `util_file_to_paths_and_names`, splits on the double-NUL, skips any record that is empty after trimming, and for each surviving record uses `split_once(MASTER_FIELD_DELIM)` — a present-but-empty field becomes `None`, an absent delimiter means a bare untagged path. The writer in `util_paths_and_names_to_file` builds the buffer by zipping the path vector against the name vector, emitting `path` alone when the tag is `None` and `path\0name` when it is `Some`:

```
// src/util/utils.rs - record assembly, then atomic swap
let records: Vec<String> = paths.iter().zip(names.iter())
    .map(|(p, n)| match n {
        Some(name) => format!(
            "{}{}{}", util_path_to_string(p),
            MASTER_FIELD_DELIM, name),
        None => util_path_to_string(p),
    })
    .collect();
// ... join with "\0\0", append a trailing "\0\0", then:
let tmp = out.with_extension("tmp");
util_overwrite_file(&tmp, &buf);
if rename(&tmp, out).is_err() {
    util_terminate_error(ERR_MASTER_WRITE);
}
```

The atomicity comes from the last three lines: the new record is written to `temprs-stack.tmp` and then `renamed` over `temprs-stack`. On a POSIX filesystem `rename` within a directory is atomic, so a reader either sees the complete old record or the complete new one, never a torn write. An empty stack writes an empty buffer rather than a stray delimiter, so a cleared stack round-trips to zero records. The concurrency problem is handled a level up, in `TempApp::new()`, which creates `temprs-stack.lock` and calls `lock_exclusive()` on it via `fs2`; because the whole program runs while holding that lock, two shells racing to mutate the stack serialize cleanly rather than clobbering each other. Corruption tolerance is the fourth guarantee: records that fail to parse — empty or with an empty path field — are skipped on read (with a `warn!` to the log) and quietly dropped on the next write, so a damaged master file self-heals rather than wedging the tool.

The CLI surface

`temprs` exposes 38 operations, catalogued in `consts.rs` and wired to `clap` in `opts.rs`. Almost every index-taking flag sets `allow_hyphen_values(true)`, which is what lets `-1`, `-2` and friends arrive as *values* rather than being mistaken for short flags — negative indexing works precisely because of that per-argument setting. The operations fall into groups.

I/O and pipeline. `-i INDEX` writes stdin into an existing entry; `-o INDEX` reads an entry to stdout; `-A INDEX` appends stdin to an entry; `-v/--verbose` (a counting flag) raises the log level to `Debug` and echoes what was written; `-q/--quiet` sets a silent flag. Port-honest note: that silent flag is stored on `TempState` and asserted extensively in the state tests, but the runtime output path does not currently consult it — `-q` parses and records intent without yet suppressing anything at runtime. With no flags, `CMD | tp` pushes a new entry and `tp FILE` pushes a file's contents; if both a pipe and a positional `FILE` are present, the pipe wins (the positional argument's help text states this explicitly).

Stack algebra. `-p/--pop` removes from the top, `-s/--shift` removes from the bottom, `-u/--unshift` pushes to the bottom, `-a/--add INDEX` inserts at a position, `-r/--remove INDEX` deletes a specific entry, `-M/--mv FROM TO` moves, `-S/--swap A B` exchanges, `-x/--dup INDEX` duplicates onto the top, `--rev` reverses the whole stack, and `--sort name|size|mtime` reorders it (bare `--sort`, via `default_missing_value("name")`, sorts by name). The reordering operations all funnel through one helper, `apply_permutation`, which rebuilds both the path vector and the name vector from an index list so that tags always follow their files:

```
pub(crate) fn apply_permutation<T: Default>(<
    vec: &mut Vec<T>, indices: &[usize]) {
    let mut taken = std::mem::take(vec);
    vec.extend(indices.iter()
        .map(|&i| std::mem::take(&mut taken[i])));
}
```

`sort_stack` computes the permutation once — comparing `file_name()`, cached file sizes, or cached mtimes depending on the key — then applies it to stack and names in lockstep. `move_tempfile` removes the source index and clamps the insertion point with `to_idx.min(len)` so an out-of-range destination lands at the end rather than panicking. `apply_permutation` is the one purely-functional helper on `TempApp`, and it is pinned by six unit tests in `app.rs` covering identity, reverse, element duplication (a repeated index leaves `T::default()` behind because of the `mem::take`), subset truncation, empty drain, and a two-element swap.

Inspection. `-l` lists entries, `-n` lists them numbered, `-L/-N` do the same with contents inlined, `-k/--count` prints the stack size, `-I/--info INDEX` prints per-entry metadata, `-d/--dir` prints the managed directory, `-m/--master` prints the master-record path, and `--path INDEX` prints the backing file path so scripts can hand it to another program (`cat "${tp --path 1}"`). The coreutils-analogue operations live here too. `--head INDEX N` and `--tail INDEX N` read the whole file into a `String` and then `.lines().take(n)` or slice the last `n` — not a streaming implementation, but correct for the scratch-sized files the tool is meant for. `--wc INDEX` prints a *line count only* (`content.lines().count()`), not the word/byte columns of real `wc`. `--size INDEX` prints raw bytes from `fs::metadata`. `-g/--grep PATTERN` does a literal substring match, prints each matching entry with per-line numbers, and exits 0 on any match and 1 otherwise so it composes in `&&` chains. `-D/--diff A B` shells out to `diff -u` and propagates its exit code (0 identical, 1 differ, 2 error). `-C/--cat` concatenates several entries in any given order (`num_args(1..)`).

Editing and lifecycle. `-e/--edit INDEX` opens an entry in `$EDITOR` (falling back to `vi`) and exits with the editor's code, `--replace INDEX PATTERN REPLACEMENT` does an in-place find-and-replace and prints the match count, `-c/--clear` purges everything, and `--expire HOURS` purges entries older than a fractional number of hours (`--expire 0.5` for thirty minutes) and prints how many it removed. Two of these carry hardening that reads like scar tissue from real bugs, documented in the source comments. `--expire` parses its argument as `f64` and explicitly rejects negative, NaN, and infinite values before the `as u64` cast — the comment records that a pre-fix `--expire=-1` saturated to a zero cutoff and silently wiped the entire stack. `--replace` rejects an empty pattern up front, because `str::replace("", repl)` splices the replacement at every byte boundary and would balloon the file to `0(len * repl.len())`.

The `-c/--clear` guard is the sharpest of the three. Clearing removes the parent directory of the master record with `remove_dir_all`, and a naive version combined with `TEMPRS_DIR=$HOME` would delete the home directory. The safe contract requires the directory's basename to contain the substring `temprs` (case-insensitively) before it will remove anything; anything else is refused outright:

```

let safe_name = parent.file_name()
  .and_then(|s| s.to_str())
  .map(|s| s.to_ascii_lowercase().contains("temprs"))
  .unwrap_or(false);
if !safe_name {
  util_terminate_error(ERR_NO_FILE);
  unreachable!()
}

```

Dual indexing by position or name

Every operation that takes an `INDEX` also takes a name. Entries can be tagged at creation with `-w/--name NAME` or renamed later with `-R/--rename OLD NEW`, and thereafter the tag is interchangeable with the number. The resolution rule is in `resolve_idx`: the argument is first parsed as a signed integer, and only if that fails is it looked up as a tag.

```

fn resolve_idx(&self, f: &str) -> Option<usize> {
  match f.parse::<i32>() {
    Ok(idx) => Some(util_transform_idx(
      idx, self.state.temp_file_stack().len())),
    Err(_) => self.state.temp_file_names().iter()
      .position(|n| n.as_deref() == Some(f)),
  }
}

```

`util_transform_idx` folds negative indices into the valid range — `-1` is the top of the stack, `-stack_size` is the bottom — and terminates with `ERR_INVALID_IDX` for anything out of bounds. Index `0` is always invalid by construction, since the stack is one-based: a positive `idx` maps to `idx - 1` and a zero would map to `-1`, which the guard rejects. Tag names must be unique across the stack and must not contain NUL bytes, which would break the master-record framing; both constraints are enforced with dedicated error constants (`ERR_INVALID_NAME`, `ERR_NAME_NUL`) at both `--name` set time and `--rename` time. Because a parse of `"1"` succeeds as an integer, a purely numeric tag name would be shadowed by index resolution — the ordering makes numbers win, which is the sensible default for a positional stack.

The terminal-output layer

One area the earlier chapter did not describe is how `temprs` decides what its output looks like. The `cyber_*` family in `utils.rs` renders one way to a terminal and another way to a pipe, keyed off a cached TTY check: `is_tty()` memoizes `io::stdout().is_terminal()` in a `thread_local! Cell<Option<bool>>` so the syscall is paid at most once per run. When stdout is a terminal, listings gain ANSI color and box glyphs — `cyber_print_content` frames a payload in a `┌ ... ┐` box, `cyber_hr` draws a cyan `███` rule, indexed listings print `[NN]` markers in yellow with `@tag` names in a second color. When stdout is a pipe, every one of those routines collapses to plain, byte-faithful text: `cyber_path` prints just the path, `cyber_hr` prints eighty dashes, `cyber_print_content` degrades to a bare `print!`. That split is what makes `tp -o 1 | jq` safe — the machine-consuming path never sees an escape code — while an interactive `tp -n` gets the decorated view.

The same aesthetic reaches the help output. `opts.rs` defines a `CYBERPUNK_TEMPLATE` help layout, a multi-line ASCII-art `BANNER` shown via clap's `before_help`, and an `AFTER` footer, all stitched from `concat!` with `env!("CARGO_PKG_VERSION")` interpolated so the version stripe in the banner never drifts from `Cargo.toml`. The version, author, and about strings all come from `env!("CARGO_PKG_*")` macros, so the CLI metadata is single-sourced from the manifest.

Here is the difference in practice — piped output stays clean and scriptable:

```

$ printf 'alpha\nbeta\ngamma\n' | tp -w greek
$ tp -k
1
$ tp --wc greek
3
$ tp --path greek
/tmp/temprs/tempfile1749145203871
$ tp -g beta
1:/tmp/temprs/tempfile1749145203871 @greek
2:beta
$ tp --expire 24
0

```

Integration into the shell and the stack

`temprs` ships a single zsh completion file, `completions/_temprs`, whose `#compdef tp temprs` line registers it for both binaries. The completion is not static: its helper `_tp_indices` reads the live `temprs-stack` master file out of `${TEMPRS_DIR:-${TMPDIR:-/tmp}/temprs}` and offers the current stack indices, file basenames, and `@`-prefixed tag names as completion candidates, so tab-completion reflects the actual contents of your stack at the moment you press Tab. It even solves a real encoding problem inline: zsh variables cannot hold NUL bytes, so the completion pipes the master record through `tr '\0' '\x1e'`, turning the `\0\0` record delimiter into `\x1e\x1e` and the `\0` field delimiter into `\x1e`, then splits on those. Tagged entries get two candidates — one keyed by index, one keyed by name — so either addressing form completes. This is the integration point that makes the tool feel native in an interactive shell rather than like a foreign command you have to remember indices for.

Two man pages are installed. `man/man1/temprs.1` is the conventional short reference — synopsis, indexing rules, options. `man/man1/temprsall.1` is a comprehensive single-document reference explicitly modeled on `zshall(1)`, folding every section into one page for readers who want to grep the whole manual at once. That choice is a deliberate nod to the zsh documentation tradition that the surrounding stack is built in.

In practice `temprs` slots between commands as a stack-aware pipe buffer:

```

$ ps aux | tp -w procs          # push, tagged
$ tp -g python procs && echo hit # grep the tagged entry
$ dmesg | tp                   # push another
$ tp -D procs -1               # diff the two by name/index
$ tp --expire 1                # drop anything older than 1h
$ tp -c                        # wipe the stack

```

Status: version, tests, CI

The crate is at **version 2.9.12** ([Cargo.toml](#) and both man-page `.TH` lines agree). It is MIT-licensed and published on [crates.io](#) with a Homebrew tap install path (`brew install temprs`) and the `cargo install temprs` route.

The test surface is large and lives mostly beside the code. A static grep of `#[test]` attributes across `src/` yields **5,065** unit tests — **4,449** of them are the generated flag-recognition tests in `opts.rs`, with the remainder spread across `consts.rs` (87), `utils.rs` (174), `state/tests.rs` (187), `apply_permutation_tests.rs` (162), and the six `apply_permutation` tests in `app.rs`. The `tests/` directory adds **585** more `#[test]` functions across integration files (`integration.rs`, `edge_cases.rs`, `expire_invalid_hours.rs`, the `contract_*` invariant suites, `master_record_tag_roundtrip.rs`, `tag_alignment_after_mutation.rs`) plus **21** shell-based gate scripts that check documentation and hygiene — final newlines, README badges and section structure, man-page synopsis sections, executable bits and shebangs on the test scripts themselves, and the absence of deprecated HTML tags, inline handlers, `http://` links, and placeholder hrefs in the docs

site. These counts are from a static grep of the source, not from a `cargo test` run in this session, so treat the totals as provisional; the authoritative numbers come from the `test result: lines of cargo test --locked`. A `criterion` bench (`benches/benchmarks.rs`) microbenchmarks the hot free functions — `transform_idx`, `path_to_string`, `time_ms`, and the 100-record master-record read/write paths (`file_to_paths_100`, `paths_to_file_100`, `overwrite_file_100_lines`, `append_file`, `file_contents_to_string_100_lines`, `lines_to_file_100`).

The CI workflow (`.github/workflows/ci.yml`) runs a long chain of gates on every push and pull request to `main`: Check, Test (on an OS matrix, `--no-fail-fast`), Format (`cargo fmt --check`), Clippy (`-D warnings`), Doc, Release Build (target matrix), plus the Docs, Polish, Semantic, Newline + README, and Structure gate jobs that run the shell scripts. A separate `release.yml` cross-compiles per-target binaries (including musl via `cross`), packages them, publishes a GitHub Release, and auto-updates the Homebrew tap formula with freshly computed SHA-256 sums.

What `tempris` costs, plainly: it keeps state on disk that outlives your shell, so a forgotten stack accumulates files until `--clear` or `--expire` reclaims them; it holds a process-lifetime exclusive lock on its master record, so overlapping invocations serialize rather than run truly in parallel; and `--head/--tail/--grep/--replace` read whole files into memory rather than streaming, which is fine for scratch data and wrong for gigabyte logs. Those are deliberate trades in favor of the guarantee the tool actually sells — that the stack never corrupts and never tears, no matter how many shells reach for it at once.

iftoprs — iftop rebuilt on ratatui and pcap

What it is and what it ports

iftoprs is a Rust reimplementation of **iftop**, the per-connection bandwidth monitor that has shipped on Unix systems since the early 2000s. The upstream tool — originally written in C by Paul Warren and Chris Lightfoot, driven by **libpcap** and a **curses** front end — answers one question: *which hosts are eating my link right now, and at what rate*. It sniffs packets off an interface, aggregates them into bidirectional flows, and paints a live table of source/destination pairs with rolling 2-second, 10-second and 40-second averages plus a header bar summarizing total throughput. **iftoprs** keeps that model intact and rebuilds the machinery underneath it in Rust: **pcap** for capture, **ratatui** + **crossterm** for the terminal UI, **tokio** channels to move packets off the capture thread, and native OS syscalls for the one thing classic **iftop** never did — attributing each flow to the process that owns it.

The format is not the invention here. Flow tables, sliding-window rate columns, and the `src ↔ dst` layout are **iftop**'s. The engineering is the port: a single statically-linked binary that carries its own process-attribution backend (no shelling out to **lsof**), a capture pipeline that auto-restarts on interface flaps, a 40-slot ring-buffer accounting model, an NDJSON headless mode for pipelines, and a **ratatui** render layer with mouse support, sparklines, hover tooltips and a config that persists to TOML. The manifest describes it plainly as an “iftop clone in Rust with ratatui TUI” (**Cargo.toml:5**). One of the 31 bundled color themes is literally named **Iftopcolor** (**src/config/theme.rs:68**, **iftoprs.default.conf**), a deliberate parity palette for people who want the original look — the enum's own doc comment notes the palette set was “ported from storageshower” (**src/config/theme.rs:4**), another MenkeTechnologies tool, so the theme machinery is shared rather than reinvented per binary.

The crate is at version **2.22.9** (**Cargo.toml:3**), Rust **edition 2024**, and pins the **stable** toolchain with **rustfmt** and **clippy** via **rust-toolchain.toml**. The README states a minimum of Rust 1.85. macOS and Linux are the supported platforms; both need **libpcap** present, and raw capture needs root. The dependency list is deliberately small and durable: **anyhow**, **clap** + **clap_complete**, **crossterm**, **dirs**, **dns-lookup**, **pcap**, **ratatui** 0.30, **regex**, **serde/serde_json**, **tokio** (full features), **toml**, **chrono**, and — only on macOS — **libproc** for the kernel socket table (**Cargo.toml**, [**target.'cfg(target_os = "macos")'.dependencies**]). The release profile builds with **opt-level = 3**, **lto = true** and **strip = true** for a lean single binary; local development uses a plain **cargo build**.

Architecture: five modules behind a thin binary

src/lib.rs exposes the crate as five submodules — **capture**, **config**, **data**, **ui**, **util** — and **src/main.rs** is the binary that wires them together. The whole tree is 22,913 lines of Rust across 21 files (via **wc -l** on **src/**/*.rs**), and the bulk of it lives in three files: **src/ui/app.rs** (4,066 lines), **src/capture/parser.rs** (3,074 lines) and **src/ui/render.rs** (2,754 lines). The rest is distributed across **resolver.rs** (1,733), **flow.rs** (1,685), **theme.rs** (1,263), **main.rs** (1,239), **tracker.rs** (1,069), **prefs.rs** (966), **history.rs** (863), **format.rs** (673) and **procinfo.rs** (459), with the CLI definition in **cli.rs** (2,344); **provenance.rs** (445), **sniffer.rs** (237) and six small module files account for the remaining 725.

capture — libpcap in, ParsedPacket out

`src/capture/sniffer.rs` owns the packet source. `start_capture` validates the requested device up front (so a typo'd interface fails immediately), then spawns a dedicated OS thread named `packet_capture` that opens the device with `snaplen(256)`, a 100 ms read timeout, and optional promiscuous mode, applies a BPF filter string if one was given, and pushes every decoded packet into a `tokio::sync::mpsc::UnboundedSender`. The `snaplen` of 256 bytes is deliberate: `iftopr` only needs L2–L4 headers to build a flow key, never payload, so it caps the copy.

The capture loop is resilience-first. If the device fails to open or the read loop hits a non-timeout pcap error, it treats it as transient, sleeps with exponential backoff starting at 250 ms and capped at `MAX_BACKOFF = 30s`, and retries — resetting the backoff on any successful reopen. `TimeoutExpired` is not an error; it just continues. The only clean exit is the receiver being dropped, i.e. the app shutting down:

```
match cap.next_packet() {
  Ok(packet) => {
    let parsed = match datalink {
      pcap::Linktype::ETHERNET =>
        parser::parse_ethernet(packet.data, local_net),
      pcap::Linktype(0) =>
        parser::parse_loopback(packet.data, local_net),
      pcap::Linktype(113) =>
        parser::parse_sll(packet.data, local_net),
      pcap::Linktype(101) =>
        parser::parse_raw(packet.data, local_net),
      _ => parser::parse_raw(packet.data, local_net),
    };
    if let Some(p) = parsed
      && tx.send(PacketEvent { parsed: p }).is_err() {
      return LoopExit::ReceiverDropped;
    }
  }
  Err(pcap::Error::TimeoutExpired) => continue,
  Err(_) => return LoopExit::TransientError,
}
```

The datalink dispatch is real link-layer handling, not an Ethernet-only shortcut. It decodes Ethernet (`DLT_EN10MB`), BSD loopback (`DLT_NULL = Linktype(0)`), Linux cooked capture (`DLT_LINUX_SLL = Linktype(113)`), what you get on the `any` pseudo-device), and raw IP (`DLT_RAW = Linktype(101)`), with raw IP as the fallback for unknown link types.

`src/capture/parser.rs` turns bytes into a `ParsedPacket { key, direction, len }`. `parse_ethernet` reads the ethertype, dispatches `0x0800` to IPv4 and `0x86DD` to IPv6, and handles `0x8100` 802.1Q VLAN tags by skipping the 4-byte tag and re-reading the inner ethertype. IPv4 parsing reads IHL to find the L4 offset, takes `total_len` from the header as the byte count, maps the protocol number through `Protocol::from_ip_next_header` (`src/data/flow.rs:44`: `6` → TCP, `17` → UDP, `1/58` → ICMP, anything else → `Other(n)`), and extracts source/destination ports for TCP and UDP. The `Protocol` enum's `Display` renders unknown protocols as `Proto(n)` — so an ESP tunnel shows as `Proto(50)`, SCTP as `Proto(132)`, with those exact cases pinned by tests (`flow.rs:124`, `flow.rs:129`).

Direction is where the local-network filter earns its place. `determine_direction` compares `src` and `dst` against the local CIDR: if the source is inside the local net and the destination is not, the packet is `Sent`; the reverse is `Received`; if both or neither are local it defaults to `Sent`. The flow key is then run through `FlowKey::normalize` (`src/data/flow.rs:66`), which canonicalizes the (addr, port) pair ordering using a `u128` comparison so that `A` → `B` and `B` → `A` collapse to the same key, flipping the recorded direction when it swaps. The comparison lifts both IPv4 and IPv6 addresses into `u128` (`u32::from(v4)` widened, or the raw v6 bytes) and compares the (addr, port) tuples, so the ordering is total and allocation-free. That is how one logical connection stays one row regardless of which side a given packet traveled.

data — the accounting model

`src/data/history.rs` is the rate engine. Each flow gets a `FlowHistory` holding two `VecDeque<u64>` ring buffers — one for bytes sent, one for received — capped at `HISTORY_SLOTS = 40` one-second slots (`history.rs:5`), plus cumulative `total_sent/total_recv`, `peak_sent/peak_recv`, `last_seen`, and optional process name/PID. `add_sent/add_recv` fold bytes into the current (back) slot, bump the cumulative counter, and stamp `last_seen = Instant::now()`. `rotate()` is called once a second: it folds the just-completed slot into the peak trackers, pushes a fresh zero slot, and evicts the front once either deque exceeds 40 entries (`history.rs:92`) — a fixed-size sliding window that never grows unbounded regardless of connection lifetime.

The 2s/10s/40s figures come from `window_total`, which sums the trailing N slots:

```
fn window_total(slots: &VecDeque<u64>, n: usize) -> f64 {
    let len = slots.len();
    if len == 0 { return 0.0; }
    let take = n.min(len);
    slots.iter().rev().take(take).sum::<u64>() as f64
}
```

So `avg_sent_2s()` is the summed byte count over the last two one-second slots, `avg_sent_10s()` over ten, `avg_sent_40s()` over forty. Those windowed sums are what feed the display columns and the JSON output; `readable_size` then multiplies by 8 when rendering in bits (the default) or leaves the byte figure alone under `--bytes`.

`src/data/tracker.rs` is the shared state: a `FlowTracker` wrapping `Arc<Mutex<FlowTrackerInner>>` with a `HashMap<FlowKey, FlowHistory>`, global cumulative and peak counters, plus `current_sent/current_recv` accumulators for the in-progress second, and a `last_rotation` timestamp. `record` (`tracker.rs:104`) locks, looks up (or default-inserts) the flow, folds in the bytes, and updates both the per-flow history and the global counters. `maybe_rotate` (`tracker.rs:131`) fires only when at least one second has elapsed: it promotes `current_sent/current_recv` into the global peak if they exceed the running maximum, zeroes the accumulators, rotates every flow's history, and — critically — prunes. Flows whose `last_seen` is older than 60 seconds are `retain`-dropped (`tracker.rs:156`), so an idle connection ages out of the table instead of accumulating forever. `snapshot()` (`tracker.rs:161`) walks the map once, building a `Vec<FlowSnapshot>` — per-flow windowed rates, cumulative totals, process info, and a combined sent+recv per-slot history vector (`s + r` per slot) for the sparkline — plus a `TotalStats` header aggregate computed in a single pass over the same snapshots. The mutex is poison-tolerant everywhere — `lock().unwrap_or_else(|e| e.into_inner())` — so a panic in one thread can't wedge the tracker.

util — attribution, resolution, formatting

`src/util/procinfo.rs` is the part classic `iftop` doesn't have, and it does the attribution without a subprocess. The table it builds is a `HashMap<(u16, u8), (u32, String)>` keyed by `(local_port, protocol)` mapping to `(pid, name)` (`procinfo.rs:14`), stored in a `static PROC_CACHE: OnceLock<Arc<Mutex<ProcTable>>>`. On macOS, `refresh_proc_table_macos` enumerates every PID via `pids_by_type(ProcFilter::All)`, sizes each process's fd buffer from `pidinfo::<BSDInfo>().pbi_nfiles`, walks its fds with `listpidinfo::<ListFDs>`, and for each socket fd reads the local port and protocol straight out of the kernel via `pidfdinfo::<SocketFDInfo>` — selecting the correct arm of the `soi_proto` union by `soi_kind` (TCP vs. datagram), converting the network-order `insi_lport` with a local `ntohs`, and skipping anything that isn't TCP(6)/UDP(17) or has a zero local port (`procinfo.rs:47–124`). The process name is resolved lazily via `name(pid)` only once a socket is confirmed, falling back to `pid:<n>` if the name lookup fails. The `Cargo.toml` comment notes this compiled-in path replaces a runtime `lsuf`. On Linux, `refresh_proc_table_linux` parses `/proc/net/tcp`, `/proc/net/tcp6`, `/proc/net/udp` and `/proc/net/udp6` for the inode → port map, then walks `/proc/[pid]/fd/` for inode → PID and reads `/proc/[pid]/comm` for the name (`procinfo.rs:127`). The table is refreshed on a 2-second cadence by the dedicated `proc-lookup` thread; `lookup_process(src, src_port, dst, dst_port, proto)` resolves a flow by trying the source port

first, then the destination port, against the port table (`src/util/mod.rs`), and returns `None` for ICMP or any non-TCP/UDP protocol (pinned by `procinfo.rs:270`).

`src/util/resolver.rs` handles `async`, cached reverse DNS via `dns-lookup`, wrapping a `HashMap<IpAddr, CacheEntry>` behind an `Arc<Mutex<>>` with a TTL-based `evict_stale` pass (`resolver.rs:120`). It also eagerly parses `/etc/services` on startup into a globally-cached `HashMap<u16, &'static str>, &'static str>` (`resolver.rs:44`, `resolver.rs:150`) for port→service-name mapping, normalizing the protocol string before lookup. `src/util/format.rs` provides `readable_size` (iftop-style `b/kb/Mb/Gb/Tb` in bit mode with a `×8` conversion, `B/KB/MB/GB/TB` in byte mode), `readable_total` for cumulative counters (which always display in bytes regardless of bit/byte mode — `format.rs:26`), and `sparkline`, which maps a slice of values onto the block glyphs , rendering a literal space for a zero-valued slot rather than the lowest block (`format.rs:62`) so idle gaps read as gaps.

ui and config

`src/ui/app.rs` holds `AppState` — sort column, active view tab (`Flows / Processes`), CLI overrides, filter and selection state, and the several popup sub-states (theme chooser, theme editor, interface chooser) — and `src/ui/render.rs` draws it with `ratatui`. Two small enums drive the flow row layout: `LineDisplay` cycles `TwoLine` → `Online` → `SentOnly` → `RecvOnly` (`app.rs:63`), and `BarStyle` cycles `Gradient` → `Solid` → `Thin` → `Ascii` with a `name()` for the status line (`app.rs:88`); `Gradient` is the default. `src/config/theme.rs` defines the `ThemeName` enum with 31 variants (verified: `ThemeName::ALL.len() == 31`, `theme.rs:335`) and their RGB triples, plus a `CustomThemeColors` struct of six palette slots for user-authored themes. `src/config/prefs.rs` is the persistence layer: a TOML file at `~/.iftoprs.conf`, created on first run if missing (`load_prefs` writes the default when the file is absent — `prefs.rs:123`), auto-saved on every in-TUI toggle. The persisted fields include `theme`, `dns_resolution`, `port_resolution`, `show_ports`, `showBars`, `use_bytes`, `show_processes`, `show_cumulative`, `bar_style`, `pinned` (a `Vec<PinnedFlow>`), `show_border`, `show_header`, `hover_tooltips`, `refresh_rate`, an `alert_threshold` in bytes/sec, an optional `interface`, and the `custom_themes` map plus `active_custom_theme` name (`prefs.rs:10–65`). Every field carries a `#[serde(default)]` so an older config missing a key still loads — a forward-compat guard against schema drift. `src/config/cli.rs` is the clap `Args` definition plus the custom help renderer and completion generation.

The CLI surface

The binary front-loads its non-TUI modes in `main()` before ever touching the terminal (`src/main.rs:32-66`): `--help` prints a custom help screen, `--version` prints `iftoprs <version>`, `--completions <shell>` emits a completion script, `--list-colors` previews all 31 themes with swatches, `--list-interfaces` lists devices and exits, and `--json` runs the headless streaming mode. Everything else falls through to the TUI. The capture, display, output and system flags are:

```

# capture
-i, --interface NAME    interface to monitor
-f, --filter EXPR       BPF filter, e.g. "tcp port 80"
-F, --net-filter CIDR   IPv4 local network, e.g. "10.0.0.0/8"
-p, --promiscuous       promiscuous mode

# display
-n, --no-dns            disable reverse DNS
-N, --no-port-names     disable port→service names
-b, --no-bars           disable bar graphs
-B, --bytes            show bytes instead of bits
-P, --hide-ports        hide ports next to hosts
-Z, --no-processes     hide the process column

# output / system
--json                 stream NDJSON, no TUI
-c, --config FILE       config path (default ~/.iftoprs.conf)
-l, --list-interfaces   list interfaces and exit
--list-colors          preview all 31 themes
--completions SHELL    zsh | bash | fish | elvish | powershell

```

The five completion shells are real, but the source of the enum is worth stating precisely: `cli.rs` imports `clap_complete::Shell` (`cli.rs:5`) and wires it directly as the `--completions` argument type, rather than defining its own — so `Zsh`, `Bash`, `Fish`, `Elvish` and `PowerShell` are `clap_complete`'s built-in variants, round-tripped through the parser and pinned by the tests at `cli.rs:450–474` and `tests/contract_args_independence_round4.rs.generate_completions` (`cli.rs:175`) simply calls `clap_complete::generate` against the built `Args::command()`. A prebuilt zsh completion also ships checked-in at `completions/_iftoprs`, starting with the required `#compdef iftoprs` header.

Two clap details are set explicitly: `disable_help_flag` and `disable_version_flag` (`cli.rs:68`) are both on, because `-h/--help` and `-V/--version` are handled by hand as plain `bool` fields so the custom cyberpunk help screen (`print_cyberpunk_help`, `cli.rs:8`) renders instead of clap's default. The `-F` net filter is parsed by `parse_net_filter` (`cli.rs:181`), which splits on `/`, parses the address and prefix, and returns `None` (with a stderr warning) on anything that isn't valid CIDR — accepting both IPv4 and IPv6 forms, verified across a spread of cases including `2001:db8::/32`, `::1/128`, `0.0.0.0/0` and the malformed `10.0.0.0/24/8`.

Typical invocations (raw capture needs root):

```

sudo iftoprs -i en0          # monitor one interface
sudo iftoprs -f "tcp port 443" # HTTPS only, via BPF
sudo iftoprs -F 10.0.0.0/8 -B # scope local net, bytes
sudo iftoprs -Z             # hide the process column
iftoprs --completions zsh > _iftoprs
iftoprs --list-colors       # theme swatches, then exit

```

JSON mode for pipelines

`--json` (`run_json_mode` in `src/main.rs:762`) stands up the same capture and proc-lookup threads but skips the TUI entirely, emitting one JSON object per line (NDJSON) at the configured refresh interval (`prefs.refresh_rate` seconds). Each line is a `JsonSnapshot` with an RFC-3339 timestamp, `flow_count`, cumulative totals, peaks, and a `flows` array. Per-flow rate fields are multiplied by 8 into bits unless `--bytes` is set, `src/dst` are rendered as `host:port` (resolved through the DNS cache), and `process_name/pid` are omitted when unknown (`[serde(skip_serializing_if = "Option::is_none")]`, `main.rs:827`):

```

sudo iftoprs --json | \
jq -c '.flows[] | {dst, sent_2s, process_name}'

```

```
{
  "timestamp": "2026-07-13T10:22:04-04:00", "flow_count": 2,
  "total_sent": 184320, "total_recv": 998400, "peak_sent": 24576.0,
  "peak_recv": 131072.0, "flows": [
    {
      "src": "host.example.com:52344",
      "dst": "cdn.example.com:443", "src_port": 52344, "dst_port": 443,
      "protocol": "TCP", "sent_2s": 98304.0, "recv_2s": 786432.0,
      "sent_10s": 410000.0, "recv_10s": 3100000.0, "sent_40s": 1200000.0,
      "recv_40s": 9800000.0, "total_sent": 184320, "total_recv": 998400,
      "process_name": "curl", "pid": 40122
    }
  ]
}
```

That makes the same accounting core usable as a data source for logging, dashboards, or ad-hoc `jq` slicing, not just an interactive view.

The interactive loop

The TUI runs at roughly 30 fps: a 33 ms tick (`run_app`, `main.rs:174`) drains the packet channel into the tracker with `try_recv`, calls `maybe_rotate`, refreshes the display snapshot only at the user's configured refresh rate, redraws with `ratatui`, and polls `crossterm` for input within the remaining tick budget. The snapshot refresh is decoupled from the render tick — the terminal repaints every ~33 ms, but `update_snapshot` only pulls a fresh flow table every `refresh_rate` seconds, so a 10-second refresh still gives a smooth, responsive UI while the numbers hold steady between updates.

Mouse capture is enabled (`main.rs:124`), and `handle_mouse` (`main.rs:652`) routes clicks: left-click selects a flow row, right-click on a row raises a tooltip with TX/RX rates, totals, process and a 40-second sparkline, right-click on a header segment raises a persistent segment tooltip, middle-click pins a flow, scroll navigates, and any mouse-move dismisses the flow tooltip. The row-to-index math accounts for the extra header row in the process view (`main.rs:699`).

Keyboard controls mirror iftop where they overlap and add a layer beyond it. The core set: `1/2/3` sort by the 2s/10s/40s columns, `</>` sort by source/destination name, `r` reverses the sort, `o` freezes the current order (so rows stop reshuffling under you), `t` cycles the two-line/one-line/sent-only/recv-only display, `b` cycles the bar style, `n/N` toggle DNS and service-name resolution, `B` toggles bits/bytes, `p` toggles ports, `U` toggles cumulative-total columns, `Z` toggles the process column, `x` toggles the border, `g` toggles the header bar, `T` toggles hover tooltips, `f` cycles the refresh rate, and `P` pauses. Navigation is vi-style: `j/k` (or arrows) move the selection, `G/End` and `Home` jump to the extremes, and `Ctrl-D/Ctrl-U` page down/up — all dispatched per active view tab. Beyond iftop it adds: `Tab` switches between the flow view and a per-process aggregated view, `Enter` drills into a process's flows (in the process tab), `/` opens a live filter, `0` clears it, `c` opens the theme chooser, `C` opens the theme editor, `i` opens the interface chooser, `F` pins a flow to the top, `y` copies the selected flow, `e` exports all flows to `~/.iftoprs.export.txt`, and `h/?` toggle the help HUD. Every display toggle calls `save_prefs()` immediately, so the config file tracks the live UI without an explicit “save.”

Modal sub-states

Four of those keys open modal input states that intercept keys before the main dispatch (`main.rs:225–427`). The **filter** (`/`) is a full mini-readline: character insert, `Backspace`, cursor `Left/Right/Home/End`, and the emacs bindings `Ctrl-W` (delete word), `Ctrl-A/Ctrl-E` (line start/end), `Ctrl-K` (kill to end) and `Ctrl-U` (clear), with a live preview that updates `screen_filter` on every keystroke; `Enter` commits, `Esc` restores the previous filter. The **theme chooser** (`c`) navigates with `j/k` or the arrows (and scroll wheel), applies each theme live as you move, and persists on `Enter`. The **theme editor** (`C`) is a six-slot RGB palette editor: `j/k` move between slots, `h/l` nudge the selected color index by 1 and `H/L` by 10 (wrapping), applying the palette live; `s/Enter` enters a naming sub-mode that writes the result into `custom_themes` under a user-supplied name (capped at 20 chars). The **interface chooser** (`i`) lists devices from `list_interfaces` and, on `Enter`, records the selection to config with a “restart to apply” status note — capture is bound at launch, so the interface change is persisted rather than hot-swapped.

Threshold alerts

The `alert_threshold` preference (bytes/sec, default 0 = off, config-file only) drives `check_alerts` (`app.rs:1107`). On each snapshot it sums each flow's 2-second sent+recv rate; any flow at or above the threshold that wasn't already alerting produces a new alert. When new alerts appear it sets `alert_state.flash = Some(Instant::now())` (a timed border flash handled by the renderer), writes an  ALERT status line naming the source and rate, and emits a terminal bell (`print!("{}", app.rs:1135)`). The alerting set is tracked across ticks so a sustained flow flashes once rather than continuously.

The process view and export

`Tab` switches to a per-process aggregate built by `aggregate_processes` (`app.rs:1612`): flows are grouped by `process_name` (unattributed flows collapse into a single `(unknown)` bucket), their 2s/10s/40s rates and cumulative totals summed, flow counts tallied, and the result sorted descending by 2-second combined rate. `Enter` on a process drills back into the flow view filtered to that process; `Esc` clears the drill-down. `e` writes a formatted plaintext report to `~/.iftoprs.export.txt` — a timestamped header, a fixed-width `SOURCE <=> DESTINATION / PROTO / TOTAL / 2s / 10s / 40s` table with labels truncated to 40 columns, and TX/RX summary lines carrying cumulative, peak, and the three windowed rates (`app.rs:1442`). `y` copies the selected flow's line to the clipboard.

Where it sits in the stack

`iftoprs` is one of the MenkeTechnologies `*rs` Rust reimplementations of Unix networking and system tools; the README cross-links it with `lsotrs`, `zshrs` and `strykelang`. It installs from crates.io (`cargo install iftoprs`) or the Homebrew tap (`brew install MenkeTechnologies/menketech/iftoprs`). Two man pages ship: `man/man1/iftoprs.1` (short reference, with `NAME/SYNOPSIS/DESCRIPTION/OPTIONS/KEYBINDINGS/EXAMPLES/ENVIRONMENT/FILES/EXIT STATUS` sections) and `man/man1/iftoprsall.1` (the full reference). Note that `iftoprsall` is a second *man* page, not a second binary; the crate builds one executable, `iftoprs`. A checked-in `iftoprs.default.conf` documents every TOML key and enumerates all 31 theme names as reference for hand-editing the config.

Parity and status

Against upstream iftop, the shared surface is the flow table, the three rolling windows, the src ↔ dst bidirectional accounting, BPF filtering, promiscuous mode, bits-vs-bytes display, and the sort/display keybinds. Beyond iftop it adds native per-flow process attribution with no external tools, a per-process aggregate view with drill-down, 31 themes plus a live RGB theme editor and user-saved custom palettes, mouse and hover tooltips, sparklines, four bar styles, four line-display modes, TOML config that auto-persists, threshold alerts (border flash plus terminal bell), an NDJSON headless mode, and an auto-restarting capture pipeline with real handling for Ethernet, loopback, Linux cooked and raw-IP link types.

Two accounting details are worth stating precisely rather than glossing as “identical to iftop.” First, the 2s/10s/40s columns hold the *summed* byte count over the trailing N one-second slots (`window_total`), rendered through `readable_size` with a ×8 bit conversion — the value is a windowed total scaled by the display, not a slot-averaged bytes/sec. Second, direction assignment leans on the local CIDR; when neither endpoint is inside the configured local net (or no net filter is active and none is auto-detected via `auto_detect_local_net`, which derives the prefix from the interface netmask and defaults to `/24`), packets default to `Sent`, which affects the TX/RX split on transit or fully-remote traffic.

CI runs on every push and PR across Ubuntu and macOS runners (libpcap-dev installed via apt on Linux) and gates on `cargo fmt --check`, `cargo clippy -D warnings`, `cargo doc` with `RUSTDOCFLAGS=-D warnings`, `cargo build/cargo test`, and a set of shell scripts under `tests/` that enforce docs/man/README/newline hygiene (final-newline checks, man-page synopsis presence, README

badge/section/https-link checks, workflow no-tabs, and a battery of docs HTML lints). Integration tests execute the built binary through `CARGO_BIN_EXE_iftoprs` rather than `cargo run`, so CLI output is read from the real process. The source carries a large embedded test suite: a `grep` for `#[test]` returns 2,286 annotations (1,741 inline in `src/`, 545 under `tests/`) — a provisional count of annotations, not a guarantee of passing runs, since it was measured by pattern match rather than a `cargo test` execution in this session.

Design commentary

Two decisions define the shape of the port. The first is the 40-slot ring buffer with 60-second idle eviction: it fixes the memory ceiling at “number of active flows × 40 u64 slots × 2 directions” and refuses to let a long-lived session’s history grow, while the eviction pass keeps the visible table bounded to genuinely recent traffic. The second is compiling the process-attribution backend into the binary rather than shelling out. Classic `iftop` has no process column at all; tools that add one typically parse `lsof` or `ss` output at runtime. `iftoprs` reads the kernel socket table directly — `libproc` syscalls on macOS, `/proc` on Linux — on a 2-second background cadence, keyed by `(local_port, protocol)`, so attribution is a hash lookup against a pre-warmed table rather than a subprocess spawn per refresh. The tradeoff is honest: attribution only resolves TCP and UDP flows, only where the local port is visible in the socket table, and it inherits the privilege limits of inspecting other processes’ fds — which is why some flows show no process even under root. Within those bounds it is a single self-contained binary that does what `iftop` did and attributes it too.

htoprs — the interactive process viewer, ported function-for-function into Rust

`htoprs` is a faithful Rust port of `htop`, the cross-platform, ncurses-based interactive process viewer that most people reach for when `top` is not enough. The tool it descends from is not new — `htop` has existed since 2004 and is standard equipment on virtually every Linux and BSD box. Nothing about the *concept* of a scrollable, colored, tree-capable process table is invented here. What is engineered here is the port itself: `htop`'s C source treated as an executable specification and translated into Rust module by module, function by function, with each translated `fn` carrying a citation back to the `<File>.c:<line>` it came from. The value proposition is not “a better process viewer” — it is “the same process viewer, byte-for-byte, rebuilt in a memory-safe, vendorable, cross-architecture Rust codebase with a mechanically-enforced parity discipline.” The upstream is `htop 3.5.1`, pinned in-repo as the `vendor/htop` submodule, and that pin is the spec of record.

The package is `htoprs` version **0.5.7** (Cargo.toml), edition 2021, licensed MIT — original Rust code that reimplements `htop`'s behaviour rather than deriving from its source. It builds one library (`htoprs`) and one binary (`htoprs`). Attribution is MenkeTechnologies.

What a “faithful port” means mechanically

The distinguishing engineering decision in this codebase is that faithfulness is not an aspiration in the README — it is a build gate. The source is split into two trees with different rules:

- `src/ported/<file>.rs` — one Rust module per `htop` C file. Every free `fn` here is supposed to be a translation of a specific C function, and each carries a `/// Port of` doc comment naming its origin. There are **1,937** such `/// Port of` citations across the ported tree (grep of `src/ported`), spread over **141** Rust modules.
- `src/extensions/<name>.rs` — `htoprs`-original code that is *not* a translation of anything in `htop`, and is therefore explicitly exempt from the port rules.

The gate lives in `build.rs` and runs on every `cargo build`, `cargo test`, and `cargo check` that touches `src/ported/`. It extracts every free `fn` name under `src/ported/` and checks it against a snapshot of `htop`'s actual C function names at `tests/data/htop_c_fn_names.txt` — a flat index of `<File>.c:<name>` pairs, **4,132** entries as checked in. A function whose name has no counterpart in that index *fails the build*, printing each violation as a `cargo:warning` line and then a `panic!` with instructions (inline the body, rename to the real C function, or — last resort — justify it in the allowlist). Rust-original helpers are rejected outright; genuine architectural helpers that have no C analog must be listed explicitly in `tests/data/fake_fn_allowlist.txt`, which at present holds **zero** entries — nothing in the ported tree needed the escape hatch.

The gate is more careful than a regex sweep would be. `collect_free_fns` in `build.rs` is a byte-level brace-counting state machine that tracks nesting depth so it can distinguish module-level free functions (depth 0) from methods inside `impl/trait` blocks (which it skips), and it enters and leaves `mod tests { / mod test {` blocks so unit-test helpers do not trip the check. Crucially it ignores braces that appear inside line comments, block comments (with nesting), string literals, raw strings (`r#"..."#` with balanced hash counts), and char literals — without that, a test that merely contains `"{"` in a string would corrupt the depth counter and misfire the gate after any harmless reordering. It strips `pub`, `pub(crate)`, `pub(super)`, `unsafe`, `async`,

and `extern "C"` prefixes before reading the name, so the gate sees the same identifier regardless of how the port declares visibility or ABI. If the ported root or the C-name snapshot is missing entirely, the build script returns silently rather than failing an unrelated workspace.

The C-name snapshot is regenerated by `tests/data/extract_c_fn_names.sh` after pulling a new upstream htop (it honors `$HTOP_C_SOURCE`, default `~/forkedRepos/htop`), so the gate tracks the real upstream rather than a hand-maintained list that drifts. Coverage is reported by `scripts/gen_port_report.py`, which walks both the C source and the Rust port and writes `docs/port_report.html` with per-file and overall numbers derived from source at run time — nothing hardcoded. Critically, a `todo!()` or `unimplemented!()` body is counted as **stubbed**, never **ported**, so scaffolding cannot inflate the coverage figure. The stub scaffold generator (`scripts/gen_port_stubs.py`) lays out the full port surface — one module per C file, one `todo!()` stub per C function named to satisfy the gate — and never overwrites an already-ported module.

Current port state

Per the README's "Current state" section and the generated `docs/report.html`, the port covers **1,069 of 1,093 C functions (97.8%)** across **130 of the 131 C files**, with **1 stub remaining**, and the TUI runs as a daily driver on macOS. (A raw grep for `todo!()/unimplemented!()` under `src/ported` returns 57 tokens; most sit in the intentionally-unported allocation families and the feature-gated BSD/PCP backends, which the report classifies as out-of-scope rather than as in-scope stubs — that interpretation is provisional, the authoritative breakdown is the generated report.) The ported surface is end-to-end: the process model and table build (`Process`, `ProcessTable`, `Table`, `Row`, `Machine`), the container and utility layer (`Vector`, `Hashtable`, `RichString`, `XUtils`), the full meter set, the UI panels and main loop, key dispatch, the `CRT` terminal layer, and the per-OS machine/process-table backends.

Both man pages still lag the code. `man/man1/htopr.1` carries a **PORT STATUS** note claiming "the port is at an early stage ... the binary does not yet present an interactive interface," and `man/man1/htoprall.1` similarly says only the `XUtils.c` layer is ported; those texts predate the 97.8% figure and should be treated as stale relative to the README's current-state numbers and the live `docs/port_report.html`.

Some C is intentionally *not* ported because it has no faithful safe-Rust analog. The `xMalloc` allocation family, the raw `Object**` dynamic-array memory machinery in `Vector.c` (`Vector_new/_insert/_add/_resizeIfNecessary`), the open-addressing bucket table in `Hashtable.c` (`Hashtable_new/_put/_get/_foreach`), the varargs formatters, and `String_freeArray` all manage allocation, bounds, probing, and lifetimes by hand — work that Rust's `Vec` and `HashMap` already own. Porting those byte-for-byte would mean reimplementing an allocator in `unsafe`, which is precisely the class of bug the port exists to eliminate. `combSort` in `Vector.c` is commented-out dead code and is not ported. These omissions are honest and documented, not coverage gaps papered over.

The dependency surface

The runtime crate graph is deliberately small and foundational, chosen to keep the build vendorable and durable. From `Cargo.toml`: `crossterm` 0.29.0 (the terminal layer), `libc` 0.2.186 (the real `getopt_long`, `sysctl`, and platform syscall surface), `nix` 0.31.3 (typed syscall wrappers), `unicode-width` 0.2.2 (column-width math for the styled string buffer, matching htop's `wcwidth` handling), and `serde` 1.0.228 with `serde_json` 1.0 for persisting preferences. `regex` 1 backs the saved-filters extension. `ratatui` 0.30.0 is pulled with `default-features = false` and only the `std` feature so it drags in *no* terminal backend — the themed help overlay, chooser, and editor render into a `ratatui::Buffer` while htoprs still drives the actual terminal through its own `crossterm`, avoiding a second, conflicting `crossterm` in the tree.

The Linux target adds three pure-Rust, no-FFI helper crates behind `cfg(target_os = "linux")`: `libmedium` 0.13.4 for hwmon sensors, `neli` 0.6 (a generic-netlink client) for the delay-accounting path that queries the kernel TASKSTATS family (`linux/LibNL.c`), and `zbus` 5 (a pure-Rust D-Bus client) for the `SystemdMeter` port (`linux/SystemdMeter.c updateViaLib`). Both `neli` and `zbus` speak their respective

wire protocols directly over a socket rather than linking a C library — a deliberate choice to keep the build free of C dependencies and vendorable long-term. `zbus`'s blocking module mirrors `htop`'s synchronous sd-bus property reads.

Module map

The `src/ported/` tree is a near-mirror of `htop`'s file layout. The generic model and UI substrate includes `process.rs`, `processtable.rs`, `table.rs`, `row.rs`, `machine.rs`, `object.rs` (`htop`'s `vtable` OOP rendered as a Rust `Object` trait with a class-chain `Object_isA`), `richstring.rs`, `vector.rs`, `hashtable.rs`, and `xutils.rs`, alongside the settings and support modules `settings.rs`, `history.rs`, `scheduling.rs`, `affinity.rs`, `userstable.rs`, `listitem.rs`, `optionitem.rs`, and `htop.rs` (the ported `main`). The `ProcessField` enum in `process.rs` transcribes `htop`'s `RowField.h` reserved-field discriminants verbatim (`PERCENT_CPU = 47`, `PERCENT_MEM = 48`, and so on), because those numeric IDs are load-bearing for column identity and must match.

The UI layer is `panel.rs`, `screenmanager.rs`, `mainpanel.rs`, `functionbar.rs`, `header.rs`, `crt.rs`, plus the setup and dialog screens: `categoriespanel.rs`, `columnspanel.rs`, `availablecolumnspanel.rs`, `colorspanel.rs`, `displayoptionspanel.rs`, `headeroptionspanel.rs`, `meterspanel.rs`, `availablemeterspanel.rs`, `screenspanel.rs`, `screentabspanel.rs`, `signalspanel.rs`, and `affinitypanel.rs`. Interaction dispatch is `action.rs`, `incset.rs` (incremental search/filter), and `lineeditor.rs`. Auxiliary screens include `envscreen.rs`, `openfilescreen.rs`, `processlocksscreen.rs`, `commandscreen.rs`, `tracescreen.rs`, `backtracescreen.rs`, `dynamicsscreen.rs`, and `infoscreen.rs`.

The meter set is fully represented: `cpumeter.rs`, `memorymeter.rs`, `swapmeter.rs`, `memoryswapmeter.rs`, `loadaveragemeter.rs`, `batterymeter.rs`, `networkiometer.rs`, `diskiometer.rs`, `gpumeter.rs`, `zfsarcmeter.rs`, `zfscompressedarcmeter.rs`, `tasksmeter.rs`, `uptimemeter.rs`, `datetimemeter.rs`, `hostnamemeter.rs`, `sysarchmeter.rs`, `filedescriptormeter.rs`, and the `dynamicmeter.rs` / `dynamiccolumn.rs` machinery for platform-defined meters. `meter.rs` holds the shared value-formatting core, including `Meter_humanUnit` (kibibytes → human-readable `K...Q` with an `inf` cap).

Platform backends live in per-OS subdirectories: `darwin/`, `linux/`, `freebsd/`, `openbsd/`, `netbsd/`, `dragonflybsd/`, `solaris/`, plus `generic/`, `pcp/`, and `unsupported/`. The Darwin backend is `darwinmachine.rs`, `darwinprocess.rs`, `darwinprocesstable.rs`, `platform.rs`, and `platformhelpers.rs`. The Linux backend is the largest — sixteen modules including `linuxmachine.rs`, `linuxprocess.rs`, `linuxprocesstable.rs`, `platform.rs`, `cgrouputils.rs`, `libnl.rs`, `libsensors.rs`, `gpu.rs`, `ioprioritypanel.rs`, and the platform meters `systemdmeter.rs`, `pressureallmeter.rs`, `selinuxmeter.rs`, `zrammeter.rs`, `openrcmeter.rs`, and `hugepagemeter.rs`. The `generic/` directory holds portable fallbacks and the FFI-gated leaves — `demangle.rs`, `unwindptrace.rs`, `openzfs_sysctl.rs`, `fdstat_sysctl.rs`, `gettime.rs`, and `uname.rs`. On Linux the port reaches the kernel through the pure-Rust `neli/zbus/libmedium` crates named above rather than through C libraries.

Terminal backend

`htop` renders through `ncurses`. `htoprs` does not — the terminal layer is `crossterm` 0.29, pure-Rust, cross-architecture, no C dependency. This is a *behavioral* port rather than a line-for-line one: the constraint is that output must be byte-for-byte identical to `htop`, not that the draw code mirror `ncurses` call-for-call. To meet that constraint the color substrate underneath the draw code is ported faithfully — `CRT.c`'s `ColorElements` enum and every `CRT_colorSchemes` entry are transcribed verbatim so that `htoprs`'s colors match `htop`'s exactly, glyph for glyph and attribute for attribute. The optional feature-gated `ncurses`-adjacent build variants `htop` offers via `./configure` are mirrored as Cargo features: `pcp` (Performance Co-Pilot backend), `demangle` (C++/Rust symbol demangling for the backtrace screen), `unwind` (libunwind-ptrace backtraces), `capabilities` (libcap capability dropping), and `hwloc` (topology-aware affinity picker). All are off by default; each declares its foreign

FFI surface by hand in `extern` blocks and is verified by primary-source reading plus the port gate rather than by linking on every platform (several of those libraries do not link on macOS). The `hwloc` feature carries an explicit, documented substrate gap: `AffinityPanel_new`'s topology-tree build reads a `host->topology` field the ported `Machine` omits (`machine.rs:46`), so until that field is wired the `topoRoot/allCpuset` values stay their null/None defaults — the Cargo.toml feature comment states this outright rather than hiding it.

The clippy-allow ledger

`src/lib.rs` carries sixteen crate-wide `#![allow(clippy::...)]` attributes, each with a doc paragraph tying it to a faithful-port reason, because CI runs `cargo clippy --all-targets -- -D warnings`. These are not blanket lint suppression — every one names the C shape it preserves. `needless_range_loop` and `explicit_counter_loop` keep the dual-index `for (i, j)` walks that mirror the C pointer arithmetic in `strncpy` and `RichString` fills. `manual_div_ceil` keeps `(cpus + 1) / 2`, the C half-split formula. `neg_cmp_op_on_partial_ord` keeps `!(rate >= 0.0)`, a deliberate NaN guard from `Row.c` that `rate < 0.0` would silently break. `manual_c_str_literals` keeps `b"kern.osrelease\0"` as the literal constant passed to `sysctlbname`, since `c"..."` would hide the NUL the C source spells out. `field_reassign_with_default` keeps `htop's *_init` routines setting fields one at a time on a freshly-`calloc`'d struct rather than collapsing into a Rust struct literal. The full list is the clearest single artifact of the port's discipline: where idiomatic Rust and the C spec disagree, the spec wins and the lint is documented, not the other way around.

CLI surface

The command-line contract mirrors `htop's getopt_long` table exactly, driven through the real `libc::getopt_long` symbol so parsing behavior matches. `parseArguments` in `src/ported/commandline.rs` marshals `argv` into a null-terminated C `char**`, builds the same `option` array `htop` builds (with `has_arg` 0/1/2 for none/required/optional), and calls `libc::getopt_long` in the same loop. The option spec is:

```
-h, --help          Print the help screen and exit
-V, --version       Print version info and exit
-d, --delay=DELAY   Update delay, tenths of a second (1..100)
-n, --max-iterations=NUMBER Exit after NUMBER refresh iterations
-s, --sort-key=COLUMN Sort by COLUMN (--sort-key help lists)
-u, --user[=USERNAME|UID] Only this user's processes (self if none)
-C, --no-color, --no-colour Monochrome mode
-M, --no-mouse      Disable mouse control
-U, --no-unicode     ASCII graph meters (not Unicode braille)
-t, --tree          Tree view
-p, --pid=PID,PID... Show only the given PIDs
-F, --filter=FILTER Fixed-string filter ("|" multi-term)
-H, --highlight-changes[=DELAY] Highlight new/old processes
    --no-meters      Hide graph meters
    --no-function-bar Hide the function bar (--no-functionbar)
    --readonly       Disable process/system-changing features
```

Two flags are intentional divergences: `htoprs -h` and `htoprs -V` route to branded help/version screens before delegating to the shared ported startup path. `src/main.rs` is a thin wrapper that scans `argv` for `-h/--help` and `-V/--version` and short-circuits — `-V` calls the ported `printVersionFlag` (which prints `htoprs 0.5.7`), `-h` calls `extensions::help::print_help` (the figlet-bannered, sectioned screen in the MenkeTechnologies house style). Everything else calls `CommandLine_run`, the ported entry that does the `getopt` parse, assembles the runtime object graph, and drives `ScreenManager_run` — the same path `htop's main` follows. The plain ported `printHelpFlag` is retained verbatim as the spec the styled screen tracks; it emits the version line, the `(C) MenkeTechnologies 2026`. copyright, the MIT licence notice, and the same aligned option list `htop` prints. There are two `CommandLine_run` definitions in `commandline.rs`, selected by `cfg`, so the non-interactive/test path and the live-TUI path share one signature.

The zsh completion (`completions/_htoprs`, `#compdef htoprs`) keeps its option set in lockstep with that `getopt` table, and it completes sort columns *dynamically* — the `sort-keys` state calls `_call_program sort-keys $words[1] --sort-key help` and parses the tool's own live column list rather than hardcoding one:

```
case $state in
  sort-keys)
    tmp=( ${f}"${_call_program sort-keys \
      $words[1] --sort-key help}" )
    tmp=( ${tmp/#[:space:]}## )
    tmp=( ${tmp//:/\\:} )
    _describe -t sort-keys 'column (key)' tmp 66 ret=0
  ;;
esac
```

The extensions layer

On top of the port sits `src/extensions/` — **21 modules** (22 files including `mod.rs`) that are `htoprs`-original and gate-exempt; this has grown past the “20 modules” the README still cites, because the `forecast` module below was added. This is where `htoprs` adds capability `htop` does not have, carefully bound to keys `htop` leaves free so that no upstream binding is shadowed. The theming system (`extensions::theme`, `colors`, `prefs`, `overlay`) provides 31 built-in 6-color palettes plus custom-theme plumbing, remapping the live `htop` UI in 256-color at the single `Ncurses::to_color` choke point and persisting the choice to `~/.config/htoprs/prefs.json`. Overlay keys: `z` opens the theme chooser, `~` the theme editor, `h`/`F1` a themed help overlay, `g` toggles the header, `B` toggles the border, and `b` cycles the bar fill-glyph style (Classic, Gradient, Solid, Thin, Ascii) through the ported `BarMeterMode_draw` fill loop.

The live monitoring suite is driven by a single thread-local `PanelState` in `extensions::panels`, fed the real ported process table each sample tick. That struct holds the persistent engines (`ring: ProcRing` per-PID history, `alerts: AlertEngine`, `filters: FilterStore`, `cpu_hist`, and an optional `baseline` snapshot), the latest sampled frame (`table`, the `firing` PID set, `selected_pid`, `cpu_peak`, `tick`), the forecast fields, and the UI state (a `Modal` enum with ten variants and a `SparkMode` with three). It takes first refusal on keystrokes — yielding to the theme overlay if that owns the frame — and dispatches on raw key bytes chosen from keys `htop` leaves unbound:

```
f  Fuzzy process finder – Enter jumps the cursor to the match
r  Regex/substring filter over comm/cmdline/user, saved named
   store (~/.config/htoprs/filters.json)
d  Snapshot: first press = baseline, next = diff
   (+started -exited -changed); w writes the snapshot JSON
o  Export the current table to JSON + CSV under ~/.config/htoprs/
A  Threshold alerts – the rule set plus every firing PID
E  Forecast – per-PID memory-exhaustion ETA (see below)
G  Braille CPU history graph (system total plus selected PID)
y  Aggregate/pivot – CPU+memory totals grouped by
   user/command/parent (Tab cycles)
:  Command palette – fuzzy-search every action by name, run it
v  Per-PID CPU sparkline: off → right-edge column → inline graph
```

The `:` palette is backed by a `CMD5` registry that pairs each action's searchable name with the key byte it injects into the run loop, so a palette pick flows through the *whole* dispatch pipeline — the palette can reach both extension actions and `htop`'s own keys (`k` kill, `t` tree, `/` search, `S` setup) with one mechanism. Two of these reach the rows themselves rather than a modal, injected at the per-row draw site in `Panel_draw` (the same extension-hook pattern the theme border uses, so no new ported `fn` is introduced): a firing-alert PID's row is recolored, and the `v` sparkline is overdrawn per row. The inline `v` graph mode (`SparkMode::Double`) makes the panel variable-height — each process grows a full-width braille CPU graph whose line count scales with its CPU usage, so `Panel_draw/Panel_onKey` project every screen-Y, page step, and scroll clamp through per-row heights (`item_height = 1 + graph_lines(cpu)` for a process, `1` otherwise). Non-process panels

keep `rowHeight = 1` and stay byte-identical to ported behavior.

Forecast: the memory-exhaustion horizon

The newest extension, `extensions::forecast`, answers a question htop cannot: not what memory a process holds *now*, but *when* that becomes a problem. Bound to `E`, it opens a modal listing every rising PID sorted by soonest ETA to exhaustion. Each refresh it fits an ordinary least-squares trend on that PID's bounded memory ring and projects the wall-clock time until its resident set crosses its own real ceiling.

The math is split so it is testable without an OS underneath it. `Trend::fit` is a pure hand-rolled closed-form slope (three sums, no clock, no I/O) over the sample indices of a PID's KiB series — positive slope means growing, and it returns `None` for fewer than two samples. `Ceiling::detect` reads the enforced limit *once* and caches it, because the enforced limit does not move during a session, and `select_ceiling` is a separate pure function that picks it in precedence order: a Linux cgroup-v2 `memory.max` if present, else the process address-space `RLIMIT_AS`, else total machine RAM. The comment is explicit that a stats crate was declined for the fit — the closed form is three sums and the durability rule prefers no new dependency. `PanelState` measures `sample_secs` live from the gap between ticks so the ETA tracks the real refresh rate (and pauses when updates pause), keeps every rising PID's `Projection` sorted by ETA, and maintains a `forecast_soon` set of PIDs whose ETA is at or under a `forecast_horizon` threshold — that set drives an optional inline row tint, toggled from within the modal, while the projections keep computing whether or not the tint is on.

Man pages, completion, and documentation

htopr ships two man pages. `htopr.1` is a short quick-reference of the CLI contract. `htoprall.1` is a comprehensive all-in-one reference modeled on `zshall(1)` — eighteen numbered sections concatenated into one greppable page covering invocation, every command-line option, the process-list columns, the header meters and their Bar/Text/Graph/LED modes, tree view, search/filter/tagging, sorting, every interactive keybinding, the function bar, the setup screen, color schemes, mouse support, signals, configuration files, environment, exit status, worked examples, and porting notes. It cross-references the rest of the fleet in SEE ALSO (`iftoprsall`, `lsofrsall`, `nmaprsall`, `zshrsall`). Both pages, as noted above, still carry the stale early-stage porting caveat and should be resynced to the 97.8% figure.

Verification, parity, and crash handling

There are **1,412** `#[test]` functions across `src/` and `tests/` (grep count) — hand-crafted unit tests pinning the C edge-case behavior each ported function must reproduce, rather than generic coverage padding. Beyond unit tests, a dedicated **parity suite** at `tests/parity/` (registered as the single `parity` cargo test target, since Cargo does not auto-discover files under that subdirectory) compares htoprs against the genuine htop in two ways. `cli_parity.rs` runs the reference C `htop` binary and `htopr` on the same arguments and diffs output byte-for-byte, modulo the deliberate rebrand handled by `canon()` (`htopr` → `htop`, differing version banner). `xutils_parity.rs` goes further: it compiles a tiny C harness (`tests/parity/cref/htop_cref.c`) *against htop's real XUtils.c* — the genuine 3.5.1 implementation, not a reimplementation — and diffs the Rust port's output against that compiled C oracle, function by pure function. The harness gates itself: `ref_available()` returns the reference path only if it exists and is the **3.5** series (`REF_HTOP_SERIES`), and the xutils path skips when the htop C source or a C compiler is absent, so CI without them stays green while a dev box with them runs the real diff. Point the binary comparison at a specific reference with `HTOP_REF=/path/to/htop`, and the source comparison with `HTOP_C_SOURCE=/path/to/htop`.

```

cargo build          # runs the port-purity gate
cargo test          # ported unit tests
cargo test --test parity # diff htoprs vs reference htop 3.5.x
cargo test --test parity cli # only the CLI cases
cargo test --test parity -- --ignored # documented gaps
python3 scripts/gen_port_report.py # regen the report

```

Because a TUI runs on the alternate screen with the tty in raw mode, a panic printed to the terminal would be painted over and lost the instant the screen is restored. htoprs therefore persists crashes to a logfile at `~/.cache/htoprs/crash.log` (honoring `$XDG_CACHE_HOME`, overridable with `$CRASH_LOG`), via `extensions::crashlog`. Two paths funnel into that one file: `log_panic`, called from a process-wide panic hook, and the ported SIGSEGV/SIGBUS handler's stderr report. Each entry records time, htoprs version, pid, thread, source location, panic message, and a full backtrace captured unconditionally — no `RUST_BACKTRACE=1` required — and entries are appended so the history survives across crashes. On a crash the restored terminal prints `htoprs: crash logged to <path>`.

Place in the stack

htoprs slots into the MenkeTechnologies CLI fleet as the process-monitoring front end, shipping a `#compdef htoprs` zsh completion whose option set is kept in lockstep with the getopt table in `commandline.rs`, plus the short and comprehensive man pages. It coexists with, rather than replaces, the C htop it ports — the parity suite depends on the original being installed as the oracle, and the xutils path links the original's `XUtils.c` directly. The honest framing is the one the README leads with: the C source is the spec, the port is faithful, and the engineering that matters is the architecture of the port, the two-tree gate that keeps it honest (a comment-and-string-aware brace state machine in `build.rs`, an empty allowlist, a 4,137-entry C-name index), the byte-for-byte parity harness that reaches down to individual C functions, the sixteen-entry clippy ledger that documents every place the spec overrides idiom, and the extension layer bolted on at keys htop leaves free — most recently the memory-exhaustion forecast. It is a re-engineering of a known tool, not a claim to have invented the process table.

storageshower — a real-time disk-usage TUI in Rust

What it is, and what it ports

`storageshower` is a terminal user interface for watching disk usage in real time. It enumerates every mounted filesystem, draws one colored usage bar per mount, refreshes on an interval, and lets you drill down into any mount to see which directories are consuming the space. It runs on macOS and Linux, is written entirely in Rust on top of `ratatui` and `crossterm`, and stores every preference you change back to a TOML config file so the next launch looks exactly like the last one you closed. The crate manifest describes it as a “Cyberpunk disk usage TUI,” and the visual language throughout — neon palettes, [`SYSTEM://DISK_MATRIX`] chrome, threshold “NOMINAL / WARNING / CRITICAL” badges — is deliberate, but underneath the styling it is a disciplined system-monitoring tool.

The lineage matters for honesty about what was invented here. `storageshower` is not a port of an external Unix utility. It is not `ncdu`, not `du`, not `df`. It is the Rust rewrite of the author’s own shell scripts, which still ship in the repository under `old_scripts/`: `storageShower.sh` carries a 2018 header and is a thin wrapper that dispatches to `storageshowerCyberpunk.sh`, a ~31 KB bash implementation. The original engineering problem — enumerate mounts, size directories, render bars, theme the whole thing — was solved first in bash. The work in this crate is the reimplementation of that concept as a compiled, threaded, tested Rust binary: the architecture, the background collectors, the drill-down scanner, the theme system, and the test suite are the original contribution, not the idea of a disk monitor. Where it overlaps conceptually with `ncdu` (recursive directory sizing with keyboard navigation) or `duf/df` (per-mount usage bars), it arrives at that overlap independently from the bash predecessor rather than by porting either tool’s code.

One thing to be precise about up front, because it is easy to assume a disk explorer has it: `storageshower` does not delete files. The `ncdu` workflow of pressing a key to remove the directory under the cursor has no analog here. The only `Delete` key handling in the codebase is inside filter-edit mode, where it removes a character from the filter string (`src/keys.rs:30`). The disk operations it does offer are open-in-file-manager, copy-path-to-clipboard, export, and bookmark — read and navigate, never destroy. That restraint is a design stance, not an oversight: the tool answers “where did my disk go” and hands you the path, then lets you act on it in whatever file manager or shell you already trust.

Architecture

The crate builds a single binary and no library consumers are expected in practice, though `lib.rs` exposes most modules publicly so the integration tests can reach them — `app`, `cli`, `columns`, `helpers`, `prefs`, `system`, `types`, and `ui` are `pub`, while `keys` and `mouse` are private modules whose tests live inline (`src/lib.rs:3`). The source is about 13,700 lines across a dozen modules:

- `ui.rs` (~3,860 lines) — the rendering layer: the disk list, drill-down view, help overlay, theme chooser, theme editor, tooltips, borders, and footer are all drawn here.
- `keys.rs` (~1,930 lines) — keyboard event handling for every mode, plus the bulk of the unit tests that live alongside the code.
- `cli.rs` (~1,550 lines) — the `clap` derive struct, the override logic, and the hand-formatted `--help`, `--version`, `--list-colors`, and `--export-theme` output.

- `system.rs` (~1,510 lines) — all platform data collection: mounts, directory sizing, I/O counters, SMART status, network-filesystem latency, system stats.
- `app.rs` (~1,320 lines) — the central `App` state machine that ties input, preferences, and rendering together.
- `mouse.rs` (~920 lines) — click, drag, scroll, and hover handling.
- `prefs.rs` (~690 lines) — load/save of the TOML config.
- `helpers.rs` (~680 lines) — formatting (`format_bytes`, `format_uptime`, `format_latency`, `format_rate`, `truncate_mount`), color math, layout helpers.
- `types.rs` (~695 lines) — the data model: `DiskEntry`, `DirEntry`, `SysStats`, `SortMode`, `BarStyle`, `ColorMode`, `UnitMode`, `ThemeColors`, `ViewMode`, `DrillSortMode`, `SmartHealth`.
- `columns.rs` (~230 lines) — column-width computation for the resizable layout.

The runtime is built around two ideas: a single-threaded render/input loop, and background worker threads that feed it through shared, mutex-guarded state.

The main loop lives in `run_app` (`src/main.rs:58`). It computes the effective refresh interval from `app.prefs.refresh_rate`, calls `app.refresh_data()` when that interval elapses, then in a scoped block reads the terminal size, subtracts the chrome height (border, header, and a two-row footer), and calls `ensure_visible` so the selection stays on screen — with a parallel `ensure_drill_visible` when the drill view is open. It re-sorts through `update_sorted`, draws one frame via `terminal.draw`, then polls `crossterm` for input with a 200 ms timeout. Key presses go to `app.handle_key`, mouse events to `app.handle_mouse` (with the current terminal width and height), `FocusLost` clears the hover state to `Default::default()`, and `Resize` is a no-op because the next draw already re-measures. This is a conventional immediate-mode TUI loop; nothing about the disk or system data collection happens on this thread.

The four non-interactive flags short-circuit before any of this: `main` (`src/main.rs:20`) checks `cli.help`, `cli.version`, `cli.list_colors`, and `cli.export_theme` in turn and returns after printing, so `--help`, `--version`, `--list-colors`, and `--export-theme` never initialize the terminal, never enable mouse capture, and never spawn the collector. Only after those checks does `main` build the initial snapshot with `collect_sys_stats` and `collect_disk_entries`, wrap it in an `Arc<Mutex<(SysStats, Vec<DiskEntry>)>>`, spawn the collector, enable `EnableMouseCapture` and `EnableFocusChange`, and enter the loop — and it disables both and calls `ratatui::restore()` on the way out regardless of how the loop returned.

The heavy data collection runs on a detached thread spawned by `spawn_bg_collector` (`src/system.rs:895`). It owns a `System` from `sysinfo`, and every three seconds it refreshes system stats, re-enumerates disks, computes I/O rates against the previous snapshot, and layers SMART health onto each entry, then writes the whole `(SysStats, Vec<DiskEntry>)` tuple into the shared mutex that the UI reads from:

```
pub fn spawn_bg_collector(
    shared: Arc<Mutex<(SysStats, Vec<DiskEntry>)>>,
) {
    std::thread::spawn(move || {
        let mut sys = System::new_all();
        let mut prev_io = read_io_counters();
        let mut prev_time = Instant::now();
        loop {
            std::thread::sleep(Duration::from_secs(3));
            sys.refresh_all();
            let stats = collect_sys_stats(&sys);
            let mut entries = collect_disk_entries();
            // ... I/O rates + SMART status applied here ...
            let mut lock = shared.lock().unwrap();
            *lock = (stats, entries);
        }
    });
}
```

The three-second cadence of the collector is independent of the user-facing refresh rate (1s / 2s / 5s / 10s,

cycled with the `f` key), which only governs how often the UI re-reads the shared snapshot. That decoupling is deliberate: expensive work like SMART queries and I/O counter reads happens at a fixed low frequency regardless of how snappy the display feels.

Disk enumeration

`collect_disk_entries` (`src/system.rs:427`) is platform-split three ways. On macOS it calls `getmntinfo(&mut mntbuf, MNT_NOWAIT)` directly through `libc`, walks the returned `statfs` array, and computes `total = f_blocks * f_bsize`, `avail = f_bavail * f_bsize`, `used = total - avail` with a saturating subtraction. On Linux it parses `/proc/mounts`. On any other Unix it falls back to `sysinfo::Disks`. Every path skips rows with an empty mount path — the doc comment notes CI runners occasionally emit unusable mount-table rows, and dropping them keeps the TUI from ever showing a blank mount. Each entry carries mount path, used/total bytes, percentage, disk kind, filesystem type, and optional latency, I/O rates, and SMART status. Two committed tests pin the contract: `collect_disk_entries_have_mount_points` asserts no entry has an empty mount, and `collect_disk_entries_pct_in_range` asserts every percentage lands in `0..=100` (`src/system.rs:1094`, `src/system.rs:1102`).

Drill-down scanning

Pressing `Enter` on a mount switches `ViewMode` to `DrillDown` and kicks off a background scan through `start_drill_scan` (`src/app.rs:234`). It sets `drill.scanning`, clears the previous entries and selection, resets the shared progress counters to zero, then spawns a thread running `scan_directory_with_progress` (`src/system.rs:232`), which reads the target directory, records the total entry count and a running processed count into two `Arc<Mutex<usize>>` handles so the UI can draw a progress bar, and for each child computes a size. Files use their own length; directories are sized recursively by `dir_size` (`src/system.rs:200`), which walks the tree with `read_dir`, accumulates with `saturating_add`, and — importantly — does not recurse through symlinks. A symlink counts as its own `symlink_metadata().len()`, not the tree it points at, which prevents cycle-driven runaway. A dedicated test, `scan_directory_symlink_cycle_completes_with_finite_sizes`, pins that behavior. Results come back sorted largest-first.

The completed scan is picked up on the render thread, not delivered by callback. Each `refresh_data` tick begins by checking `drill.scanning`; if set, it `take()`s the `Option<Vec<DirEntry>>` out of the shared `scan_result` slot, and when a value is present it swaps it into `drill.entries`, clears `scanning`, and re-sorts (`src/app.rs:194`). Navigation inside the drill view is breadcrumb-style: `drill.path` is a `Vec<String>` stack, `Enter` on a directory pushes the child path and rescans, `Backspace/Esc` pops the stack and rescans the parent — and popping past the last element returns to the disk list and clears the entries. Descent into a file, or into anything while a scan is still in flight, is ignored (`src/keys.rs:249`). Within a level, `s` sorts by size (toggling reverse if already size-sorted), `n` sorts by name, `r` flips the direction, `g/Home` and `G/End` jump to the ends, and `o` opens the current directory in the file manager.

Telemetry, I/O, SMART, and latency

Beyond raw usage, each `DiskEntry` can be annotated with live per-mount read/write throughput, drive health, and network latency. I/O rates come from `read_io_counters` (`src/system.rs:706` on macOS, with distinct Linux and fallback variants further down), which on macOS reads IOKit's `IOBlockStorageDriver` byte counters and on Linux reads sector counts from `/proc/diskstats`, mapping devices to mounts via `mount_to_device_map`, then diffing two snapshots over elapsed wall-clock time in `compute_io_rates`. SMART status is collected in `collect_smart_status` — macOS shells out to `diskutil info` and reads the `SMART Status` field, Linux reads `/sys/block/*/device/state` — and cached per base device; each variant returns `SmartHealth::{Verified, Failing, Unknown}` and the “other-OS” variant returns everything as `Unknown`. Network-filesystem latency (`measure_mount_latency`, `src/system.rs:177`) times a `read_dir` on the mount in a spawned thread with a 2-second timeout so a hung NFS/SMB/SSHFS mount cannot freeze the UI, and requires no root. `is_network_fs` (`src/system.rs:160`) recognizes `nfs`, `nfs4`, `cifs`,

`smbfs`, `afp`, `ncp`, `fuse.sshfs`, `fuse.rclone`, `fuse.s3fs`, `9p`, and `afs`, and is pinned by `is_network_fs_detects_known_types`.

Alerts, pause, and hover

`refresh_data` (`src/app.rs:193`) does more than copy the snapshot: it is also where threshold alerts fire. After swapping in the fresh disks it walks them, collects the set of mounts at or above the warn threshold, and diffs that set against the mounts it flagged on the previous tick. Any newly crossed mount produces a status line like `⚠ ALERT: /Volumes/data 92%`, flashes the alert banner by stamping `alert.flash` with the current `Instant`, and emits a terminal bell (`print!("\x07")`). Mounts that stay above the threshold do not re-alert; the alert set is rebuilt every tick so a mount that drops back below and later climbs again will alert a second time.

The `p` key sets `paused`, and a paused app returns from `refresh_data` before reading the shared snapshot — but only after the drill-scan pickup, so an in-flight drill still completes even while the disk list is frozen. Hover behaves on a timer: `hover_ready` (`src/app.rs:296`) reports a tooltip as visible only once the pointer has rested for 1000 ms, and auto-hides it after 4000 ms unless the hover came from a right-click, which pins it open. Tooltips can be toggled off entirely with `T`, which also drops a confirming status message.

The CLI surface

`storageshower` has no subcommands. It is a single invocation configured entirely by flags, parsed with `clap`'s derive API in `src/cli.rs`. The governing rule is that CLI flags override the config file, and almost every boolean flag has a `--no-` inverse so you can force a setting in either direction from the command line regardless of what the saved config says. Internally this is done with `clap`'s `overrides_with`: each visible flag and its hidden inverse override one another, so the last one on the command line wins and only one boolean survives to `apply_to` (`src/cli.rs:383`). The polarity of which half is hidden follows the default — `--no-bars`, `--no-border`, `--no-header`, `--no-used`, `--no-tooltips`, `--no-virtual`, and `--no-local` are the visible members whose positive counterparts are hidden, while `--reverse`, `--compact`, and `--full-mount` are the visible positives.

The flag families, all grounded in the `clap` struct and the README option tables:

- **Sorting:** `-s/--sort <name|pct|size>`, `-R/--reverse` (with hidden `--no-reverse`), `-l/--local-only` (with `--no-local`), `--virtual/--no-virtual`.
- **Display:** `-b/--bar-style <gradient|solid|thin|ascii>`, `--color <palette>` (30 builtins), `--theme <name>` for a custom theme, `-u/--units <human|gib|mib|bytes>`, `-k/--compact`, `-f/--full-mount`, and `--bars/--border/--header/--used/--tooltips` toggles.
- **Thresholds:** `-w/--warn <pct>` and `-C/--crit <pct>`.
- **Columns:** `--col-mount`, `--col-bar-end`, `--col-pct`, each a `u16` width with `0` meaning auto.
- **System:** `-r/--refresh <secs>`, `-c/--config <path>`, `-h/--help`, `-V/--version`.
- **Introspection:** `--list-colors` prints every builtin palette with a preview; `--export-theme` prints the current palette as TOML and combines with `--color` or `--theme` to export a specific one.

Note the layering: `-f` on the command line is `--full-mount`, while the interactive `f` key cycles the refresh rate, and `-r/--refresh` sets it from the shell — different surfaces, no collision. Representative invocations:

```
storageshower --color purple -b ascii # purple palette, ascii bars
storageshower -s pct -R                # sort by usage%, reversed
storageshower -l --no-virtual          # local physical disks only
storageshower -u gib -w 60 -C 85       # GiB units, custom limits
storageshower --list-colors            # preview builtin palettes
storageshower --export-theme --color blue # dump blue as TOML
```

Keys inside the TUI

Inside the interface the same knobs are reachable from the keyboard, and the handler in `src/keys.rs` is a mode stack: filter-edit, help, theme-chooser, theme-editor, and drill-down each capture input first, and only when none is active does the disk-list keymap run. On the disk list, `j/k` (and arrows) move the selection, `g/G` and `Home/End` jump to the ends, and `Ctrl+d/Ctrl+u` half-page while `Ctrl+g` returns to the top. Sorting is `n` (name), `u` (usage percent), and `s` (size); pressing the active sort key again flips direction, and `r` flips direction directly. Toggles are dense: `p` pauses data refresh, `l` local-only, `a` show-all, `v` bars, `d` used-vs-total, `g` header, `x` border, `m` compact, `w` full-mount, `T` tooltips. `b` cycles bar style through gradient → solid → thin → ascii, `i` cycles units through human → gib → mib → bytes, and `f` cycles the refresh rate 1 → 2 → 5 → 10 seconds.

Thresholds cycle through fixed steps rather than free entry: `t` walks the warn level 50 → 60 → 70 → 80 and wraps, and `z` walks the crit level 80 → 85 → 90 → 95 and wraps (`src/keys.rs:520`, `src/keys.rs:544`). The `/` key enters filter mode; `0` clears the filter without entering it. Filter mode is a full readline-style line editor over a `String` buffer with a cursor: `Ctrl+w` word-delete, `Ctrl+u` kill-to-start, `Ctrl+k` kill-to-end, `Ctrl+a/Home` and `Ctrl+e/End` for line motion, `Ctrl+b/Ctrl+f` and arrows for character motion, `Ctrl+h/Backspace` and `Delete` for erase, `Enter` to commit and `Esc` to restore the previous text (`src/keys.rs:14`).

The action keys operate on the selected mount. `o` opens it in the file manager (`open` on macOS, `xdg-open` on Linux, suppressed in `test_mode`), `y` copies the mount path to the clipboard, and `B` toggles a bookmark, pushing or removing the mount in `prefs.bookmarks` with a ★-prefixed status line. `e` writes a plain-text report of the whole disk matrix — host, timestamp, and a fixed-width `MOUNT / PCT / USED / TOTAL` table formatted through `format_bytes` — to `~/.storageshower.export.txt` (`src/keys.rs:621`). Mouse input is a first-class parallel path in `mouse.rs`: click to select, click again to drill, drag the column separators to resize, right-click for a pinned verbose tooltip, scroll to move the selection.

Themes and configuration

The theme system is the most developed non-monitoring feature. There are 30 builtin palettes defined as the `ColorMode` enum in `src/types.rs:31`, each with an internal name and a display name (`ColorMode::Default` renders as “Neon Sprawl,” `ColorMode::Green` as “Acid Rain,” and so on through “Zaibatsu”). A `ColorMode::ALL` slice, a `name()` map, and a `next()` method that indexes into `ALL` and wraps modulo its length drive in-app cycling, and tests assert the display names are all unique and that `next()` wraps cleanly through the full ring. On top of the builtins, users define custom themes as six-channel color sets (`ThemeColors: blue, green, purple, light_purple, royal, dark_purple`) either by hand in the config or through the live in-app editor bound to `C`:

```
[custom_themes.neonpink]
blue      = 199
green     = 46
purple    = 201
light_purple = 213
royal     = 196
dark_purple = 161

active_theme = "neonpink"
```

The theme editor is a six-slot picker over the 256-color palette. Opening it with `C` seeds all six slots from the current effective palette via `palette_for_prefs`, mapping each `Color::Indexed(n)` back to its index. `j/k` move between slots, `h/l` (and arrows) nudge the selected channel by ± 1 with `wrapping_add/wrapping_sub`, and `H/L` nudge by ± 10 — so a full sweep of the palette is a few keystrokes. `Enter` or `s` switches into naming mode, where the new theme is written into the custom-theme map under whatever name you type. The theme chooser bound to `c` is separate: it lists every builtin and custom theme, applies each as a live preview as you move through it with `j/k`, and restores the original on `Esc/q` — so browsing 30-plus palettes never commits an accidental change.

Every preference the app tracks — sort mode and direction, refresh rate, bar style, color mode, warn/crit thresholds, the visibility toggles, unit mode, column widths, the custom-theme map, the active theme, and the bookmark list — is serialized to `~/.storageshower.conf` in TOML via `serde` and restored on boot (`src/prefs.rs`). There is no explicit “save” step; the disk-list toggles call `self.save()` inline, so changing a setting persists it immediately. The first ten builtin palettes are also checked into `themes/` as ready-to-copy TOML files (`acid_rain.toml`, `data_stream.toml`, `ghost_wire.toml`, `ice_breaker.toml`, `neon_sprawl.toml`, and five more).

Formatting and units

All human-facing sizes go through `format_bytes` (`src/helpers.rs:3`), which is binary throughout: `Human` mode steps K/M/G/T at 1024, 1 048 576, 1 073 741 824, and 1 099 511 627 776 bytes with one decimal, while `GiB/MiB` force a single scale and `Bytes` prints the raw count with a `B` suffix. Rates use the same binary ladder (`format_rate`, B/s → K/s → M/s → G/s), latency reads as `<1ms`, `NNms`, or `N.Ns` past a second (`format_latency`), uptime collapses to `NdNhNm/NhNm/Nm` (`format_uptime`), and long mount paths are ellipsized with a trailing `...` by `truncate_mount`. Each of these has a cluster of unit tests immediately below it in the same module, which is where a large share of the crate’s inline test count comes from.

The RECLAIM_MAP overlay (in development)

The working tree carries an opt-in feature, `reclaim`, that is not part of the 0.28.8 release and is gated behind a Cargo feature so the default build pulls in zero extra dependencies. It adds a per-subtree estimate of how much space a directory could give back if compressed. The implementation lives in a `#[cfg(feature = "reclaim")] pub mod reclaim` inside `system.rs` (`src/system.rs:284`) and is honest about its method: it never compresses whole files. For each file it reads a bounded 64 KiB prefix (`SAMPLE_BYTES`), runs a single `lz4_flex` block over that sample to get a compression ratio, and projects `size - size/ratio` as the reclaimable bytes; a `shannon_entropy` helper is provided as a fast compressibility proxy. Directory totals fan out over `rayon` (`dir_reclaim` uses `par_iter().reduce`), symlinks are not followed to match the size scanner’s recursion policy, and unreadable files are treated as incompressible (ratio 1.0, zero reclaimable). When enabled, `scan_directory_reclaim` runs the ordinary sized scan and then `annotates` each `DirEntry`’s new `reclaimable` and `ratio` fields; the drill-down `c` key toggles the overlay, switches the drill sort to `DrillSortMode::Reclaim`, and rescans the current subtree so the estimate is populated. Two dependencies back it — `rayon 1.12` and `lz4_flex 0.13`, both marked `optional` in `Cargo.toml`. Treat this as active work: the feature is wired end to end behind the flag, but it is unreleased and the default (non-`reclaim`) build in the current working tree does not yet cover the new `DrillSortMode::Reclaim` variant in every match arm, so the shipped 0.28.8 remains the reference behavior.

Integration into the stack

`storageshower` installs as a standalone command, via the project’s Homebrew tap (`brew install storageshower`) or `cargo install storageshower`. It ships a `zsh` completion (`completions/_storageshower`, a `#compdef`-headed `_arguments`-style file) and two man pages: `storageshower.1`, the standard reference, and `storageshowerall.1`, an all-in-one page modeled on `zshall(1)` that concatenates invocation, keybinds, themes, drill-down, config, and alerts into a single greppable document. Within the wider environment it is a terminal-native monitoring surface — it reads mounts and directory sizes, opens paths in the file manager, and copies paths to the clipboard, so it slots in as the “where did my disk go” tool alongside the rest of the CLI suite rather than owning any part of the shell runtime.

Status

The version in the manifest and man pages is **0.28.8**, edition 2024 (`Cargo.toml`, `man/man1/storageshower.1` dated 2026-07-05), which implies a Rust $\geq$ 1.85 toolchain since edition 2024 stabilized there. Dependencies are deliberately foundational: `ratatui` 0.30, `crossterm` 0.29, `sysinfo` 0.38, `dirs` 6, `clap` 4, `serde` 1, `toml` 1.1, `libc` 0.2, with `rayon/lz4_flex` optional behind the unreleased `reclaim` feature. Dev-dependencies are `criterion` 0.8, `serde_json`, `tempfile`, and `toml`. The release profile is `opt-level = 3`, `lto = true`, `strip = true`.

The test surface is large and I verified the counts in this session: 1,336 integration test files under `tests/`, each a separate `*.rs` integration binary, carrying 3,101 `#[test]` functions between them (a `grep` count), plus 596 unit tests embedded in `src/` modules (also a `grep` count) — about 3,697 test functions in total, though I did not compile and run the full suite so the passing count is unverified. Criterion benchmarks live in `benches/bench.rs`; the README reports formatting operations in the tens of nanoseconds and full disk-list rendering for 200 mounts around 15.7 μ s on Apple Silicon, all **provisional** as I did not re-run the benchmarks. CI is two workflows: `ci.yml` runs library unit tests, integration tests, and doc tests (each `cargo test --locked` separately), plus `cargo fmt --all --check` and a `clippy` job, on an `ubuntu-latest` + `macos-latest` matrix; `release.yml` builds `--release --locked` across macOS and Linux targets, using `cross` for the cross-compiled aarch64 Linux build.

zcolorizer — the log stream, repainted in real time

What it is, and what it ports

`zcolorizer` is a streaming log colorizer. You pipe a log into it — from `tail -f`, from `journalctl -f`, from a file argument, from anything that produces text on a file descriptor — and it emits the same text back with ANSI escapes inserted so that dates, hostnames, IP addresses, error keywords, HTTP codes and dozens of other semantic fields each carry a color. The default look is a 256-color cyberpunk palette; everything about which text gets colored and what color it gets is user-editable through a TOML config with no recompile.

It is honest about its lineage. The `Cargo.toml` description reads “Real-time log file colorizer (ccze/pygments port)”, and the crate ships a full copy of the original C source under `vendor/ccze/` as a build-time reference. The upstream tool is `ccze` by cornet — a curses-based log colorizer from the early 2000s whose architecture is a set of format *plugins* (`mod_syslog.c`, `mod_httpd.c`, `mod_squid.c`, and so on), each of which matches whole-line PCRE patterns, paints the numbered capture groups with semantic colors, and hands the free-text remainder to a generic “word colorizer.” `zcolorizer` ports that architecture faithfully — the module set, the module-then-wordcolor precedence, the semantic token vocabulary — into Rust. The second influence, credited in the same breath, is the `pygments` “regex → token” idea: a lexer rule is a regex, and the thing it produces is a named token that a separate theme decides how to render.

The invention here is not the idea of colorizing logs; `ccze` did that. The engineering is the Rust rewrite: a three-layer decoupled architecture (rules / themes / engine), a streaming binary that is safe on binary and pre-colored input, a 47-module format library that extends `ccze`’s ~20 plugins into formats that did not exist in 2003 (systemd, JSON, Kubernetes, Postgres), a themeable color model that carries 32 palettes, a `--themes-json` surface built so a GUI theme-picker can drive the whole thing, and — the one piece with no `ccze` ancestor — a streaming statistical-novelty coloring mode that decides brightness from how often a line’s *shape* has been seen rather than from any rule. The vendored upstream C source — `ccze`, kept only as a parity reference — is explicitly excluded from the published crate (`exclude = ["/vendor", ...]` in `Cargo.toml`); the Rust is original work, MIT-licensed.

The three decoupled pieces

The design principle stated at the top of `src/lib.rs` is that three things are held apart so any one can change without disturbing the others:

- **Rules** (`src/rules.rs`) decide *which* span of text earns a semantic token — `date`, `error`, `host`, `ip`, and so on. A rule is a regex. Its **named capture groups are the tokens**: the pattern `(?P<date>\d{4}-\d\d-\d\d)\s+(?P<host>\S+)` paints the date field and the host field from a single expression, each in its own color. A rule with no named groups tags its whole match with one `token`. This is exactly the `pygments` model, and it is what lets rules live in the config as editable text.
- **Themes** (`src/theme.rs`) decide *what each token looks like*: a named map from token string to a `Style` (foreground color plus optional background and bold/dim/italic/underline/blink/reverse). Because color lives entirely in the theme, swapping the active theme recolors every log without touching a single rule.
- **Engine** (`src/engine.rs`) runs the compiled rules over each line and paints the spans they claim with the active theme.

`Colorizer::from_config` in `lib.rs` ties them together: it resolves the effective rule definitions from the config, compiles them, resolves the theme, and returns a ready `Colorizer`. A second constructor, `Colorizer::default_cyberpunk`, hands back the builtin generic rules plus the neon-sprawl theme with no config file at all, and it *expects* success on the grounds that the builtin rules and theme are always valid — the fast path a Rust host embedding the crate reaches for.

The matching engine and its ownership model

The core of `src/engine.rs` is `colorize_line`, and its central data structure is a byte-owner map. For a line of length `len`, it allocates `owner = vec![usize::MAX; len]` — one slot per byte, where `usize::MAX` means “unclaimed.” Rules run in listed order. Each rule’s matches call `claim(start, end, token)`, which walks `owner[start..end]` and sets any slot still equal to `usize::MAX` to that token’s interned id — but never overwrites a slot an earlier rule already claimed. The rendering pass then walks the owner map, groups maximal runs of equal owner, and emits one styled chunk per run; unclaimed runs get the theme’s `default` style.

This is the whole precedence story: **the first rule to claim a byte owns it**. A structured syslog-prefix rule listed before the generic bare-number rule will paint `192.168.0.1` as one `ip` span rather than letting the number rule chop out the octets — verified by the engine test `earlier_rule_wins_overlap`, which asserts the address renders as a single `38;5;135` chunk with exactly one reset. It is the same module-then-wordcolor ordering ccze used, expressed as span ownership rather than as a plugin dispatch chain.

The owner-building work is factored out of the render path into `build_owner`, a private method that runs every rule over a sanitized, non-empty line and returns `(Vec<usize>, Vec<Bstr>)` — the byte-to-token-index map plus the interned token names, all borrowing from `self.rules` under one lifetime. `colorize_line` calls `build_owner` and then `render`; the novelty path calls the same `build_owner` and then a sibling `render_intensity`. The two rendering functions are byte-for-byte the same run-grouping loop; they differ only in whether each chunk’s `Style` is first passed through `with_intensity`. Factoring the owner pass out this way is what keeps the novelty mode from being a parallel reimplement of the matcher — it reuses the exact span logic that the plain path is tested against.

Before any rule runs, every line passes through `sanitize`. Piped logs routinely carry pre-existing terminal escapes (a `tail` of an already-colored file) and raw binary bytes. If the colorizer ran a number rule over the digits inside an existing `\x1b[31m` sequence, it would inject its own SGR codes mid-escape and shred it into visible garbage. So `sanitize` strips full ANSI/VT escape sequences (CSI, OSC terminated by BEL or ST, and two-byte escapes) and every C0 control byte except tab, plus DEL and the C1 range. Tabs survive; the common case — a clean line — is returned borrowed, unchanged, with no allocation. The engine’s test suite pins all of this: OSC-8 hyperlink stripping, binary-byte dropping, a pure-control line collapsing to empty, and multibyte UTF-8 words surviving span slicing next to digit runs (span ends always land on regex match offsets, which are always char boundaries).

The color model

`src/color.rs` defines a `Color` as one of three representations, distinguished at parse time by an untagged serde enum: one of the 16 **named** ANSI colors (`"red"`, `"bright_cyan"`), a 256-color palette **index** written `{ index = 213 }`, or a 24-bit **truecolor** RGB triple written either as `"#ff00aa"` or `{ rgb = [255, 0, 170] }`. A `Style` bundles a foreground color, an optional background, and six boolean attributes. Its `prefix()` method emits the SGR opening sequence in a fixed canonical order — attributes (`1;2;3;4;5;7` for bold/dim/italic/underline/blink/reverse) then foreground then background — and `paint()` wraps text in that prefix plus a reset, returning the text verbatim when the style is plain so a colorless token emits no escape at all. Named backgrounds are foreground code + 10; indexed foregrounds use `38;5;N`, backgrounds `48;5;N`; truecolor uses `38;2;r;g;b`.

The intensity model

`Style::with_intensity(t)` is the color model's one novelty-aware method, and it is deliberately narrow: it takes a novelty score `t` in `[0, 1]` and adjusts *terminal attributes only*, never touching the foreground or background color. A token keeps its palette hue at every intensity; only its weight changes. The mapping is a clamped step function:

- `t >= 0.9` — first-seen or anomalous: set `bold` and `reverse`, clear `dim` (the line blazes in inverse video).
- `t >= 0.4` — rare: set `bold`, clear `dim`.
- `t < 0.15` — high-frequency noise: set `dim`, clear `bold` and `reverse`.
- otherwise (`0.15 <= t < 0.4`) — the style is returned unchanged.

Because the input is clamped to `[0, 1]` first, an out-of-range score degrades to one of the endpoints rather than misbehaving. Keeping color fixed and moving only weight is what makes the mode legible: a `journalctl` stream stays recognizably neon-sprawl-colored, but the eye is pulled to the bold-reverse lines and slides past the dimmed ones. This is separable from everything else — the theme, the rules, the modules all work identically; intensity is a post-filter applied to whatever `Style` a token already resolved to.

The novelty model

`src/novelty.rs` is the streaming statistics behind that intensity. `NoveltyModel` is an online frequency table: a `HashMap<u64, u32>` from a line-template hash to how many times that template has been seen, plus a running `total` and an optional decay factor. Its one hot method, `observe_and_score(hash)`, records a sighting and returns the template's novelty score in `[0, 1]`. The score is `1.0` on first sight, and thereafter `1 / (k + 1)` where `k` is the number of prior sightings — so a template's score falls monotonically `1.0, 0.5, 0.33, 0.25, ...` as it recurs. Feed that score straight into `with_intensity` and the behavior falls out for free: a first-seen line blazes, the first repeat is still bold, and once a template has been seen seven or more times its score has dropped below `0.15` and it dims into the background. The model's tests pin exactly this: `first_seen_scores_high_repeats_decay_toward_dim` asserts the score strictly decreases on every repeat and lands below `0.15` within a dozen sightings, and `distinct_templates_tracked_independently` asserts that hammering one template to noise does not bleed into an unrelated template's first-seen `1.0`.

The trick that makes this work on real logs is *templating*, and it lives in the engine as `template_hash`. Scoring the raw line would be useless — every line of a busy stream is textually unique because timestamps and IDs change — so before hashing, the engine masks out the spans whose *value* is a variable field. `VARIABLE_TOKENS` in `engine.rs` names the eight token classes treated as variable: `number`, `ip`, `address`, `pid`, `size`, `version`, `date`, and `time`. `template_hash` walks the same owner map the renderer uses; for a run owned by one of those variable classes it hashes a sentinel byte plus the *class name* instead of the literal bytes, and for everything else — literal text and non-variable tokens like `error` or `host` — it hashes the bytes verbatim. Two syslog lines that differ only in their timestamp, PID, and source IP therefore collapse to one template hash, while a line whose message text changes produces a new one. The class name is hashed rather than a single universal mask, so an `ip` span and a `number` span in the same position stay distinguishable — swapping a bare port number for an IP is a real structural change and reads as novel.

Decay is optional. `NoveltyModel::with_decay(factor)` multiplies every count by `factor` (clamped to `[0, 1]`) once every `DECAY_INTERVAL` (512) observations, dropping any template whose count rounds to zero. Batching the sweep every 512 observations keeps its `O(distinct_templates)` cost off the per-line hot path. The effect is a sliding window of attention: a template that was noise but has gone silent is eventually forgotten, so if it returns it re-lights as first-seen. The `decay_forgets_a_silenced_pattern` test hammers a template to 30 sightings, flows four decay-intervals of unrelated traffic past it, and asserts its next appearance scores `1.0` again. A `factor >= 1.0` is normalized to `None` (pure cumulative counting), so `with_decay(1.0)` is exactly `new()`.

Nothing in this mode requires a rule that knows the log format. The masking is driven entirely by which builtin

tokens the generic ruleset already assigns, so a `tail -f` of an unknown, repetitive service self-quiets while an anomalous line — a stack trace, a `SEGFAULT`, an unexpected message — surfaces in bold reverse, with no configuration at all. This is the piece of `zcolorizer` with no `ccze` lineage; `ccze` colors by pattern class only, and the statistical-intensity layer is original.

The color model in the CLI

`--novelty` turns the mode on; `--novelty-decay D` sets the decay factor and *implies* `--novelty`. The flag plumbing is in `build_novelty`: given `D`, a value in `(0, 1]` builds a decaying model, a value outside that range is rejected with `--novelty-decay: must be in (0, 1], got ...`, and with no `D` the presence of `--novelty` alone builds a non-decaying `NoveltyModel::new()`. The model lives on the `LineProcessor` and persists across `--watch` config reloads — it tracks the *stream*, not the theme, so swapping palettes mid-run must not reset the frequencies it has learned. A worked run through the default palette, with the variable fields masked so the four connection lines collapse to one template:

```
$ printf 'Connection from 10.0.0.1 port 22\n\
Connection from 10.0.0.2 port 22\n\
Connection from 10.0.0.3 port 22\n\
Connection from 10.0.0.4 port 22\n\
SEGFAULT in worker pid 991\n' | zcolorizer -C --novelty
<1;7>Connection ... 10.0.0.1 ... port 22      # first-seen: bold+reverse
<1>  Connection ... 10.0.0.2 ... port 22      # k=1, score .5:  bold
      Connection ... 10.0.0.3 ... port 22      # k=2, score .33: plain
      Connection ... 10.0.0.4 ... port 22      # k=3, score .25: plain
<1;7>SEGFAULT in worker pid 991              # new template:  blaze
```

(Attribute prefixes shown as `<...>`; hues omitted.) The four `Connection` lines differ only in their IP and are collapsed by the `ip/number` mask into one template, so their scores march `1.0 → 0.5 → 0.33 → 0.25` and their weight drops from bold-reverse to plain across four lines. The `SEGFAULT` line is a template never seen before and blazes even though it carries no `error` keyword the ruleset recognizes.

The theme grammar in depth

A theme is a *name*, a *description*, a *base* style (the fallback for any token not otherwise mapped), and a *styles* map keyed by token name. Token names are plain strings, so a config can invent new tokens freely; the well-known ones the builtin rules emit are enumerated in `theme::tokens` — a large const module covering the generic vocabulary (`date`, `time`, `host`, `ip`, `mac`, `pid`, `process`, `error`, `warning`, `debug`, `info`, `good`, `bad`, `system`, `keyword`, `email`, `uri`, `dir`, `file`, `size`, `version`, `number`, `address`, `percentage`, `signal`, `http_method`, `http_code`, `string`, and more), the `ccze`-plugin tokens (`gettime`, `getsize`, `ftp_code`, `ident`, `ctype`, `subject`, `field`, `chain`, `pkg`, `pkgstatus`, `incoming/outgoing/unique` for exim, `repeat`, `swapnum`, and the eleven `proxy_*` squid actions), and the modern-format tokens (`thread`, `level`, `facility`, `duration`, `latency`, `hash`, `json_key`).

The 32 themes are not hand-written token-by-token. Thirty-one of them are generated from *palettes* ported verbatim from the shared MenkeTechnologies theme set (`iftoprs`, itself derived from `storageshower`). A *Palette* is a kebab id, a display name, and six 256-color indices [`c1..c6`] standing for primary, brightest accent, secondary, mid, dim, and darkest. The declaration order matters: `neon-sprawl` is first and is the default (aliased `cyberpunk`), followed by `acid-rain`, `ice-breaker`, `synth-wave`, `rust-belt`, `ghost-wire`, `red-sector`, `sakura-den`, `data-stream`, `solar-flare`, `neon-noir`, `chrome-heart`, `blade-runner`, `void-walker`, `toxic-waste`, `cyber-frost`, `plasma-core`, `steel-nerve`, `dark-signal`, `glitch-pop`, `holo-shift`, `night-city`, `deep-net`, `laser-grid`, `quantum-flux`, `bio-hazard`, `darkwave`, `overlock`, `megacorp`, `zaibatsu`, and `iftopcolor`.

`theme_from_palette` is where six colors become a full styling for roughly sixty tokens. It assigns each token to a palette role and multiplies the six colors into enough distinct looks by layering bold, italic, and underline:

`date` is bold-accent, `time` plain-accent, `host` bold-secondary, `ip` plain-secondary, `uri` underlined-secondary, severity tokens lean on the brightest accent so `error` stays loud in every palette. Where two tokens would otherwise collide on the same palette slot — `process` versus `host` — the code derives an extra distinguishable shade with `shift_lighter`, an index-cube arithmetic function ported from iftoprs that steps a 256-color index one notch brighter (clamping at cube and grayscale-ramp edges). The theme tests assert every palette builds, that `error` is bold in all 31, that `process` and `host` never render identically, and that unmapped tokens fall through to `base`.

The thirty-second theme is `ccze-classic`, a hand-written 16-color map that approximates the original ccze palette (cyan dates, bold-red errors, bold-yellow warnings) for parity and nostalgia. `builtin_s()` returns the 31 generated palettes chained with `ccze-classic` — 32 in total, confirmed at runtime by `--list-themes | wc -l`. Name resolution is forgiving: `normalize` lowercases, collapses spaces and underscores to hyphens (so "Neon Sprawl", `neon_sprawl`, and `NEON-SPRAWL` all match), and treats `cyberpunk` and `default` as aliases for `neon-sprawl`.

Themes compose. A `[[themes]]` block naming an existing builtin overlays only the tokens it sets via `merged_with`, keeping the rest; a block with a fresh name defines a whole theme (set `base` so unlisted tokens still have a color). That is how the config example makes errors louder without redefining the palette:

```
[[themes]]
name = "cyberpunk"           # overlay the builtin
[themes.styles.error]
fg = "#ff003c"
bold = true
underline = true

[themes.styles.pid]
fg = "#b14bff"               # tone PIDs down to a muted violet
```

The rule library

`builtin_generic()` in `rules.rs` is the port of ccze's word-colorizer plus a handful of structured-line rules, and it is what runs on any log out of the box. It leads with named-group timestamp rules (ISO 8601, syslog `Mon DD HH:MM:SS`, and `proc(pid):`), then severity word-lists compiled case-insensitively (`ERROR|ERR|CRITICAL|FATAL|PANIC...` → `error`; `WARNING|WARN` → `warning`; and so on), ccze's `good/bad/system` word lists, HTTP methods and status codes, network identifiers (IPv4, IPv6, MAC, email, URI, hostname), absolute paths, hex addresses, sizes, versions, percentages, signal names, bare numbers, and double-quoted strings. Each `RuleDef` carries a name, a pattern, an optional whole-match token, and an `ignore_case` flag that compiles as an inline `(?i)`. The severity, `good`, `bad`, and `signal` word lists double as the semantic anchors the novelty mode leans on: because they are *not* in `VARIABLE_TOKENS`, a line's error and signal keywords survive templating verbatim, so a line that gains a `SEGFAULT` or a `panic` reads as a structurally new template even when its numbers and timestamps are masked away.

Format modules

The generic ruleset colors anything; a `module` colors a known format's structured fields precisely. A `Module` is a named bundle of `RuleDefs` whose start-anchored named-group patterns pin the shape of a whole line to a format. There are 47 modules (confirmed by `--list-modules | wc -l`), in two families. The classic ccze plugin ports live in `src/modules.rs` — 20 of them: `syslog`, `httpd`, `squid`, `dpkg`, `postfix`, `exim`, `procmail`, `proftpd`, `vsftpd`, `ftpstats`, `xferlog`, `php`, `oops`, `icecast`, `fetchmail`, `apm`, `distcc`, `sulog`, `super`, `ulogd`, each translated from its `vendor/ccze/src/mod_*.c` original with the C source as the spec. The 27 modern modules in `src/modules_modern.rs` cover formats ccze never saw: `json`, `logfmt`, `journald`, `authlog`, `kernel`, `cron`, `auditd`, `nginx`, `haproxy`, `caddy`, `traefik`, `postgres`, `mysql`, `redis`, `mongodb`, `klog`, `docker`, `spring`, `python`, `rust-log`, `rails`, `named`, `dnsmasq`, `dhcpcd`, `fail2ban`, `ufw`, and

dovecot.

Modules layer on top of the generic rules and win on overlap, but a free-text message still flows through the generic word-colorizer — ccze's exact module-then-wordcolor design, expressed here as the module rules sitting between user rules and the generic builtins in `resolve_rule_defs`. Enabling every module with `-m all` is safe because each module's structured patterns are `^`-anchored, so they only fire on lines that genuinely match that format. Where ccze chose a color dynamically by substring — squid's `TCP_HIT/TCP_MISS`, an HTTP method — the port emits small follow-up whole-match rules for each variant token, because the Rust regex crate forbids duplicate group names in one pattern (the `apm` module is split into single-field rules for the same reason, noted in its source comment).

The CLI surface and how it feeds

With no file arguments the binary reads stdin, line-buffered, so `tail -f` and `journalctl -f` stream live; with file arguments it colorizes each in turn. Color auto-disables when stdout is not a terminal unless `-C/--force-color` is given (the path for piping into `less -R`), and `--no-color` forces passthrough. The real invocations:

```
tail -f /var/log/syslog | zcolorizer
journalctl -f | zcolorizer --theme cyberpunk
zcolorizer -m httpd access.log | less -R
tail -f app.log | zcolorizer -g '(?i)error' # color, no
tail -f app.log | zcolorizer -H ERROR # dim non-matches
tail -f /var/log/syslog | zcolorizer -m auto --watch
tail -f app.log | zcolorizer --novelty # noise self-dims
tail -f app.log | zcolorizer --novelty-decay 0.9
```

`-g/--grep` filters to matching lines while keeping color, unlike `| grep`, which strips it; `-H/--highlight` keeps every line but renders the non-matches dim so matches stand out in context. Grep and highlight are compiled with `compile_filter`, which surfaces a bad regex tagged with the offending flag name rather than a bare regex-crate error. `-m/--module` is repeatable and merges with any `modules` in the config. The `auto` pseudo-module samples the first 200 lines (`SNIFF_LINES`), fingerprints the format from each module's start-anchored signature, and enables only what is present — no input is dropped, because the sniffed sample is buffered and replayed. Formats identifiable only mid-line (JSON, logfmt) have no anchored signature and are deliberately not auto-enabled; you name them with `-m json`. `-w/--watch` re-stats the config on a 200 ms throttle and rebuilds rules and theme mid-stream when the mtime changes, keeping the previous good config if a half-saved edit fails to parse — the live half of the “pick the theme from zgui” flow. When novelty is active, the reload preserves the running `NoveltyModel`, because the frequencies belong to the stream and not to the config being reloaded.

Running the syslog example through auto-detection produces the paint directly:

```
$ printf 'Jun 27 14:03:11 host sshd[1234]: Failed \
password for invalid user admin from 10.0.0.5 \
port 22\n' | zcolorizer -C -m auto
zcolorizer: -m auto detected: syslog, journald, authlog
<1;38;5;48>Jun 27 14:03:11 <1;38;5;135>host
  <1;38;5;177>sshd [<1;38;5;141>1234]:
    <1;38;5;48>Failed password for invalid user
    <1;38;5;141>admin from <38;5;135>10.0.0.5
    port <38;5;141>22
```

(Escape codes shown as `<...>` for legibility.) The date, host, process, pid, the `Failed password` bad-phrase, the user, the IP, and the trailing port number each land in their neon-sprawl token color.

Several flags exist purely to feed tooling and introspection. `--list-themes` marks the active theme with `*`; `--list-modules` prints each module and its description; `--list-rules` prints the effective ruleset after the config merge; `--dump-config` re-emits the resolved config as normalized TOML; and `--themes-json`

resolves every available theme to a full struct and emits `{ "active": ..., "themes": [ ... ] }` so a GUI picker can render real swatches from each token's style. The `-h/--help` output is the house-style cyberpunk banner (an ANSI-Shadow wordmark with a cyan-to-magenta truecolor gradient, a status box, and green `//` descriptions) rendered by hand rather than by clap — clap's own help is disabled with `disable_help_flag = true` and `-h/--help` is captured as an explicit `help` field so the hand-drawn banner runs instead. That banner carries its own — NOVELTY — section documenting `--novelty` (“intensity = statistical novelty; noise self-dims”) and `--novelty-decay D` (“age counts (0<D<=1) so silenced patterns relight”).

Integration and stack shape

The crate is built to be embedded, not only invoked. `Cargo.toml` declares `crate-type = ["rlib", "staticlib", "cdylib"]` — an rlib for native use from Tauri hosts and sibling Rust crates, plus staticlib/cdylib for a future C ABI. The library never depends on clap; the CLI binary is gated behind a `cli` feature (`default = ["cli"]`, `cli = ["dep:clap"]`), so a host linking the rlib pulls in none of the argument-parsing machinery. Dependencies are deliberately foundational and vendorable, per the meta durable-deps rule: `serde`, `serde_json`, `toml`, `regex`, `thiserror`, `dirs`, `once_cell`, and `clap` for the binary only. The release profile sets `lto = true` and `strip = true`. The default config path is `$XDG_CONFIG_HOME/zcolorizer/config.toml`, resolved through `dirs`. Together with `--themes-json`, `--dump-config`, and `--watch`, that shapes the intended workflow: a GUI enumerates themes from the JSON, writes the chosen one into the config, and the running `zcolorizer` picks it up live without a restart. The whole `NoveltyModel` type is re-exported from the crate root alongside `Colorizer`, `Color`, `Style`, `Config`, and `Theme`, so an embedding host can drive the statistical-intensity path directly without going through the CLI.

Status

Provisional, from the working tree: `Cargo.toml` version is `0.2.2` (recent commits past the `v0.2.2` tag touch the docs/HUD color schemes and the help footer, not the crate version). `cargo test` passes 94 unit tests, 17 integration tests, and 1 doc-test — 112 total, all green (the unit count grew from the earlier 91 as the `novelty` module's three tests and related engine coverage landed). Runtime enumeration confirms 47 format modules and 32 themes. The README's “31 cyberpunk palettes” and the help line's “31 cyberpunk themes” both count the generated palettes; the 32nd builtin is `ccze-classic`. Working today: the streaming CLI, the regex → token engine with span-ownership precedence and input sanitization, TOML config with custom rules and theme overrides, truecolor/256/named color, all 32 themes, all 47 modules, `-m auto` detection, `--watch` live reload, `--themes-json` for tooling, and the streaming novelty mode (`--novelty` / `--novelty-decay`) with per-template intensity, field-masked templating, and optional exponential decay that persists across watch reloads.

grcrs — a two-binary Rust port of the Generic Colouriser

`grcrs` is a faithful Rust port of `grc`, the Generic Colouriser (upstream version 1.13, by Radovan Garabík). The upstream tool is a pair of Python programs that colourise the output of arbitrary commands by matching lines against regexp/colour rules. `grcrs` reproduces that pair as two compiled binaries — `grc` (the launcher) and `grcat` (the colouriser) — keeping the same option surface, the same config-file search order, the same colourfile grammar, and the same line-carried streaming-colour behaviour. The port declares version `1.13.0` in `Cargo.toml`, aligning its version number with the upstream release it tracks.

The engineering here is not the invention of the colourising model — that model is `grc`'s, and the bundled `grc.conf` plus the 83 colourfiles under `vendor/grc/colourfiles/` are the upstream data files carried verbatim. The engineering is the Rust reimplementation of both programs: reproducing Python's `re` regexp semantics on top of `fancy-regex`, reproducing Python's `surrogateescape` byte handling so non-UTF-8 command output survives round-trip, reproducing `getopt`'s abbreviated-long-option matching, and building an end-to-end test suite whose expectations were cross-checked byte-for-byte against the reference Python `grcat`. The whole thing depends on exactly two crates: `fancy-regex 0.14` and `libc 0.2` (`Cargo.toml`), with a release profile using `lto = true` and `codegen-units = 1`. Beyond the port, the Rust version adds one directive family that upstream has no equivalent for — the streaming `trend/key/metric` extension covered further below — kept strictly additive so that any colourfile written for `grc 1.13` behaves identically here.

What it does

`grc <command>` runs a command and colourises its output. It joins the command and its arguments into a single string, scans `grc.conf` top to bottom for the first regexp that matches, loads the colourfile named on the following line, runs the command, and pipes the command's stdout (and/or stderr) through `grcat <colourfile>`, where per-line regexp-driven ANSI colouring is applied. If no regexp matches, the command runs uncoloured with inherited stdio.

```
grc ping example.com      # colourised ping
grc netstat -tulpn        # colourised netstat
grc df -h                 # colourised disk usage
grc ifconfig              # colourised interface dump
grc make                  # colourised build log
```

The two-program split is the same one upstream uses, and it matters for how the pieces compose. `grc` is only a dispatcher: it decides *which* colourfile applies and *how* to wire the child's file descriptors. `grcat` is a standalone filter that reads a colourfile and stdin and writes coloured stdout — it can be driven directly from a shell pipeline (`some-command | grcat conf.log`), which is how the colourising logic stays independent of the launch logic. The colouriser refuses direct misuse only in the trivial sense: called with anything other than exactly one argument it prints `You are not supposed to call grcatrs directly...` to stderr and exits 1 (`src/grcatrs.rs:548`), matching the upstream guard.

The launcher: `src/grcis.rs`

The `grc` binary is built from `src/grcis.rs`. Its job is option parsing, config resolution, and process orchestration.

Option parsing lives in `parse_opts` and mirrors `getopt`. Short options cluster (`-se` sets both stdout and stderr redirect); `-c` takes its value either attached (`-cconf.x`) or as the next argument (`-c conf.x`); `--` terminates option scanning so everything after it is treated as the command even if it looks like a flag; and option scanning otherwise stops at the first non-option token, which begins the command (`src/grcis.rs:42`). The long options are `stdout`, `stderr`, `config`, `colour`, and `pty`. The port reproduces `getopt`'s prefix behaviour through `resolve_long` (`src/grcis.rs:120`): an exact name wins, otherwise a *unique* prefix resolves to its option, and an ambiguous or unknown prefix returns `None` — which `grc` turns into a `help()` call and exit. So `--conf=name` resolves to `--config`, `--col=off` to `--colour`, and `--pt` to `--pty`, but `--std` is rejected because it could be either `--stdout` or `--stderr`. This is the exact `getopt` contract the Python original inherits, reproduced explicitly rather than approximated.

One consequence of routing every unrecognised long option through `help()` is that `grc --version` is not a dedicated flag at all: `resolve_long("version")` fails to resolve, `help()` fires, and the help banner — whose first line is `Generic Colouriser 1.13` — is what gets printed before the exit-0. That is deliberate and is what the CI smoke test leans on: it runs `grc --version` and greps the output for `Colouriser`, so the “version check” is really a check that the help path still emits the upstream banner string.

The colour default is computed from the terminal: `colour` initialises to `libc::isatty(1) == 1` (`src/grcis.rs:47`), and `--colour=auto` re-evaluates the same `isatty` check, so colour is on when stdout is a tty and off when piped, unless forced with `--colour=on/--colour=off`.

Config resolution runs through `conffile_list` and `find_config`. The `grc.conf` search order is `/etc`, `/usr/local/etc`, `/opt/homebrew/etc`, `$XDG_CONFIG_HOME/grc`, then `~/.grc` (`src/grcis.rs:147`), with `$XDG_CONFIG_HOME` defaulting to `~/.config` when unset. Two precedence rules matter and are reproduced deliberately. Within a single `grc.conf`, the *first* matching regexp wins — `scan_conf_for_cfile` returns the line following the first match (`src/grcis.rs:173`). Across the search path, `find_config` keeps scanning every file and lets the *last* match overwrite earlier ones (`src/grcis.rs:202`), so a higher-precedence user config beats a system config. Each non-comment, non-blank line in `grc.conf` is a candidate regexp and the next line names the colourfile — the same two-line-per-rule format the upstream file uses:

```
# ping command
^([/\w\.\.]+\w)?(io|o|n|h|arp|l2)?ping[236]?b
conf.ping

# gcc command
^([/\w\.\.]+\w)?(g?cc|[gc]\+)+\s
conf.gcc
```

The bundled `vendor/grc/grc.conf` carries 78 such regexp rules mapping onto 71 distinct colourfiles (some commands share a colourfile — `make` and `gcc` both point at `conf.gcc`, for instance). The leading `^([/\w\.\.]+\w)?` fragment recurs across nearly every rule: it lets an absolute or relative path prefix (`/usr/bin/ping`, `./configure`) match the same rule as the bare command name, which is why the launcher matches on the joined command line rather than just `argv[0]`.

Regexp dialect is bridged by `translate_regex` (`src/grcis.rs:220`). Python's `re` has no `</>` word-boundary form — those are literal `</>` characters — but `fancy_regex` reads them as GNU-style boundaries. The translator rewrites `<` and `>` to literals and leaves every other escape untouched, so configs authored for `grc` match the same strings under `fancy_regex`. This is the one place where a naïve reimplementer would silently diverge, and it is handled identically in both binaries — the launcher and the colouriser each carry their own copy of `translate_regex`, deliberately duplicated so neither can drift from the other.

Process orchestration is in `run_coloured` (`src/grcis.rs:303`). The wrapper ignores `SIGINT` (`libc::signal(SIGINT, SIG_IGN)`) so Ctrl-C reaches the child rather than killing `grc` before it can reap

the child and report its status — `grc` still waits on the child and propagates its exit code. The stdout/stderr redirection flags follow `grc`'s rule exactly: stdout is redirected by default; `-e` alone switches to stderr-only; adding `-s` re-enables stdout even alongside `-e` (`src/grcrs.rs:268`). When stdout is redirected, the child's stdout is piped into a spawned `grcat <cfile>`; when stderr is redirected, a second `grcat` reads the child's stderr and writes to a `dup` of fd 2. The `grcat` binary is located next to the running `grc` executable if a sibling `grcat` exists, otherwise it falls back to `grcat` on `PATH` (`src/grcrs.rs:248`) — so a locally built pair works without installation. After spawning, the launcher waits on the child first and only then reaps the one or two `grcat` children it started, returning the child's exit code (falling back to 0 when the status carries no code, e.g. signal death), so the visible exit status is the wrapped command's, not the colouriser's.

The `--pty` mode is marked experimental. It allocates a pseudo-terminal with `libc::openpty` (`src/grcrs.rs:390`), hands the slave end to the child's stdout so the command believes it is writing to a tty (and emits tty-style output), and feeds the master end into `grcat` as stdin. This exists for commands that suppress colour or change formatting when they detect a pipe. When `openpty` fails, the code degrades gracefully to an ordinary pipe rather than aborting.

Error output follows `grc`'s terse format: failures print `grc: <command>: <reason>` to stderr, where `<reason>` is the bare OS error string pulled through `libc::strerror` (`src/grcrs.rs:410`) to match the reference `%s: %s` formatting rather than Rust's default `io::Error` display.

The colouriser: `src/grcatrs.rs`

The `grcat` binary is built from `src/grcatrs.rs`. It reads one colourfile argument, resolves it through a search path, parses it into an ordered list of patterns, then reads stdin line by line and applies colours.

The colourfile search path in `conf_search_path` (`src/grcatrs.rs:231`) is: the empty prefix (so an absolute or relative name passed by `grc -c` is honoured as-is), then `$XDG_DATA_HOME/grc/`, `$XDG_CONFIG_HOME/grc/`, `~/.grc/`, `/usr/local/share/grc/`, `/usr/share/grc/`, and `/opt/homebrew/share/grc/`. The first existing non-directory file wins. `$XDG_DATA_HOME` defaults to `~/.local/share` and `$XDG_CONFIG_HOME` to `~/.config`. A name that resolves nowhere prints `config file [name] not found` to stderr and exits 1.

Each parsed block becomes a `Pattern` (`src/grcatrs.rs:18`): the compiled `Regex`, an optional per-group `colours` vector, the `count` directive string, and the optional `command`, `skip`, `replace`, and `concat` side effects — plus the three `grcrs`-extension fields (`trend`, `key`, `metric`) discussed later. A colourfile is a sequence of these blocks parsed by `parse_config` (`src/grcatrs.rs:261`). A block is a run of `keyword=value` lines; a line starting with `#` or blank is skipped inside a block, and a line whose first character is *not* an ASCII letter ends the block — which is why the `=====` separators in the shipped configs work as block delimiters. The recognised keywords are held in a ten-entry `KEYWORDS` array (`src/grcatrs.rs:265`): `regex`, `colours`, `count`, `command`, `skip`, `replace`, `concat`, plus the extensions `trend`, `key`, and `metric`. US spellings `color/colors` and the singular `colour` are all aliased to `colours` (`src/grcatrs.rs:300`), and an unrecognised keyword is a hard parse error. The `count` directive defaults to `more`. A block with no `regex` contributes no pattern.

The `colours` value is comma-split into one entry per capture group; within a group, whitespace-separated tokens are resolved and concatenated, so `bold red` composes the bold and red escapes into one string. Token resolution is `get_colour` (`src/grcatrs.rs:132`): a named token maps through a static ANSI table (`colour_table`, `src/grcatrs.rs:73`) that carries `none`, `default`, `bold`, `underline`, `blink`, `reverse`, `concealed`, the eight base foregrounds and `on_*` backgrounds, `beep`, the sentinel tokens `previous` and `unchanged`, the non-standard `dark/italic/rapidblink/strikethrough`, and the aixterm `bright_*/on_bright_*` codes. The bright codes are emitted with a standard-code prefix (`bright_red` → `\x1b[31;91m`) so a terminal without aixterm support degrades to the base colour instead of showing nothing. A token that is not in the table but is a `"..."` quoted literal is unescaped as a Python string literal would be — `unescape` (`src/grcatrs.rs:144`) decodes octal (`\033`), hex (`\xNN`), and the common single-char escapes, so configs that write `colours="\033[38;5;208m"` for 256-colour codes work unchanged. The octal decoder stops after

three digits and the hex decoder after two, matching Python's literal rules exactly — pinned by `unescape_octal_stops_after_three_digits` and `unescape_hex_caps_at_two_digits_and_bell`.

Here is a shipped colourfile — `vendor/grc/colourfiles/conf.df` — showing the block structure, the `=====` separators, and per-group colours:

```
# FS
regexp=^(?!Filesystem)(\[[-\w\d.]+\s
colours=blue,bold blue
=====
# Size 'G'
regexp=\s\d*[. ,]?dGi?\s|\b\d{7,9}\b
colours=red
=====
# Use 90-97%
regexp=\s9[0-7]%\s
colours=red
=====
# Use 98-100%
regexp=\s9[89]%\s|100%\s
colours=bold red
```

The `regexp` sources exercise features that force the choice of `fancy-regex` over the standard `regex` crate: the negative lookahead `(?!Filesystem)` and the lookaheads used elsewhere in the config set are not expressible in `regex`, which is why `fancy-regex` (a backtracking engine) is the dependency.

The main loop

The main loop (`src/grcatrs.rs:604`) reads with `read_until(b'\n')` into a raw byte buffer, strips a single trailing `\n/r`, and decodes leniently through `decode_units` (`src/grcatrs.rs:478`). This is the port's reproduction of Python's `surrogateescape`: valid UTF-8 decodes 1:1, and each *undecodable* byte becomes exactly one `U+FFFD` unit while its original byte is preserved in a parallel `orig` vector. One replacement char per bad byte keeps character indices aligned with what the Python reference produces, and keeping the source bytes means output can re-emit the invalid byte unchanged rather than the lossy replacement character. The net effect: colourising never corrupts non-UTF-8 command output, and this is pinned by dedicated tests (`invalid_utf8_bytes_round_trip_unchanged`, `invalid_utf8_byte_inside_coloured_span_round_trips` in `tests/cli.rs`, plus the unit tests `decode_units_maps_each_invalid_byte_to_one_roundtrip_unit` and `decode_units_splits_multibyte_invalid_run_per_byte`).

For each line, every pattern is tried in order. Matching uses `captures_from_pos` so a pattern can match repeatedly across a line, driven by the `count` directive. The directive semantics are reproduced from `grcat`:

- `more` (default) — colour every match on the line, advancing past each; a zero-width match advances one char boundary to avoid an infinite loop. The reference Python `grcat` loops forever on a whole-line `.*` under `more`; the port's boundary advance is what lets `zero_width_whole_line_pattern_terminates` pass.
- `once` — colour only the first match, then jump to end of line.
- `stop` — colour this match and stop trying further patterns on this line.
- `block` / `unblock` — set or clear a block-colour flag that carries across subsequent lines; while `blockflag` is set, whole lines are painted `blockcolour` regardless of per-line matches. A `block` match is internally converted to a `stop` so no later pattern runs on the block-start line (`src/grcatrs.rs:662`), which is why `count_block_halts_subsequent_patterns` expects a following `skip=yes` pattern *not* to suppress the line.
- `previous` — resolve to whatever the prior pattern's count was.

Colours are laid down into a per-character `cline` array of escape strings (`src/grcatrs.rs:752`), which is where the two carried-across-lines sentinels are honoured: `unchanged` leaves a group's characters untouched, and `previous/prev` reuses the colour from the previous line's matching group via the `prevcolour` state

variable. Group spans are accumulated first into a `clist` of `(char_start, char_end, colour)` triples by `add2list` (`src/grcatrs.rs:519`); groups beyond the colour list reuse colour 0, and non-participating groups are skipped. The final line is assembled in `nline` (`src/grcatrs.rs:785`) by emitting a colour escape only when it changes from the previous character's escape (run-length collapsing the ANSI codes), then the character's *original* bytes from `orig` — except after a `replace` rewrite, where `orig` no longer aligns with the mutated line and the decoded chars are used instead. A trailing `default` reset and newline close each line. `EPIPE` (downstream closed) breaks the loop cleanly, and `grcat` also ignores `SIGINT` itself so the launcher's forwarding scheme is the single owner of Ctrl-C.

The per-block side effects are also reproduced: `command` runs a shell command via `sh -c` on a match; `skip=yes|1|true` suppresses the line from output; `concat` appends the matched line to a file; and `replace` rewrites the line, with Python `\N` backrefs converted to the `${N}` form `fancy-regex` expects (`convert_backrefs`, `src/grcatrs.rs:429`), doubling any literal `$` to `$$` so it is not misread as a group reference. When a block carries a side effect but no `colours`, the inner loop breaks after applying it once, matching `grcat`'s single-shot semantics for pure side-effect blocks.

Streaming trend colour (grcrs extension)

The one place `grcrs` goes past the upstream grammar is a directive family for colouring a number by its *motion* over the stream rather than by which regexp matched it. Three keywords participate — `trend`, `key`, and `metric` (`src/grcatrs.rs:18` fields, parsed at `src/grcatrs.rs:332`). `trend=` is a comma-separated list of `direction:colour` pairs where the directions are `rising`, `falling`, and `steady`; each resolves through the same `get_colour` token machinery into a `TrendSpec` (`src/grcatrs.rs:39`). `key=` names the capture group whose text keys a per-key last-value map, and `metric=` names the capture group whose text is parsed as an `f64`. On each match, `trend_colour` (`src/grcatrs.rs:48`) compares the parsed value against the last value seen for that key and recolours the metric group green/red/dim (or whatever the spec says) by the sign of the delta, then stores the new value. The first observation of a key and any repeated equal value both count as `steady`; a group that does not parse as a number is left with its ordinary `colours` entry. The per-pattern map is composited into a single key of the form `pattern_idx \0 group_text` so two different patterns cannot alias one another's series.

```
regexp=(\w+)\s+(\d+)
colours=default,default
trend=rising:green,falling:red,steady:dark
key=1
metric=2
```

Under this block, group 1 (a name) keys the map and group 2 (a number) is recoloured by its own trend under that name, so each name's series climbs and falls independently — verified at the unit level: `parse_config_trend_key_metric` confirms the spec resolves to `rising=\x1b[32m`, `falling=\x1b[31m`, `steady=\x1b[2m`, and `trend_colour_is_isolated_per_key` confirms the per-key isolation. The extension is strictly additive: a colourfile that never mentions `trend/key/metric` takes the plain `add2list` path (`src/grcatrs.rs:690`) and behaves exactly as upstream `grcat`.

Worked examples

The end-to-end tests double as the clearest specification, because each asserts the exact ANSI byte stream. Basic single-match wrapping, with the `\x1b[0m` reset that `grcat` emits ahead of every colour run so previous state is cleared:

```
config: regexp=foo\ncolours=red
input:  a foo b
output: \x1b[0ma \x1b[0m\x1b[31mfoo\x1b[0m b\x1b[0m
```

A lookahead — the feature that forces `fancy-regex` — colouring a number only when `Mar` follows it:

```
config:  regexp=\d+(?=\sMar)\n colours=green
input:   size 344 Mar
output:  \x1b[0msize \x1b[0m\x1b[32m344\x1b[0m Mar\x1b[0m
```

`count=block/unblock` carrying colour state across lines, from `block_state_carries_across_lines`:

```
before      -> \x1b[0mbefore\x1b[0m
START here  -> \x1b[3m\x1b[30mSTART here\x1b[0m
inside block -> \x1b[3m\x1b[30minside block\x1b[0m
END now     -> \x1b[0m\x1b[0mEND\x1b[0m now\x1b[0m
after       -> \x1b[0mafter\x1b[0m
```

And the invalid-byte round-trip, where the raw `0xff` survives inside a coloured span rather than becoming a replacement character (`invalid_utf8_byte_inside_coloured_span_round_trips`, `regexp=o.n` over `no\xffne`):

```
output bytes: \x1b[0mn\x1b[0m\x1b[31mo\xffn\x1b[0me\x1b[0m
```

Running the full launcher, an absolute `-c` config skips the `grc.conf` scan so the test needs no system install:

```
grc -c "$CONF" --colour=on sh -c "printf 'a foo b\n'"
# -> \x1b[0ma \x1b[0m\x1b[31mfoo\x1b[0m b\x1b[0m
```

Man pages, config data, and packaging

The runtime data — `grc.conf` and the colourfiles — is not bundled into the binaries; it lives in the `vendor/grc` submodule and must be installed on one of the search paths. Section-1 man pages for both binaries ship in `man/grc.1` and `man/grcat.1`; the `grcat.1` page documents all ten colourfile keywords, including the `trend/key/metric` extension flagged there explicitly as “grcrs extension” and “not present in upstream grc”. The pages can be read without installing (`man ./man/grc.1`).

The release pipeline (`.github/workflows/release.yml`) fires on `v*` tags and builds a six-target matrix: `aarch64/x86_64 apple-darwin`, `x86_64/aarch64 unknown-linux-gnu`, and the two static `unknown-linux-musl` variants (the musl targets are marked `allow_failure` so a musl toolchain hiccup does not block the release). Each target’s package step stages the two stripped binaries alongside `grc.conf` and every `colourfiles/conf.*` into one tarball, so the archive carries the runtime data the binaries expect to find at their search paths. A `publish` job creates the GitHub release from the artifacts, and a `homebrew` job computes per-target SHA-256 sums and regenerates the `Grcrs` formula in the `MenkeTechnologies/homebrew-menketech` tap — emitting `on_macos/on_linux` blocks only for the arches that actually built, referencing the musl tarballs as trailing comments (Homebrew has no libc gate), and installing `grc/grcat` to `bin`, `grc.conf` to `etc`, and the colourfiles to `share/grc`. That matches the `/opt/homebrew/...` entries baked into the search paths, so `brew install menketech/menketech/grcrs` yields a working install without manual data placement. In the wider stack, `grc` is the front for the shell’s command colourising, invoked either directly or through the aliasing shims the upstream project ships (`grc.zsh`, `grc.sh`, `grc.fish`) that wrap common commands.

Status and parity

Parity against upstream is enforced by the test suite, currently **71 tests** total (counted by `#[test]` attributes): 17 launcher unit tests in `src/grcrs.rs` (option parsing, abbreviated long options, regexp translation, `grc.conf` matching and precedence), 27 colouriser unit tests in `src/grcatrs.rs` (colour table, string unescape, back-ref conversion, config-block parsing, `surrogateescape` decoding, and the streaming-trend logic), and 27 end-to-end tests in `tests/cli.rs` that spawn the built binaries and assert on the exact coloured bytes. The end-to-end tests cover the observable behaviours directly: `count=stop/once/more/block`, `unchanged` and `previous` colours, `replace` then colour, quoted-literal colours, `</>` literals, `skip`, `concat`, CRLF

stripping, multibyte char spans, stderr redirection, exit-status propagation, and the invalid-UTF-8 round-trip cases. CI ([.github/workflows/ci.yml](#)) runs four jobs: `cargo fmt --all --check`, `cargo clippy --all-targets --locked -- -D warnings`, `cargo doc --locked --no-deps` under `RUSTDOCFLAGS=-D warnings`, and a build+test matrix on `ubuntu-latest` and `macos-latest` that finishes with the `grc --version | grep -q Colouriser` smoke test.

The licence is **MIT**, stated consistently across the source tree: `Cargo.toml` declares `license = "MIT"`, the README's badge and licence section both read MIT, the `LICENSE` file carries the MIT text, and the Homebrew formula generated by `release.yml` writes `license "MIT"`. `grcrs` is original Rust code — it reimplements `grc`'s behaviour and its config-rule model rather than deriving from `grc`'s source, and the upstream tree kept under `vendor/grc/` is a read-only reference for parity work, not what the binaries are built from.

zpwrcode-host and zwire-host — the browser-adjacent command line

The two subjects of this final chapter sit at an awkward border. They are launched, most of the time, by a web browser, and they exist because a browser extension needs a way to reach the local machine. That lineage tempts you to file them under “browser tooling” and move on. But a browser extension is a bundle of manifest JSON and service-worker JavaScript that Chrome loads; it has no binary you build, no `--help`, no prompt, and no life outside the browser process. The two things this chapter covers — `zpwrcode-host` and `zwire-host` — are Rust programs. You compile them with `cargo`, they carry `--help` and `--version`, they run on macOS, Linux, and (for `zwire-host`) Windows, and at least one of their run modes has nothing to do with a browser at all. They belong in a book about the command line because that is what they are: command-line binaries that happen to also speak the browser’s native-messaging dialect. The extension is the client; these are the tools.

Native messaging, in one paragraph

Chrome’s native-messaging mechanism is the reason both binaries exist. An extension declares the `nativeMessaging` permission, and the browser is told — through a small JSON manifest installed in a per-browser directory — that a given host name maps to an executable on disk with a fixed set of allowed extension origins. When the extension calls `chrome.runtime.sendMessage` or `chrome.runtime.connectNative`, Chrome spawns that executable and talks to it over stdin/stdout using `u32`-length-prefixed JSON frames. Neither of these binaries invented that protocol, and the chapter does not claim they did; the engineering is what each does on top of it. `zpwrcode-host` ports a specific upstream protocol into Rust and adds actions to it. `zwire-host` takes the single stdio transport Chrome gives you and unifies it with two more (a local socket and a client), so the same capabilities are reachable whether the caller is a browser or a shell script.

zwire-host — one dispatcher over three transports

`zwire-host` is v0.3.14, MIT-licensed (`zwire-host/Cargo.toml:3-8`). One crate produces two artifacts: a `[[bin]]` named `zwire-host` and a `[[lib]]` named `zwire_host` (`zwire-host/Cargo.toml:15-21`). The binary is a five-line entry point — `main` collects args and calls `zwire_host::run` (`zwire-host/src/main.rs`) — so that every other Rust program can embed the exact same host by pulling in the library. The docstring is blunt about the scope creep that gave the crate its name: it began as “the Chrome native-messaging host for the zwire HUD” and became “a universal local endpoint” reachable “from a browser extension *and* from tmux, emacs, desktop apps, plugins, and any language” (`zwire-host/src/lib.rs:4-12`).

The three run modes are selected by the first positional argument in `run` (`zwire-host/src/lib.rs`):

```
# 1. native messaging on stdio – no subcommand, what Chrome spawns
zwire-host

# 2. the NDJSON socket daemon
zwire-host serve

# 3. a one-shot client against that daemon
zwire-host call '{"cmd":"hostinfo"}'
```

Modes 2 and 3 are ordinary CLI usage with no browser in sight. The daemon listens on a Unix domain socket on macOS/Linux and a named pipe on Windows (`zwire-host/src/transport.rs`), speaking newline-delimited JSON — “the lingua franca every tool already has” (`zwire-host/src/lib.rs:16-18`). The socket path defaults to a runtime dir, never the persistent state dir, and is overridable with `$ZWIRE_HOST_SOCKET`; the code walks `$XDG_RUNTIME_DIR`, then macOS `$TMPDIR`, then a per-user name in `/tmp`, binding it `0600` because “the socket exposes `exec/fs/pty`, so no other local user may connect” (`zwire-host/src/lib.rs`, `zwire-host/src/transport.rs`). A client is trivial to drive from a shell:

```
zwire-host serve &
zwire-host call '{"cmd":"fs_walk","path":"/src","ext":"rs"}'
echo '{"cmd":"exec","program":"git","args":["status"]}' \
| zwire-host call
zwire-host call --stream '{"cmd":"sysinfo_start"}'
```

The last line shows the streaming variant: `--stream` relays every reply frame instead of just the first, which is how `sysinfo` and `PTY` output flow back (`zwire-host/src/lib.rs`, `help body`). Beyond the local socket, `serve` can also open a TCP listener guarded by a shared `--token` and dial named `--peer` hosts, federating the bus across machines (`zwire-host/src/transport.rs`, `ServeConfig`) — but the token gate exists precisely because local socket clients are trusted and TCP ones are not.

Every transport feeds the same dispatcher. A `Session` is per-connection state, and `Session::handle` takes one decoded request plus the connection’s output sink; “both the Chrome stdio loop and the socket daemon drive it the same way, so every capability is reachable from every client with no per-transport code” (`zwire-host/src/session.rs:1-7`). The capability set is advertised in the `hello` reply so a client can feature-test, and it reflects which optional features were compiled in (`zwire-host/src/session.rs:14-43`):

```
hello kv fs exec jobs bus peer watch
open clipboard notify hostinfo scheme ui palette
sysinfo procs      (feature "sysinfo-caps")
pty                (feature "pty")
```

The command arms behind those tags include `kv_get/kv_set/kv_merge` for a namespaced key/value store, `fs_*` crawl and watch, `exec`, `job_*`, `ps/kill/which`, `pty_spawn/pty_write/pty_resize/pty_kill`, `sysinfo_start/sysinfo_once`, plus `open/clipboard_get/clipboard_set/notify/hostinfo` (`zwire-host/src/session.rs:188-538`). The optional capabilities are real Cargo features: `sysinfo-caps` pulls `sysinfo`, `pty` pulls `portable-pty`, and both are in the default set, but an embedder can drop them (`zwire-host/Cargo.toml:24-40`).

The pub/sub bus

The capability that turns the host from an RPC endpoint into a coordination hub is the bus (`zwire-host/src/bus.rs`). Any long-lived connection can `sub` to a topic and receive `{"ev":"pub",...}` frames whenever another connection `pubs` to the same topic. Because the broker is process-global, “a message published by the browser HUD reaches a subscribed `zmax`, `tmux` widget, or plugin — they never talk to each other directly, only through the host” (`zwire-host/src/bus.rs:1-9`). The daemon also publishes on well-known topics itself — `scheme`, `ui`, `palette`, `schemes` — so a new subscriber gets a snapshot on subscribe and every change afterward, which is live theme sync with no polling. Delivery is best-effort fan-out; publish collects target sinks under the broker lock and writes to them outside it, so IO never deadlocks a publisher that is also a subscriber (`zwire-host/src/bus.rs:79-95`).

zpwchrome-host — a browserpass port plus additive actions

`zpwchrome-host` is v0.10.2, MIT-licensed (`zpwchrome-host/Cargo.toml:1-6`). It is a Rust port of the Go program `browserpass-native`, the native host that the browserpass browser extension talks to for UNIX `pass` integration. The port is deliberate and disciplined: `src/ported/` is a 1:1 mirror of the upstream Go source, one Rust file per Go file, PascalCase identifiers preserved verbatim, every function carrying a `/// Port of <name>() from <go_file>:<line>` comment, drift caught by pinning tests (`zpwchrome-host/README.md:27-39`). It implements every action in browserpass-native's `PROTOCOL.md v3.1.2` — `configure`, `list`, `tree`, `fetch`, `save`, `delete`, `echo` — with error codes 10–32 pinned to the same integers and the version reported as `3.1.2` (`zpwchrome-host/README.md:11-25`). Because it is wire-compatible, it is a drop-in replacement for the Go binary and can be registered under the upstream host name for the stock browserpass extension. The shipped code is original MIT Rust; the lineage is credited, and that is the whole of the debt.

The original engineering lives in `src/extensions/`, which the README calls additive because the upstream extension never sends these action names, so wire compatibility is preserved (`zpwchrome-host/README.md:41-43`). Three additive families matter most:

- `otp` — shells `pass otp <entry>` with `PASSWORD_STORE_DIR` set to the matching store and returns the current TOTP code.
- `search` — host-side fuzzy plus substring scoring across every configured store, faster than shipping the whole tree to the client for `zfz`.
- `dl.*` — a segmented download manager, the largest addition, described below.

The dispatch order in `main` sends any `dl.`-prefixed action to the download manager first, then `otp/search`, then `run.spawn`, then `host.crawl/host.exec`, then `zcite.save`, and only then falls through to the ported browserpass switch (`zpwchrome-host/src/bin/zpwchrome_host.rs:362-421`). The message is parsed twice — once as a raw `Value` for the extension actions that take their own shape, once as the ported request struct — so the strict port stays free of extension fields (`zpwchrome-host/src/bin/zpwchrome_host.rs:316-320`).

The download manager's file-state model

`dl.add` does not download in the host process. It spawns a detached worker — `zpwchrome-host --dl-worker <gid>` — and the process model is a CLI subcommand of the same binary (`zpwchrome-host/src/bin/zpwchrome_host.rs:246-257`). State for each job lives in a JSON file at `$XDG_CACHE_HOME/zpwchrome/dl/gid_NNNNNN.json` (`zpwchrome-host/src/extensions/dl.rs:104-105`), and that file, not a long-lived connection, is the coordination medium. This matters because native messaging spawns a fresh host process per one-shot message; there is no persistent port for downloads. Every control action — `dl.pause`, `dl.resume`, `dl.cancel`, `dl.remove`, `dl.clear`, `dl.list` — reads or mutates flags in the state file, and the worker polls those flags between chunks. The `JobState` struct carries `gid`, a `status` string (`pending|active|paused|done|failed|cancelled`), `paused` and `cancelled` bools, a `paused_offset_ms` accumulator so a resumed job reports accurate elapsed time, and a `worker_pid` (`zpwchrome-host/src/extensions/dl.rs:50-83`). On `dl.resume`, the host checks `kill(worker_pid, 0)`: a live worker just needs the paused flag cleared, a dead PID means the service worker was suspended and a fresh worker must be spawned — self-healing without the extension knowing (`zpwchrome-host/README.md:74-83`). Segmentation kicks in only when the total is at least `MIN_SEGMENT_BYTES` (1 MiB), the server sends `Accept-Ranges`, and more than one segment is requested; otherwise it degrades to a single stream (`zpwchrome-host/src/extensions/dl.rs:39-44`, README download-manager section). Spawn isolation is load-bearing: `setsid()` plus closing every `fd >= 3` before `exec`, because otherwise the worker inherits Chrome's native-messaging stdout pipe and Chrome reports “Native host has exited” even on success (`zpwchrome-host/src/bin/zpwchrome_host.rs`, README worker-process-model). The HTTP client is `ureq` with `rustls`, chosen so there is no `aria2`, no system OpenSSL, and no Go toolchain at

runtime (zpwchrome-host/Cargo.toml, zpwchrome-host/README.md:9).

Embedding zwire-host as a library

The `host.crawl` and `host.exec` actions are where the two binaries meet. Rather than re-implement a recursive filesystem walk or a capture-stdout exec, `zpwchrome-host` depends on the `zwire-host` crate and calls `zwire_host::api::walk` and `zwire_host::api::exec` — “the same code path the `zwire-host` daemon runs” (`zpwchrome-host/src/extensions/host.rs:1-8`). The dependency is declared `default-features = false` (`zpwchrome-host/Cargo.toml`, dependency comment), which is why the feature split in `zwire-host` exists: the embedder wants only the light filesystem/exec half and none of the heavy `portable-pty` or `sysinfo` weight, verified with `cargo tree`. So one of these two CLI tools is literally built out of the other, at the library layer, with the transport and heavy capabilities compiled out.

The wiring that joins the extension to both hosts

The `zpwchrome` extension — manifest v3, an MV3 service worker, the client and not a subject here — opens two native-messaging channels, and that is why the two hosts belong in the same chapter. For `pass`, `OTP`, `search`, and `downloads` it uses one-shot `sendNativeMessage` to the host name `com.menketotechnologies.zpwchrome`, which resolves to `zpwchrome-host`; each call spawns a fresh host process with no long-lived port (`zpwchrome/background.js:3726-3789`). Separately, for theme sync, it opens a persistent `connectNative` port to `com.zwire.hud`, which resolves to `zwire-host`, and `subs` to the `scheme/ui/palette/schemes` topics on the bus; the host answers with a snapshot and then pushes every change from any app in the fleet (`zpwchrome/background.js:4287-4354`). That is the same pub/sub bus described above, consumed from JavaScript. If `zwire-host` is not present — the extension loaded in vanilla Chrome — the port disconnects and the extension backs off harmlessly, its own theme still working from storage (`zpwchrome/background.js:4333-4354`).

The manifest side of that wiring is what `--install` writes. `zpwchrome-host --install <ext-id>` drops `com.menketotechnologies.zpwchrome.json` into every Chromium-family config dir it finds, plus `zwire`’s own profile dir, with `allowed_origins` scoped to the calling extension so only it can spawn the host (`zpwchrome-host/src/bin/zpwchrome_host.rs:258-283, 513-634`). The written manifest is the standard native-messaging shape:

```
{
  "name": "com.menketotechnologies.zpwchrome",
  "description": "zpwchrome native host (BP protocol)",
  "path": "/Users/user/.cargo/bin/zpwchrome-host",
  "type": "stdio",
  "allowed_origins": [
    "chrome-extension://<ext-id>/"
  ]
}
```

Installation is `cargo install`, upgrade is `cargo install --force`, and the path baked into the manifest survives a Homebrew keg change because `stable_exe_path` rewrites a versioned Cellar path back onto the version-independent `opt` symlink — a keg path would die on the next `brew upgrade` and silently drop every download back to Chrome’s own downloader (`zpwchrome-host/src/bin/zpwchrome_host.rs:491-511, zpwchrome-host/README.md:103-132`). None of that — `cargo install`, `--install`, the `--help` banner, the keg-path repair — has an analogue in the extension. It is all command-line surface, which is the point of the distinction this chapter opened with.

Both binaries carry the same styled `--help` screen and a `--version/version` path: `zpwchrome-host --version` prints the browserpass protocol version (3.1.2) it speaks, distinct from its own crate version (`zpwchrome-host/src/bin/zpwchrome_host.rs:307-309, 454-456`), and `zwire-host version` prints its crate version (`zwire-host/src/lib.rs, run`). Test coverage is reported at 121 tests for `zpwchrome-host` including a 2 MiB segmented download against a local Range-capable HTTP server

([zpwchrome-host/README.md:179-188](#), count provisional — taken from the README, not verified by a build here). The version numbers above are read from each crate's `Cargo.toml` rather than confirmed against a compiled artifact, so treat them as the declared versions. What is not provisional is the shape of the thing: two Rust programs, three transports between them, one embedded inside the other, both reachable from a browser and one of them equally reachable from a shell.

Why one dispatcher over three transports is the whole design

The structural claim `zwire-host` makes is that a capability — walk a directory, run a command, spawn a PTY, read system statistics — should be written once and be reachable from every kind of caller. The mechanism is a single `Session` type whose `handle` takes one decoded request plus the connection's output sink, and whose source comment states the invariant plainly: both the Chrome stdio loop and the socket daemon drive it the same way, so every capability is reachable from every client with no per-transport code.

That is easy to write down and easy to violate, and it is worth being specific about what the design has to get right for the claim to hold. The transports differ in three ways that leak into a naive implementation. Framing differs — Chrome sends `u32`-length-prefixed JSON, the socket speaks newline-delimited JSON. Lifetime differs — a one-shot `sendNativeMessage` spawns a process that handles one message and exits, while a socket connection may live for hours and carry a subscription. And trust differs — a local socket bound `0600` is reachable only by the same user, while a TCP peer is not, which is why the TCP listener is gated behind a shared `--token` and the local one is not.

The way all three are kept out of `Session` is that each is handled strictly above it. Framing is the transport's job; `Session` receives a decoded request. Lifetime is expressed through the sink rather than through the handler — a handler that wants to stream writes multiple frames to the sink it was given, and whether those frames go to a long-lived socket or to a stdio pipe Chrome is holding open is not its concern. Trust is a bind-time and accept-time property, resolved before a request is ever decoded. The result is that the capability arms — `kv_*`, `fs_*`, `exec`, `job_*`, `pty_*`, `sysinfo_*`, `open`, `clipboard_*`, `notify`, `hostinfo` — contain no transport awareness at all, and adding a transport would not touch any of them.

The `hello` reply is the other half of the design and it is the part most local protocols skip. Capabilities are *advertised*, and the advertised set reflects which optional Cargo features were compiled in — `sysinfo-caps` pulls `sysinfo`, `pty` pulls `portable-pty`, both default-on but droppable. A client therefore feature-tests rather than guessing, which is what makes the same protocol usable against a full daemon and against the stripped build `zpwchrome-host` embeds with `default-features = false`. Without the advertisement, an embedder dropping `pty` would turn a working client into one that fails on an unknown command with no way to have known better.

The security posture of a local capability bus

A daemon that exposes `exec`, filesystem traversal, and PTY spawn to anything that can connect to it is, precisely, a remote code execution service. The posture that makes that acceptable is worth stating explicitly because it is assembled from several small decisions that only work together.

The socket lives in a runtime directory, never the persistent state directory — the code walks `$XDG_RUNTIME_DIR`, then macOS `$TMPDIR`, then a per-user name in `/tmp`, and `$ZWIRE_HOST_SOCK` overrides. Runtime directories are cleared between boots and are per-user, which means a stale socket does not survive into a session that did not create it. It is bound `0600`, with the source comment giving the reason in one line: the socket exposes `exec`, filesystem, and PTY, so no other local user may connect. That single mode bit is the entire local authorisation model, and it is the right one — it delegates to the OS's user boundary rather than inventing an authentication scheme inside a local IPC protocol.

TCP is where that delegation stops working, because a TCP peer is not authenticated by a filesystem permission.

So TCP is not the same thing with a different address family: it is opt-in, it is gated behind a shared `--token`, and the `--peer` federation dials named hosts explicitly rather than discovering them. The asymmetry — local clients trusted by construction, TCP clients required to present a token — is the correct reading of the two situations and it is stated as the reason the token gate exists.

The browser side gets a third mechanism, because a browser extension is neither a local user nor a network peer. `zpwchrome-host --install <ext-id>` writes the native-messaging manifest with `allowed_origins` scoped to the calling extension, so Chrome will only spawn the host for that extension ID. That is an authorisation decision enforced by the browser before the host process exists, which is the only place it can be enforced — once Chrome has spawned the binary and connected its stdio, the host has no way to ask who is on the other end.

Three callers, three different trust mechanisms, one dispatcher behind all of them. The generalisable point is that unifying the *capability* surface across transports does not mean unifying the *trust* model, and a design that tried to would end up either rejecting local clients that should be trusted or accepting network clients that should not.

Failure modes peculiar to a browser-spawned process

Being launched by a browser imposes constraints that no other program in this book has to think about, and both hosts carry scars from them.

The first is that the parent may vanish. Chrome's MV3 service workers are suspended aggressively, which takes down the context that spawned a one-shot host. For `zpwchrome-host`'s downloads this is not an edge case, it is the expected steady state on a long download, and it is the reason job state lives in a file and the reason `dl.resume` checks `kill(worker_pid, 0)` before deciding whether to clear a flag or spawn a fresh worker. A design that kept job state in the host process's memory would work perfectly in every test and fail on every download long enough to matter.

The second is that stdio is a protocol channel, not a console. Anything written to stdout is a native-messaging frame, so a stray `println!` is not a log line, it is a protocol violation that Chrome will reject. The sharpest consequence is the inherited-descriptor problem the chapter already records: a spawned worker that does not close descriptors `>= 3` inherits Chrome's native-messaging stdout pipe, and Chrome — which decides the host has exited by observing the pipe — reports “Native host has exited” even on a completely successful run. `setsid()` plus closing every fd `>= 3` before exec is the fix, and the failure it prevents is one that presents as a bug in the extension rather than in the host.

The third is that the binary's *path* is part of the installed configuration. The native-messaging manifest names an absolute path, so anything that moves the binary silently breaks the integration — and a Homebrew keg path moves on every upgrade. `stable_exe_path` rewrites a versioned Cellar path onto the version-independent `opt` symlink before the manifest is written, precisely so a `brew upgrade` does not leave Chrome pointing at a path that no longer exists. The observable failure without that repair is not an error message; it is downloads quietly falling back to Chrome's own downloader, which is the class of failure that takes weeks to notice.

The fourth is a versioning subtlety with no analogue elsewhere in the fleet. `zpwchrome-host -version` prints `3.1.2` — the browserpass protocol version it speaks — which is deliberately *not* its crate version. Two different consumers ask two different questions: the extension needs to know which protocol dialect it is talking to, and a package manager needs to know which build is installed. Collapsing them into one string would force one consumer to parse the other's answer. The book's version tables list the crate versions for exactly this reason, with the protocol version recorded separately.

Matching the upstream — the parser-compatibility layer, tool by tool

The shape-of-the-fleet chapter made the argument-parsing point in a paragraph: everyone matches the upstream parser, using `clap` where `clap` can express the grammar and refusing or pre-processing it where it cannot. That paragraph is true, but it hides the interesting part — the specific ways each parser is *wrong* for a modern derive-based CLI, and the specific machinery each tool grows to be wrong in exactly the same way the original was. This chapter reads all of it across the grain. It is the one place in the book that treats the eight tools' front ends as a single subject, because the front end is where a port either earns the word “drop-in” or forfeits it, and the failure mode — accepting a real upstream command line and doing something subtly different with it — is invisible until a fifteen-year-old script breaks.

Why `clap` is the wrong shape for a ported CLI

`clap` 4's derive parser is an excellent argument parser for a program designed around it. It expects options to be separated from their arguments (or joined with `=`), it expects short flags to cluster only when they are all boolean, it reserves `--help/--version`, and it errors on anything it does not recognize. Every one of those reasonable defaults is a liability for a tool whose grammar was frozen decades ago by a different parser. Isov's grammar is not `getopt-clean`; Nmap's short options are glued (`-sS`, `-P580`); `htop` and `grc` were written against `getopt/getopt_long`, whose abbreviated-long-option and option-scanning rules `clap` does not reproduce; and every one of these tools has flags that a script passes and that must be *accepted and ignored* rather than rejected. So the fleet splits into three postures toward `clap`, and the split is not stylistic — it tracks how far each upstream grammar sits from what `clap` can express.

Refuse `clap` entirely (`htoprs`, `grcrs`). Two of the five reimplementations do not let `clap` near their argv, because their upstream parser is the specification and `clap` cannot reproduce it. `htoprs` calls the real `libc::getopt_long` symbol — the identical library routine `htop`'s own `getopt_long` fills — marshalling `argv` into a null-terminated C `char**` and building the same `option` array with `has_arg` 0/1/2, so the parse is not merely similar to `htop`'s, it *is* `htop`'s, produced by the same C code (`src/ported/commandline.rs`). `grcrs` hand-mirrors `getopt` in `parse_opts`, reproducing two subtleties `clap` gets wrong: abbreviated long options resolved through `resolve_long` (an exact name wins, a unique prefix resolves, an ambiguous prefix is refused — `--conf=name` → `--config`, `--col=off` → `--colour`, but `--std` rejected as ambiguous between `--stdout/--stderr`), and option scanning that stops at the first non-option token and treats everything after `--` as the command even when it looks like a flag (`src/grcrs.rs:42`, `:120`).

Pre-process, then hand `clap` a clean line (`lssofrs`, `nmaprs`). Two more keep `clap` downstream but run a hand-written normalizer *before* it, because their grammar is glued or `+-`prefixed rather than `getopt-abbreviated`. `lssofrs`'s `normalize_lsof_args` (`cli.rs:368`) splits `--clusters` `getopt-style`, expands the `+L/+d/+D` family, peels attached arguments (`-o9`, `-sTCP:LISTEN`, `+L1`), maps each Isov option to its `clap` equivalent or to nothing, and passes everything after `--` untouched. `nmaprs`'s `expand_nmap_style_argv` (`argv_expand.rs`) walks `argv` token by token and separates Nmap's compressed grammar — `-sS` → `--scan-type S`, `-Pn` → `--no-ping`, `-P580,443` → `--ping-S 80,443`, `-T4` → `--timing 4`, stacked `-vv/-dd` collapsed to `--verbosity=N/--debug=N` by counting letters — with the match-arm ordering commented as load-bearing (`-s0` before the generic `-sX` arm, `-P0` before the generic `-P*` arm).

Use `clap` as designed (`iftoprs`, `storageshower`, `temprs`, `zcolorizer`). The remaining tools have grammars `clap` can express directly, so they lean on the derive parser and add only targeted overrides. `iftoprs`

and `zcolorizer` set `disable_help_flag/disable_version_flag` so their hand-drawn cyberpunk help screens render instead of clap's default (`iftop.rs:68`; `zcolorizer` captures `-h/--help` as an explicit field). `storageshower` uses clap's `overrides_with` to pair every visible flag with a hidden `--no-` inverse so the last one on the line wins and only one boolean survives to `apply_to` (`cli.rs:383`). `temprs` sets `allow_hyphen_values(true)` on almost every index-taking flag, which is the single setting that lets `-1`, `-2` arrive as *values* rather than being read as short flags — negative stack indexing works precisely because of it.

The accept-and-ignore contract

The subtlest obligation is the one that does nothing visible: accepting flags that have no meaning in the port so that scripts passing them keep running rather than erroring. `lsf` is the clearest case and its normalizer is where the contract lives. Single-char options `-b -O -C -M -X -E -L` are consumed inside a cluster and dropped (`cli.rs:452`); `-z -Z -S -f` drop any attached argument and end the cluster (`:456`); `-A -k -m -D -e` are treated as taking a required argument so the following token is swallowed if none is attached (`:460`); on the `+` side, `+w +f +E +M +X +m` are silent toggles and `+e` consumes a separate argument (`:564`). A script that runs `lsf -b -O` (avoid kernel blocking, bypass overhead) runs under `lsf` without error even though neither flag has an implementation behind it. The book's own phrase for this is exact and worth keeping: the flags are *honored syntactically, not behaviorally*. `nmaprs` does the same at a different layer — its `effective_*` accessors and the extended long-option set from `nmap.cc` (`--resolve-all`, `--defeat-rst-ratelimit`, `--nsock-engine`, `--release-memory`, and the rest) are accepted for parity and logged at `debug` via `trace_nmap_compat` rather than acted on, so an Nmap command line that leans on them parses and scans rather than aborting.

The reason this is the correct trade, and not a lie, is that the alternative is worse in exactly the way a port exists to avoid. A parser that rejects `-b` because the port has no kernel-blocking behavior forces every caller to audit and rewrite their invocations — which is the migration cost the drop-in replacement was supposed to eliminate. Accepting the flag and doing nothing is the honest behavior *provided the tool says so*, which every chapter does: `lsf`'s parity section names `-b/-O/-D` as syntactic no-ops, and `nmaprs`'s capability matrix marks the accepted-but-unwired options.

Where the parsers deliberately diverge

Two divergences are intentional and both are about branding, not grammar. `htoprs -h` and `htoprs -V` short-circuit in a thin `src/main.rs` wrapper before the ported `getopt` path ever runs — `-V` calls the ported `printVersionFlag` (prints `htoprs 0.5.7`), `-h` calls `extensions::help::print_help` (the figlet-bannered house-style screen) — while the plain ported `printHelpFlag` is retained verbatim as the spec the styled screen tracks. `nmaprs` intercepts clap's help/version errors and routes them into `help_tp::print_help/print_version` so the styled CYBERPUNK help appears instead of clap's plain renderer, and re-derives the binary basename from `argv[0]` so `nms` shows `nms` in its help. These are the only places the fleet's parsers do something the upstream did not, and in both cases the upstream behavior is still reachable underneath — the divergence is a presentation layer over an unchanged parse.

The one-token overload problem

The hardest single thing a ported parser does is disambiguate a flag whose meaning depends on what follows it, because that is where a token-by-token pre-processor and a grammar-driven parser disagree most. `lsf`'s `-s` is the canonical example and `normalize_lsof_args` handles all three readings: `-s` with no argument means “always show size,” `-s` with a colon spec (`-sTCP:LISTEN`) filters sockets by state, and `-s` followed by a non-colon rest is size-always with the cluster continuing (`cli.rs:479`). `-o` optionally takes a leading digit count for offset precision (`-o9` = nine digits), peeled character by character and re-emitted as `--offset-always` plus `--offset-digits` (`:465`). `+L` alone lists the NLINK column while `+L1` additionally selects files with link count below 1 — the unlinked-but-open case (`:521`). None of these is expressible as a clap argument type;

they exist only because a human wrote a state machine that reads the same bytes `lsf`'s own option loop reads. The lesson the fleet encodes is that a compatible parser is not a compatible *option set* — it is a compatible *reading* of the exact byte sequence the upstream accepted, overloads and all, and the only way to get it right is to treat the original parser as the spec and reproduce its decisions rather than its intentions.

The `argv[0]` problem, and what a second binary name costs the parser

Three tools in the fleet ship under two names — `lsf`/`lsf`, `nmaprs`/`nms`, `temprs`/`tp` — and a fourth, `grcrs`, ships two names that are genuinely two programs. That decision looks like a packaging detail and is really a parser detail, because the moment a binary can be invoked under more than one name the parser has to decide what `argv[0]` means to it. There are only three coherent answers and the fleet uses two of them.

The first answer is *ignore it*: the alias is a byte-identical second artifact and the parse is unaffected. `lsf` takes this line and takes it literally — its normalizer copies `argv[0]` verbatim into the rewritten argument vector so that whatever the shell used to spawn the process survives untouched into `clap`, and `lsf -i 4TCP:22` and `lsf -i 4TCP:22` produce the same normalized line character for character. The property that buys is testability: the normalizer's test cases never have to be written twice, because there is no code path where the program name changes a parsing decision.

The second answer is *read it for presentation only*. `nmaprs` re-derives the basename from `argv[0]` so that `nms --help` prints usage lines that say `nms` rather than `nmaprs`, which matters more than it sounds like it should: a help screen that names a binary the user does not have on `PATH` is a help screen that produces a wrong copy-paste. The discipline here is that the derived name reaches the *renderer* and nothing else. It does not reach the option table, it does not select a dialect, and it does not change which flags are accepted. That boundary is what keeps a two-name binary from quietly becoming two programs.

The third answer — dispatch on `argv[0]` to select behavior, the way `busybox` does — appears nowhere in this fleet, and `grcrs` is the reason it is worth naming as a rejected option. `grc` and `grcat` really are different programs with different grammars: `grc` scans options until the first non-option token and treats the rest as a command line to run, while `grcat` takes a colourfile argument and reads a stream from `stdin`. Upstream `grc` ships them as two programs and `grcrs` ships them as two `[[bin]]` targets with two `main` functions and two hand-written parsers, rather than as one binary that inspects its own name. The cost is a little duplicated scaffolding — each carries its own `translate_regex`, for instance — and the benefit is that neither parser has a branch in it that exists only to answer “which program am I today.”

What the completion system needs from a compatible parser

A parser that reproduces a decades-old grammar has a second consumer besides the user: the shell completion function. This is where compatibility work stops being purely defensive, because a completion is a *second, independent* encoding of the same option table, and the two encodings drift unless something holds them together.

The fleet holds them together three different ways, and the three are worth ranking. The weakest is a hand-written table that a human updates alongside the parser — the default, and the arrangement most of the checked-in `<tool>` functions start from. It works exactly as long as the human remembers. The middle strength is generating the completion from the parser: `iftoprs` can emit completions at runtime for five shells straight out of `clap_complete::Shell`, so the emitted function is derived from the same `clap` definition the binary parses with and cannot describe a flag the binary does not have. The strongest is a completion that asks the binary at completion time: `htoprs`'s sort-key state calls `_call_program sort-keys $words[1] --sort-key help` and parses the tool's live column list, so adding a column to the port updates the completion

with no completion edit at all, and `temprs's _tp_indices` reads the running stack's master file so the offered indices and `@`-tags are the ones currently on the stack.

That last case is the one where the compatibility layer and the completion layer touch most directly, and it shows what a NUL-delimited on-disk format costs downstream. zsh variables cannot hold a NUL byte, so a completion function that wants to read `temprs's` master record cannot simply `$(cat)` it — the helper pipes the record through `tr '\0' '\x1e'` and splits on the substituted byte. The format was chosen because NUL is the one byte a Unix path cannot contain, which is exactly the property that makes it unreadable to the shell. Neither decision is wrong; the interesting part is that the on-disk format's correctness argument and the completion's implementation constraint are in direct tension, and the resolution is one `tr` in one helper function rather than a weaker format.

How you test a parser you did not design

The last thing a compatibility layer owes is a way to be checked, and the fleet splits sharply here between the tools that can diff against a running upstream and the tools that cannot. `grcis` can: its end-to-end tests spawn the built binaries and assert on the exact coloured byte stream, with expectations cross-checked against the reference Python `grcat`, which makes the upstream a literal oracle for the output side of the grammar. `htoprs` goes one better and removes the question entirely on the parse side, because it calls the real `libc::getopt_long` — there is no second implementation to diff, so the class of bug where a hand-written getopt clone mishandles an edge case cannot exist.

`lsotrs` and `nmaprs` are in the harder position: their normalizers are original code with no upstream binary to diff against, because the thing being reproduced is an internal decision (how does `lsot` read `-sTCP:LISTEN?`) rather than an observable output. What both do instead is pin the *transformation*: the test asserts that a given upstream-style argv normalizes to a specific expanded argv, which turns an invisible internal decision into a written-down, reviewable mapping. `lsotrs's cli.rs` carries well over a hundred parse-combination cases of exactly this shape, and `nmaprs's expand_nmap_style_argv` has its match-arm ordering documented as load-bearing — `-s0` before the generic `-sX` arm, `-P0` before the generic `-P*` arm — precisely because a reordering is a silent behavior change that no type checker will catch. `temprs's` enormous generated flag battery is the brute-force version of the same instinct: 4,449 machine- emitted cases asserting that each flag spelling is recognized as the operation it names. That is inventory rather than insight, and the chapter says so, but for a 38-operation surface where every operation takes an index that might be negative, mechanical coverage of the recognition table is the correct thing to automate.

The general rule the fleet encodes is that a compatibility parser needs its tests written against the *transformation*, not the outcome. Asserting that `lsf -i 4TCP:22` lists the right sockets is a test of the whole program; asserting that it normalizes to the expected clap line is a test of the compatibility layer, and only the second one fails in a way that tells you where to look.

The output contract — TTY-awareness, ANSI, and the cyberpunk chrome

Every tool in this book is a filter in the Unix sense — `argv` and `stdin` in, bytes out — and the single convention that lets eight independently-authored programs compose into one pipeline is not a framework or a shared crate. It is an agreement about output: colour and interactive chrome are gated on whether the relevant descriptor is a terminal, and they vanish the moment output is piped. The shape-of-the-fleet chapter stated the rule; this chapter is the mechanism behind it, because the mechanism is where the rule is either honored precisely or broken silently, and a tool that leaks an escape code into a `jq` pipeline is not composable no matter how good its colours look on a terminal.

The `isatty` gate, eight ways

The gate is the same idea everywhere — ask the OS whether a descriptor is a TTY, and branch — but the tools reach for it through four different surfaces, and the differences are instructive.

- `grc.rs` initialises its colour default straight from the C call: `colour = libc::isatty(1) == 1` (`src/grc.rs:47`), and `--colour=auto` re-evaluates the same `isatty` check, so colour is on to a terminal and off to a pipe unless forced with `--colour=on/--colour=off`. It is the most literal reproduction because it is porting `grc`'s own `isatty`-driven default.
- `temprs` branches on the Rust `IsTerminal` trait at both ends of the pipeline: `input()` reads `stdin().is_terminal()` so `CMD | tp` reads the pipe while a bare `tp` at a TTY waits for typed input, and `output()` mirrors it with `stdout().is_terminal()`, choosing the boxed cyberpunk renderer for a terminal and a plain byte-faithful `print!` for a pipe. It then *memoizes* the stdout check in a `thread_local! Cell<Option<bool>>` (`is_tty()` in `utils.rs`) so the syscall is paid at most once per run rather than once per rendered chunk.
- `lsOfrs` gates the whole columnar-output styling on the check (`main.rs:57`): off a TTY it drops its colours and cyberpunk headers and prints plain, script-safe text, and — importantly — its live modes never enter the alternate screen non-interactively; they print one snapshot and exit. The `--color` flag forces `always/never/auto` over the auto-detected default.
- `zcolorizer` auto-disables colour off a TTY unless `-C/--force-color` forces it (the path for piping into `less -R`), and `--no-color` forces passthrough. `nmap.rs`'s styled help suppresses all escapes on a non-TTY or when `NO_COLOR` is set.

The `NO_COLOR` environment convention appears in `nmap.rs` explicitly, and the pattern across the fleet is the union of three signals: is the descriptor a TTY, did the user force colour on or off, and is `NO_COLOR` set. Any one of them can turn colour off; only an explicit force turns it on against a pipe.

Two output paths from one renderer

The tools that draw decorated output do not have a “colour” path and a separate “plain” path that can drift apart — they have one renderer that collapses. `temprs`'s `cyber_*` family in `utils.rs` is the clearest specimen. To a terminal, `cyber_print_content` frames a payload in a `┌ ... ┐` box, `cyber_hr` draws a cyan

 rule, and indexed listings print `[NN]` markers in yellow with `@tag` names in a second colour. To a pipe, every one of those routines degrades to plain, byte-faithful text: `cyber_path` prints just the path, `cyber_hr` prints eighty dashes, `cyber_print_content` becomes a bare `print!`. That single-source split is what makes `tp -o 1 | jq` safe — the machine-consuming path is not a re-implementation that might forget an escape, it is the same function with the decoration compiled out by a branch. `grc`'s colouriser carries the analogous guarantee at the byte level: `grcat` emits a colour escape only when it changes from the previous character's escape (run-length collapsing the ANSI codes) and re-emits the character's *original* bytes, so an uncoloured token produces no escape at all and a non-UTF-8 byte survives round-trip.

Structured output for the pipeline that wants data, not colour

TTY-awareness handles the human-versus-pipe axis, but two tools go further and emit structured data specifically for downstream tooling, which is a different contract: not “plain text when piped” but “machine-parseable format on request.” `iftoprs --json` (`run_json_mode`, `main.rs:762`) stands up the same capture and proc-lookup threads, skips the TUI entirely, and emits one JSON object per line — NDJSON, one `JsonSnapshot` per refresh with an RFC-3339 timestamp, flow count, cumulative totals, peaks, and a `flows` array, with `process_name/pid` omitted when unknown via `skip_serializing_if`. `nmaprs` writes Nmap-compatible XML (`-oX`, an `<nmaprun>` root with the full element set) and grepable (`-o6`) output so existing XML- and grep-consuming tooling has a chance of parsing it. `lsofrs` adds `-J/--json` (serde array) and `--csv` (RFC 4180) alongside its lsof field-output mode (`-F`), and `temprs`'s `--path INDEX` exists purely to hand a backing file path to another program (`cat "${tp --path 1}"`). The pipeline stages the book demonstrates fall directly out of these:

```
sudo iftoprs --json | jq -c '.flows[] | {dst, sent_2s}'
kill "${lsf -t -c nginx}"          # bare PIDs feed kill
ps aux | tp -w procs               # push tagged onto the stack
tp -g python procs && echo hit     # grep the entry, compose in &&
journalctl -f | zcolorizer        # repaint a live stream
```

The cyberpunk chrome, and where it is not allowed to go

The house visual style — neon 256-colour palettes, `[ SYSTEM://DISK_MATRIX ]` and `NMAPRS // GRID SCANNER` chrome, figlet/ANSI-Shadow wordmarks with cyan-to-magenta truecolor gradients, NOMINAL/WARNING/CRITICAL badges — is real and shared, and the tools that draw help screens by hand (`nmaprs`'s `help_tp`, `zcolorizer`'s hand-drawn banner, `temprs`'s `CYBERPUNK_TEMPLATE`, `htoprs`'s figlet help, `storageshower`'s hand-formatted `--help`) all disable clap's default renderer to run it. But the chrome is bounded by the same TTY gate as everything else, and this is the discipline that keeps it from being a liability: `nmaprs`'s styled help suppresses every escape on a non-TTY, `zcolorizer`'s banner is only drawn on the interactive path, and every tool's error output drops to the terse `<tool>: <thing>: <reason>` shape on stderr regardless of chrome. `grc` even reproduces the upstream's error *formatting* specifically — `grc: <command>: <reason>` where `<reason>` is the bare OS string pulled through `libc::strerror` (`src/grc.rs:410`) to match the reference `%s: %s` rather than Rust's default `io::Error` display. The chrome lives on the human path and nowhere else; the moment a descriptor is a pipe, all eight tools become quiet, plain, script-safe filters, which is the only reason the aesthetic is affordable at all.

Errors, exit status, and the second channel

Colour policy governs stdout. Errors are a separate contract on a separate descriptor, and the fleet's convention there is narrower than the colour one and enforced by nothing but consistency: a diagnostic goes to stderr, in the shape `<tool>: <thing>: <reason>`, with no chrome, no suggestions, and no “for help, run `—help`” trailer. The reason to write it that way is that stderr is the one stream that survives every pipeline the user builds — `cmd | jq, cmd > file, cmd | grcat conf.log` — so it is the only place a tool can say something and be confident it was seen without corrupting what a downstream program is parsing.

`grcrs` is the tool that treats this most literally, because its upstream's error text is part of the observable behaviour it ports. It formats failures as `grc: <command>: <reason>` where `<reason>` is the bare OS string pulled through `libc::strerror`, rather than Rust's default `io::Error` display, so a script matching on the reference tool's stderr keeps matching. That is a small thing that reveals a large one: for a port, the error text is API. Anyone who has written a shell script with `2>&1 | grep 'No such file'` in it has depended on a string a C author picked, and a rewrite that improves the wording breaks the script as surely as a renamed flag would.

Exit status is the other half of the second channel and it is thinner across the fleet than the stderr shape is. `lsOfrs` publishes a real contract in its man page — `0` on success, `1` on error or no match — which is the lsof-shaped meaning where “found nothing” is a *failure* because the caller was asking a question. `temprs's -g PATTERN` deliberately exits `0` or `1` on the same grep-shaped logic, which is what makes `tp -g python procs && echo hit` a legal sentence in a shell rather than a construction that needs an `if` and a subshell. Those two are the ones the book documents precisely, and the honest statement about the rest is that their status conventions are the ordinary zero-on-success shape without a documented enumeration of failure codes. A tool whose exit status is load-bearing to a pipeline should say so in its man page; where the book does not report one, the reader should not assume a rich taxonomy.

What run-length escape collapsing actually guarantees

`grcat` emits a colour escape only when the escape changes from the previous character's, and re-emits each character's original bytes. Stated that way it sounds like an output-size optimisation, and it is one, but the load-bearing property is different and worth stating precisely, because it is what makes the colouriser safe to put in the middle of a pipeline that also contains non-colour-aware programs.

The guarantee is that the byte stream is the input's bytes with escape sequences *interleaved*, never substituted. An uncoloured token produces no escape at all, which means passing a file through `grcat` with a colourfile that matches nothing yields the file back byte for byte. A byte that is not valid UTF-8 survives, because the colouriser reasons in bytes and re-emits the original bytes rather than a replacement character — the behaviour `grcrs` pins with the invalid-UTF-8 round-trip tests that reproduce upstream's `surrogateescape` handling. And a CR that upstream strips is stripped identically, because the line-ending handling is part of what is being reproduced rather than part of what is being improved.

The reason to insist on this is that the alternative — decode to a string, restyle, re-encode — is the obvious implementation and it is lossy in three distinct ways at once. It corrupts invalid UTF-8, it normalises line endings, and it makes the output a *rendering* of the input rather than the input plus decoration. Once the output is a rendering, `cmd | grcat conf.log | grep foo` is no longer guaranteed to find what `cmd | grep foo` would find, and the colouriser has stopped being a filter and become a formatter. The distinction is invisible until the day it is not.

Structured output is an interface, with the obligations of one

`--json`, `-oX`, `-J`, `--csv`, and `-F` are a different promise from “plain text when piped.” Plain-when-piped is a courtesy; a structured format is an interface that something else is written against, and the fleet’s structured outputs divide into two kinds with different obligations.

The first kind is *someone else’s format*, and the obligation is fidelity. `nmaprs`’s `-oX` writes an `<nmaprun>` document with the Nmap element set, and `-oG` writes grepable output, for one reason: the existing corpus of XML-consuming and grep-consuming tooling. The value of that output is entirely in how closely it matches what the parsers on the other end already expect, so the format has no room for improvement in it — a better-designed XML schema would be worth exactly nothing. `lsofrs`’s `-F` field output is the same situation at a smaller scale, down to the `0t<offset>` encoding and the `TST=<state>` field prefix, and `-0` exists for the same reason it exists in `lsof`: so a consumer can split on NUL and be immune to filenames containing newlines.

The second kind is *the tool’s own format*, and the obligation is stability plus a shape that survives streaming. `iftoprs --json` is the cleanest example: it emits NDJSON, one `JsonSnapshot` object per refresh, rather than one enormous JSON array closed at exit. That choice is what makes `sudo iftoprs --json | jq` work on a live capture — a consumer sees a complete, parseable value every refresh instead of waiting for a document that only terminates when the tool does. The same design carries a smaller decision worth copying: `process_name` and `pid` are omitted via `skip_serializing_if` when unknown rather than emitted as `null` or as an empty string, so a consumer distinguishes “no process attributed” from “process named empty string” without a sentinel convention. `lsofrs`’s `-J` and `--csv` sit in the same category, and `temprrs`’s `--path INDEX` is the degenerate case of it — a structured output consisting of exactly one field, existing so that `cat "$(tp --path 1)"` hands a path to a program that knows nothing about stacks.

The common obligation across both kinds is that structured output must not be gated on the TTY check. Colour is; format is not. A tool that quietly changed its JSON shape when connected to a terminal would be unusable interactively for exactly the debugging case where a human wants to look at what a script will see, and none of the tools here does it.

The theme engine – the 31-palette lineage and `shift_lighter`

The shape-of-the-fleet chapter called the shared theme set “a genuine shared component, not merely a shared aesthetic,” and traced its path: the 256-colour palette set originates in `storageshower`, is carried into `iftoprs`, `lsofrs`, and `htoprs`, and is inherited explicitly by `zcolorizer`. That is worth a chapter of its own, because it is the clearest example in the whole book of code — not just convention — moving between tools, and because the details of *how* thirty-odd palettes stay consistent across five independently-shipped binaries reveal a small, sharp piece of engineering that recurs verbatim: `shift_lighter`.

One palette set, five consumers, an enum-length invariant

The palette set is a fixed roster of named 256-colour schemes running from `NeonSprawl` through `Zaibatsu`. The count is not identical across the tools, and the differences are meaningful rather than sloppy:

Tool	Palettes	Notes
<code>storageshower</code>	30 <code>ColorMode</code> variants	the origin; display names “Neon Sprawl”...“Zaibatsu”
<code>iftoprs</code>	31	adds the parity palette <code>Iftopcolor</code>
<code>lsofrs</code>	31	<code>ThemeName::ALL.len() == 31</code> , asserted (<code>theme.rs:860</code>)
<code>htoprs</code>	31 six-colour palettes	applied at the <code>Ncurses::to_color</code> choke point
<code>zcolorizer</code>	31 generated + 1 hand-written	<code>ccze-classic</code> added for parity → 32 total

The important discipline here is not the count but the *assertion* of the count. `lsofrs` pins `ThemeName::ALL.len() == 31` with a compile-adjacent test, and `iftoprs` verifies `ThemeName::ALL.len() == 31` (`theme.rs:335`); `zcolorizer` confirms 32 at runtime via `--list-themes | wc -l`. A palette roster that is supposed to match across five tools is exactly the kind of thing that drifts silently — someone adds a scheme to one binary and forgets the others — so the tools that carry the set assert its length rather than trusting it, which is the same posture the parity chapters take toward test counts and the port gate takes toward function names.

The six-colour model

Every palette in the set is six 256-colour indices, and the six slots carry a fixed meaning that lets one palette drive an entire UI. `zcolorizer` names them most precisely — a `Palette` is a kebab id, a display name, and six indices `[c1..c6]` standing for *primary*, *brightest accent*, *secondary*, *mid*, *dim*, *darkest*. `storageshower`’s

custom-theme struct spells the same six as named channels (`blue`, `green`, `purple`, `light_purple`, `royal`, `dark_purple`), and its theme editor is a six-slot picker over the 256-colour palette seeded from the current effective palette via `palette_for_prefs`. `iftoprs`'s `CustomThemeColors` is six palette slots; `htoprs`'s are six-colour palettes; `lssofrs`'s theme editor builds custom six-colour palettes and writes them to `~/.lssofrs.conf`. Six is the shared width because it is the smallest number that gives a UI a primary, two accents, and a three-step dim ramp — enough to colour a bar, a header, a selected row, and a muted secondary without the palette becoming a full 16-colour theme the user has to design.

shift_lighter — the function that moves with the palettes

The single piece of code that most clearly *is* shared rather than merely resembling a shared idea is `shift_lighter`, an index-cube arithmetic helper that steps a 256-colour index one notch brighter, clamping at the cube and grayscale-ramp edges. The book records its migration explicitly: it “moves with the palettes from `iftoprs` into `zcolorizer`.” Its purpose is collision avoidance. When a palette assigns two logically-distinct tokens to the same slot — `zcolorizer`'s `process` versus `host`, both landing on the secondary colour — rendering them identically would lose information, so `theme_from_palette` derives an extra distinguishable shade by pushing one of them one step brighter through `shift_lighter` rather than by adding a seventh palette colour. The theme tests pin the outcome: `zcolorizer` asserts `process` and `host` never render identically across all 31 generated palettes, and that `error` is bold in every one. The elegance is that the palette stays six colours wide — the extra shades are *computed* from the six at render time, not stored — so a user editing six slots still controls the whole look, and the derived shades track their parents automatically.

Generated, not hand-authored

The reason a six-colour palette can style sixty tokens is that the mapping from palette to token-styling is a function, not a table. `zcolorizer`'s `theme_from_palette` is the fully-worked example: it assigns each token to a palette role and multiplies the six colours into enough distinct looks by layering bold, italic, and underline — `date` is bold-accent, `time` plain-accent, `host` bold-secondary, `ip` plain-secondary, `uri` underlined-secondary, and the severity tokens lean on the brightest accent so `error` stays loud in every palette. Thirty-one of `zcolorizer`'s thirty-two themes are generated this way from the ported palettes; only `ccze-classic` is hand-written, a 16-colour map approximating the original ccze look (cyan dates, bold-red errors, bold-yellow warnings) for parity and nostalgia. The same generate-don't-enumerate principle is why `storageshower` can ship 30 builtins while checking only the first ten into `themes/` as ready-to-copy TOML — the rest are computed from their `ColorMode` variant, and a `next()` method that indexes into `ColorMode::ALL` and wraps modulo its length drives in-app cycling with the display-name-uniqueness and clean-wrap asserted by tests.

Persistence, and forward-compatible config

A theme is only useful if it survives a restart, and the fleet's persistence story is uniform with two honest exceptions covered in the packaging chapter. The three TUI tools that keep user preferences auto-save the selected theme (and every other toggle) to a TOML dotfile on every in-app change with no explicit save step — `~/.lssofrs.conf`, `~/.iftoprs.conf`, `~/.storageshower.conf` — while `htoprs` stores JSON under `~/.config/htoprs/prefs.json` and `zcolorizer` uses TOML under XDG at `$XDG_CONFIG_HOME/zcolorizer/config.toml`. The load path is where the forward-compat guard lives: every persisted field carries `#[serde(default)]`, so a config written by an older build that is missing a newer key still loads, the missing field falling back to its default rather than erroring. `lssofrs`'s `load()` goes further and silently returns defaults on any parse error at all (`config.rs:88`), and its `save()` is compiled out under `#[cfg(test)]` so the test suite never touches the user's real config. For a file the tool rewrites itself on every toggle, tolerant loading is not laxness — it is the only behavior that lets the palette roster and the config schema evolve without stranding a user on a config their new binary refuses to read.

Why 256 colours and not truecolor

Every palette in the shared set is six *256-colour indices*, not six RGB triples, and on a modern terminal that looks like a needless constraint. The constraint is deliberate and it buys three things that are hard to get any other way.

The first is that a 256-colour index is a value the terminal owns, not a value the application dictates. When a user has themed their terminal — and every user of a tool like this has — the low sixteen indices resolve to *their* palette, which means a tool's "dim" reads as dim in their scheme rather than as a fixed grey that fights it. Emitting truecolor overrides that relationship in both directions: it looks identical everywhere, including on the terminals where identical is wrong.

The second is arithmetic. The 256-colour space has structure — sixteen system colours, a 6×6×6 cube, and a 24-step grayscale ramp — and that structure is what makes `shift_lighter` possible at all. Stepping an index one notch brighter with clamping at the cube and ramp edges is a small closed-form operation on an integer. The truecolor equivalent is colour-space work: to lighten an sRGB triple correctly you convert out of gamma, adjust luminance, convert back, and clamp per channel, and you have imported a colour library into a tool whose dependency floor is `libc` and `clap`. The six-index model keeps a derived shade to a few lines of integer arithmetic that any reader of the source can verify by hand.

The third is that indices are trivially serialisable and trivially editable. A custom theme in `~/.storageshower.conf` is six integers; a user editing it by hand is picking from a fixed, enumerable, well-documented set rather than typing hex and hoping. And an index survives a round trip through a config file, a completion, a `--export-theme` dump, and a GUI picker fed by `--themes-json` without any risk of a rounding difference between the writer and the reader.

The cost is real and worth naming: six indices cannot express a gradient, so the bar styles that want one (`storageshower's -b gradient`) are composing from the palette's existing steps rather than interpolating freely, and two palettes that are conceptually distinct can collide on a shared index in a way two RGB triples would not. `shift_lighter` exists because of exactly that collision case. The trade the fleet made is fewer expressible colours in exchange for integer arithmetic, terminal-respecting resolution, and a config format a human can edit — which is the right trade for a terminal tool and the wrong one for a graphics application, and the fleet is not a graphics application.

The editor is the harder half

Shipping thirty-one palettes is a data problem. Letting a user build a thirty-second one, live, inside a running TUI is a UI problem, and it is the part of the theme system that actually needed designing. Three tools carry a six-slot editor bound to `l` (`lsofrs`, `iftoprs`, `storageshower`) and `htoprs` carries one at `~`, and the shape they converge on answers four questions that any live editor has to answer.

What does the editor start from? Not a blank slate. `storageshower` seeds the editor from the current effective palette via `palette_for_prefs`, which makes the first interaction a tweak rather than a construction — the user is dragging one slot away from a scheme they already chose, not assembling six colours from nothing.

What is the preview? The application itself. Because the six slots feed the same `theme_from_palette`-style derivation the built-in palettes feed, changing slot three repaints every element that derives from slot three at once, in situ, with real data underneath it. This is the property that makes a six-slot model worth keeping even where a larger model would be more expressive: a six-wide palette is small enough that a user can hold the whole mapping in their head while editing it, and large enough to drive a full UI.

When is it saved? Immediately, with no save step. Every one of the persisting tools writes the config on every in-app toggle, which is the right default for a preference file — the failure mode of forgetting to save is worse than the failure mode of an accidental change you can toggle back — and it is why the load path has to be as forgiving as it is.

What happens to the built-ins? They stay. A custom theme is an entry in a named map (`[custom_themes.neonpink]`) plus an `active_theme` pointer, not a mutation of a built-in, so the thirty-one shipped palettes are immutable and a user can always get back to a known state by pointing `active_theme` elsewhere. That structure is also what lets the enum-length assertion mean something: the built-in roster is a fixed array in the binary that user configuration cannot extend or shorten, so `ThemeName::ALL.len() == 31` is a statement about the program rather than about the machine it is running on.

A shared roster is a distributed invariant

The theme set is the only piece of real shared *state* in a fleet of otherwise independent binaries, and it is shared by copying rather than by a common crate. That is worth being explicit about, because copying is how most cross-project sharing actually happens and the fleet's handling of it is the interesting part. There is no `menketech-theme` crate that five tools depend on. The palettes were authored in `storageshower`, carried into `iftoprs`, `lsofrs`, and `htoprs`, and inherited by `zcolorizer` with a source comment naming its provenance explicitly — “ported verbatim from the shared MenkeTechnologies theme set (`iftoprs`, itself derived from `storageshower`).”

Copying has a well-known failure mode: one copy gains a palette and the others do not, and the roster silently diverges. The fleet's response is not to centralise but to assert. `lsofrs` pins `ThemeName::ALL.len() == 31`, `iftoprs` pins the same, `zcolorizer` confirms 32 at runtime through `--list-themes`, and `zcolorizer`'s theme tests additionally assert semantic properties that must hold across all 31 generated palettes at once — `process` and `host` never render identically, `error` is bold in every one. Those assertions do not prevent divergence; they make divergence *loud in the tool where it happens*, which for a copied component is the achievable goal. Adding a palette to `iftoprs` and forgetting `lsofrs` does not break `lsofrs`; it breaks the next person's expectation, and the count assertion is what turns that expectation into something a test failure can carry.

The reason a shared crate was not the answer is the dependency floor. Five binaries taking a dependency on a sixth crate purely to share thirty-one arrays of six integers imports version skew, a release-ordering constraint, and a publishing obligation, in exchange for eliminating a class of drift that costs one assertion to detect. For a component this small and this stable, the assertion is cheaper than the coupling. That calculus would invert immediately if the shared thing were logic rather than data — and note that the one piece of shared *logic*, `shift_lighter`, is a handful of lines of integer arithmetic whose correctness a reader can check locally in either copy.

Talking to the kernel — procfs, libproc, pcap, raw packets, sysctl

The system tools in this book do not learn about the machine by running other programs. That is a single design value stated once in the shape-of-the-fleet chapter — the anti-subprocess discipline — and it is worth a chapter because it is where the ports do their least glamorous and most load-bearing work: reading the kernel directly, per operating system, by hand, from the same interfaces the C originals read. The payoff of doing it this way is a single self-contained binary with no runtime dependency on `lsof`, `ss`, `netstat`, or `diskutil` being installed and on `PATH`; the cost is that each tool carries a per-OS backend tree and hand-parses kernel data structures that a shell-out would have let it treat as opaque text. This chapter reads those backends across the grain.

The Linux path: hand-parsed procfs, no crate

Everywhere Linux appears in the fleet, the data comes from the `/proc` and `/sys` pseudo-file systems, parsed by hand against the files — and, notably, **none of the tools uses the `procfs` crate**. The parsing is done directly:

- `lsofrs` walks `/proc/<pid>/fd` for numbered descriptors and reads the `cwd`, `root`, and `exe` symlinks into the `cwd/rtd/txt` pseudo-descriptors `lsof` users expect, correlating sockets through a prebuilt socket map keyed by inode. Because `/proc/net/tcp` exposes queue occupancy but not the socket buffer limits, `send_buf_size/recv_buf_size` stay `None` on Linux — a gap the stall classifier degrades around rather than fabricating.
- `iftoprs` parses `/proc/net/tcp`, `/proc/net/tcp6`, `/proc/net/udp`, and `/proc/net/udp6` for the inode → port map, then walks `/proc/[pid]/fd/` for inode → PID and reads `/proc/[pid]/comm` for the process name (`procinfo.rs:127`) — the whole process-attribution table built without a subprocess, the `Cargo.toml` comment noting this compiled-in path replaces a runtime `lsof`.
- `storageshower` reads `/proc/mounts` for the mount table, `/proc/diskstats` for sector counts (mapping devices to mounts via `mount_to_device_map`), and `/sys/block/*/device/state` for SMART-style health.
- `htoprs` carries the full Linux backend `htop` carries (sixteen modules), reading the kernel through three pure-Rust, no-FFI crates rather than C libraries: `libmedium` for hwmon sensors, `neli` speaking generic-netlink for the TASKSTATS delay-accounting path (`linux/LibNL.c`), and `zbus` speaking D-Bus directly for the `SystemdMeter` (`linux/SystemdMeter.c`). Both `neli` and `zbus` speak their wire protocols directly over a socket rather than linking a C library — a deliberate choice to keep the build free of C dependencies and vendorable long-term.

The through-line is that `procfs` is a text (and symlink) interface, and the fleet treats it as one it parses itself. There is no shared `procfs` abstraction; each tool reads exactly the files it needs, which keeps the parsing auditable against the kernel's documented `/proc` format rather than against a crate's model of it.

The macOS path: direct libproc and sysctl FFI

macOS has no `/proc`, so the same data comes from `libproc` and `sysctl`, declared as raw FFI and called against `repr(C)` structs matched to the kernel headers:

- `lsoprof` declares `proc_listpids` and `proc_pidinfo` as raw `extern "C"` and calls them twice — first to size the PID buffer, then to fill it — with no shelling out and no parsing of another tool's output. Because macOS exposes socket send/rcv buffer *limits* alongside occupancy, the Darwin path fills the `send_buf_size/rcv_buf_size` fields that the stall classifier needs (which is why `--stall` uses a fill-ratio rule on macOS and a trend rule on Linux).
- `iftoprs` uses the `libproc` crate to read each process's open sockets via `pidfdinfo::<SocketFDInfo>`, selecting the correct arm of the `soi_proto` union by `soi_kind` (TCP vs datagram), converting the network-order `insi_lport` with a local `ntohs`, and skipping anything that is not TCP(6)/UDP(17) or has a zero local port (`procinfo.rs:47–124`).
- `storageshower` calls `getmntinfo(&mut mntbuf, MNT_NOWAIT)` through `libc`, walks the returned `statfs` array to compute `total/avail/used` with saturating subtraction, and reads IOKit's `IOBlockStorageDriver` byte counters for I/O rates; SMART status on macOS is the one place it does shell out (`diskutil info`, reading the `SMART Status` field), which the chapter names rather than hides.
- `htoprs` carries the Darwin backend (`darwinmachine.rs`, `darwinprocess.rs`, `darwinprocesstable.rs`) and uses `libc::sysctlbyname` with byte-string constants spelled out including their NUL (`b"kern.osrelease\0"`, one of the sixteen documented clippy-allow entries) so the literal matches the C source exactly.

The `repr(C)` struct-matching is the discipline that makes this safe: the FFI is not “call a syscall and hope,” it is a struct laid out to match the kernel header, filled by the kernel, and read field by field — the same contract the C original relies on, transcribed rather than reinvented.

The wire: pcap and raw packets, and graceful privilege degradation

Two tools go below the process table to the network itself, and they are the only two that need elevated privilege, so they are also the two that degrade most carefully when they do not have it.

`iftoprs` is the only tool that uses `pcap`. It opens the interface with a 256-byte snaplen — enough for L2–L4 headers, never payload, so it caps the copy — and decodes four real link types (`DLT_EN10MB` Ethernet, `DLT_NULL` BSD loopback, `DLT_LINUX_SLL` Linux cooked capture, `DLT_RAW` raw IP) rather than assuming Ethernet, pushing decoded packets across a `tokio::sync::mpsc` channel off the capture thread. Its capture loop is resilience-first: a device-open failure or a non-timeout pcap error is treated as transient, retried with exponential backoff from 250 ms capped at 30 s, resetting on any successful reopen, with `TimeoutExpired` treated as “continue” rather than an error and the receiver being dropped as the only clean exit.

`nmaprs` is the only tool that constructs raw packets, via `pnet/pnet_sys`, and the only one with a genuinely async core (`tokio` with `full`). It detects `geteuid()` and silently falls back from SYN to TCP-connect scan when unprivileged; `--privileged` forces raw mode and `--unprivileged` forces connect mode, rejecting raw-only techniques (`--s0`, `--sI`, SCTP) with an error rather than a wrong answer. Its raw paths run a dedicated rcv thread per address family, register the reply key in a `DashMap` *before* each send to avoid a rcv race, and shard work across up to 16 independent pipelines per family. Interestingly — a point the connect-scan chapter makes — `nmaprs`'s *default* connect scan does not use async sockets at all but a worker pool of OS threads running blocking `connect_timeout`, because the blocking probe is three syscalls (`socket + connect + close`) where the async path adds an `ioctl(FIONBIO)` and a reactor registration, and the lock-free `UnsafeCell` result array lets each worker write its own slot exactly once with no contention.

Both privileged tools share the same posture: capability is detected, not assumed, and absence of privilege produces a lesser-but-correct result (connect scan instead of SYN, retry instead of crash) rather than an error or a fabricated answer.

Bounded memory and poison tolerance

Two properties recur across the kernel-facing backends because reading a live kernel means reading an unbounded, mutating source. The first is a fixed memory ceiling: `iftoprs`'s accounting model is a 40-slot ring buffer per flow (two `VecDeque<u64>`, sent and received, capped at `HISTORY_SLOTS = 40`) with 60-second idle eviction, so history cannot grow with connection lifetime and idle flows age out of the table; `storageshower`'s directory sizer walks with `read_dir` and `saturating_add` and refuses to recurse through symlinks (a symlink counts as its own `symlink_metadata().len()`), which is what stops a cycle from driving runaway — pinned by `scan_directory_symlink_cycle_completes_with_finite_sizes`. The second is poison tolerance: `iftoprs`'s shared `FlowTracker` locks its mutex everywhere with `lock().unwrap_or_else(|e| e.into_inner())` so a panic in one thread cannot wedge the tracker. Both properties exist because the data source never stops changing, and a tool that reads it on a background thread and renders it on another has to bound what it keeps and survive a partial failure on either side — which is exactly the shape every one of these backends takes.

What the pseudo-filesystem will not tell you

The instructive part of reading kernel interfaces by hand is not what they expose — it is what they do not, because that is where a tool has to choose between fabricating an answer, refusing to answer, and answering a different question. The fleet contains one worked example of each, and they are worth lining up.

The clean case is `lsufrs`'s socket buffer accounting. On macOS the kernel exposes both socket buffer *occupancy* and the buffer *limits*, so the `--stall` classifier can compute a fill ratio and say “this send buffer has been at or above eighty percent of its capacity across the last several samples and is not draining.” On Linux, `/proc/net/tcp` publishes the queue occupancy but not the limit, so `send_buf_size` and `recv_buf_size` are simply `None` on that platform. The three available responses are: guess a limit from a system default and compute a ratio anyway; refuse to classify on Linux; or answer a different, weaker question. `lsufrs` takes the third — on Linux the classifier switches to a trend rule, calling TX stalled when the queue is persistently non-zero and not draining, and RX starved when it is persistently non-zero and strictly growing. That is a genuinely different predicate with genuinely different false-positive behaviour, and the book states it as such rather than presenting one `--stall` that means the same thing everywhere. The failure this avoids is the worst kind in a diagnostic tool: a number that is present, plausible, comparable across machines, and derived from an assumption the user never sees.

The second case is a shell-out that survives review. `storageshower` reads its mount table, its capacity numbers, and its I/O counters through `getmntinfo` and IOKit on macOS with no subprocess, and then reads SMART status by running `diskutil info` and parsing the `SMART Status` field. That is a real exception to the anti-subprocess discipline, and the chapter names it rather than hiding it. The reason it is the right exception is worth extracting as a rule: the subprocess is not on the hot path, it is not in the render loop, its absence degrades one field rather than the tool, and the alternative is linking a privileged, version-volatile Apple framework to obtain a single string. A shell-out that costs a field's freshness is a different thing from a shell-out that costs the tool its self-containedness, and the distinction is exactly the one that justifies `iftoprs`'s manifest comment noting that its compiled-in `libproc` path *replaces* a runtime `lsuf` — there, the subprocess was per refresh, on the hot path, and load-bearing.

The third case is data that exists but is racing you. `/proc/<pid>/fd` is a directory whose entries can vanish between the `read_dir` and the `readlink`, and every PID in a list gathered a moment ago may be gone by the time it is opened. There is no way to take a consistent snapshot of a live process table through procsfs, and the only correct posture is to treat every per-PID read as independently fallible and to keep going. This is why the process-walking code in these tools has no global error path for “a PID disappeared” — a vanished process is not an error condition, it is the normal case, and a tool that aborted a scan because a short-lived process exited during it would be unusable on exactly the busy machine where you need it.

Counters, units, and the arithmetic of a live source

Kernel data comes as monotonically increasing counters far more often than as rates, and every tool that displays a rate is doing arithmetic the kernel did not do for it. That arithmetic is where units get lost, and the fleet contains a precise, documented example of what it costs to be exact about it.

`iftoprs`'s three columns are labelled the way `iftop`'s are — 2s, 10s, 40s — and the book states what they actually contain: *summed byte counts over trailing one-second slots, scaled to bits*, not slot-averaged bytes per second. Those are different quantities and the difference is not a rounding detail. A sum over a window and an average rate over a window differ by the window length, they respond differently to a partially-elapsed window at startup, and they differ again at the moment a flow is evicted. The chapter's decision to state the formula rather than call the columns "the same as `iftop`" is the honest version of a parity claim, and it is the version that lets a reader who is comparing two tools side by side account for a discrepancy instead of filing a bug.

The bounded-history structure underneath is the second half of the same story. A per-flow ring of forty one-second slots (two `VecDeque<u64>`, sent and received, capped at `HISTORY_SLOTS = 40`) is what makes the 40s column computable at all, and it fixes the memory cost of a flow at a constant regardless of how long the flow lives. Sixty-second idle eviction then bounds the *number* of flows. Both bounds are necessary and they bound different axes: without the ring, a long-lived connection's history grows without limit; without eviction, a busy host's flow table does. A tool reading a live source needs both, and the useful generalisation is that any accumulator over a kernel counter should have a stated retention policy on both axes before it ships.

Counter width and reset are the third arithmetic hazard, and the fleet's defensive posture toward it is visible in the small: `storageshower`'s directory sizing uses `saturating_add` rather than `+`, and its capacity computation uses saturating subtraction over the `statfs` fields. Neither is defending against a realistic overflow on a modern disk. Both are defending against the case where the kernel hands back a field combination that does not satisfy the invariant the arithmetic assumes — an `avail` larger than `total` on some filesystem type, a counter that reset under you — and the choice is between a panic in a render loop and a clamped, obviously-wrong number. In a TUI that redraws several times a second, the clamped number is the correct choice, because it is visible, survivable, and gone on the next refresh.

The backends a port carries and cannot test

`htoprs` carries the full per-OS backend tree that `htop` itself carries — `darwin/`, `linux/`, `freebsd/`, `openbsd/`, `netbsd/`, `dragonflybsd/`, `solaris/` — and this deserves its own note because it is the clearest case in the book of the port method producing something the porting effort cannot directly verify. The module count is high (141 ported modules) largely because of those seven trees, and the honest characterisation the appendix gives is that the count "is a property of faithfulness to the upstream's structure, not of feature sprawl."

The structural argument for porting a backend you cannot exercise is that the port's specification is the C source, not the running program, and the C source for `openbsd/OpenBSDProcessTable.c` exists whether or not an OpenBSD box does. Skipping it would mean the function-name gate's inventory no longer describes `htop`, the coverage report's denominator would be a subset chosen by hardware availability, and the day someone does build on OpenBSD the work would start from nothing. Porting it means the coverage figure means what it says and the structure stays isomorphic to the upstream, which is the entire premise of a 1:1 port directory.

The cost is equally clear and should not be minimised: a ported backend that has never been compiled against its target platform's headers, let alone run, is transcription rather than tested code. The parity harness cannot reach it — the byte-diff oracle runs the reference C `htop` and `htoprs` on the same machine, so it exercises exactly one backend per run. What the gate *does* guarantee about the untested trees is narrow and worth stating exactly: that every function in them corresponds to a real `htop` C function, that none is a `todo!()` masquerading as ported (the report generator counts a `todo!()` body as stubbed regardless of context), and that the module layout matches. That is a real guarantee about structure and a null guarantee about

behaviour, and the difference between those two is the difference between the port report's number and a claim of cross-platform support.

The comparison worth drawing is with `lsotrs`, which does not carry backends for platforms it does not support and instead prints an unsupported-platform notice to stderr and returns nothing outside macOS, Linux, and FreeBSD. Neither posture is wrong; they follow from different specifications. `lsotrs`'s spec is lsof's observable behaviour, so a platform it cannot produce that behaviour on is simply out of scope. `htotrs`'s spec is htop's C source, and the source covers seven platforms, so the port covers seven — with the parity apparatus honest about which one it actually checked.

The concurrency map – threads, channels, locks, and the process boundary

Eight tools, four concurrency substrates, and one of them is a filesystem. That sentence is the whole chapter in compressed form, and it is worth unpacking because the fleet's choices here are unusually varied for a set of programs built by one organisation under one set of house rules — which means the variation is carrying information. Nobody picked `rayon` for `lsd` and OS threads for `nmap`'s default scan and a single thread for `htop` out of habit. Each choice follows from what the tool is waiting on, and reading the four together is the fastest way to internalise a decision procedure that is usually learned by getting it wrong twice.

Four substrates and the question that selects between them

The selecting question is not “how much parallelism do I want.” It is *what is the work blocked on, and does the answer's ordering matter?* Cross the two axes and the fleet's four substrates fall out of the grid.

Data-parallel over a finite set, CPU- and syscall-bound, order irrelevant → `rayon`. `lsd` gathers open files across every process on the machine. The set of PIDs is enumerable up front, each PID's work is independent, none of it blocks on a network, and the output is sorted or grouped afterward anyway. That is precisely the shape a work-stealing data-parallel pool is built for, and it is why `rayon` appears in `lsd`'s manifest and in none of the other seven system tools' hot paths. The tell is the finite, up-front set: `rayon` is at its best when you can hand it a collection, and worst when work arrives over time.

Many concurrent waits on the network, order irrelevant, results collected → either an async reactor or a thread pool, and the choice is subtler than the folklore. `nmap` is the tool with a genuine reason to be async and it carries `tokio` with `full`, yet its *default* TCP-connect scan does not use async sockets. It runs a worker pool of OS threads doing blocking `connect_timeout`. The reasoning the chapter gives is a syscall count: a blocking probe is `socket + connect + close`, three syscalls, where the async path adds an `ioctl(FIONBIO)` to set non-blocking and a reactor registration on top of the same three. When the per-probe work is that small, the reactor's bookkeeping is a meaningful fraction of the probe, and the reactor's advantage — not burning a thread per outstanding wait — only starts paying when the number of outstanding waits exceeds what the OS scheduler handles comfortably. At `-T3`'s 256 workers it does not.

Where `nmap` does go async is where the work per unit stops being three syscalls: a SOCKS or HTTP-CONNECT proxy probe is a state machine with several round trips and a parser, and there the reactor earns its overhead. So the tool carries both and picks per technique. This is the single most transferable piece of engineering judgment in the chapter, and it inverts the usual reflex: the async runtime is not the default that blocking calls are an exception to. The blocking call is the default, and async is what you reach for when the per-unit work is large enough to amortise a registration.

Continuous arrival from an external source, order matters within a stream → dedicated OS threads plus channels. `iftop` has work that does not arrive as a collection: packets show up when they show up, forever. It runs a dedicated `packet-capture` thread and a `proc-lookup` thread, pushes decoded packets across a `tokio::sync::mpsc` channel off the capture thread, and keeps the render loop on a 33 ms tick that never does expensive work inline. `storageshow` has the same shape with a slower clock: a `spawn_bg_collector` thread refreshes disks and system statistics on a three-second cadence entirely independent of the user-facing refresh rate, while the UI polls `crossterm` with a 200 ms timeout. In both cases the structural

point is the same and it is a UI point, not a throughput point — collection cadence and render cadence are decoupled, so the cost of collection cannot make the interface feel slow.

Strictly sequential work → one thread, and say so. `htoprs` draws through a single render loop because `htop` does, and its correctness criterion is byte-identical output to a program with one thread of control. `grcis` and `zcolorizer` read stdin line-buffered on one thread because a colouriser's job is a stream transform where output order *is* the semantics; parallelising it would require reordering machinery whose only payoff would be on an input that a pipe is already rate-limiting. `temprs` is single-threaded because its work is a few file operations. Three of the eight tools are the ones that decided not to, and that is the correct proportion for a fleet of Unix filters.

Shared state, and the two locks that matter

Threads that share data need a discipline for it, and the fleet's two interesting cases are opposite in almost every respect.

`iftoprs`'s `FlowTracker` is the in-process case: an `Arc<Mutex<...>>` written by the capture and proc-lookup threads and read by the render loop. The detail that makes it worth studying is how it locks — `lock().unwrap_or_else(|e| e.into_inner())`, everywhere. Rust's mutex poisons on a panic-while-held, and the default `unwrap()` turns any panic in any thread into a cascading panic in every thread that subsequently touches the shared state. For a bandwidth monitor, the correct posture is the opposite: a panic in packet decoding of one malformed frame should not take down the display of every other flow. Recovering the inner value says, explicitly, that the invariant this mutex protects is a *statistics table* whose partial corruption is survivable and self-correcting on the next refresh — which is true here and would be false for, say, an allocator's free list. The generalisation is that poison-recovery is not a shortcut around handling errors; it is a claim about the data, and it is legitimate exactly when the claim is.

`temprs`'s lock is the cross-process case and it is the more unusual one, because `temprs` has no threads at all. Its concurrency is several *shells* mutating one stack at the same moment. `TempApp::new()` creates `temprs-stack.lock` and takes an exclusive `fs2` advisory lock on it, held for the whole program run, so two shells racing to push serialise cleanly. Underneath that, the master record is written to `temprs-stack.tmp` and `renamed` over `temprs-stack`, which on a POSIX filesystem is atomic within a directory, so a reader sees either the complete old record or the complete new one and never a torn write.

The pairing is the interesting part. The lock is what makes concurrent *mutation* correct; the atomic rename is what makes concurrent *reading* correct without the reader taking the lock at all. That second property is what lets the `zsh` completion read the master file directly at Tab time — completion is a read, the rename guarantees it sees a whole record, and requiring the completion to take an advisory lock would have made pressing Tab able to block on a `tp` invocation in another pane. Two mechanisms, two distinct guarantees, and the cheaper one covers the higher-frequency path.

Backpressure, or its deliberate absence

Every producer-consumer pair has to answer what happens when the consumer falls behind, and the fleet gives three different answers on purpose.

`iftoprs` answers with a bound on *retained data* rather than on the channel: the forty-slot per-flow ring and the sixty-second idle eviction mean that a render loop falling behind loses resolution, not memory. The oldest slot rolls off and the display shows a slightly staler window. That is the right answer for a monitor, where the newest data is the valuable data and there is no obligation to show every packet.

`zwire-host`'s bus answers with best-effort fan-out and a deliberate lock discipline: `publish` collects the target sinks under the broker lock and then writes to them *outside* it, so a slow subscriber cannot hold the broker while it drains. A publisher that is also a subscriber therefore cannot deadlock against itself, which is not a hypothetical for a pub/sub hub where a browser HUD, an editor, and a multiplexer widget are all connected

at once. Best-effort means a subscriber that cannot keep up loses messages rather than stalling the bus — the right trade for theme-sync and status topics, and a trade that would be wrong for a message queue with delivery obligations.

`zpwchrome-host`'s download manager answers by removing the channel entirely. There is no long-lived connection between the extension and a running download, because Chrome's one-shot native messaging spawns a fresh host process per message; a persistent in-memory queue would die with the process. So the coordination medium is a JSON state file per job under the cache directory, the worker polls flags in it between chunks, and every control action — pause, resume, cancel, remove — is a mutation of that file. Backpressure is not a concept here because there is no stream between the parties; there is a file, and both sides read it when they need to. Choosing a filesystem as the concurrency substrate is unusual enough that it looks like a workaround, and it is worth recognising as a legitimate design: it is the only substrate that survives both parties exiting.

The process boundary as a concurrency primitive

Two of the tools reach for a process boundary where a thread would have been the obvious choice, and both do it for isolation rather than parallelism.

`zpwchrome-host` spawns its download worker as a detached process — `zpwchrome-host --dl-worker <gid>` — with `setsid()` and every file descriptor `>= 3` closed before exec. The fd-closing is not hygiene, it is load-bearing: without it the worker inherits Chrome's native-messaging stdout pipe, Chrome observes the pipe staying open after the host exits, and reports "Native host has exited" even on a successful download. That is a failure mode with no analogue in a threaded design, and it is the price of the isolation — a download that survives the host process exiting, which is exactly the property the design needs.

`htoprs`'s crash handling is the other side of a boundary, this time between the program and the terminal. A TUI on the alternate screen with the tty in raw mode cannot report a panic to the user: the message is painted over the instant the screen is restored. So crashes are persisted to `~/.cache/htoprs/crash.log` from two funnels — a process-wide panic hook and the ported SIGSEGV/SIGBUS handler's stderr report — with time, version, pid, thread, source location, message, and an unconditionally-captured backtrace, appended so history survives repeated crashes, and the restored terminal prints one line saying where it went. Getting diagnostics *out of* a process whose output channel is a screen you are about to destroy is the same class of problem as getting a download to outlive the process that started it, and both are solved by putting the durable thing on the filesystem rather than in a stream.

Failure, degradation, and the discipline of saying no

The interesting behaviour of a systems tool is not what it does when everything is available. It is what it does when the kernel refuses, the device disappears, the config is corrupt, the peer never answers, or the caller asks for something that is not there. This chapter reads the fleet's failure behaviour across the grain, because the choices are consistent enough to constitute a doctrine and the doctrine is worth stating: prefer a lesser-but-correct answer to a fabricated one, prefer retrying a transient failure to aborting, prefer refusing a request you cannot satisfy to satisfying a different one silently, and never let a recoverable condition in one part of the data kill the rest of it.

Missing privilege: detect, degrade, and refuse the impossible

Two tools need elevated privilege and both treat its absence as a first-class state rather than an error.

`nmaprs` calls `geteuid()` and silently falls back from SYN half-open to TCP connect scan when unprivileged. The fallback is silent because it is the correct answer to the question the user asked — “scan these ports” — obtained a different way, with different observable characteristics on the wire and the same port state as the outcome. What the tool will *not* do is the corresponding thing for techniques that have no unprivileged analogue: `--unprivileged` combined with a raw-only technique (`--s0`, `--sI`, SCTP) is rejected with an error rather than quietly substituting something else. That pair of behaviours is the whole doctrine in miniature. Where a lesser-but-correct answer exists, produce it. Where it does not, refuse, because the alternative is a scan report whose method is not what the user believes it is — and a scan report is exactly the kind of artifact that gets pasted into a ticket and read six months later by someone who was not there.

`iftoprs` needs raw capture privilege and its non-privileged behaviour is governed by the same retry logic as any other device failure, described below. The important structural detail is that the privilege check is not a startup gate that either passes or aborts; capture failure is a runtime condition the loop is built to survive, which is what makes the tool usable in the case where an interface appears later or permissions change under it.

Transient failure: retry with a ceiling, and define the exit

`iftoprs`'s capture loop is the fleet's reference implementation of resilience to an external resource. A device-open failure or a non-timeout pcap error is treated as transient and retried with exponential backoff from 250 ms capped at 30 s, the backoff resetting on any successful reopen, with `TimeoutExpired` treated as “continue” rather than as an error, and the receiver being dropped as the only clean exit.

Four decisions are packed into that sentence and each is a place a simpler implementation goes wrong. The 250 ms floor keeps a transient failure — an interface bouncing during a DHCP renewal — from costing the user a visible gap. The 30 s ceiling keeps a persistent failure — an interface that was unplugged — from backing off to an interval so long the tool never notices it coming back. Resetting on success is what makes the backoff a response to the *current* failure rather than a running tally of every failure since launch, which is the bug you get by incrementing a counter that nothing decrements. And treating `TimeoutExpired` as a normal continuation

rather than an error is the difference between a capture loop that works on an idle interface and one that backs off to thirty seconds because nothing was talking.

The exit condition deserves the same attention. “The receiver being dropped is the only clean exit” means the capture thread’s lifetime is defined by its consumer, not by a shutdown flag it polls, not by a signal handler, and not by a timeout. When the UI goes away, the channel’s receive end drops, the send fails, and the thread ends. That is a single, unambiguous, race-free termination condition, and it is a considerably better design than a `should_stop: AtomicBool` that every loop in the program has to remember to check.

Corrupt or absent state: self-heal, do not wedge

Persisted state is the failure surface users actually hit, because it is the one that survives a restart. Three tools handle it and all three choose recovery over diagnosis.

`lssofrs`’s config `load()` reads `~/.lssofrs.conf` and silently returns defaults on *any* parse error. There is no error message, no rename-to-`.bak`, and no prompt. For a file the tool rewrites itself on every in-app toggle, this is correct: the file is derived state, not user-authored source, and the cost of losing it is one theme selection while the cost of refusing to start is the tool. The `#[serde(default)]` guard on every field handles the milder version of the same problem — a config written by an older build that lacks a newer key loads with that key at its default rather than erroring — which is what lets the palette roster and the config schema evolve without stranding a user on a configuration their new binary will not read.

`temprs`’s master record handles corruption at record granularity rather than file granularity, which is the more sophisticated version. Records that fail to parse — empty, or with an empty path field — are skipped on read with a warning to the log and quietly dropped on the next write, so a damaged master file self-heals into a valid one containing everything that was still readable. The alternative behaviours are both worse: refusing to read the file wedges a tool whose entire purpose is holding onto things you did not want to lose, and reading it strictly would let one bad record destroy a stack of good ones.

The third case runs the other direction, and it is the one place in the fleet where the answer is a flat refusal. `temprs --clear` deletes the working directory’s contents, and it refuses to run unless the directory’s basename contains `temprs`. `--expire HOURS` rejects a negative, NaN, or infinite argument *before* the float-to-integer cast rather than after, because the cast is where the nonsense becomes a plausible-looking number. `--replace` rejects an empty pattern. None of those is a recoverable condition — they are requests the tool declines — and the distinction from the config case is exactly the distinction that matters: recover from state you own, refuse instructions you cannot execute safely. A tool that recovers from a bad config and also “recovers” from `--clear` pointed at a home directory has not applied a principle, it has applied a habit.

Absent data: the third answer, and naming it

The subtlest failure is the one where everything worked and the answer is empty, because there are two very different reasons for an empty result: nothing matched, or the thing you asked about does not exist. Conflating them is the default behaviour of almost every filter, and `lssofrs` reproduces `lssof`’s solution to it. Under `-V`, after filtering, `unfound_specs` walks the positive `-p`, `-c`, `-u`, and file specs and prints a `no PID found / no command found / no user found / no file found` line to `stderr` for each spec that matched no process — skipping exclusions and command regexes, because “the negation matched nothing” and “the regex matched nothing” are not errors in the same sense. A directory argument counts as matched when any held file lives beneath it, so `-V /tmp` is satisfied by `/tmp/anything`.

The design worth extracting is that the report goes to `stderr` while the empty result goes to `stdout`. A script that pipes `stdout` gets an empty stream and a clean parse; a human watching the terminal gets told which of their four filters was the one that eliminated everything. That is the only arrangement that serves both consumers, and it costs one pass over the spec lists after the filter.

`lssofrs --stall` carries the parallel case for a live analyzer: only currently- seen stalled sockets are printed,

and a quiet system reports `No stalled sockets detected`. — an explicit statement rather than silence. Silence from a diagnostic tool is indistinguishable from the tool being broken, which is why the empty case gets a sentence.

Partial failure inside a live source

The last category is failure that affects part of the data while the rest stays valid, and it is where the poison-tolerant lock discipline from the concurrency chapter meets the bounded-memory discipline from the kernel chapter.

`storageshower`'s directory sizer is the cleanest example. It walks with `read_dir` and `saturating_add`, and it refuses to recurse through symlinks — a symlink counts as its own `symlink_metadata().len()` rather than as the size of what it points at. That single rule is what makes the walk terminate on a directory tree containing a cycle, and it is pinned by a test whose name states the property directly: `scan_directory_symlink_cycle_completes_with_finite_sizes`. Note what the rule is *not*: it is not cycle detection, there is no visited-inode set, and there is no depth limit. It is a structural choice — do not traverse the edge type that can form a cycle — which is cheaper than detection and cannot be defeated by a cycle the detector has not seen yet. The cost is that a symlinked directory's contents are not counted, which is also, arguably, the more useful answer for a disk-usage tool: the bytes live where the real directory is.

`nmaprs`'s recv-side race handling is the same instinct applied to a network. Its raw paths run a dedicated receive thread per address family and register each reply key in a `DashMap` *before* the corresponding send, not after. The failure that ordering prevents is the one that is invisible in testing and constant in production: a reply arriving before the sender finished bookkeeping, being matched against nothing, and being dropped — which presents as a port inconsistently reported as filtered. Registering first costs nothing and removes the race by construction rather than by widening a timeout.

`zpwchrome-host`'s self-healing resume is the process-level version. On `dl.resume` the host checks `kill(worker_pid, 0)`: a live worker just needs its paused flag cleared, while a dead PID means the browser's service worker was suspended and took the worker's parent context with it, so a fresh worker is spawned against the same state file and picks up from the recorded offset. The extension is not told any of this happened. That is the correct division of labour — the client asked for the download to continue, and which of the two mechanisms delivers that is an implementation detail — and it is only available because the job's state lives in a file rather than in the memory of whichever process happened to start it.

The cost model — where the time goes, and why the shapes differ

The benchmark chapter's rule is that a measured number belongs to a machine, a workload, and whoever ran it. This chapter is the complement: not what any tool measured, but where the time *structurally* goes in each of them, which is the part that transfers between machines and the part that explains why a ratio moves when a workload changes. No numbers are introduced here that the book has not already reported; the subject is mechanism.

Four cost regimes

The eight tools fall into four regimes, and knowing which regime a tool is in tells you immediately which optimisations are available to it and which are theatre.

Startup-dominated. A tool that does a small amount of work per invocation pays for process creation, dynamic linking, and whatever initialisation it does before the first useful instruction — and if the useful work is small enough, those fixed costs *are* the runtime. This is the regime the book's own explanation of `nmaprs`'s benchmark shape lives in: the largest reported advantages appear on small scans, where Nmap's startup cost (Lua and NSE initialisation, libpcap, database loads) dominates, and the advantage narrows on full 65536-port sweeps where probe work dominates. Nothing about that shape is specific to scanning. Any tool whose competitor initialises an interpreter or loads databases at startup shows the same curve: a large ratio on small inputs that decays toward the ratio of the actual work as the input grows. The corollary is the useful part — if a tool's advantage is startup-shaped, the optimisation that matters is *not doing initialisation*, and no amount of tuning the inner loop will move the small-input case.

Syscall-dominated. When per-unit work is a handful of kernel transitions, the count of transitions is the cost model. `nmaprs`'s connect probe is the worked example the book already carries: `socket + connect + close` is three syscalls, and the async path adds an `ioctl(FIONBIO)` and a reactor registration on top of the same three. Adding two operations to a three-operation unit is a large relative change, which is why the default connect scan uses blocking calls on a thread pool. The same reasoning appears in miniature in `temprs`'s `is_tty()`, which memoizes the stdout terminal check in a `thread_local! Cell<Option<bool>>` so the syscall is paid once per run rather than once per rendered chunk. That is a trivially small optimisation in absolute terms and it is the correct one in kind: identify the operation inside a loop that does not vary, and hoist it.

Copy-dominated. Tools that move bytes pay for the bytes. `iftoprs` opens its pcap handle with a 256-byte snaplen — enough for L2 through L4 headers, never payload — which caps the per-packet copy out of the kernel at a constant regardless of whether the packet is a 64-byte ACK or a full-MTU data segment. On a saturated link that is the difference between copying the link's full throughput into userspace and copying a fixed small multiple of the packet rate. The decision is available only because the tool genuinely does not need payload: it accounts and attributes, it does not inspect. A tool that needed payload would have no such option, which is why “cap the copy” is a design decision made at the requirements stage rather than an optimisation applied later.

Render-dominated. For a TUI, the cost that the user perceives is the frame, and the frame is dominated by whatever the draw path does synchronously. All four TUI tools respond identically: the render/input loop never does expensive work inline. Collection lives on background threads writing into shared state the UI reads. `storageshower`'s background collector runs on a three-second cadence while the UI polls `crossterm` with a 200 ms timeout; `iftoprs` runs a 33 ms tick with capture and proc-lookup on their own threads. The property

this buys is that the *interface*'s responsiveness is independent of the *data*'s cost — a slow `getmntinfo` on a machine with many mounts delays the numbers, not the keystroke.

The costs that were designed away

Several of the fleet's more interesting performance properties are not optimisations at all. They are consequences of a data representation chosen so the expensive operation never occurs.

`zcolorizer`'s and the theme system's generated palettes are one. Thirty-one themes styling roughly sixty tokens each is not a table of eighteen hundred entries — it is a function, `theme_from_palette`, over six indices, and the derived shades (`shift_lighter` for the `process/host` collision) are computed at render time rather than stored. The saving is not primarily memory; it is that there is no table to keep consistent, which is why the semantic assertions (`process` never equals `host` across all 31; `error` bold in every one) can be written as properties over the generator rather than as thirty-one hand-checked fixtures.

`grcat`'s run-length escape collapsing is another. Emitting an escape only when it changes from the previous character's is described in the output chapter as a correctness property — an uncoloured token produces no bytes — and it is simultaneously the reason the output of colouring a large log is not several times the size of the log. Both properties fall out of one rule, which is the signature of a representation chosen well.

The absence of memory mapping across all eight tools is the third and the most counterintuitive, because `mmap` is the reflexive answer for a systems tool in Rust. There is no `memmap/memmap2` anywhere in the set. The reason is that no tool in this book reads a large file: `grcrs`, `zcolorizer`, and `tempis` read stdin line-buffered, and everything else gathers from kernel APIs and pseudo- filesystems where the data is generated on read and mapping is not on offer. Mapping is the right tool for random access into a large, stable, file-backed byte range — the shape `awkr` has, and the book records that `awkr` does memory-map regular files, which is exactly the contrast that makes the fleet's uniform absence of it meaningful rather than an oversight.

Where the release profile actually helps

Six of the eight tune for a lean shipped binary with `lto = true` and `strip = true`, most adding `opt-level = 3` and `codegen-units = 1`, and some setting `panic = "abort"`. It is worth being precise about which regime each of those settings serves, because the settings are frequently applied as a block without anyone asking.

`opt-level = 3` and `codegen-units = 1` buy inner-loop quality, which matters for the copy- and syscall-dominated regimes and matters very little for a tool that spends its life blocked on a pipe. `lto = true` buys cross-crate inlining and, significantly for the startup-dominated regime, a smaller binary — less to page in before `main`. `strip = true` is purely size, and size is a startup cost on a cold cache. `panic = "abort"` removes unwinding tables and the landing pads, which is size again plus a small code-quality gain; it is also a semantic decision, since it eliminates the possibility of catching a panic, and it is notable that the tools setting it are ones with no long-lived shared state to protect. `htoprs`, whose panic hook writes a crash log with a full backtrace, is not among them — which is the correct pairing, since a crash-logging panic hook and `panic = "abort"` want different things from the runtime.

The honest summary is that the profile block is a size-and-startup optimisation first and an inner-loop optimisation second, and for a fleet of short-lived CLI tools that ordering is right. The one setting that would be a mistake to generalise is `panic = "abort"`, because it is the one with observable semantics.

What the cost model implies for the reader

Three practical readings follow.

If you are comparing one of these tools against the thing it replaces on a small input, you are largely measuring startup, and the ratio you get will not survive scaling the input. If you are comparing on a large input, you are measuring the inner loop and the syscall count, and the ratio you get is the one that generalises. The book's own benchmark reporting already shows this shape and attributes it correctly, which is the model to copy when reading anyone else's numbers.

If you are wondering why a TUI feels slow, the question is which of the four regimes the slowness is in, and the decoupled-collection architecture means the answer is almost never the render path. A frame that takes too long in one of these tools is a frame that is doing collection work it should have delegated — which is precisely the bug the shared architecture is shaped to prevent.

And if you are extending one of these tools, the regime tells you where the budget is. Adding a field to `iftoprs`'s NDJSON snapshot costs render-path serialisation, which is bounded and small. Adding a per-packet computation costs capture-path work at packet rate, which is the copy-dominated regime and the one place in that program where a small constant matters. The two changes look equally trivial in a diff and differ by orders of magnitude in where they land.

Measuring the fleet — benchmark and test methodology, honestly

Every per-tool chapter in this book reports numbers — versions, test counts, benchmark ratios — and every one of them marks those numbers as provisional. That is not hedging for its own sake. It is a methodology, stated once here so the per-tool chapters can keep repeating the caveat without re-deriving it, and it is worth reading as a subject because the way the fleet handles its own numbers is the clearest evidence of the honesty the book keeps claiming. A book that printed “1,468 tests, all passing” and “5–21× faster than Isof” as flat facts would be lying in two distinct ways, and the tools’ own discipline names both.

What “provisional” means, precisely

A test count in this book is, almost everywhere, a count of `#[test]` (and `#[tokio::test]`) *attributes* found by `grep` over the source tree — not the `test result:` line of a `cargo test` run. Those are different quantities, and the gap between them is real. A `grep` of attributes counts intentions: it includes `#[ignore]`d tests, it includes helper tests, it cannot tell a passing test from a failing one, and in at least one case it counts a generation. `temprs`’s ~5,065 unit tests are dominated by 4,449 *generated* flag-recognition tests in `opts.rs` — a machine-emitted battery, not 4,449 hand-written cases — which is why the chapter states the breakdown rather than the headline. The correct reading of every test number in this book is therefore “count of test *attributes* present in the source, as of the read, treat as inventory not as a green-suite claim; the authoritative figure is the `test result:` line of `cargo test --locked`.” The one place the book verified counts in-session is `storageshower` (3,101 in `tests/` + 596 in `src/` at the time of that chapter; 3,114 + 622 on a later re-count), and that crate is now also the one place where the `grep` count has been checked against a completed run — the chapter *The suite that would not finish, finished* reports `3,727 passed; 0 failed; 0 ignored` and a nine-attribute gap between the two ways of counting.

The consolidated table of these numbers lives in Appendix C. What matters here is the rule that produced them: a number goes in the book with the tool that generated it named (`grep`, `wc -l`, runtime enumeration, or a `cargo test` run) so a reader can tell an intention from a result.

Benchmark numbers are the author’s, on one machine

The benchmark claims are held to the same standard and it is a strict one. Three tools report head-to-head figures, and all three are marked as the project’s own measurements on one machine and configuration, not reproduced for the book:

- `lsufrs` — the README publishes hyperfine benchmarks claiming a 5–21× advantage over `Isof` 4.91 and `Isofng` on macOS, on a machine with roughly 470 processes and 5,000 open files (consistent with the 431/4,032 snapshot the chapter captures). Not reproduced here; read as the author’s reported figures.
- `nmaprs` — hyperfine comparisons against `nmap` 7.99 on macOS arm64, all against `127.0.0.1` with `-n -Pn --min-rtt-timeout 50ms --max-retries 0` to `/dev/null`. The headline is 1.5–5.1× faster across port counts, with the largest gains (3–5×) on small scans where `Nmap`’s startup cost dominates and narrowing to 1.5–1.6× on full 65536-port sweeps where probe work dominates. A Criterion microbenchmark of a three-closed-port localhost connect scan reports ~74–82 μs.

- `storageshower` — the README reports formatting operations in the tens of nanoseconds and full disk-list rendering for 200 mounts around 15.7 μ s on Apple Silicon.

The uniform posture — “these are the author’s own measurements on one machine; treat them as directional rather than as guaranteed cross-platform figures” — is not false modesty. A benchmark ratio is a measurement of two binaries on one machine under one workload, and reprinting it as a property of the tool (“lsofrs is 5–21× faster than lsof”) drops every qualifier that makes it true. The book’s rule is to keep the qualifiers attached: which upstream version, which machine, which workload, and who measured it. Where the chapter can explain *why* a number is plausible it does — nmaprs’s small-scan advantage is attributed to Nmap’s startup cost (Lua/NSE init, libpcap, database loads) that a Rust binary with none of that pays once and cheaply — but the explanation is a mechanism, not a re-measurement, and the chapter says so.

The parity spectrum: only one tool enforces it at build time

The most over-statable thing in the whole book is “the fleet’s parity discipline,” because it is tempting to describe it as a shared harness and it is not one. Parity varies enormously across the tools, and reading it as a single thing would paper over the difference between a build gate and an aspiration. The honest spectrum, strongest to weakest:

1. `htoprs` — **a build gate plus a byte-diff oracle**. The port-purity gate in `build.rs` runs on every `cargo build/test/check`: it extracts every free `fn` name under `src/ported/` and checks it against a 4,132-entry snapshot of htop’s real C function names, and a function with no upstream counterpart *fails the build*. It fires even under `cargo test --test X` so a reimplementer cannot route around it, the allowlist for genuine architectural helpers holds zero entries, and the brace-counting collector is comment-, string-, and raw-string-aware so it cannot be tricked by a “{” in a test. On top sits `tests/parity/cli_parity.rs` diffs htoprs against the real C htop byte-for-byte (modulo the deliberate rebrand via `canon()`), and `xutils_parity.rs` compiles a C harness against htop’s *genuine* `XUtils.c` and diffs the Rust port against that compiled oracle function by function. Both skip gracefully when no htop 3.5.x or C compiler is present, so CI stays green. This is the strictest apparatus in the book by a wide margin.
2. `grcrs` — **a byte-for-byte oracle without a build gate**. Its 27 end-to-end tests in `tests/cli.rs` spawn the built binaries and assert the exact coloured byte stream, with expectations cross-checked against the reference Python `grcat` — covering `count=stop/once/more/block`, `unchanged/previous` colours, `replace-then-colour`, `\</\>` literals, CRLF stripping, and the invalid-UTF-8 round-trips that pin its `surrogateescape` reproduction.
3. `ztmux (Part I)` — **a parity suite plus an anti-drift gate**. Reported at 1188/1188 cases passing against the vendored tmux, with a `tests/ported_fn_names_match_c.rs` gate that fails the build on a free `fn` with no tmux C counterpart. It earned the number by root-causing real divergences (a `#{l:...}` crash from a dropped pointer increment, layout rounding, a macOS `#{pane_current_command}` case).
4. `nmaprs`, `zcolorizer` — **documented parity, contract-tested**. `nmaprs` documents parity as a capability matrix (which port-state each scan technique produces) backed by contract tests over the port database, scanflags, OS-DB parsing, and target expansion. `zcolorizer` keeps ccze’s C source vendored as reference and pins its precedence model (`earlier_rule_wins_overlap`) and sanitization behaviour with engine tests — but it colours formats ccze never saw, so byte-parity is not even the goal for most of its surface.
5. `lsofrs`, `iftoprs` — **reproduce the grammar and vocabulary, no running-upstream diff**. `lsofrs` reproduces lsof’s option grammar, type vocabulary, and output formats and asserts them with its own tests but does not diff against a running `lsof`. `iftoprs` ships a parity palette and mirrors iftop’s keybinds and flow-table model, but its 2s/10s/40s columns are *summed byte counts over trailing one-second slots scaled to bits*, not slot-averaged bytes/sec — a difference the chapter states precisely rather than calling it identical.
6. `storageshower`, `tempis` — **originals, so nothing to be parity-tested against**. Their large suites pin *their own* invariants — symlink-cycle termination in `storageshower`, master-record tag alignment and round-trips in `tempis` — which is a different kind of discipline entirely.

The single sentence that keeps this honest is the one the parity chapter already uses: treat a test count as a count of intentions until a `test result:` line confirms it, and treat “parity” as whatever the specific tool actually enforces — a build gate for one tool, a byte oracle for two, a capability matrix for two, grammar-reproduction for two, and self-invariants for the two originals. There is no fleet-wide parity harness, and saying there is would be the exact kind of convenient generalization the shape-of-the-fleet chapter warned against.

The audit tools are themselves guarded

One detail deserves its own note because it is where a coverage number could most easily become a lie. `htoprs`'s coverage figure is not typed into a README — it is derived from source at run time by `scripts/gen_port_report.py`, which walks both the C source and the Rust port and writes `docs/port_report.html` with per-file and overall numbers, and which counts a `todo!()` or `unimplemented!()` body as **stubbed, never ported**. That last rule is the load-bearing one: it means scaffolding cannot inflate the coverage figure, because the generator classifies an unimplemented body as a gap no matter how the module around it is named. The stub scaffold generator (`scripts/gen_port_stubs.py`) lays out the full port surface — one module per C file, one `todo!()` stub per C function named to satisfy the gate — and never overwrites an already-ported module, so the port surface is enumerated without the enumeration masquerading as completion. A report that derives its number from source at run time, and that scores a stub as a stub, is the correct shape for an audit tool: the number moves only when the code moves, which is exactly what makes it worth printing.

What a benchmark of a system tool is actually measuring

The provisional-numbers rule keeps the book honest about *whose* numbers these are. There is a second honesty question that the rule does not reach, and it is the more useful one for a reader deciding whether a reported ratio predicts anything about their machine: what, mechanically, is being compared?

For the three tools with head-to-head figures, the answer differs in kind.

`nmaprs`'s comparison is the most controlled and also the most explicitly scoped: `hyperfine` against `nmap` 7.99, all against `127.0.0.1`, with `-n -Pn --min-rtt-timeout 50ms --max-retries 0` and output to `/dev/null`. Every one of those flags is removing a variable. `-n` removes DNS, `-Pn` removes host discovery, the RTT and retry settings pin the timing behaviour so the two programs are not being compared on their adaptive-timing heuristics, and `/dev/null` removes output formatting. What is left is close to a measurement of probe issue and reply handling — plus, unavoidably, startup. The loopback target is the crucial scoping fact: on `127.0.0.1` the network is not a variable, which makes the measurement reproducible and makes it *not* a prediction about scanning a real network, where the wire is the bottleneck and both programs would converge. That is not a flaw in the benchmark; it is what the benchmark is for. It measures the part of the work the rewrite controls.

`lsoprof`'s comparison is a whole-program wall-clock measurement — `hyperfine` against `Isof` 4.91 and `Isofng` on a machine with roughly 470 processes and 5,000 open files. That is a realistic workload rather than a controlled one, which makes it a better predictor of a user's experience and a worse instrument for attributing the difference. The reported span is wide (5–21×), and a wide span is itself information: it says the ratio depends heavily on which invocation was measured, which is exactly what you would expect from a comparison where some invocations are dominated by the process walk and others by output formatting.

`storageshower`'s figures are the third kind entirely: microbenchmarks of internal operations — formatting in the tens of nanoseconds, a 200-mount disk list rendering in single-digit microseconds — with no competitor at all. There is nothing to be faster *than*; the numbers exist to establish that the render path is nowhere near the frame budget, which is the claim a TUI actually needs to make. Reading them as a comparison with `df` would be a category error.

The reader's rule that falls out of this is worth stating: before using a ratio, identify which of the three kinds it is.

A controlled comparison predicts the component it isolates. A whole-program comparison predicts the workload it ran. A microbenchmark predicts nothing about a competitor and everything about whether an internal path is a bottleneck. Most published performance claims are the second kind reported as though they were the first.

The shape of a suite worth its runtime

The test-attribute counts in this book span two orders of magnitude, and the counts are close to uninformative on their own — which the methodology section already says. What is informative is the *shape* of each suite, and the fleet contains four distinguishable shapes with different value per test.

Oracle tests are worth the most, because they cannot be wrong about the expected value. `htoprs`'s `xutils_parity.rs` compiles a C harness against `htop`'s genuine `XUtils.c` and diffs the Rust port against that compiled oracle function by function; `cli_parity.rs` runs the reference binary and diffs byte-for-byte modulo the deliberate rebrand. `grcrs`'s end-to-end tests assert exact coloured byte streams cross-checked against the reference Python `grcat`. In all three the expectation is *derived* from the specification rather than transcribed by a human who might have transcribed it wrong. The cost is that they need the reference present, which is why all of them skip gracefully when it is absent — and that graceful skip is itself a design decision with a cost, since a suite that goes green by skipping is a suite that can silently stop testing anything.

Property tests are the next most valuable, because they state an invariant rather than an instance. `storageshower`'s `scan_directory_symlink_cycle_completes_with_finite_sizes` names a termination property. `zcolorizer`'s theme assertions — `process` and `host` never render identically across all 31 generated palettes, `error` is bold in every one — are properties over a generator, which is why they are worth writing once instead of thirty-one times. `lsofrs`'s and `iftoprs`'s `ThemeName::ALL.len() == 31` is the degenerate but genuinely useful case: a property whose whole content is that a number has not drifted.

Transformation tests pin an internal decision that has no observable counterpart. This is where the compatibility parsers live — asserting that a given upstream-style argv normalizes to a specific expanded argv — and their value is diagnostic rather than protective: when one fails, it names the layer.

Recognition batteries are the cheapest per test and the most numerous. `temprs`'s 4,449 generated flag tests are the fleet's example, and the chapter is right to state the breakdown rather than the headline, because 4,449 machine-emitted assertions that each flag spelling maps to the operation it names is one test repeated with different data. It is still worth having — a 38-operation surface with signed index arguments has real combinatorial risk in the recognition table — but a reader comparing `temprs`'s count against `grcrs`'s 71 and concluding anything about relative rigour has compared a generated battery against a suite that is mostly oracle tests, and reached exactly the wrong conclusion.

The single number that would actually be comparable across eight projects does not exist, which is why the appendix publishes the counts with their provenance attached and declines to rank them.

Anatomy of a port — from vendored C to a shipped binary

Five of the tools in this book reimplement a named Unix program, and the per-tool chapters describe what each one achieved. This chapter describes the *method* — the sequence of decisions a port goes through, in the order they have to be made, with the fleet's own instances as the worked examples. It is written as a method rather than a history because the sequence is reusable, and because the failure modes at each step are specific enough to be worth naming in advance.

Step one: decide what the specification is

The first decision determines every subsequent one, and there are exactly two answers. The specification is either the upstream's *source* or the upstream's *observable behaviour*, and the fleet contains clean examples of both.

`htoprs` takes the source. Its README leads with “the C source is the spec,” and everything follows: one Rust module per htop C file, function names matching C function names, a build gate enforcing the correspondence, per-OS backend trees carried because the C carries them. `ztmux` (covered in its own volume) takes the same line, with the tmux C sources vendored read-only and SHA-pinned and every ported function carrying a back-link comment to its C origin.

`lsofrs` takes the behaviour. It reproduces lsof's option grammar, its type and state vocabularies, and its output formats down to the `0t<offset>` encoding and the `TST=<state>` prefix — and its internal architecture (a `rayon` gather, a `TuiMode` trait, a stall classifier) owes nothing to lsof's. `grcrs` is the same posture applied to a smaller surface: it reproduces grc's getopt behaviour, its colourfile format, and its exact coloured output bytes, without mirroring the Python's structure.

The choice is not a matter of rigour — both are rigorous — it is a matter of what the port is *for*. A source-spec port buys the ability to track upstream mechanically: when htop changes, the C-name index changes, and the gate tells you where. A behaviour-spec port buys freedom to build a better internal architecture, at the cost of having no mechanical way to know when upstream has moved. Picking source-spec for a tool whose upstream is dormant, or behaviour-spec for one that changes weekly, is the way this decision goes wrong.

Step two: vendor the reference, and forbid it from building

Whichever spec is chosen, the upstream source is needed for reading, and the fleet's convention is uniform: keep it under a `vendor/` tree, read-only, and build nothing from it. `zcolorizer` carries a full copy of ccze's C under `vendor/ccze/` as a build-time reference. `ztmux` vendors tmux SHA-pinned and declares its own `[workspace]` specifically so Cargo never walks into the reference. The book's licensing section states the rule that makes this safe: the binary is built from the tool's own `src/`, never from `vendor/`, the vendored trees are excluded from published crates, and every tool is MIT with original Rust.

The failure this arrangement prevents is subtle and expensive: a vendored tree that is *in* the build graph is a licence question, a build-portability question, and a C-toolchain dependency all at once. Keeping it strictly out of the graph means the vendored source is documentation that happens to be compilable elsewhere, which

is exactly what a specification should be. The one place the fleet deliberately crosses this line is `htoprs's xutils_parity.rs`, which compiles a harness against htop's real `XUtils.c` — and it crosses it inside a test that skips when the source or a C compiler is absent, never in the build of the shipped binary.

Step three: enumerate the surface before implementing any of it

The instinct on starting a port is to implement the interesting function first. The method the fleet uses does the opposite: generate the whole surface as stubs, then fill them in. `htoprs's scripts/gen_port_stubs.py` lays out one module per C file and one `todo!()` stub per C function, named to satisfy the gate; `vimlrs's scripts/gen_port_stubs.sh` emits one cited stub per not-yet-ported C function for the same reason.

Three properties come from doing it in this order, and none is available if you implement first. The port surface is *enumerated*, so the denominator of any coverage figure is real rather than a subset someone happened to touch. The gate can be turned on from day one, because every name that exists corresponds to an upstream name. And the remaining work is visible as a list rather than as a feeling. The critical accompanying rule is the one the report generator enforces: a `todo!()` or `unimplemented!()` body counts as **stubbed**, **never ported**, so the scaffold cannot inflate the number it was created to measure.

Step four: gate the names, mechanically, on every build

`htoprs's build.rs` extracts every free `fn` name under `src/ported/` and checks it against a 4,132-entry snapshot of htop's real C function names; a function with no upstream counterpart fails the build. Three implementation details make it a real gate rather than a gesture, and each corresponds to a way a weaker version gets bypassed.

It runs on `cargo build`, `test`, and `check`, and it fires even under `cargo test --test X` — so a developer cannot route around it by working in a test target. Its allowlist for genuine architectural helpers holds zero entries, which is the number that keeps an allowlist from becoming the place exceptions accumulate; `ztmux's` equivalent gate keeps pre-existing wrappers in an allowlist explicitly framed as a burn-down list rather than a permanent carve-out. And the collector is a comment-, string-, and raw-string-aware brace state machine, so it cannot be fooled by a `"{"` inside a test fixture — a naive brace-counting parser miscounts on exactly the kind of string literal a test file is full of, and a gate that miscounts is a gate that either fails spuriously or passes wrongly.

The complement to the gate is the citation: every free `fn` under `src/ported/` carries a `/// Port of` doc comment (1,937 of them in `htoprs`; `zshrs` uses the richer `/// Port of <cname>() from Src/<file>.c:NNNN` form). The gate proves the name exists upstream; the citation records *where*, which is what a reader needs when the ported function's behaviour looks wrong and the question is whether the port is wrong or the original was surprising.

Step five: build an oracle, and let it skip

A gate proves structure. Only an oracle proves behaviour, and the fleet's three oracle designs are worth distinguishing because they cost different amounts.

The cheapest is *diff against the running upstream*: `htoprs's cli_parity.rs` runs the reference C htop and `htoprs` on the same arguments and compares bytes, with a `canon()` function normalising the deliberate rebrand. `grcrs` does the same shape against the reference Python `grcat`. The cost is that the reference must be installed, and the subtlety is the rebrand: a port that renames itself has to define precisely which differences are intentional, and a `canon()` function is that definition made executable.

The most expensive and most precise is *link the upstream's real code as the oracle*: `xutils_parity.rs` compiles a C harness against htop's genuine `XUtils.c` and diffs function by pure function. This only works for pure functions with no OS dependency, which is a small slice of any program — and inside that slice it is unanswerable, because both sides of the comparison are running the same specification.

The third is *capability-matrix parity*, which is what you build when byte-parity is not the goal. `nmaprs` documents which port state each scan technique produces and contract-tests the port database, scanflags, OS-DB parsing, and target expansion. `zcolorizer` pins its precedence model (`earlier_rule_wins_overlap`) and its input sanitisation with engine tests, and does not attempt byte-parity across the formats ccze never saw. Both are honest reductions in scope rather than gaps.

Every one of these skips gracefully when its reference is unavailable, so CI stays green — and that decision needs its own guard, because a suite whose oracle silently stops running looks identical to a suite whose oracle passes. The guards the fleet uses are version-pinned availability checks (`ref_available()` returns the reference path only if it exists and is the 3.5 series) plus environment overrides (`HTOP_REF`, `HTOP_C_SOURCE`) so a developer can point the harness at a known-good reference deliberately.

Step six: decide where the port stops, and write it down

Every port stops somewhere, and the method's final discipline is that the stopping points are enumerated rather than discovered by a user.

`htoprs` does not port htop's hand-rolled allocator families — `xMalloc`, `Vector.c`'s raw `Object**` machinery, `Hashtable.c`'s open-addressing table — because reimplementing them faithfully would mean writing an allocator in `unsafe`, which is the bug class the port exists to eliminate. `Vec` and `HashMap` stand in. That is a stop with a stated reason, and the reason is a principle rather than an excuse: a faithful port of a memory-safety hazard defeats the purpose of porting.

`nmaprs` does not embed the NSE Lua runtime, its OS detection is partial, and its version detection skips Perl-only regex constructs — all recorded in a capability matrix, with the README pointing anyone needing authoritative NSE or OS-DB behaviour back at Nmap itself. `lsotrs` stops at the kernel-tuning flags, accepting `-b`, `-0`, and `-D` syntactically while implementing nothing behind them.

The accept-and-ignore surface is the specific form this takes at the CLI, and it is worth restating as part of the method because it is the step most often skipped: a port that rejects flags it does not implement forces every caller to audit their invocations, which is the migration cost the port existed to remove. Accepting them is correct *provided the tool says so* — which is why the capability matrix and the parity section are not documentation chores appended at the end but the artifact that makes the accept-and-ignore contract honest rather than deceptive.

What the method looks like when there is no upstream

Three tools in this book are originals, and the useful observation is which parts of the method survive the absence of a specification. The gate does not — there are no upstream names to check against. The oracle does not — there is nothing to diff. What survives is the enumeration discipline and the property assertions: `storageshower` pins symlink-cycle termination, `temprs` pins master-record tag alignment and round-trips, and both carry large suites of their own invariants.

That is a genuinely weaker position and the book says so. A port has an external authority that can tell it it is wrong; an original has only the invariants its author thought to write down. The compensating discipline is to write the invariant as a property over generated inputs rather than as an example — the difference between `scan_directory_symlink_cycle_completes_with_finite_sizes` and a fixture that happens to contain a symlink — and to be explicit, as `zcolorizer` is, about which parts of a hybrid tool are ported (ccze's plugin- then-word-colorizer architecture, pinned by engine tests) and which are original (the 47-module

format library, the themeable colour model), because those two halves earn confidence in completely different ways.

Packaging the fleet – binaries, completions, man pages, release

Below the architecture sits a layer of packaging conventions that make the fleet feel like one product installed from one place rather than eight separate downloads, and — as with everything else in this book — the conventions come with honest exceptions that are worth naming rather than smoothing over. This chapter collects the packaging surface: how each tool ships its binaries, its shell completions, its man pages, its config, and its releases.

The short-alias binary convention

Frequently-typed tools get a two- or three-character second name built from the same source, byte-identical to the long form. `lsofrs` also builds `lsf`, `nmaprs` also builds `nms`, and `temprs` also builds `tp` — each declared as a second `[[bin]]` target pointing at the same entry point, so the alias is not a symlink or a shell alias but a second compiled artifact that parses `argv` identically. `lsofrs` copies `argv[0]` verbatim through its normalizer so `lsf` and `lsofrs` behave the same, and `nmaprs` re-derives the basename so `nms --help` shows `nms`. The rationale is muscle memory: a tool you reach for dozens of times an hour needs a name your fingers can hit without thinking, and `temprs`'s chapter is blunt that `tp` “exists purely because a temporary-file tool that you reach for dozens of times an hour needs a two-keystroke name.”

`grcrs` is the exception that proves the convention is about role, not length. It builds two binaries — `grc` the launcher and `grcat` the colouriser — that are *functionally distinct* rather than aliases, mirroring the same split upstream `grc` uses so the colouring logic stays a standalone filter you can drive straight from a pipeline (`some-command | grcat conf.log`). The remaining four tools (`iftoprs`, `htoprs`, `storageshower`, `zcolorizer`) ship a single binary. And the two native hosts each produce one binary plus a library: `zwire-host` builds the `zwire-host` binary and the `zwire_host` lib so other Rust programs can embed the same host, and `zpwirchrome-host` similarly exposes its internals — the download worker is a `--dl-worker` subcommand of the same binary rather than a second artifact.

Shell completions, and the two that are live

Six of the eight ship a checked-in zsh completion under `completions/_<tool>` with a `#compdef` header: `lsofrs`, `nmaprs`, `iftoprs`, `htoprs`, `storageshower`, and `temprs`. Several are kept in lockstep with the parser, and two are genuinely dynamic rather than static tables:

- `lsofrs`'s `_lsofrs` is a real `#compdef lsofrs lsf` covering both names, built on `_arguments -C -s -S` with dedicated state handlers (`->filedes` for `-d`, `->addresses` for `-i`, `->states` for `-s`, `->fields` for `-F`) that reuse the shell's own `_pids/_users/_pgids` completers with comma-sequencing.
- `htoprs`'s completion keeps its option set in lockstep with the getopt table in `commandline.rs` and completes sort columns *dynamically* — the `sort-keys` state calls `_call_program sort-keys $words[1] --sort-key help` and parses the tool's own live column list rather than hardcoding one, so the completion cannot drift from the binary's real column set.
- `temprs`'s `_temprs` (`#compdef tp temprs`) is *live against the running stack*: its `_tp_indices` helper reads the actual `temprs-stack` master file out of `${TEMPRS_DIR:-...}` and offers the current indices, file basenames, and `@`-prefixed tag names, so tab-completion reflects the contents of your stack at the

moment you press Tab. It even solves the NUL problem inline — zsh variables cannot hold NUL, so it pipes the master record through `tr '\0' '\x1e'` before splitting.

`iftoprs` both ships a checked-in `_iftoprs` and can emit completions at runtime for five shells (`--completions zsh|bash|fish|elvish|powershell`), the shell set coming from `clap_complete::Shell` directly rather than a hand-rolled enum. `zcolorizer` and `grcrs` ship no in-tree completion — `grcrs` relies on the upstream `grc.zsh/grc.sh/grc.fish` shims that wrap common commands, which is the same integration path upstream `grc` uses.

The zshall-modelled man-page pair

The same six that ship completions also ship the paired man-page convention: a short `<tool>.1` and a comprehensive `<tool>all.1` modelled on `zshall(1)`, folding invocation, keybinds, themes, and architecture into one greppable document. `lsotrs.1` is the short reference and `lsotrsall.1` documents every flag, mode, theme, and architectural note; `lsotrs.1` even defines a real exit-status contract (`0` success, `1` on error or no match) and a `SEE ALSO` pointing at `lsot(8)`, `ss(8)`, `fstat(1)`, `netstat(1)`, `pstree(1)`, `top(1)`, and `proc(5)`. `htoprsall.1` is eighteen concatenated sections and cross-references the rest of the fleet in `SEE ALSO` (`iftoprsall`, `lsotrsall`, `nmaprsall`, `zshrsall`), which is the detail that makes the pair a *fleet* convention rather than a per-tool habit — the manuals point at each other. The exceptions again: `grcrs` ships per-binary `grc.1` and `grcat.1` (the `grcat.1` page documenting all ten colourfile keywords including the `trend/key/metric` extension, flagged there explicitly as “grcrs extension”), and `zcolorizer` ships no `man/*.1` in the tree searched here. Two chapters also note their man pages have gone *stale* — `htoprs`’s both carry an early-stage PORT STATUS caveat that predates the 97.8% figure and should be resynced — which the book records rather than hides, because a stale man page is a real defect even when the code is finished.

Config persistence, split by interactivity

Where a tool keeps user preferences follows one rule — interactive tools persist, non-interactive tools do not — with the storage location split by tool:

Tool	Config	Format
<code>lsotrs</code>	<code>~/.lsotrs.conf</code>	TOML, auto-saved on every toggle
<code>iftoprs</code>	<code>~/.iftoprs.conf</code>	TOML, auto-saved
<code>storageshower</code>	<code>~/.storageshower.conf</code>	TOML, auto-saved
<code>htoprs</code>	<code>~/.config/htoprs/prefs.json</code>	JSON
<code>zcolorizer</code>	<code>\$XDG_CONFIG_HOME/zcolorizer/config.toml</code>	TOML (XDG)
<code>nmaprs</code> , <code>grcrs</code> , <code>temprs</code>	none	—

The three non-interactive tools keep no preference file: `grcrs` reads the upstream `grc.conf` data files (shared config, not per-user state), and `temprs` keeps its *stack* on disk (`$TMPDIR/temprs`, overridable with `TEMPRS_DIR`) rather than a settings file. Every persisted config, as the theme chapter noted, guards each field with `#[serde(default)]` for forward-compatibility.

Release, distribution, and the licence floor

Distribution is uniform: every tool is published to crates.io (`cargo install <tool>`) and carries a Homebrew tap install path (`brew install <tool>` or the fully-qualified `MenkeTechnologies/menketech/<tool>`), and the two native hosts install via `cargo install` with `--force` for upgrade. The release automation rhymes across the fleet — a `release.yml` that fires on `v*` tags, cross-compiles a per-target matrix (macOS aarch64/x86_64, Linux gnu, and musl via `cross`, with the musl targets typically marked `allow_failure` so a toolchain hiccup does not block a release), packages per-target tarballs, publishes a GitHub Release, and auto-updates the Homebrew tap formula with freshly computed SHA-256 sums. `grcrs`'s release is the most elaborate because it must ship *data*: each tarball stages the two stripped binaries alongside `grc.conf` and every `colourfiles/conf.*` so the archive carries the runtime data the binaries expect at their search paths, and the generated formula installs `grc/grcat` to `bin`, `grc.conf` to `etc`, and the colourfiles to `share/grc` — matching the `/opt/homebrew/...` entries baked into the search paths so `brew install` yields a working install with no manual data placement.

The licence floor is the simplest fact in the book and the one most worth stating plainly: every tool in the fleet is MIT, the Rust in each is original code, and where a tool reimplements a Unix program it keeps the upstream source under a read-only `vendor/` tree purely as a parity reference — the binary is built from the tool's own `src/`, never from `vendor/`, and the vendored trees are excluded from the published crates. There is no copyleft anywhere in the CLI fleet and no per-tool licence divergence to track. That uniformity is not incidental to the packaging story; it is what lets the whole shelf install from one tap under one licence, which is the concrete meaning of “one product, eight binaries.”

Installing on top of the tool you replace

A drop-in replacement has an installation problem that an original does not: the thing it replaces is probably already installed, probably earlier on `PATH`, and possibly depended on by the parity suite. The fleet's answer is that a port never takes the upstream's name, and the consequences of that decision run further than they look.

Every reimplementation ships under its own name — `lsotrs`, `nmaprs`, `iftotrs`, `htotrs` — plus, for the frequently-typed ones, a short alias that is also not the upstream's name (`lsf`, `nms`). `grcrs` is the apparent exception, shipping binaries called `grc` and `grcat`, and it is not really one: those names are the upstream's because `grc`'s integration path *is* the name. The shell shims that wrap common commands (`grc.zsh`, `grc.sh`, `grc.fish`) call `grc` by name, and a colouriser installed as `grcrs` would be invisible to every one of them.

Adoption is therefore the user's explicit act, which is what `lsotrs`'s chapter means when it says aliasing `lsot=lsf` is the intended path. Three things follow. The upstream stays available, which matters because it is the oracle — `htotrs`'s parity suite depends on the real `htop` being installed, and `xutils_parity.rs` links its `XUtils.c`. Reverting is a shell-level operation with no reinstall. And a script that genuinely needs the original's unimplemented behaviour — `-b`, `-0`, the NSE runtime — can call it by name from inside an environment where the interactive alias points at the port.

The accept-and-ignore contract is what makes the alias safe to set globally rather than per-invocation. Without it, aliasing `lsot=lsf` breaks the first script that passes a flag the port does not implement; with it, the flag is consumed and the script runs. The packaging decision and the parser decision are the same decision viewed from two ends, and neither is sufficient alone.

Eight crates, eight version numbers, and no shared release

The ledger's version table has no pattern in it — `lsotrs` at 4.9.1, `nmaprs` at 0.1.9, `temprs` at 2.9.12, `iftotrs` at 2.22.9, `htotrs` at 0.5.7, `storageshower` at 0.28.8, `zcolorizer` at 0.2.2, `grcrs` at 1.13.0 — and the absence of a pattern is the design. There is no fleet version, no synchronised release, and no lockstep bump,

because there is no build-time coupling between any two of these crates. They share conventions and a copied palette table; they do not share a dependency edge.

Two of the numbers carry meaning beyond their own history and are worth reading. `grcrs` at 1.13.0 tracks the upstream `grc` version it reproduces — its help banner leads `Generic Colouriser 1.13` — so the version is a *parity* statement about which upstream the colourfile semantics match. `lsofrs` at 4.9.1 sits in the neighbourhood of `lsof`'s own version series, which makes the same kind of gesture less formally. Everything else is ordinary independent semver, and `nmaprs` at 0.1.9 alongside a capability matrix full of documented gaps is a version number doing exactly what a pre-1.0 number is for.

The one real coupling in the set is the pair of native hosts, where `zpwchrome-host` depends on the `zwire-host` crate with `default-features = false`. That is a genuine build-graph edge and it is managed the ordinary way, through the dependency declaration, not through release coordination — which is precisely why the feature split exists, so the embedder can take the light filesystem and exec half without inheriting `portable-pty` and `sysinfo`.

The argument against a fleet-wide version is the one the theme chapter makes about a shared crate: coupling that buys tidiness costs release ordering. Eight independently versioned crates can each ship the day their fix is ready. A fleet version would mean a `zcolorizer` typo fix waits on whatever is unfinished in `nmaprs`, in exchange for a number on a README that no build system reads.

What is actually in the tarball

The release automation rhymes across the fleet — a `release.yml` firing on `v*` tags, a cross-compiled per-target matrix (macOS aarch64 and x86_64, Linux gnu, and musl via `cross` with the musl targets typically `allow_failure` so a toolchain hiccup does not block a release), per-target tarballs, a GitHub Release, and an auto-updated Homebrew formula with freshly computed SHA-256 sums — and the variation between tools is entirely about what has to travel *alongside* the binary.

For most of the fleet that is nothing: the binary is self-contained, the completions and man pages are installed from the tap formula, and the config file is created on first write. `grcrs` is the tool that breaks this, and its release is the most elaborate for one reason — it must ship *data*. `grcat` is useless without colourfiles, and the search paths for `grc.conf` and `colourfiles/conf.*` are compiled into the binaries. So each tarball stages the two stripped binaries alongside `grc.conf` and every `colourfiles/conf.*`, and the generated formula installs `grc` and `grcat` to `bin`, `grc.conf` to `etc`, and the colourfiles to `share/grc` — matching the `/opt/homebrew/...` entries baked into the search paths so `brew install` yields a working install with no manual placement.

That is the general shape of the problem: a binary with compiled-in data paths has a packaging contract, and the formula and the source have to agree about it or the install is broken in a way no test in the repository will catch. The fleet's mitigation is that the formula is *generated* by the same release automation that builds the tarball, so the two cannot be edited apart.

Two smaller distribution notes complete the picture. Every tool is published to crates.io, so `cargo install <tool>` is a first-class path that does not go through the tap at all — which for a tool with data files means `cargo install grcrs` gets binaries whose data search paths may not be populated, and is the reason the tap exists as the recommended path rather than as a convenience. And the two native hosts install by `cargo install` with `--force` for upgrade, because their installation has a second step no formula can perform: `zpwchrome-host --install <ext-id>` has to write native-messaging manifests into per-browser configuration directories, keyed to an extension ID that only the user knows.

The security posture of tools that read the whole machine

Nothing in this book is a security product, and that is precisely why the security posture is worth a chapter. A process viewer, a descriptor lister, a packet-flow monitor, a disk walker, and two colourisers are, between them, a set of programs that observe most of what a machine is doing and then render it onto a terminal. Each of those two halves — observing and rendering — carries a distinct class of exposure, and the fleet's answers to them are made of small decisions scattered across eight crates rather than a single policy. Pulling them together in one place is the only way to see whether they add up.

The observation surface, stated plainly

Start with what these tools can see when you run them, because the honest answer is more than most users think about. `lsOfirs` enumerates open descriptors and resolves the `cwd`, `root`, and `exe` symlinks of every process it can read. `htoprs` reads the process table. `iftoprs` parses `/proc/net/tcp`, `/proc/net/tcp6`, `/proc/net/udp`, `/proc/net/udp6`, `/proc/[pid]/fd/`, and `/proc/[pid]/comm` on Linux, and reads each process's open sockets through `pidfdinfo/SocketFDInfo` on macOS, in order to attribute a flow to a program. `storageshower` walks filesystems. Every one of those is a read of information that the kernel will hand over to whoever asks with sufficient privilege, and the tools do not add an access-control layer of their own — they inherit the kernel's and report what they get.

That inheritance is the correct design, and it needs to be said out loud because the alternative is worse in a specific way. A tool that filtered its own output by some internal notion of what the user “should” see would be lying about the system state while looking authoritative, and every one of these programs is used for diagnosis, where a quiet omission is the most expensive possible failure. So the rule across the fleet is that the visible data set is exactly the kernel's answer to this user, and the tools make the *vantage point* legible rather than editing the results. That is why the degradation behaviour documented elsewhere in this book — showing an explicit absent marker for an unattributable flow rather than guessing — matters as a security property as well as a correctness one: the user can tell the difference between “this process owns no sockets” and “you cannot see this process's sockets from here.”

The observation surface also sets the blast radius of a bug. A parser defect in `iftoprs`'s pcap decoder is being fed bytes chosen by whoever is on the other end of the wire; a parser defect in a `procfs` reader is being fed bytes chosen by whoever can name a process. Both are attacker-influenced inputs reaching a program that is frequently run under `sudo`. This is the strongest engineering argument for the whole porting effort, and it is worth stating in its sharpest form: the upstreams these tools reproduce are C programs that parse hostile input as root. `htoprs`'s refusal to port the `xMalloc` allocation family, `Vector.c`'s raw `Object**` machinery, and `Hashtable.c`'s open-addressing table is recorded in this book as a fidelity decision — reimplementing them would mean writing an allocator in `unsafe`, precisely the class of bug the port exists to eliminate — but the security consequence of the same decision is that the bounds checks on the data structures holding attacker-influenced strings are the compiler's rather than a human's.

Privilege, and where it comes from

Two of the tools genuinely need privilege for part of their surface. `iftoprs` needs to open a capture device. `nmaprs` needs raw sockets for its SYN, protocol, idle, and SCTP techniques. The rest need privilege only in

the ordinary sense that reading another user's `/proc` entries requires being that user or being root.

The install paths this book documents — `cargo install`, a Homebrew formula, a release tarball of per-target binaries — deliver an ordinary unprivileged executable. No `setuid` step and no capability-granting step appears in any of them, which means privilege at run time comes from the invoking user, normally via `sudo`. That is a deliberate place to land, and its cost and benefit are both concrete. The cost is friction: `sudo iftoprs` on every invocation, and a config file written as root the first time you forget. The benefit is that the security review surface of the fleet does not include the hardest problem in Unix privilege management — a program that starts privileged and must decide what to drop, when, and in what order, while an attacker controls its environment, its `argv`, and its inherited descriptors. A `setuid` binary has to be correct about all of that before it does anything useful. An ordinary binary run under `sudo` inherits a sanitised environment from `sudo` itself, which is an implementation someone else maintains and audits.

`nmaprs`'s privilege handling is the model for what a non-`setuid` tool should do with the privilege it may or may not have. It detects `geteuid()` and silently falls back from SYN to TCP-connect scan when unprivileged, so the common case just works. `--privileged` forces raw mode and `--unprivileged` forces connect mode, and in the latter it *rejects* the raw-only techniques (`--s0`, `--sI`, SCTP) with an error rather than silently substituting a different scan. The distinction between those two behaviours is the important one. Falling back from SYN to connect is legitimate because both answer the same question with different accuracy characteristics; silently substituting connect for a protocol scan would answer a different question entirely, and the user would have no way to know. The general rule that falls out is worth carrying to any tool with an optional privileged path: degrade when the degraded answer is a worse answer to the same question, refuse when it would be an answer to a different one.

Configuration files that are code

The sharpest local-privilege consideration in the fleet is not in the system tools at all. It is in `grcrs`, and it is inherited from the upstream by design.

A `grcat` colourfile block may carry a `command=` keyword, and on a match that command is run through `sh -c`. That is a documented upstream directive and the port reproduces it, correctly — a port that dropped it would break every colourfile that uses it. But it means a colourfile is not data; it is a program, and the search path that resolves colourfiles is therefore a search path for code. That path is, in order: the empty prefix so an absolute or relative name passed by `grc -c` is honoured as given, then `$XDG_DATA_HOME/grc/`, `$XDG_CONFIG_HOME/grc/`, `~/.grc/`, `/usr/local/share/grc/`, `/usr/share/grc/`, and `/opt/homebrew/share/grc/`, with the first existing non-directory file winning.

Read that list as an attacker would. The three user-writable prefixes come *before* the three system ones, which is the right precedence for a tool whose whole purpose is user-authored colour rules, and it means anything that can write a file into `~/.grc/` can pre-empt a system colourfile of the same name. The `grc.conf` side has the mirrored property with an inverted rule: the search order is `/etc`, `/usr/local/etc`, `/opt/homebrew/etc`, `$XDG_CONFIG_HOME/grc`, then `~/.grc`, and `find_config` keeps scanning every file and lets the *last* match overwrite earlier ones, so the user config beats the system config there too. Within a single `grc.conf`, by contrast, the *first* matching regexp wins. Two opposite precedence rules in one tool is exactly the kind of thing a reader should distrust until they have seen the reason, and the reason is that the two levels answer different questions: within a file, order expresses specificity; across files, order expresses authority.

The practical consequences are worth being blunt about. Running `grc some-cmd` executes whatever `command=` directives the resolved colourfile carries, with your privileges, in your environment. Running it under `sudo` does the same with root's. The mitigation is not a feature the tool can add without breaking compatibility — it is operational: treat `~/.grc/` and `$XDG_CONFIG_HOME/grc/` with the same suspicion as a shell startup file, because they have the same power, and prefer `grcat` driven explicitly from a pipeline over `grc` under `sudo` when the colourfile is not one you wrote.

One more detail sits underneath this. The vendored data set carries 83 colourfiles under `vendor/grc/colourfiles/`, while `grc.conf`'s 78 regexp rules name 71 distinct ones. The gap is not

a defect — some colourfiles are shared, `make` and `gcc` both resolving to `conf.gcc`, and some ship for explicit use rather than automatic dispatch. But it means there are colourfiles on disk that no automatic rule will ever select, reachable only through `grc -c` or a direct `grcat` invocation, and an audit that reads only `grc.conf` will not have looked at all of the code that ships.

The colouriser as an injection surface

A colouriser reads bytes from an untrusted stream and writes bytes to your terminal. Terminals interpret some of those bytes as commands. That is the whole of the problem, and it is not hypothetical: a log line containing escape sequences is a log line that can move the cursor, repaint earlier output, or change the window title, and on terminals with the relevant features enabled it can do considerably more.

The fleet's two colourisers take deliberately different positions on this, and both positions are correct for the tool that holds them.

`grcat` re-emits each character's *original* bytes and interleaves colour escapes around them. This book has already established why: the byte stream is the input's bytes with escapes interleaved, never substituted, which is what makes `cmd | grcat conf.log | grep foo` find what `cmd | grep foo` would find, and what makes invalid UTF-8 survive the round trip. The security reading of that same guarantee is that `grcat` is transparent to escape sequences already present in its input, by construction. It cannot strip them without ceasing to be a filter and becoming a formatter, and it cannot strip them without diverging from the upstream whose behaviour is its specification. So it does not, and the correct statement of its contract is that it adds decoration to a stream and makes no promise about the safety of what was already there.

`zcolorizer` is not under that constraint, because full byte-parity with ccze is not the goal for most of its surface, and it pins input-sanitization behaviour with engine tests. That is the right split, and the reason it is not an inconsistency: a tool whose specification is another tool's observable output cannot sanitise; a tool that owns its own output contract can, and should.

The generalisable lesson is about where a mitigation belongs. Neither colouriser is the last program in the pipeline — the terminal is — and the terminal is the only component that knows which escape sequences it will act on. A filter that tried to strip “dangerous” escapes would be maintaining a denylist against an unbounded and terminal-specific set, would break legitimate pre-coloured input, and would still not be able to promise safety. The honest posture is the one the fleet takes: state the transparency plainly, and let the operator decide whether a given stream should be piped into a terminal at all.

Shared-directory state

`temprs` is the one tool in the fleet whose normal operation leaves persistent state in a directory that other users may be able to see. Its stack lives in `$TMPDIR/temprs` by default, and every entry is a real file containing whatever you piped in — command output, API responses, configuration fragments, anything. That data is protected by nothing except the permissions of the files and the directory, which is to say by the process umask and by whether `$TMPDIR` is a per-user directory or a shared one. Anyone reaching for `temprs` as a scratch space for sensitive output should set `TEMPRS_DIR` to a location they control rather than relying on the default, and should use `--expire` and `-c/--clear` as a matter of routine rather than as a cleanup afterthought, because the tool's selling point is precisely that the stack outlives the shell.

Two pieces of the design earn their keep here. The first is the `-c/--clear` guard: clearing removes the parent directory of the master record with `remove_dir_all`, and the safe contract requires the directory's basename to contain the substring `temprs`, case-insensitively, before anything is removed. That check exists because `TEMPRS_DIR` is a user-supplied path feeding a recursive delete, and `TEMPRS_DIR=$HOME` followed by `tp -c` is a catastrophe any user could inflict on themselves by accident. The guard is crude — a directory named `not-temprs-really` passes it — and crude is the right calibration, because its job is to catch the accident, not to defeat an adversary who is already choosing the argument.

The second is the process-lifetime exclusive lock. `TempApp::new()` creates `temprs-stack.lock` and calls `lock_exclusive()` on it, and the handle is held in an underscore-prefixed field for its RAII side effect alone, releasing when the process exits. The concurrency argument for this is made elsewhere in the book; the integrity argument is that the atomic `rename` of `temprs-stack.tmp` over `temprs-stack` guarantees no reader sees a torn record, and the lock guarantees no two writers interleave their read-modify-write cycles. Together those are what let the master record be trusted after an abrupt kill, which is what makes the self-healing startup path — prune entries whose files no longer exist, rewrite, continue — a repair rather than a guess.

Scanning is an act, not an observation

Everything else in this book reads. `nmaprs` writes: it sends packets to hosts that did not ask for them, and on many networks that is a policy violation, an incident, or a crime depending on jurisdiction and ownership. The tool's design reflects the seriousness in the one way a tool can — its capability matrix documents which port-state mappings each technique produces, so a user knows what their traffic will look like and what conclusions the results support — but no amount of engineering makes the authorisation question the program's to answer. The book's worked session that audits a subnet is written against address space the reader is presumed to own, and that presumption is load-bearing.

What the posture does not include

Two absences are worth naming so nobody assumes otherwise. There is no cryptographic verification anywhere in the fleet's normal operation: no signed config files, no integrity checking of the vendored data files at run time, and no authentication on any of the tools in this book beyond the socket-mode and token mechanisms already documented for the two native hosts. And there is no sandbox: these are ordinary processes with the invoking user's full authority, which is the only way `lsoprof` could do its job in the first place.

What the fleet does have, and what is worth more than either of those for this class of program, is a small and durable dependency floor. `grcirs` depends on exactly two crates. Nothing in the fleet fetches from the network at build time. Only two of the eight carry a `build.rs` at all — `nmaprs`, to compile one C++ object for its OS-fingerprint expression matcher, and `htoprof`, to run its anti-drift gate. A supply chain you can read in an afternoon is a security property, and it is one of the few that improves rather than decays with time.

Bytes, characters, and columns — three coordinate systems and the bugs between them

Every program in this book moves text from a source that produces bytes to a destination that displays glyphs, and in between it has to answer questions like “where does this match start,” “how wide is this column,” and “did I write back what I read.” Those questions live in three different coordinate systems — bytes, Unicode scalar values, and display columns — and essentially every text bug in a terminal program is a place where one of them was silently substituted for another. The fleet contains clean specimens of all three layers being handled deliberately, which makes it a good vehicle for saying what the layers actually are and where the seams between them run.

The byte layer, and the things that are not text at all

The lowest layer is where a program treats its data as an opaque sequence of octets that must survive unchanged. Three places in the fleet operate there, and each does so because the data genuinely is not text.

`tempres`’s master record is the purest case. It stores filesystem paths, and a Unix path is a sequence of bytes containing neither NUL nor `/` — not required to be valid UTF-8, not required to be valid in any encoding, and free to contain newlines, tabs, and control characters. The record format solves framing by using the one byte that cannot occur: a single `\0` separates a path from its optional tag, and `\0\0` separates records. That choice buys unambiguous parsing for any path the filesystem can produce, and it costs one thing, which the tool then pays explicitly — tag names must not contain NUL, enforced with a dedicated error at both `--name` and `--rename` time, because a tag with an embedded NUL would forge a record boundary.

The cost shows up again at the shell boundary, and the fix there is the most instructive detail in the whole design. Zsh variables cannot hold NUL bytes, so the live completion helper cannot simply read the master record into a variable and split it. It pipes the record through `tr '\0' '\x1e'`, turning `\0` into `\x1e\x1e` and `\0` into `\x1e`, and splits on those instead. That is a transcoding between two framing alphabets, performed because one participant in the pipeline cannot represent the other’s delimiter — a byte-layer problem with a byte-layer solution, and a useful reminder that “use NUL as the delimiter” is correct for files and impossible for shell variables.

`lsOfrs`’s `-0` exists for the identical reason one level up: it emits NUL-separated field output so a consumer can split on NUL and be immune to filenames containing newlines. The upstream shipped that flag because the naïve line-oriented parse of `lsOf` output is wrong for any file whose name contains a newline, and any tool that lists filenames and does not offer a NUL-delimited mode is quietly telling its users to write a parser that will eventually be wrong.

`grcat`’s handling is the third specimen and the most demanding, because it has to be simultaneously byte-faithful and character-addressable. Its `decode_units` routine decodes valid UTF-8 one-to-one and turns each *undecodable* byte into exactly one `U+FFFD` unit while preserving the original byte in a parallel `orig` vector. Two properties fall out of that shape, and they are the whole reason for it. One replacement unit per bad byte keeps character indices aligned with what the reference Python implementation produces under `surrogateescape`, which is what makes regex match offsets agree between the two implementations. And keeping the source

bytes means output re-emits the invalid byte rather than the lossy replacement, so colourising cannot corrupt non-UTF-8 command output.

That is a parallel-array design, with the usual hazard of parallel arrays: the two must stay index-aligned or the output is garbage. The port's own code records where the alignment breaks — after a `replace` rewrite, `orig` no longer corresponds to the mutated line, so the assembly path uses the decoded characters instead. That exception is not a wart; it is the honest consequence of a directive that rewrites the line, and the alternative, tracking byte provenance through a regexp substitution, is not implementable in a form anyone would want to maintain.

The character layer, and why indices are a dialect

The middle layer counts Unicode scalar values, and the fleet's clearest use of it is `grcat`'s per-character `cline` array of escape strings. Colours are laid down into that array by character position; group spans are accumulated as `(char_start, char_end, colour)` triples; the final line is assembled by walking the array and emitting an escape only when it differs from the previous character's. Every index in that machinery is a character index, not a byte index.

The reason is compatibility rather than aesthetics. The tool being reproduced is a Python program, and Python's `re` module indexes matches by character position in a `str`. A Rust implementation that used byte offsets would produce identical results for pure ASCII and diverge the moment a multi-byte character appeared before a match — the colour would land on the wrong span, off by the number of continuation bytes that preceded it. This is one of the few places in the whole port where a reasonable implementation choice silently produces wrong output for inputs that a fixture set of ASCII lines would never catch, which is exactly why the round-trip tests for invalid UTF-8 and multi-byte runs are the ones worth having.

The general form of the lesson: when you port a program, its *index dialect* is part of its interface. A tool that reports offsets, or that positions decoration by offset, has committed to a unit of measure, and translating the algorithm without translating the unit is the most invisible way to be wrong.

The column layer, where the terminal disagrees with everyone

The third coordinate system is display columns, and it is the one where a program's model can be correct at both lower layers and still produce a broken screen. A terminal renders a grid of cells. The number of cells a string occupies is neither its byte length nor its character count. East Asian wide characters occupy two cells. Combining marks occupy zero, attaching to the preceding character. Zero-width joiners and variation selectors occupy zero. Control characters occupy an unpredictable number, frequently zero, sometimes with side effects. And ANSI escape sequences — the exact thing every tool in this book emits — occupy zero cells while consuming a great many bytes.

That last one is the case the fleet hits constantly, because every tool here draws boxes. `tempris` frames content in  and rules it with a cyan  bar. The system tools draw columnar tables with headers and badges. Any of that decoration computed by measuring a string that contains colour escapes will be wrong by the length of the escapes, and the failure presents as a box whose right edge wanders as the content's colouring changes — visible immediately, but only to someone looking at the case where a coloured value appears.

The structural defence, and the one the fleet's design supports, is to keep the layout computation and the colour application in different passes over different representations. `grcat` demonstrates it most explicitly: the `cline` array holds one escape *per character position*, and escapes are emitted during assembly rather than being embedded in the string being indexed. Layout reasons about positions; colour is applied at emission. Nothing in that pipeline ever measures a string that has escapes in it, because at the moment measurement matters the escapes do not exist yet.

The same separation is why the theme engine and the layout engine are different subsystems in the TUI tools rather than one. A palette entry is a colour, not a prefix baked into a label. The ratatui-based tools get this from the widget model, which tracks style and content separately; the tools that draw by hand have to maintain it by discipline.

Wide and combining characters are a harder problem than escapes, and here the book should be careful to say what it knows. Non-ASCII text reaches these tools through several doors: process names and command lines in `htoprs` and `lsofrs`, filenames in `storageshower`, hostnames and reverse-DNS results in `iftoprs` and `nmaprs`, and arbitrary log content in the colourisers. What this book documents about the handling is what the per-tool chapters record — for instance, `lsofrs`'s `+c` command-column width with `+c0` for unlimited, a column-budget mechanism inherited from the upstream. It does not document a width-measurement implementation for any tool, and this chapter will not invent one. The honest statement is the general one: a column budget expressed in characters and a column budget expressed in cells are different budgets, they agree on ASCII, and a reader auditing a specific tool's behaviour on CJK or combining-mark input should test it rather than assume from the presence of a width flag.

Filenames are not strings, and the fleet's answer to that

A recurring consequence of the byte layer is that the two tools which print paths in bulk — `lsofrs` and `storageshower` — are printing data that is not guaranteed to be text. The fleet's answer is not to solve the problem in the renderer but to offer an output mode where the problem does not arise. `lsofrs`'s `-0`, its `-F` field output, `--csv` with RFC 4180 quoting, `-J` JSON, and `temprs`'s `--path INDEX` all exist so that a consumer can receive a path through a framing that survives whatever the path contains. The human-readable columnar view is allowed to be a rendering, because a human is looking at it; the machine-readable modes are held to a stricter standard, because something is parsing them.

That division is the practical resolution of the whole three-layer problem, and it is worth stating as a rule. Choose the coordinate system by consumer: bytes for anything that will be parsed or written back, characters for anything whose positions must agree with another implementation's, and columns only at the last moment, for the pixels. Programs get into trouble when a value computed for one consumer is reused for another — a character count used as a column width, a rendered string re-parsed as data — and every one of those is a seam the fleet's designs keep visible rather than papering over.

Signals, terminal state, and the obligation to hand the terminal back

A Unix program that draws on a terminal borrows something it did not create and must return. A program in a pipeline participates in a signal contract it did not write alone. Both obligations are invisible when everything succeeds and extremely visible when it does not — a crashed TUI leaves a shell in raw mode with no echo, and a Ctrl-C that kills the wrong member of a pipeline leaves an orphaned child writing to a terminal nobody is reading. The fleet contains worked examples of both problems, and the reasoning behind them generalises well past these eight tools.

Signal disposition is a pipeline-wide contract

The clearest specimen is `grc`, because it is two cooperating processes with a third — the wrapped command — in between, and Ctrl-C has to do the right thing for all of them.

When you press Ctrl-C at a terminal, the kernel delivers SIGINT to every process in the foreground process group, not to “the program you are looking at.” Under `grc some-command`, that group contains the `grc` launcher, the wrapped command, and the one or two `grcat` colourisers the launcher spawned. The naive outcome is that all of them die at once, in an order nobody controls, and the visible result is a partially-coloured last line, no exit status propagated, and possibly a process left behind.

The port’s answer is to designate an owner. `run_coloured` ignores SIGINT with `libc::signal(SIGINT, SIG_IGN)` so that Ctrl-C reaches the child rather than killing `grc` before it can reap the child and report its status, and `grcat` also ignores SIGINT itself, so the launcher’s forwarding scheme is the single owner of Ctrl-C. Read the two decisions together and the design is legible: the wrapped command is the only member of the group that acts on the interrupt, and the two wrapper processes stay alive specifically so that the shutdown is ordered. The launcher waits on the child first and only then reaps the colourisers it started, and returns the child’s exit code, falling back to zero when the status carries no code — which is what happens when the child dies by signal.

The property that makes this correct rather than merely clever is that the observable behaviour matches what you would get without the wrapper. Ctrl-C interrupts your command; your command’s exit status is what the shell sees; whatever your command had written is fully coloured and flushed before anything exits. A wrapper that is transparent under signals is doing the same job as a wrapper transparent under exit status and stderr formatting, and it is the same argument this book makes about the `grc: <command>: <reason>` error text: for a port, everything observable is API, and signal behaviour is observable.

SIGPIPE, EPIPE, and a default that surprises Rust programs

The other signal every filter meets is SIGPIPE, and it has a wrinkle specific to Rust that is worth stating because it produces a class of bug reports that look like the program is broken.

In C, writing to a pipe whose read end has closed delivers SIGPIPE, whose default disposition terminates the process. That is why `yes | head -1` exits promptly: `head` closes, `yes` takes the signal, `yes` dies, and nobody thinks about it. The Rust standard library sets SIGPIPE to ignored at startup, which means the write

does not kill the process — it returns an `EPIPE` error instead, and the program must decide what to do. A program that ignores the error keeps running, keeps producing output nobody reads, and does not exit until its own input is exhausted.

`grcat` handles this explicitly: `EPIPE` — downstream closed — breaks the main loop cleanly. That single decision is what makes a long-running command piped through `grcat` into `head` terminate the way a user expects instead of running until the source dries up. It is also why the handling has to be in the loop rather than in a signal handler: since the signal is ignored, the only place the condition is observable is the return value of the write.

The generalisable rule for anything in a pipeline: treat a write error on stdout as a normal termination condition, not as an error to report. Printing a diagnostic about a broken pipe to stderr is worse than silence, because the common case is that the user deliberately closed the pipe, and a tool that complains about being cut off by `head` is a tool that gets wrapped in a stderr redirect to `/dev/null`, which then hides the errors that mattered.

Resize: the event you can poll instead of catch

Terminal resize arrives as `SIGWINCH`, and handling it in a signal handler is a familiar source of trouble — the handler runs at an arbitrary point, may not safely touch most state, and must communicate with the main loop through something async-signal-safe. The TUI tools in this fleet mostly do not have to solve that problem, because the terminal backend converts it into an ordinary event.

`storageshower`'s loop is the documented example, and its handling of the resize event is a single word: `Resize` is a no-op, because the next draw already re-measures. The loop reads the terminal size at the top of each iteration, subtracts the chrome height, calls `ensure_visible` so the selection stays on screen, re-sorts, draws one frame, and polls `crossterm` for input with a 200 ms timeout. Nothing in that sequence depends on having been told the size changed; it re-derives the layout from the current size every frame, so a resize is absorbed by the next iteration whether or not the event was delivered.

That is worth naming as a design principle rather than an implementation detail: a render loop that recomputes layout from measured state each frame does not need resize notifications for correctness, only for latency. The event, when it arrives, does nothing except wake the poll early. Compare the alternative — cache the geometry, update it on `SIGWINCH` — which is faster and which fails permanently the one time a notification is missed or arrives during initialisation. The tools that run a tick anyway, like `iftoprs` at a 33 ms tick, pay nothing for the recompute and get the robustness free.

Raw mode and the alternate screen are borrowed, and the loan has terms

Entering a TUI means putting the tty in raw mode — no line buffering, no echo, no kernel-level signal generation from keystrokes — and switching to the alternate screen so the user's scrollbar survives. Both are global mutations of a resource owned by the terminal emulator and shared with the shell that will run after you exit. The obligation is to restore both, on every exit path, exactly once.

“Every exit path” is the part that is easy to get wrong, because the paths are more numerous than they look: a normal quit, an error return from deep in the event loop, a panic in the draw code, a panic on a background thread that poisons a lock the UI thread then unwraps, a fatal signal. The fleet's documented answers cluster around getting diagnostics *out* of that situation rather than around preventing it, and `htoprs`'s crash handling is the case study this book records: a TUI on the alternate screen with the tty in raw mode cannot report a panic to the user, because the message is painted over the instant the screen is restored, so crashes go to `~/.cache/htoprs/crash.log` from two funnels — a process-wide panic hook and the ported `SIGSEGV/SIGBUS` handler's stderr report — and the restored terminal prints one line saying where the report went.

Two details of that arrangement are worth drawing out because they are the general shape of the problem. First, the ordering is fixed and non-obvious: the diagnostic must be *written* before the screen is restored, and *announced* after, because the announcement is the only thing the user will still be able to see. Second, the two funnels exist because they are reachable from different states. A panic hook runs on the panicking thread with the runtime intact and can allocate, format, and capture a backtrace. A signal handler for SIGSEGV runs after the process has already done something illegal, where the set of operations that are safe is close to “write to a file descriptor.” Wanting both paths to end in the same logfile is natural; implementing them as one function is not possible, and the fact that this book records two funnels into one file rather than one handler for both is the honest shape of that constraint.

There is a corollary about interactive tools that non-interactive users hit constantly: a program that unconditionally enters the alternate screen is unusable in a pipeline, in a script, and under any harness that captures output. `lsotrs` gets this right by making the mode conditional rather than the entry conditional — its live modes never enter the alternate screen non-interactively; they print one snapshot and exit. That is better than refusing to run and better than running and producing escape soup, and it is the same collapse-to-plain principle the output-contract chapter describes, applied to terminal state rather than to colour.

Suspend, and what the shell expects back

The last piece of terminal-state etiquette is job control. Ctrl-Z at a terminal in cooked mode generates SIGTSTP and the shell takes the foreground back — but a TUI has put the terminal in raw mode precisely so that keystrokes are not interpreted by the kernel, which means Ctrl-Z arrives as a byte in the input stream rather than as a signal, and it is the application’s decision whether to act on it. Whatever the tool decides, the invariant is the same one as for exit: if control returns to the shell, the terminal must be back in the state the shell expects, and if the tool resumes, it must re-enter raw mode and redraw, because another program may have used the terminal in between.

This book does not document a suspend implementation for any of the four TUI tools, and it will not claim one. What it can say is what the invariant is and why the failure is memorable: a tool that returns to the shell without restoring cooked mode leaves the user typing into a shell that does not echo, recoverable with a blind `reset`, and it is the single most common way a terminal program earns a reputation for being broken.

Namespaces, containers, and the boundaries of a view

Everything the system tools report is a view through an interface the kernel maintains, and on Linux those interfaces are namespaced. `/proc` shows the process namespace of the reader. `/proc/net/*` shows the network namespace of the reader. `/proc/mounts` shows the mount namespace of the reader. None of that is a caveat about containers specifically — it is the definition of what those files are — but containers are where a user first meets the consequence, because they are the common case in which the reader's namespaces differ from the ones the interesting processes live in. This chapter is about what that does to the four system tools, and about the discipline it implies for anyone reading their output.

`/proc` is a view, not the truth

`lssofs` walks `/proc/<pid>/fd` and reads the `cwd`, `root`, and `exe` symlinks; `htoprs` reads the process table; `iftoprs` reads `/proc/[pid]/fd/` and `/proc/[pid]/comm` to attribute flows. All three enumerate whatever `/proc` contains, and what `/proc` contains is determined by the PID namespace of the process doing the reading and by the mount that is visible at `/proc`.

Run any of them inside a container and they see the container's processes, with the container's PID numbering. Run them on the host and they see everything, including the containerised processes, but with host PIDs that do not match the numbers anyone inside the container reports. Neither view is wrong and neither is complete: they are answers to two different questions, and the pathological outcome is a user who takes a PID from one and uses it against the other.

The symlink reads have a sharper version of the same problem. `/proc/<pid>/root` exists precisely because a process's root may differ from yours, and `exe` points into that process's mount namespace. Reading the target gives a path that is meaningful in the *target's* namespace, and a path that is meaningful there may not resolve — or may resolve to something entirely different — in yours. A tool that prints those targets is printing correct data in a frame of reference it cannot label with anything the user would recognise. The practical rule for reading `lssofs` output on a container host is that the paths are the process's paths, and comparing them to your own filesystem is a category error unless you have established that the two share a mount namespace.

One more Linux-specific behaviour is worth knowing, because it produces output that looks like a bug in the tool. The `hidepid` mount option on `/proc` restricts what an unprivileged reader can see of other users' processes — at its stricter setting, other users' `/proc/<pid>` directories are not even visible. A process viewer run under such a mount reports a process table that is complete for the user and radically incomplete for the machine, without any error, because from the tool's point of view the entries simply are not there. This is the strongest practical argument for the fleet's convention of showing an explicit absent marker rather than a guess: the difference between “no such information” and “not visible from here” is exactly the difference the user needs, and only a tool that refuses to fill in blanks preserves it.

Network namespaces break flow attribution structurally

`iftoprs` is the tool this bites hardest, because it is the one that joins two independently namespaced sources, and understanding why requires stating what its attribution actually does. It captures packets on an interface with `pcap`, and separately parses `/proc/net/tcp`, `/proc/net/tcp6`, `/proc/net/udp`, and `/proc/net/udp6` to map a connection four-tuple onto an inode, then scans `/proc/[pid]/fd/` to map that inode onto a process, then reads `/proc/[pid]/comm` for its name. That is a join across two independent sources, and both sources are namespaced — but they are namespaced along the same axis only when the capture interface and the socket tables belong to the same network namespace.

They frequently do not. A container with its own network namespace has its own `/proc/net/tcp`, and its sockets do not appear in the host's. Traffic from that container observed on a host bridge or on the host's uplink is real traffic whose endpoint sockets are invisible to a host-side reader of `/proc/net/*`. The join finds nothing, and the flow is correctly reported as unattributed.

The important consequence is that this is not a defect to be fixed by better parsing. No host-visible table maps a container's socket to a container's process without entering that container's namespaces, and a tool that did enter them would need privilege it does not otherwise require and would be solving a container-orchestration problem rather than a traffic-monitoring one. The correct engineering response is the one the fleet already takes: report the flow with its byte counts, which are fully accurate because they come from the capture, and mark the process attribution absent, which is fully accurate because it is. Because the JSON output omits `process_name` and `pid` via `skip_serializing_if` when unknown rather than emitting an empty string, a downstream consumer can distinguish the two cases without a sentinel convention — and in a containerised environment that distinction is not an edge case, it is most of the data.

The same reasoning inverts cleanly. Run `iftoprs` inside a container and the capture sees only that namespace's interfaces while `/proc/net/*` is the container's, so the join works again and the attribution is good — for the container's traffic only. Choosing the vantage point is choosing which question you are asking, and no vantage point answers both.

Mount namespaces, overlays, and what a filesystem walk means

`storageshower` reads `/proc/mounts` on Linux and calls `getmntinfo` on macOS, then walks and measures. Inside a container, `/proc/mounts` shows the container's mount table, which typically means an overlay root, a handful of bind mounts, and tmpfs for the usual runtime directories. Every number the tool then reports is correct for that view and describes something different from what the same numbers would mean on the host.

Two specific distortions are worth naming. An overlay filesystem's reported free space is the free space of the underlying upper-layer filesystem, which is shared with every other container using the same backing store — so “this container has forty gigabytes free” is a statement about a resource a different container can consume. And a bind mount of a host directory into a container appears as a mount whose usage figures are the host filesystem's, so a disk-usage view inside the container will attribute host-scale numbers to a path that holds very little of the container's own data. Neither is a measurement error; both are what the kernel reports, and both mislead a reader who assumes the figures describe a private filesystem.

The `du`-versus-`df` divergence the book's first worked session covers has a container-specific member of the same family, worth adding to the reader's list: on an overlay, files present in the lower layer but deleted in the upper still consume backing-store space while being absent from a walk of the merged view. A tool that walks and a tool that asks the filesystem for its totals will disagree, for a reason that is neither tool's fault, and knowing which of the two you are reading is the whole of the diagnosis.

pcap needs more than a namespace

Capture adds a privilege dimension on top of the namespace one. Opening a capture device requires privilege that a container does not have by default, and the usual container security posture drops exactly the capability

involved. The observable result of running `iftoprs` in such a container is a device open failure, which its capture loop treats as transient and retries with exponential backoff — correct behaviour for a device that has genuinely gone away, and a retry loop that will never succeed against a permission problem that is structural.

That is a real tension and worth stating rather than glossing: retry-with-ceiling is the right policy for transient failure and the wrong policy for a permanent one, and a device open that fails for lack of privilege is permanent within the life of the process. The reason the fleet's design still lands correctly is the distinction the failure chapter draws — missing privilege is detected and refused up front where it is detectable, and only genuinely ambiguous device errors reach the retry path.

macOS: no namespaces, a different partition

The macOS side of every tool takes a different path — `proc_listpids` and `proc_pidinfo` in `lsofrs`, the `libproc` crate's `pidfdinfo/SocketFDInfo` in `iftoprs`, `getmntinfo` and IOKit block-storage counters in `storageshower` — and none of the namespace reasoning above applies, because the kernel does not partition its process or network views that way. What it does have is a system-integrity boundary that removes information from even a privileged reader's view for certain protected processes, plus a per-process entitlement model. The result is a different shape of the same phenomenon: the view is partial for reasons the tool cannot see or report on, and the correct behaviour is again to show what was obtained and mark what was not.

The generalisable conclusion across both operating systems fits in one sentence: a system tool's output is a measurement taken from a vantage point, and the tool's job is to make the measurement accurate and the vantage point legible, not to pretend the vantage point does not exist. Every design decision this chapter touches — absent markers instead of guesses, omitted JSON keys instead of empty strings, refusing rather than substituting when privilege is missing — is a different expression of that one obligation.

Extending the fleet without breaking the port

Anyone who uses these tools for long will want to change one, and the fleet is unusually explicit about which changes are safe. That explicitness is not politeness — in one case it is enforced by a gate that fails the build. This chapter is the map of the extension points: where a change is data, where it is code in a sanctioned tree, and where it is a change to a port and therefore subject to the fidelity rules.

The two-tree rule, generalised

The organising idea across the ported tools is that a crate contains two kinds of code with two different specifications, and they live in different directories. `htoprs` splits into `src/ported/<file>.rs`, one module per htop C file with every free `fn` carrying a `/// Port of` doc comment, and `src/extensions/<name>.rs` for htoprs-original work. `zshrs` uses the same split. `ztmux` uses it, with 130 extension modules under `src/extensions/`.

The specification for the first tree is the upstream's source. The specification for the second is whatever the author decided, and the only constraint is that it must not change the behaviour of the first. That is why `htoprs`'s extension features are bound to keys htop leaves free, and why `ztmux`'s extensions are gated behind `set -g toggles so @ztmux-ratatui off` restores the byte-for-byte plain-tmux draw path. In both cases the escape hatch is not “we tested that it does not interfere” — it is a switch that removes the extension from the execution path entirely, so the parity oracle can run against a build that has the new code compiled in.

The enforcement is what makes this more than a convention. `htoprs`'s `build.rs` runs on every `cargo build`, `test`, and `check`, extracts every free `fn` name under `src/ported/`, and checks it against a snapshot of htop's real C function names; a function with no upstream counterpart fails the build, and it fires even under `cargo test --test X` so a reimplementer cannot be routed around it. `ztmux` carries the analogous gate in `tests/ported_fn_names_match_c.rs`, with pre-existing wrappers frozen in an allowlist to burn down.

For a contributor the practical consequence is a single question asked before writing anything: does this function exist upstream? If yes, it belongs in the ported tree, it carries its citation, and its behaviour is not yours to choose. If no, it belongs in the extensions tree, and putting it in the ported tree will not compile. There is no third option, and that is the point — the gate converts an architectural intention into a mechanical fact, which is the only form of architectural intention that survives a year of commits.

The rule that follows for anyone considering a small improvement to a ported function: an improvement to a port is a divergence from the specification. If htop rounds a percentage a particular way, or lsof formats an offset as `0t<offset>`, or grc's help banner leads with `Generic Colouriser 1.13`, those are not defects to be corrected. They are the thing being reproduced, and changing any of them breaks something downstream that this book has repeatedly shown to be real — a script matching on stderr text, a parser expecting a field prefix, a CI smoke test grepping for `Colouriser`.

Data-file extension points, which need no code at all

Two tools are extensible without touching Rust, and they are the ones most people will actually want to extend.

`grcrs` reads the upstream data files: a `grc.conf` of two-line rules and a directory of colourfiles. Adding support for a new command means adding a regexp line and a colourfile name to a `grc.conf` in one of the searched directories, and dropping the colourfile into one of the searched colourfile directories. The precedence rules covered in the security chapter are the whole of what you need in order to place the files: within a `grc.conf`, the first matching regexp wins, so a more specific rule must precede a more general one; across the config search path, the last match wins, so a user config beats a system config.

The colourfile grammar itself is where most of the expressive power is, and it repays reading the shipped files before writing one. The block structure is a run of `keyword=value` lines terminated by any line whose first character is not an ASCII letter — which is why the `=====` separators in the shipped configs work as delimiters, and why a stray leading space can silently end a block. The recognised keywords are held in a ten-entry `KEYWORDS` array: `regexp`, `colours`, `count`, `command`, `skip`, `replace`, `concat`, plus the `grcrs` extensions `trend`, `key`, and `metric`. `color`, `colors`, and the singular `colour` all alias to `colours`; an unrecognised keyword is a hard parse error rather than a warning, which is the right choice for a config format where a typo would otherwise mean a rule that silently does nothing.

Two grammar details save time. The `colours` value is comma-split into one entry per capture group, and within a group whitespace-separated tokens are resolved and concatenated, so `bold red` composes into one escape. And any token that is not in the colour table but is a quoted literal is unescaped as a Python string literal would be, decoding octal, hex, and the common single-character escapes — which is how a colourfile writes a 256-colour code as an explicit `\033[...m` sequence and gets exactly the escape it asked for. The octal decoder stops after three digits and the hex decoder after two, matching Python's literal rules, so a colour escape followed immediately by a digit does not silently swallow it.

The `trend/key/metric` family is the one directive set with no upstream equivalent, and its design is the template for how to extend a ported grammar: strictly additive, so a colourfile that never mentions the new keywords takes the plain path and behaves exactly as upstream. That property is what makes the extension safe to ship in a tool whose value proposition is compatibility. A new keyword costs an entry in the keyword table and nothing else; a changed meaning for an existing keyword would cost the compatibility guarantee entirely.

`zcolorizer`'s format library is the second data-shaped extension point, though its modules are Rust rather than config. Its three-layer decomposition — rules, themes, engine — is what makes a new format an addition to one layer instead of a change to three, and the format modules it already carries for `systemd`, `JSON`, `Kubernetes`, and `Postgres` are the worked examples of the shape a new one takes.

Palettes, and an invariant that will fail your build

The 256-colour palette set is shared across `storageshower`, `iftoprs`, `lsofrs`, `htoprs`, and `zcolorizer`, and the theme chapter records how it travelled: 30 `ColorMode` variants originating in `storageshower`, 31 in `iftoprs` after the parity palette `Iftopcolor` was added, 31 in `lsofrs` asserted by an enum-length check, 31 six-colour palettes in `htoprs`, and 31 generated palettes in `zcolorizer` plus a hand-written `ccze-classic` for parity with the tool it colours after.

Adding a palette therefore means adding an enum variant in one crate and reckoning with the roster in the others. The enum-length assertion in `lsofrs` is the mechanism that makes this a test failure rather than a silent drift, and it is the pattern worth copying: a shared roster maintained by convention across five crates will diverge, and the only thing that stops it is a check that fails when the count changes. The check does not know what the right count is; it knows that the count changing is an event requiring a human to have looked.

The other half of adding a palette is `shift_lighter`, the six-index cube-arithmetic helper that moves with the palettes from `iftoprs` into `zcolorizer`. A palette is not six arbitrary colours — it is a set with a derived relationship, and a new palette that does not respect the derivation will look correct in the one place it was tested and wrong in the tool that computes a lighter variant from it.

Where extension is not the right answer

Two categories of change should not be made as extensions at all.

The first is anything that changes what a parity oracle compares. `grcs`'s end-to-end tests assert on exact coloured bytes, with expectations cross-checked against the reference Python `grcat`. A change that alters the byte stream for any existing colourfile is not an extension — it is a fork of the specification, and the test suite will say so immediately, which is the desired outcome.

The second is anything that adds a runtime dependency the fleet's dependency floor would not accept. That floor is a design constraint with a specific rationale — crates that will still build cleanly years from now — and it is why `grcs` depends on exactly two crates and why none of the eight depends on a memory-mapping crate. An extension genuinely worth a new dependency is worth the argument for it; an extension that quietly adds three transitive trees to a tool whose value includes being buildable in a decade has spent something that does not appear in the diff.

Diagnosing the fleet – a symptom index

The worked-sessions chapter walks forward from a question to an answer. This one walks backward from a surprise to its cause, because the failure reports these tools generate are dominated by a small number of situations where the tool is behaving exactly as designed and the design is not what the user assumed. Each entry below states the symptom, the mechanism behind it, and the check that separates it from the alternatives.

The colour disappeared

The first thing to establish is whether colour was suppressed or never generated, because the two have different causes and different checks.

Colour suppressed is the common case and it is almost always the TTY gate working correctly. Every tool here disables colour when its output descriptor is not a terminal, which means colour disappears under `| less`, under `| tee`, under command substitution, under `>` redirection, and inside any test harness or CI job that captures output. The union of signals is the same across the fleet: is the descriptor a TTY, did the user force colour on or off, and is `NO_COLOR` set — any one can turn colour off, and only an explicit force turns it on against a pipe. So the check is to force it and see: `--color=always` on `lsOfirs`, `--colour=on` for `grc`, `-C/--force-color` for `zcolorizer`. If forcing restores the colour, the gate was doing its job and the fix is to keep forcing it — which is exactly what a pager wants, since `less -R` renders the escapes when told to.

```
lsOfirs -i --color=always | less -R
journalctl -f | zcolorizer -C | less -R
```

Colour never generated is the `grc` case and it has an entirely different diagnosis; it is covered two entries below.

The inverse symptom — escape sequences appearing as literal garbage in a file or in a downstream program — means colour was forced somewhere it should not have been. Look for a `--force-color`, a `--colour=on`, or a shell alias supplying one, because the automatic path does not produce this.

lsOfirs or htoprs shows less than you expected

An empty or short result from a tool that enumerates processes is nearly always a visibility question rather than a filter question, and the fastest way to separate them is to re-run the same command under `sudo` and compare the row count.

If privilege changes the answer, you were reading a partial view, and the reasons are the ones the namespace chapter covers: another user's processes are not readable, `hidepid` may be restricting `/proc`, or you are on one side of a container boundary and the processes are on the other. If privilege does not change the answer, the filter is doing something you did not intend — and the most common cause with `lsOfirs` specifically is the option grammar. The normalizer accepts and silently discards a set of upstream flags with no meaning in this implementation: `-b -O -C -M -X -E -L` are consumed within a cluster and dropped, `-z -Z -S -f` drop any attached argument and end the cluster, and `-A -k -m -D -e` are treated as taking a required argument, so the following argv token is swallowed if none is attached.

That last group is the one that produces a genuinely confusing result. Pass one of those options without an attached argument and the next token on your command line is consumed as its argument — so a filter or a

path you intended as a separate argument silently disappears, and the tool runs a broader query that looks like it ignored you. The check is to move the suspect flag to the end of the command line, or remove it, and see whether the filter starts working.

grc colours nothing

`grc` decides which colourfile applies by joining the command and its arguments into a single string and scanning `grc.conf` top to bottom for the first regexp that matches. If no regexp matches, the command runs uncoloured with inherited stdio — no error, no diagnostic, just a normal-looking command. That silence is correct behaviour for a wrapper that must not break the commands it does not recognise, and it is why “grc did nothing” is the most common report.

Three causes, in order of likelihood. The command genuinely has no rule: check whether a colourfile exists for it at all, remembering that the vendored set carries more colourfiles than `grc.conf` names, so a colourfile may exist while no rule selects it. The match is failing on the path form: the leading `^([\w\.] + \ )?` fragment recurring across nearly every rule exists so that an absolute path matches the same rule as the bare command name, but a command invoked in a form that fragment does not cover will not match. Or a higher-precedence `grc.conf` is overriding: `find_config` scans every file in the search path and lets the last match win, so a rule in `~/grc` overrides one in `/etc`, and a user config you forgot about will beat the system one.

The direct check bypasses the launcher entirely. Run the command and pipe it into the colouriser yourself, which tests the colourfile without involving config resolution at all. If that colours correctly, the problem is dispatch; if it does not, the problem is the colourfile or the match.

```
some-command | grcat conf.something
```

One non-symptom worth knowing so it does not become a bug report: `grc --version` prints the help banner rather than a version line. `resolve_long("version")` does not resolve, `help()` fires, and the banner — whose first line is `Generic Colouriser 1.13` — is what appears. That is deliberate, and the CI smoke test greps that output for `Colouriser` precisely because the help path is where the version string lives.

iftoprs shows traffic with no process names

This is the structural case from the namespace chapter and it deserves a place in the symptom index because it looks like a failure and is not. Byte counts come from the packet capture; process attribution comes from a join against `/proc/net/*` and `/proc/[pid]/fd/`. When those two sources are in different network namespaces — a containerised endpoint observed from the host being the usual case — the join finds nothing, and the flow is reported with accurate counters and no owner.

The check is whether the connection appears in the socket table you are reading: if the four-tuple is absent from `/proc/net/tcp` on the machine where `iftoprs` is running, no amount of privilege will attribute it, because the information is not in that namespace. The fix, if you need attribution, is to run the tool from the vantage point where the sockets live rather than the one where the packets are convenient to capture — and to accept that a single run cannot give you both for cross-namespace traffic.

A different no-process symptom has a mundane cause: short-lived connections whose process exits between the capture and the lookup. There is no repair for that race in a sampling design, and it is one more reason the JSON output omits the key rather than emitting an empty string, so a consumer can count the unattributed flows rather than mistaking them for a process with no name.

iftoprs shows nothing at all

No flows, as distinct from flows without names, points at the capture rather than the join. Capture needs privilege to open the device, so the first check is `sudo`. The second is the interface: capture is per-device, and

the traffic you are looking for may be on a different one — a bridge, a virtual interface, or the loopback for local connections.

Loopback deserves its own sentence because it surprises people. Traffic between two processes on the same machine over the loopback address does not appear on the physical interface at all, and capturing it means capturing on loopback, which uses a different link-layer encapsulation. `iftoprs` decodes Ethernet, BSD loopback, Linux cooked capture, and raw IP link types precisely so that this works, but you have to point it at the right device.

Inside a container, an open failure is likely structural rather than transient: the capability required to open a capture device is usually dropped by default, and the retry-with-backoff path that correctly handles a device disappearing will not turn a permission problem into a success.

The `temprs` stack looks empty or wrong

Three mechanisms produce this, and each is distinguishable in one command.

The directory is different from the one you wrote to. The stack lives in `$TMPDIR/temprs` by default and `TEMPRS_DIR` overrides it wholesale, so a different `TMPDIR` in the current shell means a different stack. `tp -d` prints the managed directory and `tp -m` prints the master-record path; comparing them across two shells settles it immediately.

Entries were pruned because their files vanished. On startup `TempApp::new()` zips the stored paths against their names, filters out entries whose path no longer exists, and rewrites the pruned record. If something cleaned the temporary directory — a reboot, a periodic cleaner, a manual `rm` — the entries are removed on the next `tp` invocation rather than lingering as dangling indices. This is the self-healing property working, and its visible symptom is a stack that shrank without you removing anything.

An index resolved to something you did not mean. The resolution rule parses the argument as a signed integer first and only falls back to a tag lookup if that fails, so a purely numeric tag name is shadowed by index resolution — `1` is always index 1, never a tag named `1`. Negative indices count from the top, `-1` being the newest, and index `0` is invalid by construction since the stack is one-based. `tp -n` lists entries numbered, which is the fastest way to see what the numbers currently point at.

```
tp -d      # which directory is this shell using
tp -m      # which master record
tp -n      # what the indices currently point at
```

The box drawing is broken

Misaligned frames, a right edge that wanders, or replacement-character boxes in place of the `▢` glyphs have three separate causes and the fix differs for each.

Replacement characters or spacing artefacts in place of box glyphs is a font or locale problem, not a tool problem: the terminal is being asked to render characters it has no glyph for, or the locale is not UTF-8 and the multi-byte sequences are being interpreted as individual bytes. Check the locale before anything else.

A right edge that moves as content changes is a width-computation problem, and the diagnostic question is what is in the content that is not one column per unit: escape sequences, wide characters, or combining marks. The text chapter covers why those three are the recurring offenders.

Content that overflows the frame is the ordinary case of a value longer than the column budget. `lsOfirs` inherits an explicit control for this from the upstream — `+c` sets the command-column width and `+c0` means unlimited — which is worth reaching for before concluding the layout is broken.

Output stopped in the middle

A pipeline that ends early, with partial output and no error, is usually the downstream closing. `grcat` breaks its main loop cleanly on `EPIPE`, which is what makes a long-running command piped through it into `head` terminate promptly instead of running until the source is exhausted — so truncation at exactly the point the downstream stopped reading is the design working.

Truncation at an unrelated point is a different matter, and the first thing to check is whether the *source* died. Because the launcher waits on the wrapped child and only then reaps its colourisers, and returns the child's exit code — falling back to zero when the status carries no code, which is what happens when the child dies by signal — a command killed by a signal under `grc` can present as a successful-looking short run. Checking the exit status distinguishes a clean short output from a signal death only when the child exited normally; when it did not, the fallback to zero is deliberate and the truncation itself is the evidence.

Everything is slower than expected

The cost-model chapter explains where time goes in each tool; the diagnostic shortcut is that the tools have different bottlenecks and the wrong intuition sends people optimising the wrong thing. A process or descriptor enumeration is dominated by the number of syscalls against the pseudo-filesystem, so it scales with process and descriptor count and is barely affected by CPU speed. A filesystem walk is dominated by directory-entry I/O and cache state — the second run of the same walk is a fundamentally different measurement from the first. A capture is dominated by packet rate and is unaffected by how many flows are displayed. And a colouriser is dominated by regexp matching against every pattern for every line, so a colourfile's pattern count is a direct multiplier on its throughput.

One measurement mistake is worth calling out because it invalidates results silently: benchmarking any of these against a live system means the workload is changing underneath the measurement. A process viewer measured on a busy machine is measuring the machine as much as the tool, which is why this book's methodology chapter marks its numbers as the author's own, on one machine, and declines to present them as reproducible constants.

The environment contract – variables, precedence, and the state a tool keeps

Every program here is configured three ways at once: by its command line, by files on disk, and by the environment it inherits. The command line is documented per tool in the reference appendix and the config files are covered in the packaging chapter. The environment is the layer nobody writes down, and it is the one that produces the confusing bugs, because an environment variable set in a shell profile years ago is invisible at the moment the command runs and outlives every attempt to reproduce a problem. This chapter collects what the fleet reads from its environment, what precedence the layers have, and what state each tool leaves behind — three questions that turn out to be one question about where a program's configuration actually lives.

Three kinds of variable

The variables the fleet reads divide cleanly by who owns them, and the division predicts how each should be treated.

The first kind is a *system convention* the tools consume but did not invent: `$HOME`, `$TMPDIR`, `$EDITOR`, `$XDG_CONFIG_HOME`, `$XDG_DATA_HOME`, `$XDG_CACHE_HOME`, `$XDG_RUNTIME_DIR`, and `NO_COLOR`. A tool's only obligation with these is to honour them and to apply the standard fallback when they are unset. The fleet does this consistently: `$XDG_CONFIG_HOME` defaults to `~/.config` and `$XDG_DATA_HOME` to `~/.local/share` in `grc`'s two search paths; `htop`'s crash log honours `$XDG_CACHE_HOME`; `temp` falls back from `$TMPDIR` through the standard temp-directory resolution; `temp`'s `--edit` falls back from `$EDITOR` to `vi`. None of those fallbacks is interesting, which is the point — a system convention that a tool implements with a surprise in it is a bug regardless of how defensible the surprise is.

The second kind is a *tool-owned override*, and the fleet's examples are `TEMPRS_DIR`, which replaces the managed directory wholesale, `$CRASH_LOG`, which relocates `htop`'s crash file, and `$ZWIRE_HOST_SOCKET`, which overrides the socket path the host binds. These exist because the default location is derived from something the user may not control, and each is a full override rather than a modifier — the value replaces the computed path instead of adjusting it. That is the right shape for a path override, because a modifier (“use this prefix”) leaves the user reasoning about a composition rule, while a replacement leaves them reading the value they typed.

The third kind is a *test and diagnostic hook*, and it is worth separating because it should never be needed in normal operation. `HTOP_REF` points the parity harness at a specific reference `htop` binary, and `HTOP_C_SOURCE` points the xutils comparison at an `htop` source tree. Both exist so a developer can run a diff against a build that is not the one on `PATH`, and neither affects the shipped program's behaviour. The same category, in the covered fleet's tools, includes `AWKRS_CACHE=0` and `AWKRS_FUSEVM=0` for disabling a cache and a JIT offload path, and `VIMLRS_JIT_STATS=1/VIMLRS_NO_JIT=1` for reporting and disabling trace compilation. What makes these a category rather than a list is the shared property that every one of them is an *escape hatch that makes the tool slower or more verbose*, never faster or quieter. A diagnostic variable that improves behaviour is a feature that should have been on by default.

Precedence, and the one ordering that must never invert

Across the three configuration layers, the fleet's ordering is the conventional one: the command line beats the environment, and the environment beats the config file. `lsotrs`'s `--color` flag forces `always/never/auto` over the auto-detected default; `grcrs`'s `--colour=on|off|auto` overrides the `isatty` result; `zcolorizer`'s `-C/--force-color` overrides the automatic disable. In each case the explicit instruction wins over the inferred one, which is the only ordering a user can reason about, because it is the only one where doing something deliberately has a predictable effect.

The colour case has a wrinkle worth stating precisely, because it is the one place the fleet does not follow simple precedence. The signals are: is the descriptor a TTY, did the user force colour on or off, and is `NO_COLOR` set. Any of them can turn colour off, and only an explicit force turns it on. That is not strict precedence — it is a veto model, where suppression is easier than enablement — and it is correct for exactly one reason: the cost of wrongly emitting escapes into a file that something else parses is much higher than the cost of wrongly printing plain text to a terminal. When the asymmetry of two failure modes is that large, a symmetric precedence rule is the wrong tool.

The config-file layer has its own internal precedence, and the fleet's two different answers to it are covered in the security chapter: within a `grc.conf`, first match wins because order expresses specificity; across the search path, last match wins because order expresses authority. Anyone extending a config-driven tool should decide which of those two meanings their ordering carries before writing the loop, because the two are indistinguishable in a single-file test and diverge permanently the moment a second file exists.

What each tool leaves behind

The other half of the environment question is state: what exists on disk after the process exits. The fleet splits three ways, and knowing which group a tool is in tells you what to back up, what to clean, and what will surprise you on a new machine.

The largest group keeps *preferences only*. `lsotrs`, `iftotrs`, and `storageshower` each keep a TOML dotfile in `$HOME` — `~/.lsotrs.conf`, `~/.iftotrs.conf`, `~/.storageshower.conf` — auto-saved on every in-app toggle with no explicit save step. `htotrs` breaks the pattern by storing JSON under `~/.config/htotrs/prefs.json`, and `zcolorizer` uses TOML under XDG at `$XDG_CONFIG_HOME/zcolorizer/config.toml`. The auto-save behaviour is worth a note for anyone diagnosing “the tool came up with settings I did not choose”: in these three, changing a setting in the UI *is* the save, so a palette picked to see what it looked like is a palette that persists. That is the right default for a tool used interactively dozens of times a day and it means the config file is a record of the last thing you touched, not of a decision you made.

The second group keeps *no per-user state*. `nmaprs`, `grcrs`, and `temprs` all maintain no preference file. `grcrs` reads the upstream `grc.conf` data files, but those are shared configuration rather than per-user state — a distinction that matters because it means removing your home directory changes nothing about how `grc` dispatches, while removing `/etc/grc.conf` changes everything.

`temprs` is the interesting member of that group, because keeping no preferences is not the same as keeping nothing: it keeps a great deal of *data*. The stack directory accumulates real files containing real content, and the master record accumulates the ordering and the tags. Nothing expires on its own. That is the deliberate trade the tool's chapter states plainly — a forgotten stack accumulates files until `--clear` or `--expire` reclaims them — and the environment consequence is that `TEMPRS_DIR` is not a cosmetic preference but a decision about where a growing pile of your command output lives.

The two native hosts add a fourth pattern that no other tool in the book has: state whose *location* is part of an external system's configuration. The native-messaging manifest names an absolute path to the host binary, so the binary's location is configuration held by the browser, which is why `stable_exe_path` rewrites a versioned Cellar path onto the version-independent `opt` symlink before writing the manifest. Nothing else in the fleet has a consumer that records where the program is; everything else is found through `PATH` and can move freely.

The reproducibility checklist this implies

Put the three layers together and the practical consequence is a short list of things to capture when a behaviour will not reproduce, and it is worth having because the first four are invisible in a transcript of the commands that were run. Which of `TMPDIR`, `TEMPRES_DIR`, `NO_COLOR`, `EDITOR`, and the four `XDG_*` variables are set, and to what. Whether a per-user config file exists for the tool, and — for the three auto-saving tools — what the last interactive toggle left in it. Which `glibc.conf` files exist across the whole search path, not just the one you edited. And whether the descriptor the tool wrote to was a terminal, which is a property of how the command was invoked rather than of the command itself.

That list is short because the fleet's environment surface is deliberately small. Every variable recorded above tells a tool where to look or whether to decorate, never what to do; the book records no `TOOL_OPTS`-style variable injecting default arguments into any of the eight, which is the single most common way a CLI tool becomes irreproducible. On the evidence here, a user's shell can change where these programs read and write, and can turn their colour off. It cannot change what they do.

Identity – the keys these tools use, and the ones that get reused

Every row a monitoring tool prints is an assertion of identity. The row says *this* process, *this* descriptor, *this* connection, *this* stack entry — and between one frame and the next, the tool has to decide whether the thing it is looking at now is the same thing it was looking at before. That decision is made by a key, the key is a tuple of values read out of a mutable system, and every component of every such tuple in this fleet can be reused by the kernel or by a peer while the tool is running. This chapter collects the keys the eight tools actually use, states what each one is stable against, and names the specific reuse that breaks it.

The reason this deserves a chapter of its own is that key selection is invisible in the output. A column layout does not tell you whether two frames' worth of rows were matched on a PID, a `(pid, fd)` pair, or a normalised connection tuple, and the difference only becomes observable in the one situation where the answer matters: when something underneath changed identity without changing appearance.

The five keys, and what each is stable against

Reading across the fleet, the identity decisions the earlier chapters record fall into five distinct keys.

The first is the bare **PID**. `lsof`'s selection filters use it (`-p`), its `--follow` mode follows a single PID's descriptors in real time, and its `--top N` ranks processes by descriptor count. A PID is stable for exactly as long as the process lives, and not one instant longer. The kernel is free to hand the same number to an unrelated program afterwards, and on a machine with high process churn — a build, a CI runner, a shell in a loop — it will. A PID is therefore a fine key for a single-shot scan, where the whole question is answered inside one gather, and a poor key for anything that compares two moments.

The second is the `(pid, fd)` pair, which is what `lsof`'s `--stall` classifier uses to accumulate a per-descriptor history. This is a strictly stronger key than a PID and still not a strong one, because both components recycle: the process can exit and its number be reissued, and within a single long-lived process a descriptor number is released on `close()` and reissued by the next `open()`. The `lsof` chapter records the mitigation the tool actually implements — if the command name attached to a `(pid, fd)` changes, the accumulated history is cleared and the verdict resets. That is a real defence and it is worth being precise about its reach. It catches PID reuse across *different* programs, which is the common case, because the new occupant of the number almost always has a different command name. It does not catch PID reuse by the same program, which is exactly what a supervised service restarting in a tight loop produces, and it does not catch descriptor reuse inside one process at all, because the command name does not change when a process closes one socket and opens another.

The third is the `(pid, fd, name)` triple, which is what `lsof`'s `DeltaTracker` records on each iteration of a `-r` repeat loop in order to classify rows as **New**, **Gone**, or **Unchanged**. Adding the name to the pair is what makes descriptor reuse visible: a process that closes one log file on fd 7 and opens a different one on fd 7 produces a **Gone** and a **New** under the triple, where the pair would have reported an unchanged row. The two keys sit in the same crate and are chosen differently on purpose, and the difference is the clearest worked example in the book of a key selected to fit its question rather than copied from the neighbouring module. `--stall` wants continuity of a *buffer*'s behaviour over time, so it takes the coarser key and accepts the name changing underneath; `--delta` wants to report change, so it takes the finer key and treats a substitution as two events.

The fourth is the **normalised flow key**, `iftopr's FlowKey::normalize`. A connection is inherently symmetric — packets travel $A \rightarrow B$ and $B \rightarrow A$ and belong to one row — so the key has to canonicalise the pair ordering, and the implementation lifts both endpoints into `u128` (a widened `u32::from` for IPv4, the raw bytes for IPv6) and compares the `(addr, port)` tuples so the ordering is total and allocation-free, flipping the recorded direction when the comparison swaps the operands. Two things are worth extracting. The canonicalisation is what makes one logical connection one row regardless of which side a given packet travelled, which is a correctness property and not a display nicety. And the key is stable against everything *except* the case where the same address-and-port pair is reused, which for TCP is what the `TIME_WAIT` state exists to make unlikely and does not make impossible. A short-lived connection between the same two endpoints on the same ports, reopened inside the retention window, joins the previous connection's byte history. That is the correct trade for a bandwidth monitor — the question is “how much traffic between these endpoints,” not “how many connections” — but it is a trade, and a reader debugging a surprising 40s figure should know it was made.

The fifth is the **template hash**, `zcolorizer's` identity for a *class* of log line. It is a `u64` over a masked form of the line: spans owned by one of the variable token classes named in `VARIABLE_TOKENS` — `number`, `ip`, `address`, `pid`, `size`, `version`, `date`, `time`, which is eight classes — contribute a sentinel byte plus the class name rather than their literal bytes, so two syslog lines differing only in timestamp, PID, and source address collapse to one hash. This key is unlike the other four in an important way: it is not attempting to identify a kernel object at all, it is attempting to identify a *pattern*, and its failure mode is not reuse by the system but collision in the hash. Two genuinely distinct templates that hash equal share a frequency count, and the visible consequence is that one of them never reads as novel. At `u64` width over the population of distinct templates a log stream produces, that is not a practical concern; it is worth naming only because it is the one key in the fleet whose failure is arithmetic rather than environmental.

Inodes, paths, and the file that has no name

The file identity problem deserves separate treatment because Unix gives you two different identities for a file and the fleet uses both.

A path is a name in a directory; an inode is the file. `lsotrs` on Linux correlates sockets through a prebuilt socket map keyed by inode, because that is the join available — `/proc/net/tcp` and `/proc/[pid]/fd` share the inode number and nothing else. On the display side, the same tool renders paths, and its `OpenFile` carries a `name_append` field, which is where `(deleted)` lands. Those two facts describe the whole of the deleted-file situation: the inode is alive, its path is gone, and the tool has to show you something in a column headed by a name that no longer resolves.

This is what makes `+l1` a real diagnostic rather than a curiosity. `+l` alone lists the link-count column; `+l1` additionally selects files whose link count is below one, which is the set of files that have no directory entry anywhere and are held open regardless — the classic case of a disk that will not free space until a process is restarted. The identity insight underneath is that the *only* handle you have on such a file is the `(pid, fd)` that holds it. There is no path to `stat`, no name to pass to another tool, and the reason `lsf -t` exists in the form it does — bare PIDs, one per line, suitable for `kill $(...)` — is that for an unlinked open file, acting on the holder is the entire remediation vocabulary.

`temprs` inverts this arrangement and is instructive for it. Its entries have three identities at once: a **path** (`tempfile<milliseconds>` inside the managed directory), a **stack index** (positive from the bottom, negative from the top, so `-1` is the newest), and an optional **tag** name assigned by the user. The index is the least stable of the three by design — every push rennumbers the positive indices — which is why the tag exists, and why the completion function reads the live master record rather than a static list. The path is the durable identity and the tag is the durable *human* identity; the index is a convenience whose instability is the point, in the same way that the top of a stack is a position rather than a name.

The self-healing behaviour at startup is the same idea from the other direction. `TempApp::new()` zips stored paths against their names, filters with `.filter(|(p, _)| p.exists())`, and rewrites the pruned record. That makes the filesystem authoritative over the master record: an entry whose path was deleted by something else stops existing as an identity on the very next invocation. The record is a cache of an identity that lives on

disk, not the identity itself, and building it that way is what lets an external deletion inside the temp directory be a supported operation rather than a corruption.

Counters as identity: the idle scan

The strangest identity in the fleet is `nmaprs`'s idle scan, which does not identify an object at all — it identifies a *change in a remote counter*. The technique reads the zombie's IP-ID delta and treats a delta of two or more as `open`. The entire result rests on the assumption that the zombie's IP-ID sequence is a global counter incremented once per packet the zombie emits, so that a delta of two means the zombie emitted a packet the scanner did not solicit.

Every part of that is an identity claim about a number the scanner does not control, and it degrades in the obvious ways: a zombie whose IP-ID counter is per-destination or randomised produces no signal, and a zombie that is talking to anyone else while the scan runs produces deltas that mean nothing. This belongs in an identity chapter because it is the limiting case of the pattern the whole chapter describes. Every key here is an inference from an observable to an object, the inference is only as good as the system's promise to keep the observable attached to that object, and the idle scan is the one place where the promise is not even the local kernel's to make.

What to do about it

Three rules fall out of the five keys, and they generalise past this fleet.

Choose the key from the question, not from the neighbouring module. `lsopro` carries two different keys over the same underlying rows because `--stall` and `--delta` ask different questions, and a codebase that had unified them on “whichever key we wrote first” would have shipped one of two bugs: a stall history that resets on unrelated descriptor churn, or a delta report that misses file substitution.

State what the key is stable against, in the documentation, next to the feature. “Flows are keyed by the normalised connection tuple” is a one-line statement that answers a whole class of support question in advance, and its absence is why bandwidth monitors accumulate bug reports about connections that “merged.”

Where the key can be invalidated by an observable, check the observable. The command-name check on `(pid, fd)` is cheap, it is already in the gathered data, and it converts the most common reuse case from a silent wrong answer into a reset. That is the whole design pattern: a key you cannot make unique, plus a cheap witness that it changed, is most of the way to a key you can trust.

Sampling – what a polled view can and cannot see

Several tools in this book present a live view of a system that is changing while they look at it, and most of them do so by taking samples at an interval. That is the only implementable design for most of these data sources, and it has a consequence the output never states: everything that begins and ends between two samples did not happen as far as the tool is concerned. This chapter is about where that boundary falls in each tool, which tool escapes it, and how to read a live display with the sampling design in mind.

Two different things are called “refresh”

The first thing to separate is the cadence at which data is *collected* from the cadence at which the screen is *drawn*, because in this fleet they are almost never the same number, and confusing them produces wrong conclusions in both directions.

`storageshower` runs three distinct clocks. The background collector thread spawned by `spawn_bg_collector` refreshes disks and system statistics on a three-second cadence. The user-facing refresh interval is separately selectable among one, two, five, and ten seconds and is cycled with the `f` key. And the render loop polls `crossterm` for input with a two-hundred-millisecond timeout. Those are three numbers with three different meanings, and the earlier chapter states the relationship explicitly: the collector’s cadence is independent of the user-facing refresh rate. Setting the visible refresh to one second therefore does not make the disk numbers one second old — it makes the screen redraw with whatever the three-second collector last produced. That is not a defect; it is the decoupling that keeps a slow `getmntinfo` on a many-mount machine from delaying a keystroke. But a reader who believes “refresh interval” means “data age” will misread the display.

`iftoprs` runs four cadences, and enumerating them is the clearest single picture of a layered sampling design in the fleet:

```
33 ms  TUI tick – drain the packet channel, maybe rotate, draw
1 s    history rotation – close the current slot, open a new one
2 s    process-table refresh in the proc-lookup thread
60 s   idle eviction – drop flows unseen for a minute
```

Each number governs a different question. The 33 ms tick is responsiveness. The one-second rotation is the quantum of the 2s/10s/40s columns — which, as the kernel chapter states precisely, are summed byte counts over trailing one-second slots scaled to bits rather than slot-averaged rates. The two-second process refresh is the freshness of the attribution join. The sixty-second eviction is the memory bound on the number of flows.

The one tool that is not sampling its primary source

`iftoprs` is also the fleet’s one genuine hybrid, and it is the reason this chapter can make a comparison rather than only a warning. Its byte counts do not come from a sample: they come from `pcap`, which delivers every packet the capture matched. There is no interval at which traffic can hide, and a flow that lived four milliseconds and moved one packet contributes exactly as faithfully as a flow that ran all day.

Its *attribution*, though, is a poll. The process table is a join against `/proc/net/*` and `/proc/[pid]/fd/` on Linux, or the libproc socket walk on macOS, refreshed by a dedicated thread on a two-second cadence. So

the program's two halves have different epistemic status, and the observable consequence is the one the symptom index already records: short-lived connections whose process exits between the capture and the lookup produce accurate counters with no owner. That is not a bug to be fixed by tightening the poll — halving the interval halves the window and does not close it — and the correct response is the one the tool takes, which is to omit the key from the JSON output rather than emit an empty string, so a consumer can count unattributed flows instead of mistaking them for a process with an empty name.

The generalisable statement is that a tool built on two data sources inherits the weaker source's guarantees for anything that joins them. `iftoprs` has event-complete byte accounting and sampled ownership, and every row it prints is event-complete in one column and best-effort in another.

Where the interval swallows the event

Three of `lsofirs`'s live modes are interval-driven, and each has a characteristic blind spot worth naming.

`--watch <path>` reports `+OPEN` and `-CLOSE` events for one path. Those events are necessarily *derived* rather than observed — nothing in this fleet subscribes to a filesystem-open notification interface, and `lsofirs`'s only data source is its per-platform gather — so an event is the difference between two gathers. A descriptor opened and closed inside one interval appears in neither gather and produces no event at all. For the common use — a socket or a lock file held for a while, and you want to know who took it — this is fine. For a process that opens and closes the same path in a tight loop, the mode reports nothing while the path is being hammered, which reads as “nobody is touching this file” and means the opposite.

`--leak-detect` takes an `interval,threshold` specification and defaults to a five-second interval with a three-consecutive-increase threshold, clamping the threshold to a minimum of two. The clamp is the interesting part. A threshold of one would fire on any single-sample increase, which on a real machine is noise — descriptor counts move constantly. Requiring consecutive increases is a low-pass filter implemented in the crudest possible way, and it is the right crudeness for the problem: a genuine descriptor leak is monotone over minutes and trivially clears three consecutive samples, while ordinary churn does not. The cost is latency to detection, which is the interval times the threshold, and the cost of a longer interval is that a leak fast enough to exhaust the table between samples is detected once, at the end.

`--stall` is the most explicitly sampled of the three and states its constants: a bounded per-descriptor history of sixteen entries, a minimum of three consecutive samples before it will commit to a verdict, and an eighty-percent fill ratio as the threshold for “near capacity.” The refusal to classify before three samples is a direct statement about aliasing: a single observation of a full send buffer is not evidence of a stall, because a healthy buffer under load is frequently full. Only persistence distinguishes the two, and persistence is not observable in one sample no matter how good the sample is.

Active sampling: probing a remote counter

`nmaprs`'s OS fingerprinting is the one design in this book that generates its own samples rather than observing someone else's, and it inverts the usual constraint. The SEQ phase sends `NUM_SEQ_SAMPLES = 6` probes spaced by `OS_SEQ_PROBE_DELAY_MS = 100`, and derives sequence-rate values (`GCD`, `ISR`) plus the relation classifications from the responses.

The spacing is not a politeness interval, it is the measurement. The quantity being estimated is a *rate* — how fast the target's sequence generator advances — and a rate estimate from samples taken too close together is dominated by clock granularity, while samples taken too far apart risk the counter wrapping. A fixed inter-probe delay with a fixed sample count is what makes the resulting numbers comparable against a reference database populated by a tool that used the same delay and count. This is the one place in the fleet where the sampling parameters are not a tuning knob at all: changing them would not make the tool faster or slower, it would make its measurements incomparable with the database it scores against.

Reading a sampled display

Four practical rules follow, and they are worth carrying to any live tool, not only these.

Ask which number is the data cadence. If the tool exposes one interval control, find out whether it drives collection or only redraw. In `storageshower` it drives redraw and the collector runs on its own clock; in `lsofrs`'s repeat modes the interval drives the gather itself, because there is no separate collector thread.

Assume that anything shorter-lived than the interval is invisible, and design the observation around that rather than around the tool. If you need to catch a file being opened and closed rapidly, a polled descriptor lister is the wrong instrument no matter how short you make its interval; you want an event source, which for network traffic is what `iftoprs` already has and for filesystem opens is something outside this fleet entirely.

Distinguish “no evidence” from “evidence of none.” A `--watch` that prints nothing and a `--stall` that prints `No stalled sockets detected.` are different claims: the second is a positive statement about every socket currently seen, made after at least three samples; the first is the absence of a derived event and is consistent with heavy activity below the sampling floor.

And when a live view disagrees with a single-shot one, suspect the window before suspecting the reading. Summed-over-a-window and averaged-over-a-window differ by the window length, a partially elapsed window at startup differs from a full one, and a flow evicted at sixty seconds of idleness takes its history with it. All three produce a smaller number than a naive reader expects, and none of them is a counting error.

Clocks – monotonic time, wall time, and the places they are mixed

Time enters these programs in four distinct roles, and the roles want different clocks. Something has to pace probes and rotate windows; something has to name files and stamp output; something has to decide whether a cached artifact is stale; and something has to schedule the next redraw. The first and last want a monotonic clock, which only ever moves forward and is unaffected by the administrator, the time daemon, or the timezone database. The second and third want a wall clock, because their outputs are compared against values produced by other programs on other machines. Mixing them is not a theoretical hazard; it is the ordinary way a program acquires a bug that appears twice a year.

Where the fleet uses the monotonic clock

The interval machinery uses `Instant` throughout, and the earlier chapters record the specific sites.

`iftoprs` stamps `last_seen = Instant::now()` on every fold of bytes into a flow's history, and the sixty-second idle eviction is a comparison against that stamp. `maybe_rotate` fires only when at least one second has elapsed, measured against a `last_rotation` timestamp held in the tracker. Both are right: the question “has a second passed” is a question about elapsed duration, and an answer derived from a wall clock would rotate twice or not at all when the machine's clock was stepped.

`nmaprs`'s `ProbeRatePacer` is the sharpest case, because it is a rate limiter and rate limiters are where wall-clock use produces the most spectacular failures. The pacer installs a shared `Mutex<Instant>` spacing gate, and only when `--max-rate` is set — a `--min-rate` floor alone adds no mutex at all. Two properties follow from the `Instant`. A clock step backwards during a scan cannot produce a negative elapsed time and therefore cannot produce an unbounded burst, which is the classic wall-clock rate-limiter failure. And the same pacer instance is shared across the concurrent IPv4 and IPv6 senders, so the ceiling is global rather than per-family — a property about the *rate* rather than the clock, but one that only means anything if the clock underneath is monotone.

The render loops use durations rather than timestamps by construction: `crossterm`'s poll takes a timeout, `iftoprs`'s tick is a 33 ms budget, `storageshower` polls with a 200 ms timeout. None of those reads a clock the user can set, which is why none of these tools does anything strange when the machine comes back from suspend or an administrator corrects the date.

Where the fleet uses the wall clock, and must

Three roles genuinely need wall time, and each carries a hazard the monotonic clock does not have.

The first is **naming**. `temprs` names each stored file `tempfile<milliseconds>` from `util_time_ms()`, and that millisecond timestamp is the on-disk identity of the entry. This is a deliberate and defensible choice — the name sorts chronologically, it is human-legible, and it needs no coordination — and it has consequences worth stating plainly. Two pushes inside the same millisecond contend for one name, which is a narrow window and a real one for a tool invoked from a shell loop. A clock stepped backwards produces names that sort out of creation order, which matters because that sort is what ties the name to the stack's chronology. And the identity survives only as long as the file does: because the filesystem is authoritative and the startup prune drops entries whose path no longer exists, a name is a name for exactly as long as its file is there.

The second is **display**. `lsOfirs` carries `chrono` for timestamps, and anything a user reads and compares against a log line elsewhere has to be in the same frame of reference as that log line — which is wall time, in the local zone, with all the ambiguity that entails during a daylight-saving transition. There is no alternative here: a monotonic timestamp is meaningless to a reader and incomparable across processes.

The third, and the one with real teeth, is **cache invalidation by modification time**. The covered-fleet chapter records that `awkrs` memoises compiled programs to an rkyv shard and invalidates on source *or* *binary* mtime change. Modification times are wall-clock values written by whatever produced the file, which makes this a correctness argument resting on a clock the program does not control. Three cases strain it and all three occur in practice: a checkout or archive extraction that restores an older mtime onto newer content, leaving a stale entry valid; a filesystem whose mtime granularity is coarser than the interval between two edits, so a second edit inside the same tick is invisible; and a clock correction that moves mtimes relative to each other. Including the *binary*'s mtime in the key is the mitigation that matters most, because it means a rebuilt tool invalidates everything at once — which converts “the compiler changed and the cache did not notice” from a silent wrong answer into a full recompile. The general rule worth extracting: an mtime-keyed cache should include the mtime of the thing that does the interpreting, not only of the thing being interpreted.

The rule the fleet follows

Stated as a rule, the split is: **durations from the monotonic clock, identities and cross-process comparisons from the wall clock, and never the reverse**. Every site recorded above obeys it. Eviction, rotation, pacing, and poll timeouts are all durations, and all use `Instant` or a timeout parameter. Filenames, displayed timestamps, and cache keys are all identities or cross-process comparisons, and all use wall time.

The failure this prevents is specific and worth naming so it can be recognised elsewhere. A rate limiter, an eviction policy, or a retry backoff written against the wall clock behaves correctly in every test and then, on one machine, once, does something unbounded — a burst of thousands of probes, a flow table that never evicts, a backoff that returns immediately — because a time daemon stepped the clock while it was running. It is not reproducible, it does not appear in a profile, and the only defence is to have used the right clock the first time.

Two clock facts that are not about clocks

Two related items belong here because readers hit them and assume a time bug.

The first is the partially-elapsed window. `iftoprs`'s 40s column sums the trailing forty one-second slots, and for the first forty seconds after a flow appears there are fewer than forty slots to sum. `window_total` takes `n.min(len)`, so the column is correct for the slots that exist and *smaller* than its steady-state value during the ramp. Nothing is wrong; the window is young. The same applies at the other end, where a flow evicted after sixty seconds of idleness removes its history rather than decaying it.

The second is documentation dates. The man pages carry dates — `lsOfirs.1` is dated `2026-07-02` and `nmaprs`'s pages carry a May 2026 date — and those are build-time wall-clock facts recorded in a file, which makes them the one timestamp in the whole system guaranteed to go stale. They date the document, not the binary, and the version string next to them is the thing to trust.

Determinism — what these tools promise to produce twice

A tool that gathers in parallel and prints in order is making a promise, and the promise is not free. Every parallel stage in this fleet has somewhere a point at which nondeterministic completion order is converted back into a deterministic output order, and the interesting engineering is where that point sits and what it costs. This chapter collects those points, identifies the places where output is deterministic for a reason other than sorting, and names the one output in the fleet that is deliberately not reproducible at all.

The three ways order is restored

Sort at the end. `lsoprof` gathers processes in parallel with rayon across PIDs — so the order in which processes land in the result vector is a function of thread scheduling — and `gather_and_filter` finishes by sorting by PID for stable output. That is the simplest possible answer and the right one at this scale: the sort is over a few hundred to a few thousand entries, it happens once per gather, and it makes the output a pure function of the system state regardless of how many cores did the reading.

Order the container, not the results. `lsoprof`'s `--ports` groups rows into a `BTreeMap<u16, ...>` keyed by port, which sorts the output ascending by port number as a property of the data structure rather than of a later pass. This is worth contrasting with `--net-map`, which the same chapter records as grouping by remote host into a `HashMap`. The two containers make different promises: a `BTreeMap` iterates in key order, a `HashMap` does not. That is not a defect claim about `--net-map` — grouping and output ordering are separate steps, and the ordering may be established elsewhere — but it is exactly the question a reader extending either mode should ask first, because the difference is invisible in a single-host test and becomes visible the moment two runs of the same command are diffed.

Partition so that order is preserved locally. `nmaprs` uses this twice, and it is the most sophisticated of the three. Target expansion runs in parallel across specifications through `expand_specs_ordered` — the name states the contract — so multiple `-iL` files and command-line targets resolve concurrently but print in input order. And `split_into_syn_chunks` divides the ordered work into balanced contiguous chunks so that each SYN shard keeps port ordering locally. Contiguity is the whole trick: if each shard owns a contiguous run of the ordered work, then concatenating the shards' results in shard order reproduces the original order with no comparison at all. The cost is that balance is by count rather than by completion time, so a shard that draws a slow range finishes last; the benefit is that reassembly is free. The same instinct appears in `trace.rs`, which caps traceroute concurrency at `MAX_TRACEROUTE_PARALLEL = 32` and preserves scan output order.

Where determinism is a parity requirement

For two tools, deterministic output is not a nicety — it is the thing being tested, because the test is a byte comparison against another program.

`htoprs`'s `tests/parity/` suite runs the same inputs through the reference C `htop` and through `htoprs` and compares output byte for byte. That suite can only exist if the ported program's output is a deterministic function of its input, and it is one reason the ported draw path drives `crossterm` directly rather than going through `ratatui` — the constraint is byte-identical output, and every layer between the model and the bytes is a layer that can introduce a difference. `gicrs` has the same property on a smaller surface: its end-to-end

tests spawn the built binaries and assert on the exact coloured byte stream, with expectations cross-checked against the reference Python `grcat`, and among the behaviours they pin are the invalid-UTF-8 round-trips. Byte-exact tests over arbitrary input bytes are only writable because the colouriser's output is a pure function of its input, including for input that is not text.

The escape-collapsing rule in `grcat` is worth naming as a determinism property specifically. A colour escape is emitted only when it changes from the previous character's escape, and the character's original bytes are re-emitted unchanged. That makes the output canonical: there is exactly one byte sequence the colouriser can produce for a given input and colourfile, not a family of equivalent sequences differing in redundant escapes. Without canonicity a byte-diff oracle would need a normaliser, and a normaliser is a second implementation of the thing being tested.

The parallel case with a static safety check

The covered-fleet chapter records `awkrs` parallelising record processing via `rayon` with *deterministic output reordering when a static parallel-safety check passes*, controlled by `-j/--threads` with a default of one. Three design decisions are stacked in that sentence and each is worth extracting.

Default one. Parallelism is opt-in, so the default behaviour of a tool whose programs may have arbitrary side effects is the sequential semantics its language specifies.

A static check, not a dynamic one. The tool decides *before* running whether the program's structure permits parallel execution with reordered output. A dynamic check would have to detect a violation mid-run, at which point the choice is between aborting a partially complete job and producing wrong output.

Reordering, not racing. When the check passes, output is reordered back into record order rather than emitted in completion order. So the promise is not "parallel produces some valid order," it is "parallel produces the same bytes as sequential" — which is the only promise that makes `-j 8` a safe flag to add to an existing pipeline.

The one output that depends on its own past

Several live modes here classify by comparing against earlier samples — `lsoids`'s `--delta`, `--leak-detect`, and `--stall` all do — but those report a *change*, and a change is inherently a two-sample question. `zcolorizer`'s novelty mode is a different thing: it renders a single line differently depending on what came before it, so the same input produces different output on a second pass through one process.

The score is `1.0` on first sight of a template and $1/(k+1)$ after `k` prior sightings, so a line rendered at the start of a stream blazes and the same line rendered after a hundred repetitions dims into the background.

That is not a determinism failure; it is a different specification. The function being computed is not `line → colour` but `(line, stream history) → colour`, and it is deterministic in both arguments — which is what the model's own tests assert: that the score strictly decreases on every repeat and lands below `0.15` within a dozen sightings, and that hammering one template to noise does not bleed into an unrelated template's first-seen `1.0`. Optional decay adds a third argument, position in the observation count, since the sweep runs once every `DECAY_INTERVAL` (512) observations.

The reason to include it in a determinism chapter is the contrast. Every other output in this fleet is reproducible given the same system state; this one is reproducible given the same system state *and the same history*, and its documentation has to say so or a user will file the difference as a bug. A tool whose output depends on its own past owes the reader that sentence.

What a reader should check

If you are diffing two runs of one of these tools and seeing spurious differences, four things account for nearly all of them.

The system changed. This is overwhelmingly the answer for a process or socket lister, and the way to rule it out is to diff a filtered view narrow enough to be stable — a single PID's descriptors rather than the whole table.

A container without an ordering guarantee is being iterated. This is the [BTreeMap](#)-versus-[HashMap](#) question above, and the check is whether the differing lines are the *same set* in a different order or a genuinely different set.

A window is partially elapsed. A rate column read within the first window after a flow appears is smaller than its steady state, which reads as a difference between runs and is a difference between ages.

The output is history-dependent by design. In this fleet that is exactly one mode, but the general habit of asking “is this a function of the input alone” is what finds it quickly.

The unsafe surface — every departure from safe Rust, and what pays for it

A memory-safe rewrite of a C program is not a program with no unsafe code. It is a program whose unsafe code is small, enumerable, and concentrated at the boundary where the operating system's C ABI begins — and whose safety therefore rests on a set of invariants a reader can actually check, rather than on the whole program being correct. This chapter enumerates that boundary as the earlier chapters record it, sorts it into categories that fail differently, and states the invariant each site depends on.

Five categories, and why the distinction matters

Every departure from safe Rust recorded in this book falls into one of five kinds, and the kinds are worth separating because they have different failure modes and different defences.

One: a foreign function declaration that must match the platform. `lsOfs`'s macOS backend declares `proc_listpids` and `proc_pidinfo` as raw `extern "C"` and calls them against `repr(C)` structs matched to the kernel headers. The unsafety here is not in the call; it is in the *declaration*. If a struct field is the wrong width, in the wrong order, or missing a padding member the real header has, the call succeeds and returns plausible garbage. Nothing in Rust checks a hand-written `extern` block against the header it claims to mirror, which makes this the category with the quietest failure in the fleet.

Two: a C library call whose contract is documented elsewhere. `gicrs` computes its colour default from `libc::isatty(1) == 1`; `storageshower` calls `getmntinfo(&mut mntbuf, MNT_NOWAIT)` through `libc` and walks the returned `statfs` array; `htoprs` calls the real `libc::getopt_long`, marshalling `argv` into a null-terminated C `char**` and building the same `option` array with `has_arg` values of 0, 1, and 2. These cross the ABI, and the invariant in each case is a documented precondition — a valid descriptor number, a properly sized buffer, a correctly terminated argument vector.

Three: aliasing and ownership claims the compiler cannot check. `nmaprs`'s connect-scan worker pool writes results into a `ResultSlots` array of `UnsafeCell<MaybeUninit<PortLine>>` and carries an `unsafe impl Send/Sync`. The justification is documented in the source and is a claim about the *algorithm*: each slot is written by exactly one worker and read only after all workers join. This is the category where the unsafety is genuinely load-bearing for performance, and the category where a later, innocent-looking change — a second pass over the work vector, a retry that revisits an index — silently invalidates the proof.

Four: initialisation. `MaybeUninit` is in that same type for a reason: the result array is allocated without initialising its elements, and a read of a slot that was never written is undefined behaviour rather than a wrong value. The invariant is not only “one writer per slot,” it is “every slot is written before any slot is read.”

Five: a foreign object file compiled into the build. `nmaprs`'s `build.rs` compiles `c/expr_match.cpp` through the `cc` crate at `opt_level(3)` as `nmap_expr_match`, the OS-fingerprint expression matcher. Everything inside that object is outside Rust's model entirely, and the only defence is the interface — what is passed in, and what is trusted on the way out.

The two-call sizing pattern, and its window

The macOS enumeration path is worth reading closely because it contains a hazard that is structural rather than incidental. The `lsotrs` chapter records the sequence: call `proc_listpids` first to size the PID buffer, then again to fill it.

That is the standard C idiom for a variable-length kernel query and it has a race built into it. Between the sizing call and the filling call, processes are created and destroyed, so the true count can grow past the buffer that was just allocated for it. The correct handling is to treat the second call's return as authoritative — to use the count of bytes actually written rather than the count the first call predicted — and to accept truncation as a normal outcome rather than an error.

This connects directly to the kernel chapter's observation that there is no way to take a consistent snapshot of a live process table, and that a vanished process is the normal case rather than an error condition. The two-call pattern is where that fact first bites, and it bites in the one place where getting it wrong is a buffer overrun rather than a missing row.

Where the panic strategy meets the aliasing proof

There is a real interaction between two facts stated in separate chapters, and it is worth drawing out because it is the kind of connection that is invisible when each is read alone.

The `ResultSlots` invariant is “each slot is written by exactly one worker and read only after all workers join.” A worker that panics partway through does not write its slot. Under unwinding, that panic propagates out of the worker, the scope ends, and the read-after-join step is reached with at least one slot uninitialised — which is category four's failure, undefined behaviour rather than a missing row.

`nmpr's` release profile sets `panic = "abort"`. Under abort there is no unwinding path at all: a panicking worker terminates the process and the read is never reached. The cost-model chapter already notes that `panic = "abort"` is the one profile setting with observable semantics, and that the tools setting it are ones with no long-lived shared state to protect; this is the sharper version of the same point, because here the setting is not merely compatible with the design but reinforces a soundness argument.

Two caveats keep this honest. The setting is on the *release* profile, which is what ships and is not what a plain `cargo build` produces, so a developer build does not carry the property. And nothing in the sources recorded here says the soundness argument was written to depend on the panic strategy — the documented justification is the one-writer-per-slot invariant, and the panic strategy is a separate decision that happens to reinforce it. The reading to take away is not “this is unsound in a debug build”; it is that an unsafe block's invariant should be stated in terms that hold under every panic strategy, because the profile is a build configuration and the invariant is a property of the code.

The interface that is deliberately not unsafe

The most interesting decision in this chapter is a negative one. `htoprs` deliberately does not port `htop's` `xMalloc` allocation family, the raw `Object**` dynamic-array machinery in `Vector.c`, or the open-addressing bucket table in `Hashtable.c`, because porting them faithfully would mean reimplementing an allocator in *unsafe* — precisely the class of bug the port exists to eliminate. `Vec` and `HashMap` stand in, and the omission is documented rather than hidden.

This is the correct application of a rule that is easy to state and hard to follow: a port's fidelity obligation is to *behaviour*, and hand-rolled memory management is not behaviour, it is a technique for achieving it in a language that had no alternative. Transcribing those files into unsafe Rust would have produced a program that was byte-identical in output, structurally identical to the upstream, and carrying the upstream's entire memory-safety exposure into a rewrite whose stated purpose was to remove it.

The mechanical consequence is visible in the anti-drift gate, and it illustrates how a build-time invariant has to be designed around the exceptions a project intends to have. The gate's `collect_free_fns` strips `pub`, `pub(crate)`, `pub(super)`, `unsafe`, `async`, and `extern "C"` prefixes before reading a function name — six prefixes — so the gate sees the same identifier regardless of how a port declares visibility or ABI. That means the gate is deliberately blind to whether a ported function is unsafe: it enforces *which* functions exist, never *how* they are implemented. Those are the right divisions of labour, and it is worth noticing that no automated check recorded in this book counts unsafe blocks or gates their growth.

What a reviewer should ask at each site

The categories give a checklist short enough to run on any diff that adds a boundary crossing.

For a declaration (category one): what header is this claiming to mirror, on which platforms, and is there a test that would fail if the layout were wrong? The general answer for FFI struct layout is a size-and-offset assertion, and its absence means the layout is checked only by whether the numbers look plausible.

For a library call (category two): what is the documented precondition, and what does the function do on failure? A mount-table call returning zero entries and an `isatty` returning a value other than one are ordinary outcomes with defined meanings, and treating either as an error is a bug in the other direction.

For an aliasing claim (category three): write the invariant down next to the `unsafe impl`, in terms of the algorithm, and make it falsifiable. “Each slot is written by exactly one worker and read only after all workers join” is a good invariant precisely because a reviewer can check a diff against it.

For initialisation (category four): is every element written on every path, including early-exit and error paths? This is the invariant most often broken by adding a `break`.

And for foreign objects (category five): the interface is the whole review surface, because nothing inside is checkable from the Rust side.

Byte order, ABI, and the architecture a port is actually built for

A port that reads kernel structures and constructs network packets is a program whose correctness depends on the layout of memory and the order of bytes, and both of those are properties of the target rather than of the source. This fleet ships across at least four targets — macOS on aarch64 and x86_64, Linux on x86_64 and aarch64, with musl variants in at least one release matrix — which means every layout assumption is made four or more times, and only the ones that are wrong on the machine the author uses get caught by running the tool.

Three kinds of number, three different rules

The single most useful distinction for reading this material is that the numbers these tools decode are not all the same kind of number, and the rules differ.

Wire values are standardised and big-endian. `iftoprs`'s packet parser reads the ethertype and dispatches `0x0800` to IPv4 and `0x86DD` to IPv6, handles `0x8100` VLAN tags by skipping the four-byte tag and re-reading the inner ethertype, reads IHL to find the L4 offset, and takes `total_len` from the IPv4 header as the byte count. Every one of those fields is defined by a protocol specification, has the same value on every machine that has ever implemented the protocol, and arrives in network byte order. The rule for this class is simple and absolute: convert on read, never assume the host matches, and the conversion is the same code on every target.

The clearest instance of that rule in the fleet is on the *other* side of the kernel boundary. `iftoprs`'s macOS attribution path reads a socket's local port via `pidfdinfo::<SocketFDInfo>`, selects the correct arm of the `soi_proto` union by `soi_kind`, and converts the network-order `insi_lport` with a local `ntohs`. That is the interesting case precisely because the value did not come off the wire — it came from a kernel structure — and it is *still* in network order, because the kernel stores it as it arrived. A reader who assumed “kernel structure implies host order” would read every port number byte-swapped, and on a little-endian machine that failure is loud (port 80 reads as 20480) rather than subtle, which is the only mercy in this category.

Protocol numbers are standardised and endianness-free. `iftoprs`'s `Protocol::from_ip_next_header` maps 6 to TCP, 17 to UDP, 1 and 58 to ICMP, and anything else to `Other(n)`, whose `Display` renders as `Proto(n)` — so an ESP tunnel shows as `Proto(50)` and SCTP as `Proto(132)`, with those exact cases pinned by tests. These are single-octet assignments: there is no byte order to get wrong, and the value is identical on every platform and every kernel. The rule for this class is that the mapping is a data question, not a portability question, and the only design decision is what to do about unrecognised values.

Kernel enum values are per-platform and unstandardised. `lsotrs`'s `TcpState` maps the kernel TCP states to `lsotrs`'s spelling, and `from_raw` decodes numeric kernel codes 0 through 10 — eleven codes for the eleven states the type enumerates. Unlike a protocol number, nothing outside a particular kernel's headers assigns those integers. They are an internal enumeration exposed through an interface, and a different operating system is entitled to a different numbering. This is the class where a cross-platform port is most exposed, because the values look exactly like the standardised ones and behave completely differently.

The Unknown arm as an ABI escape hatch

The fleet's answer to the third class is consistent and worth naming as a pattern, because it appears in two crates with three different payload types.

`lsOfrs`'s `FileType` enumerates the Isof type vocabulary and carries an `Unknown(String)` fallback for raw type codes. Its `TcpState` carries an `Unknown(i32)` catch-all alongside the decoded states. `iftoprs`'s `Protocol` carries `Other(n)` and renders it as `Proto(n)`.

The pattern is: decode what you know into a typed variant, and preserve what you do not know *in its original form* rather than collapsing it to a default. The three payload types are each right for their domain — a string for a type code that is textual upstream, a signed integer for a kernel enum, a number for a protocol assignment — and the shared property is that no information is destroyed.

What this buys is precisely portability. A kernel that grows a twelfth TCP state produces an `Unknown` carrying the raw code rather than a wrong state or a panic, and the number is in the output for a human to interpret. A capture carrying a protocol the parser has never seen produces `Proto(50)` rather than being dropped, and its bytes are still counted. The alternative designs — panic, silently map to a default, or drop the row — each convert an unknown into either a crash or a lie. This is the same discipline the kernel chapter describes as answering a different question rather than fabricating one, applied at the type level.

Struct layout is the quiet one

The hazard with no good escape hatch is `repr(C)` struct layout in a hand-declared FFI block. `lsOfrs`'s Darwin path calls `libproc` against `repr(C)` structs matched to the kernel headers; `storageshower` walks the `statfs` array `getmntinfo` returns.

`repr(C)` guarantees that Rust lays a struct out using the platform C ABI's rules — field order preserved, alignment and padding as C would compute them. What it cannot guarantee is that the *fields you declared* are the fields the header declares. Three things vary by target and are easy to get wrong: the width of a type that is not fixed-size in C (a `long`, a `time_t`, an `off_t`), the alignment of a wide member on a 32-bit target, and the presence of padding or reserved members that a header includes and a hand transcription omits.

The consequence is that a struct transcribed correctly for one target can be wrong for another while compiling cleanly on both, and the failure is silent garbage rather than a crash. The defence is mechanical and cheap — assert the size of each FFI struct and the offset of each field against the platform's expectations, so a mismatch fails a test rather than producing a plausible number — and it is worth applying to every declaration in category one of the previous chapter.

Where the release matrix meets the source

The release automation is where architecture stops being abstract. The packaging chapter records the fleet-wide shape: a `release.yml` firing on `v*` tags, cross-compiling a per-target matrix of macOS aarch64 and x86_64 and Linux gnu, with musl via `cross` and typically marked `allow_failure`. `grcis`'s matrix is recorded concretely as six targets — `aarch64` and `x86_64 apple-darwin`, `x86_64` and `aarch64 unknown-linux-gnu`, and the two static `unknown-linux-musl` variants — and `storageshower`'s release workflow uses `cross` for the cross-compiled aarch64 Linux build.

Two consequences follow that are easy to miss.

The first is that a cross-compiled target is built but not exercised. A matrix entry proves the code compiles for that triple against that target's headers, which is a genuine and non-trivial check — it is exactly the check that catches a type width differing on another platform. It does not prove the binary produces correct output there, and the distinction is the same one the kernel chapter draws about `htoprs` carrying seven backend trees and being able to parity-test exactly one per run.

The second is that the C library is part of the target, not just the platform. `htoprs` parses its command line by calling the real `libc::getopt_long`, which is deliberate and buys a strong property: there is no second implementation of `getopt` to diff against, so the class of bug where a hand-written clone mishandles an edge case cannot exist. The corollary is that the parse is a property of whichever `libc` the binary links. A `glibc` build and a `musl` build call different code for the same declaration, and to whatever extent those implementations differ in their handling of edge cases — abbreviation, permutation of non-option arguments, error reporting — the shipped behaviour differs with them. The sources here record `glibc`'s `musl` targets and do not record `htoprs`'s target matrix, so whether this is live for `htoprs` today is not something this book answers. The structural point stands regardless, and it generalises: *any* decision to delegate behaviour to the platform C library converts a source-level guarantee into a per-target one.

The reading rule

For anyone extending one of these tools, the architecture question resolves into three checks that are quick to run and rarely run.

For every number crossing a boundary, ask which of the three classes it is in. Wire value: convert unconditionally. Standardised assignment: no conversion, and handle the unknown case. Kernel enum: this value is not portable, and it needs an `Unknown` arm and a comment naming the platform whose headers it came from.

For every `repr(C)` declaration, ask what proves it matches. If the answer is “someone read the header,” the declaration is checked on exactly the platforms where someone has run the tool and noticed a wrong number.

And for every behaviour delegated to a C library, ask whether the release matrix contains a target with a different implementation. If it does, the behaviour is per-target and the documentation should say which target it describes.

Names — turning a number into a label, and the three ways it goes wrong

Nearly every column these tools print is a name the kernel did not give them. The kernel supplies a numeric user id, an address, a port, a process id; the display shows a login name, a hostname, a service name, a command. Each of those translations is a lookup against a source outside the program, and every one of them can be slow, absent, or wrong. This chapter collects how the fleet performs those lookups, what each caches, and why three separate flags exist to switch them off.

Four resolutions, four different sources

The fleet performs four kinds of name resolution and they have almost nothing in common except their position in the pipeline.

uid to login name. `lsoprof` resolves this through the `users` crate and memoises the result per `Process` in a `OnceCell`, so the `getpwuid` lookup runs at most once regardless of how many open files that process holds. The `Process` type even hand-implements `Clone` so a populated username cache is carried into the clone rather than recomputed. This is the cheapest of the four by a wide margin in the common case and the most expensive in the uncommon one: a local passwd file is a fast read, and a directory-backed name service is a network round trip behind an interface that gives no indication of which you have.

address to hostname. `iftoprs` handles reverse DNS asynchronously through `dns-lookup`, caching into a `HashMap<IpAddr, CacheEntry>` behind an `Arc<Mutex<>>` with a TTL-based `evict_stale` pass. `nmaprs` has the richest surface here, with `hickory-resolver` for `--dns-servers`, `--system-dns` to force the OS resolver, `-n` to disable resolution, and `--resolve-all` in its extended option set. This is the slowest and least trustworthy of the four, for reasons below.

port to service name. `iftoprs` parses `/etc/services` eagerly at startup into a globally-cached `HashMap<u16, &'static str>, &'static str>` keyed by port and protocol, normalising the protocol string before lookup. Note what the value type says: `&'static str` means the table is parsed once and never freed, which is the right shape for a lookup consulted on every render that never changes during a run.

pid to command name. This is the one that is not really a name service at all. `iftoprs` reads `/proc/[pid]/comm` on Linux, and on macOS resolves the process name lazily via `name(pid)` only once a socket is confirmed, falling back to `pid:<n>` if the name lookup fails.

The three failure modes

Each of these can fail in three distinct ways, and the fleet's designs are legible once the three are separated.

It can be slow. DNS is the extreme case: an unresolvable address against an unreachable resolver costs the resolver's full timeout, and a display with fifty distinct remote addresses could cost fifty of them. This is why `iftoprs`'s resolver is asynchronous and cached rather than synchronous — the render path must never block on a name — and it is why the port and user tables are eager or memoised rather than looked up per row. The rule these three designs share is that no name resolution happens on the render path: it is either resolved before the path (the eager `/etc/services` parse), resolved once and cached (the `OnceCell` username), or resolved off-thread with the row rendered from whatever the cache currently holds.

It can be absent. A uid with no passwd entry, a port with no `/etc/services` line, a pid whose process exited before the lookup. The fleet's answers here are consistent with its answers everywhere else: `iftoprs` falls back to `pid:<n>` when a process name lookup fails, which is a label honest about being unresolved rather than an empty string that reads as a name. The same instinct appears in the JSON output omitting the process key entirely for unattributed flows, so a consumer can count them.

It can be wrong. This is the one that gets least attention and matters most for the network tools. A PTR record is published by whoever controls the reverse zone for an address, which is generally the address's operator and not the operator of whatever is answering on it. A reverse lookup therefore returns a name *asserted by the remote party*, and treating it as identifying is a category error. For a bandwidth monitor this is mostly harmless — you are reading the name to orient yourself, not to make a decision — but for a scanner writing a report someone will act on, it is exactly the sort of thing that should be labelled as what it is. The related hazard is that `/etc/services` says what a port number is *conventionally* for, not what is listening on it, which is precisely the gap `-sV` version detection exists to close by asking the service itself.

Why the no-resolve flag keeps being reinvented

Three flags in this fleet turn resolution off — `iftoprs`'s `-n`, `--no-dns`, its `-N` for port-to-service names, and `nmaprs`'s `-n` — and they are reproducing a convention that network tools have converged on independently for decades. It is worth being explicit about why, because the reasons are not the same in each case.

Speed is the reason people reach for it. A scan or a capture against a network of unresolvable addresses spends most of its wall clock in the resolver, and the flag removes that cost entirely.

Determinism is the better reason. Resolution results depend on a cache, a resolver, and a network — three things outside the program — so a run with resolution enabled is not reproducible even against an unchanged local system. For anything being diffed between two runs, the no-resolve flag is what makes the comparison meaningful.

Disclosure is the reason that applies to exactly one of these tools. A reverse lookup is a query to a nameserver, and a scanner that resolves every address it touches leaks the target list to whoever runs the resolver. This is one of the places where a security-relevant behaviour is controlled by a flag whose name mentions only names.

The `-N` case is the interesting one, because port-to-service resolution has none of those three costs — it is a local table read, it is fully deterministic, and it discloses nothing. It has a fourth cost instead: the name is often wrong in the only way that matters, since a service on port 8080 is not necessarily HTTP, and a display that says otherwise has substituted a convention for an observation. Turning it off is not a performance decision, it is a decision to see the number rather than the guess.

Caching, and the two invalidation policies

The fleet uses two different cache policies for the four resolutions, and the choice tracks whether the underlying mapping can change during a run.

Resolutions that cannot meaningfully change are cached forever. The `/etc/services` table is parsed once at startup into `&'static str` values, because the file will not change under a running process in any way anyone cares about. A `Process`'s username is memoised in a `OnceCell` for the lifetime of that value, because a process's uid does not change.

Resolutions that can change are cached with a TTL. `iftoprs`'s reverse-DNS cache carries a `CacheEntry` with a TTL-based `evict_stale` pass, because DNS records have expiries and a long-running monitor would otherwise pin a name that has moved. That is the correct distinction, and the general rule it encodes is that cache policy should follow the *authority* of the underlying data: a local file read at startup is as authoritative as it will ever be, while a network lookup is a lease.

The one thing neither policy handles is a mapping that changes to something *e/*se while a cached value is in use — a uid reassigned, an address reused by a different host. For the lifetime of these processes that is not a practical concern, and the reason to name it is that a long-lived daemon built on the same patterns would need to care, and would need the TTL policy on all four rather than on two of them.

The fleet at the prompt – seven worked sessions

Every preceding chapter reads the tools by structure. This one reads them by task. Each session below is a real diagnostic shape — the kind of question that sends someone to a terminal at an inconvenient hour — worked with the flags the book has already documented, with the reasoning about *why* each step is the next one. Nothing here introduces a capability; the value is in the sequencing, which is the part that the per-tool chapters cannot show because it spans tools.

One: the filesystem is full and `du` disagrees

The report is that a volume is at capacity. `du` over the mount adds up to substantially less than the capacity number, which is the classic signature of space held by a file that has been unlinked but is still open — the directory entry is gone so the walk cannot see it, but the blocks are not freed until the last descriptor closes.

```
storageshower -s pct -l -w 85 -C 95
lsf +L1
lsf +L1 -u www-data
```

`storageshower` first, and specifically with `-l` to drop virtual filesystems and `-s pct` to sort by percentage full, because the first question is whether one volume is the problem or the machine is generally tight. Setting `-w/-C` thresholds is not cosmetic here: the NOMINAL/WARNING/CRITICAL badge is what makes a many-mount display scannable at a glance, and the thresholds also cycle from inside the TUI (`t` walks warn, `z` walks crit) if the defaults are wrong for this host.

Then `lsf +L1`. This is the entire reason the `+L` grammar is worth reproducing faithfully: `+L` alone lists the NLINK column, and `+L1` selects files whose link count is below one — unlinked, still open, still holding blocks. Adding `-u` or `-c` narrows to a suspected service. The fix is to restart the holder or, if the descriptor is known and the process cooperates, to truncate through `/proc/<pid>/fd`; the diagnosis is complete the moment `+L1` names a PID.

`lsf --stale` is the adjacent query worth knowing about, for the case where the answer is not unlinked files but a process holding descriptors on a filesystem that has been replaced underneath it.

Two: a process is leaking descriptors

The symptom is an `EMFILE` in a service log, or a slow climb in a monitoring graph. The question is which descriptors are growing and whether the growth is monotonic or a burst.

```
lsf --summary
lsf --leak-detect=10,5 -c myservice
lsf --delta -r 2 -p 4242
```

`--summary` first for the type histogram and the top-N-by-FD table, because the shape of the answer differs completely depending on whether the growth is in sockets, pipes, or regular files. A histogram dominated by `IPv4` points at connection handling; one dominated by `PIPE` points at subprocess plumbing not being reaped.

`--leak-detect` then makes the growth claim precise. The spec is `interval,threshold` — here a 10-second sampling interval and a requirement of five consecutive increases before it reports — and the threshold is what separates a leak from a busy period. The defaults are a 5-second interval and three consecutive increases, with the threshold clamped to a minimum of two, which is the smallest value that means anything at all: a single increase is not a trend.

`--delta -r 2` is the confirmation step and the one that actually names the bug. It repeats every two seconds and colours the descriptors that appeared since the last frame, listing the ones that vanished after the table. A leak shows as a steady stream of new descriptors with an empty gone-list, and the *names* of the new descriptors are the diagnosis — repeated sockets to one endpoint is a connection-pool bug, repeated opens of one path is a file handle not being closed on an error branch.

Three: attributing a bandwidth spike

The link is saturated and the question is which process is responsible. This is the query `iftop` exists for and where the process column earns its place.

```
sudo iftoprs -i en0 -n -N
sudo iftoprs -i en0 -f "tcp port 443" -F 192.0.2.0/24
sudo iftoprs -i en0 --json | jq -c '.flows[] | {dst, sent_2s}'
```

Start with reverse-DNS and service-name resolution *off* (`-n -N`). On a saturated link with many distinct peers, resolution is the thing that makes the display stutter, and the addresses are more useful than the names during triage anyway. Inside the TUI, `Tab` switches between the flow view and the process view, which is the switch that converts “this address pair is heavy” into “this program is heavy”; `1/2/3` re-sort by the 2s, 10s, and 40s columns, and the choice matters — sorting by 2s finds the current burst, sorting by 40s finds the sustained consumer, and a flow that leads one and not the other is telling you something about its shape.

Narrowing comes second. `-f` takes a BPF expression, so the filtering happens in the kernel before the packet is ever copied to userspace, which is categorically cheaper than filtering in the display. `-F` sets the IPv4 local-network scope, which changes what counts as inbound versus outbound and is the flag people forget on hosts with more than one subnet.

The `--json` line is the same capture with no TUI at all, emitting one NDJSON object per refresh. This is the form to reach for when the spike is intermittent and the answer is to leave something running: it is a live stream a `jq` filter consumes incrementally, so a redirect to a file is a usable record rather than a JSON document that only becomes parseable when the tool exits.

Four: a connection that is technically up and going nowhere

Throughput has collapsed but nothing is erroring, and the ambiguity is whether the local process has stopped reading, the peer has stopped reading, or the network is dropping. That distinction lives in the socket buffers.

```
lsf --stall -r 2 -c myservice
lsf -T -p 4242
lsf -i 4TCP:443 -a -c myservice
```

`--stall` is the classifier and it is meant to be paired with `-r`, because the verdict is a statement about a trend rather than about a sample — it requires at least three consecutive samples before it will commit, keeps a bounded per-FD history, and treats occupancy at or above eighty percent of the buffer limit as near capacity. `TX-STALLED` means the local process keeps writing and the send buffer is not draining: the peer or the network is not consuming. `RX-STARVED` means bytes have arrived and the receive buffer is growing because the local process is not reading them fast enough — an application-level problem, usually a blocked worker. Those two verdicts point at opposite ends of the system, which is exactly the disambiguation the situation needs.

Two platform facts belong in the reading. On macOS the classifier has both occupancy and buffer limits and uses a fill-ratio rule; on Linux `/proc/net/tcp` gives occupancy but not the limit, so it falls back to a trend rule — persistently non-zero and not draining for TX, persistently non-zero and strictly growing for RX. The verdicts mean slightly different things on the two platforms and the tool does not pretend otherwise. Second, PID reuse is handled: if the command name on a (`pid`, `fd`) changes, the history is cleared and the verdict resets, so a restarted service does not inherit its predecessor's diagnosis.

`-T (--tcp-info)` adds the per-connection kernel detail, and the third line is the general pattern for narrowing: `-a` flips selection from OR to AND, so `-i 4TCP:443 -a -c myservice` means “IPv4 TCP port 443 *and* this command” rather than the union. Forgetting `-a` is the single most common way an Isof-family invocation returns far more than expected.

Five: auditing a subnet, resumably

An inventory scan of a block, run under a maintenance window, with output that has to be readable both by a person and by whatever tooling already exists.

```
nms -sn 192.0.2.0/24 -oG discovery.gnmap
nms -iL live.txt -sS --top-ports 200 -T4 -oA audit
nms --resume audit.state
```

Discovery first and ports never, which is what `-sn` means. Separating discovery from scanning is worth the extra step on any block larger than a handful of hosts, because the host list is the artifact you want to review before committing to a port scan, and `-oG` produces exactly the grepable form that turns into an input list with one `awk`.

The scan then runs against `-iL`, with the technique chosen deliberately. `-sS` requires privilege; unprivileged, the tool detects `geteuid()` and falls back to a connect scan silently and correctly. If the audit's method matters — and for an audit it usually does — state it explicitly with `--privileged` or `--unprivileged` rather than letting the environment decide, and note that `--unprivileged` will *reject* raw-only techniques (`--s0`, `--sI`, SCTP) rather than substituting something else. `--top-ports 200` scans by the embedded frequency table rather than by numeric range, which is a different and usually better two hundred ports than `1-200`. `-oA` writes the normal, grepable, and XML forms together, and the XML is the one to keep: it is the format with existing consumers.

`--resume` is the flag that makes a long scan survivable. A checkpointed scan that is interrupted resumes from its state file rather than restarting, which matters more on a scheduled window than any timing template does. On timing: `-T4` is a template, and the templates set worker counts and timeouts together — `-T3`, the default, maps to 256 workers and a 1500 ms timeout — so reaching for `-T5` on a congested link produces retransmits and a *slower* scan with worse data, which is the standard way this flag is misused.

Six: triaging a log stream and a command's output

Two different colouring problems that are easy to conflate, and the fleet uses two different tools for them because they are not the same problem.

```
journalctl -f | zcolorizer -m auto --novelty
kubectl logs -f pod | zcolorizer -m json -g ERROR
grc tail -f /var/log/syslog
some-command | grcat conf.log
```

`zcolorizer` is for a *stream whose format it can recognise*. `-m auto` selects format modules by inspection; `-m json` pins one when the stream is known and inspection would waste effort or guess wrong. `-g PATTERN` filters to matching lines while keeping the colouring, and `-H PATTERN` is its more useful sibling in triage — it keeps every line and dims the non-matches, which preserves the context around a hit instead of destroying it. `--novelty` is the flag that has no analogue anywhere else in the fleet: intensity is driven by statistical

novelty, so a message repeated a thousand times self-dims and the line that has appeared once stands out. On a firehose that is the difference between reading and scrolling. `--novelty-decay D` ages the counts so a message that was common an hour ago and has stopped becomes novel again — which is what you want when watching a system change state.

`grc` and `grcat` are for a *command whose output has a colourfile*. `grc tail -f /var/log/syslog` runs the command through the launcher, which selects the colourfile by matching the command line against `grc.conf`. `grcat conf.log` skips the launcher entirely and colours a pipeline directly, which is the form to use inside a script or when the command line is not one `grc.conf` recognises. Two behaviours are worth remembering at the prompt: option scanning stops at the first non-option token, so everything after the command name belongs to the command and not to `grc` — `grc tail -f file` passes `-f` to `tail`, which is why `--` is rarely needed — and abbreviated long options resolve by unique prefix, so `--conf=x` works and `--std` is refused as ambiguous between `--stdout` and `--stderr`.

Colour policy composes with both: piping either into `less -R` needs the force flag (`-C` for `zcolorizer`, `--colour=on` for `grc`), because the isatty gate correctly turns colour off when the next stage is a pipe.

Seven: a before-and-after comparison, without inventing filenames

The everyday workflow that `temprs` exists for: capture state, change something, capture again, diff — without hand-managing paths in `/tmp` or losing the captures when the shell exits.

```
lsf -p 4242 | tp -w before
systemctl restart myservice
lsf -p 4242 | tp -w after
tp -D before after
tp --wc before ; tp --wc after
tp -g CLOSE_WAIT after && echo "still leaking"
tp --expire 24
```

Everything here turns on the same property: entries are addressed by *tag* or by *signed index*, never by path. `-w NAME` pushes the piped input tagged, and every subsequent operation takes the tag interchangeably with an index, where `-1` is the top of the stack. Negative indices work because almost every index-taking flag sets `allow_hyphen_values(true)` — without that setting `-1` would be parsed as a short flag, and the whole addressing scheme would need a different spelling.

`-D A B` shells out to `diff -u` between two entries; `--wc` gives a line count; `-g PATTERN` is a literal substring search that exits 0 or 1, which is what makes the `&& echo` composition legal in a shell rather than requiring an `if`. `-e` opens an entry in `$EDITOR`, `--replace IDX PAT REPL` rewrites it in place, and `--path IDX` prints the backing file path for the case where another program needs a real file (`cat "${tp --path before}"`).

Two lifecycle notes. The stack survives the shell — it lives under `$TMPDIR/temprs`, overridable with `TEMPRS_DIR` — which is the point: the capture you took in one pane is addressable from another. And it therefore needs pruning, which is what `--expire HOURS` is for; the argument is validated against negative, NaN, and infinite values *before* the cast rather than *after*, and `--clear` refuses outright unless the working directory's basename contains `temprs`, which are the two places this tool declines a request rather than recovering from it.

Tab completion is worth mentioning at the end because it changes how the workflow feels: the `zsh` completion reads the running stack's master file, so pressing Tab after `tp -D` offers the actual tags and indices currently present rather than a static list.

What the seven have in common

Read together, the sessions expose the same three properties from the architecture chapters, now in their operational form.

Selection filters compose, and they compose the same way in every mode. `lsf --tree -u root`, `lsf --stale -u www-data --json`, and `lsf --ports -u root` are all the same filter language applied to different views, which is why a session can start broad and narrow without learning a second syntax at each step.

The TTY gate is what makes the sessions scriptable without modification. Each of these can be run interactively and then pasted into a script, and the only thing that changes is that the chrome disappears — the live modes print one snapshot instead of entering the alternate screen, the colours drop, and the structured formats are unaffected because format is not gated on the terminal.

And every step that produces a number produces it with a stated method. The `--stall` verdict says which rule it used on which platform, the leak detector states its interval and threshold, `iftoprs`'s columns are documented as summed byte counts over trailing one-second slots scaled to bits rather than as slot-averaged rates. At three in the morning, the difference between a tool that answers and a tool that answers *and tells you what it measured* is the difference between a diagnosis and a guess that happened to be right.

The shell-side fleet — four tools that ship no compiled code

Every tool covered so far compiles. Four do not, and they are part of the same stack: `revolver`, `fasd-simple`, `gh_reveal`, and `kubectl-aliases` are shell scripts, a POSIX `sh` program, a generated alias file, and the plugin shims that load them. They get a chapter because leaving them out would misdescribe the fleet — the shell is where all of this runs, and four of the tools in it never leave the shell — and because they are the clearest available demonstration of the same disciplines the Rust ports apply, expressed in a language with no compiler to catch anything.

The measurements in this chapter were taken directly from the working trees. The sizes:

```
$ wc -l revolver/bin/revolver revolver/src/_revolver \
    fasd-simple/bin/fasd gh_reveal/bin/reveal gh_reveal/_reveal \
    kubectl-aliases/generate_aliases.py kubectl-aliases/.kubectl_aliases
318 revolver/bin/revolver
 34 revolver/src/_revolver
620 fasd-simple/bin/fasd
 85 gh_reveal/bin/reveal
 33 gh_reveal/_reveal
183 kubectl-aliases/generate_aliases.py
804 kubectl-aliases/.kubectl_aliases
```

Four programs, 2,077 lines of executable text between them including the completions and the generator. The whole shell-side fleet is smaller than `grc`'s two Rust binaries.

revolver — a spinner that owns a state file

`revolver` is a progress spinner for `zsh` scripts, and its interesting property is that the spinner is a *separate process* driven through a state file rather than a subshell writing to the same terminal. `bin/revolver` opens with an associative array of spinner definitions, each a delay followed by its frames:

```
local -A _revolver_spinners
_revolver_spinners=(
  'dots' '0.08 █ █ █ █ █ █ █ █ █ █'
  'line' '0.13 - \\ | /'
  ...
)
```

The catalogue holds 55 styles, counted directly out of the table:

```
$ perl -ne "print \"\${1}\n\" if /^ '([A-Za-z0-9_]+)' /" bin/revolver | wc -l
55
```

The public surface is four commands and three options, and `revolver --help` prints exactly that: `start`, `update`, `stop`, `demo`, with `-h/--help`, `-v/--version`, and `-s/--style`. `revolver --version` reports `0.2.0`. Parsing is `zparseopts -D` rather than a hand-rolled loop, which is the `zsh`-native choice and the reason the option handling is nine lines instead of forty.

The state lives at `${REVOLVER_DIR:-${ZDOTDIR:-$HOME}/.revolver}/${PPID}` — one file per *parent* pro-

cess, holding the spinner PID followed by the current message. `start` creates the file and forks `_revolver_process $PPID &!`; `update` rewrites the message into the same file; `stop` reads the PID out of it, kills that process, and removes the file. The spinner loop's two exit conditions are the whole orphan-prevention design: it exits if the state file has been removed, and it exits if `kill -s 0 $pid` against its parent fails. Neither check requires the parent to cooperate on the way out, which is what makes the design survive a script that dies without calling `stop`.

The one rough edge is visible from the outside and the test suite pins it. An unrecognised style reaches a line that calls `color`, a function from a shell framework the script no longer ships with:

```
$ revolver -s nosuchstyle start x
revolver:32: command not found: color
$ echo $?
1
```

The call site is `bin/revolver:299`, `echo $(color red "Spinner '$style' is not recognised")`. The message is lost, the exit status is not, and the suite pins the exit status rather than the text — which is the right pin to have written, because it is the part a caller can act on.

fasd-simple – 620 lines of POSIX sh that generate their own integration

`fasd-simple` tracks the files and directories you have used and ranks them by “frequency”. What matters architecturally is that `bin/fasd` is `#!/usr/bin/env sh` — not `zsh`, not `bash` — and that it does not integrate with any shell directly. It *prints* the integration and lets the shell `eval` it. `fasd --init` takes a list of mode names and emits the corresponding shell code:

```
auto posix-alias tcsh-alias zsh-hook bash-hook posix-hook tcsh-hook
zsh-ccomp zsh-wcomp zsh-ccomp-install zsh-wcomp-install
bash-ccomp bash-ccomp-install
```

Thirteen emitters, of which `auto` is a dispatcher that picks the right set for the detected shell. The plugin file is thereby able to be seventeen lines: it runs `fasd --init auto` into `$ZSH_CACHE_DIR/fasd-init-cache`, regenerating only when the `fasd` binary is newer than the cache, and sources the cache. That is the same “pay the cost once, keep the artifact, invalidate on mtime” pattern the compiled tools use for bytecode caches, implemented with `-nt`.

Configuration is twelve environment variables with defaults set in one block — `_FASD_DATA`, `_FASD_BLACKLIST`, `_FASD_SHIFT`, `_FASD_IGNORE`, `_FASD_SINK`, `_FASD_TRACK_PWD`, `_FASD_MAX`, `_FASD_BACKENDS`, `_FASD_FUZZY`, `_FASD_VIMINFO`, `_FASD_RECENTLY_USED_XBEL`, and `_FASD_AWK`, the last of which is resolved by probing `mawk`, `gawk`, `original-awk`, `nawk`, `awk` in order and taking the first that runs. The ranking arithmetic is an `awk` program, which is why the probe exists at all.

The security-relevant surface is `--sanitize`, because the shell hooks build a command line from history and feed it through `--sanitize` before an `eval`. The implementation is two substitutions at `bin/fasd:274`:

```
--sanitize) shift; printf %s\\n "$*" | \
sed 's/\([^\]\)\${ *[^ ]* *\([^\]\)*\})*/\1\2/g
s/\([^\]\)\[!&;<>$`{}]\{1,\}/\1 /g'
;;
```

The first collapses command substitution, the second replaces runs of shell metacharacters with a single space. Driven from outside, the effect is visible:

```
$ sh bin/fasd --sanitize 'foo; rm -rf /'
foo rm -rf /
```

The semicolon is gone and the words survive, which is exactly the intended outcome: what reaches `--proc` is

a list of words to rank, and a word list cannot execute anything when the hook evaluates it. The four tests in `tests/t-sanitize-query.zsh` drive the binary as a subprocess with adversarial input rather than grepping the source for the `sed` expression, and their header says why — a re-vendor that changes the transformation has to fail even if the source text still matches.

gh_reveal — eighty-five lines and four private functions

`gh_reveal` opens the GitHub (or Heroku) page for the current repository's remotes. The whole program is `bin/reveal`, 85 lines of bash defining four helpers inside the `reveal` function — an open-command detector, two git-context predicates, and a zsh probe — and then a pipeline that turns `git remote -v` output into URLs.

Two details are worth extracting because they are decisions, not accidents. The first is that every exit path `unset -f` the private helpers, so sourcing the file and calling `reveal` leaves no functions behind in the shell's namespace; one of the 27 tests in `tests/t-reveal.zsh` exists solely to pin that all four are unset on every branch. The second is that the remote-to-URL pipeline strips a port suffix with `perl -pe 's@:\d{1,5}/@/@"'` rather than `sed`, which is the portable choice for a program that must behave identically under BSD and GNU userland.

The dispatch has three branches: not a git directory and no arguments opens the account's repository list; not a git directory with arguments recurses into each argument in a subshell; inside a repository, the argument list becomes an alternation matched against the remote names. `$GITHUB_ACCOUNT` overrides `git config user.name` for the first branch, which is the escape hatch for a machine whose git identity is not the account you browse as.

The completion, `_reveal`, is 33 lines and reads the live remote table into a map of name to URLs, offering an `All:` entry alongside each named remote. It declines to complete anything when the current directory is not a repository, emitting `_message "$PWD is not a git dir"` instead of an empty list — the difference between “nothing matches” and “this question does not apply here”.

kubectl-aliases — 790 aliases nobody typed

`kubectl-aliases` is the odd one: the shipped artifact is generated, and the generator is 183 lines of Python that enumerate the valid combinations of a command, a global flag, twelve operations, eight resource types, seven argument flags, and three positional-argument flags, subject to per-item requirement and incompatibility lists. The plugin sources the generated file and errors to stderr if it is missing; that is the entire runtime.

The claim that the checked-in file is the generator's output is verifiable, and verifying it is a one-liner:

```
$ python3 generate_aliases.py > /tmp/kal.txt && diff .kubectl_aliases /tmp/kal.txt \
&& echo IDENTICAL
IDENTICAL
$ grep -c '^alias ' .kubectl_aliases
790
$ perl -ne "print \"\$1\n\" if /^alias ([^=]+)=/" .kubectl_aliases | sort -u | wc -l
790
```

Byte-identical regeneration, 790 aliases, and 790 distinct alias names — no collisions, despite the generator's own source carrying a `# caution: reusing the alias` comment on the second `all` entry, which is disambiguated by the incompatibility lists rather than by renaming. The longest names are seventeen characters (`ksysgdepwslowidel` and two siblings), which is what an enumeration over six orthogonal axes produces when every axis is optional.

The value of a generated alias file is that it cannot drift from its own rules; the cost is that it cannot be hand-edited, and the twenty tests in `tests/t-aliases.zsh` are written around that constraint — they pin the

plugin's `${0:h}`-relative lookup, the stderr branch, and properties of the generated file rather than the text of individual aliases.

What the four have in common

All four are zsh plugins following the same plugin-standard preamble (`0="${0:${0:##$ZSH_ARGZERO}}:-${(%):-%N}"`), all four keep their tests in `tests/*.zsh` under the same runner, and all four suites pass:

Repository	test files	@test blocks	zunit result
revolver	3	40	40 passed, 0 failed
fasd-simple	4	29	29 passed, 0 failed
gh_reveal	3	33	33 passed, 0 failed
kubectl-aliases	1	20	20 passed, 0 failed

The counts were taken with `grep -c '^@test '` per file and the results by running `zunit tests/*.zsh` in each repository with `TERM=xterm COLUMNS=120 LINES=40`. The `@test` count and the passed count agree exactly in all four, which is the shell-side equivalent of a suite with no `#[ignore]` and no `#[cfg]`-gated tests: every test written is a test run.

The suites also share a shape. Roughly half of the blocks in `revolver`, `gh_reveal`, and `kubectl-aliases` are *contract pins over source text* — they assert that the plugin file appends to `fpath`, that a shebang is what it is, that the file parses under `zsh -n`. That is a weaker kind of test than driving the program, and the repositories know it: `gh_reveal` carries a separate `t-reveal-exec.zsh` with three tests that stub the browser opener and assert the URL actually produced, and `fasd-simple` carries `t-sanitize-query.zsh` whose header explicitly contrasts itself with “grep-the-source contract pins”. The pattern to take from this is that a shell project with no type system gets its floor from parse checks and its confidence from execution, and the two need to be in separate files so nobody mistakes the floor for the confidence.

The dependency ledger – fifty-eight direct crates and what the lockfiles make of them

The shape-of-the-fleet chapter names the dependency floor as a shared convention. This chapter reads the manifests and the lockfiles and reports what is actually there: which crates each tool declares, how many packages the resolver pulls in behind them, where the resolved versions agree, and where they do not. The distinction matters because a “durable dependency” rule is a claim about the direct edges, and the build is a property of the transitive closure.

The direct edges

Parsing the ten `Cargo.toml` files gives the following, counting `[dependencies]` separately from `[dev-dependencies]`, `[build-dependencies]`, and the `[target.'cfg(...)' .dependencies]` blocks:

Crate	direct	dev	build	target-specific
<code>lsofrs</code>	13	0	0	0
<code>nmaprs</code>	25	4	1	0
<code>temprs</code>	5	1	0	0
<code>iftoprs</code>	14	0	0	4 (macOS)
<code>htoprs</code>	8	0	0	3 (Linux)
<code>storageshower</code>	10	4	0	0
<code>zcolorizer</code>	8	0	0	0
<code>grcrs</code>	2	0	0	0
<code>zwire-host</code>	7	1	0	1 (Windows)
<code>zpwchrome-host</code>	5	0	0	1 (Unix)

Across all ten there are 58 distinct direct dependency names. Nine of them appear in four or more crates and constitute the fleet's actual shared floor:

Crate	used by
<code>serde</code>	8
<code>serde_json</code>	7
<code>clap</code>	6
<code>libc</code>	6
<code>dirs</code>	5
<code>regex</code>	5
<code>toml</code>	5
<code>crossterm</code>	4
<code>ratatui</code>	4

Seven more are shared by exactly two (`anyhow`, `chrono`, `nix`, `rayon`, `sysinfo`, `thiserror`, `tokio`), and the remaining forty-two appear once. The distribution is the interesting part: the fleet is not built on a broad common platform, it is built on a *narrow* one — serialization, argument parsing, libc, regex, and the terminal pair — with everything else specific to one tool's problem. `pcap` appears only in `iftoprs`, `pnet` only in `nmaprs`, `lz4_flex` only in `storageshower`, `portable-pty` only in `zwire-host`.

The three target-specific blocks are worth reading as a group because they are the only place the fleet's platform assumptions are declared rather than `cfg!`-branched in code. `iftoprs` pulls `core-foundation`, `core-foundation-sys`, `libproc`, and `security-framework` only under `cfg(target_os = "macos")`; `htoprs` pulls `libmedium`, `neli`, and `zbus` only under `cfg(target_os = "linux")`; `zwire-host` pulls `interprocess` only under `cfg(windows)`. A reader who wants to know what a tool needs from a platform can read those three blocks before reading any code.

One edge is internal: `zpwchrome-host` declares `zwire-host` as a dependency. That is the only tool-on-tool edge in the ten manifests, and it is the mechanism behind the shared native-messaging and socket machinery the two hosts' chapter describes.

What the resolver makes of them

Direct edges are a design statement; the lockfile is the build. Counting `[[package]]` entries per `Cargo.lock`:

Crate	direct	locked packages	ratio
<code>gcrs</code>	2	9	4.5×
<code>zcolorizer</code>	8	54	6.8×
<code>temprs</code>	5	100	20×
<code>zpwchrome-host</code>	5	102	20×
<code>htoprs</code>	8	166	21×
<code>lsofrs</code>	13	242	19×
<code>nmaprs</code>	25	256	10×
<code>storageshower</code>	10	271	27×
<code>iftoprs</code>	14	273	20×
<code>zwire-host</code>	7	397	57×

`gcrs` is the outlier at the bottom and it is worth saying why: two direct dependencies, `fancy-regex` and `libc`, resolve to nine packages total. That is the smallest closure in the book and it is the reason `gcrs` can be built on a machine with nothing cached in under a minute. `zwire-host` is the outlier at the top for an equally simple reason — it depends on `tauri`, and a GUI toolkit is a platform, not a library.

The number that predicts pain, though, is not the closure size; it is how many packages appear at more than one version:

Crate	packages resolved at 2+ versions
<code>temprs</code> , <code>zcolorizer</code> , <code>gcrs</code>	0
<code>htoprs</code> , <code>zpwchrome-host</code>	2
<code>lsofrs</code> , <code>storageshower</code>	10
<code>nmaprs</code>	11
<code>iftoprs</code>	18
<code>zwire-host</code>	35

Three crates have a fully unified graph. At the other end, `zwire-host`'s thirty-five duplicated packages are almost entirely the Windows support surface (`windows`, `windows-core`, `windows-sys`, `windows-targets`, and the six per-architecture `windows_*` crates) plus two generations of the `toml` family — which is what

pulling a cross-platform GUI framework into a graph costs, paid in compile time on a machine that will never build for Windows.

Two duplications recur across the fleet and are worth naming because they are everyone's, not anyone's fault: `hashbrown` appears twice in six of the ten graphs, and the `thiserror/thiserror-impl` pair appears twice in five. Both are ecosystem-wide transitions in progress, visible in a lockfile long before any tool's own manifest mentions them.

Where the versions agree

The most useful measurement here is the one that shows the durable-dependency rule working. Reading the resolved version of each shared crate out of all ten lockfiles:

Crate	resolved versions across the ten lockfiles
<code>serde</code>	1.0.228 — identical in all 9 graphs containing it
<code>serde_json</code>	1.0.150 — identical in all 7
<code>libc</code>	0.2.186 — identical in all 10
<code>clap</code>	4.6.1 — identical in all 6
<code>dirs</code>	6.0.0 — identical in all 5
<code>crossterm</code>	0.29.0 — identical in all 4
<code>ratatui</code>	0.30.1 in three, 0.30.2 in <code>htoprs</code>
<code>regex</code>	1.12.3 in five, 1.12.4 in two, 1.13.0 in <code>zwire-host</code>
<code>toml</code>	1.1.2 in three, 0.8.23 in three, three versions in <code>zwire-host</code>

Six of the nine shared crates resolve to a single version across every repository that uses them, and they are the six with the broadest reach. That is not enforced by any workspace — these are ten independent crates with ten independent lockfiles, one of them (`zcolorizer`) declaring a bare `[workspace]` explicitly to *avoid* being absorbed into a host workspace — so the agreement is a consequence of the manifests declaring loose major-version requirements (`serde = "1"`, `regex = "1"`, `libc = "0.2"`) and the lockfiles being refreshed on a similar cadence.

The three that disagree disagree in instructive ways. `ratatui`'s split is a single patch release and means nothing. `regex`'s three versions span two minor releases and mean only that the lockfiles were last touched at different times. `toml`'s split is the real one: three crates are on the 1.x line and three are still on 0.8, and those are different APIs. A tool moving a config-file change between repositories has to know which side of that line each repository is on, and no amount of “we all use `toml`” at the manifest level tells them.

The reading

The ledger supports the fleet's dependency claim and qualifies it. The claim — foundational crates, chosen for durability — holds at the direct edge: the nine shared dependencies are `serde`, `serde_json`, `clap`, `libc`, `dirs`, `regex`, `toml`, `crossterm`, `ratatui`, and every one of them is a crate whose absence would be a bigger event than its presence. The qualification is that the direct edge is where the choosing happens and the lockfile is where the *consequences* show up, and the consequences vary by two orders of magnitude across the fleet: 9 packages for `grcrs`, 397 for `zwire-host`. A durability rule applied to direct dependencies is a real rule with a real effect, and it does not constrain the closure at all.

The umbrella hygiene gate – twenty-one identical scripts and what they refuse

Five of the repositories in this book carry a `tests/` directory containing, in addition to their language-specific tests, twenty-one bash scripts. They are not about the tool. They check the *documentation*: the README, the man pages, the generated HTML under `docs/`, the GitHub Actions workflow, and the manifest. They are the least glamorous artifact in the fleet and the most reusable, and they are worth a chapter because they are the only mechanism in the book that enforces a convention across repositories rather than within one.

They are the same twenty-one files

The first thing to establish is that these are not five similar suites; they are one suite present in five places — `lsoprof`, `nmaprs`, `temprs`, `iftoprs`, and `zpwirchrome`, the last of which contains no Rust at all. Comparing every script in `lsoprof/tests` with its namesake in the others:

```
$ for f in $(find lsoprof/tests -name '*.sh' | xargs -n1 basename); do
  cmp -s lsoprof/tests/$f nmaprs/tests/$f && echo "IDENT $f" || echo "DIFF $f"
done
IDENT cargo-final-newline.sh
IDENT docs-final-newline.sh
IDENT docs-has-body-tag.sh
...
IDENT workflow-no-tabs.sh
```

Twenty-one `IDENT`, zero `DIFF`, and the same result running the comparison against `temprs`, `iftoprs`, and `zpwirchrome`. Byte-identical in all five repositories — including the browser-extension repository, whose other tests are JavaScript and whose only stake in `Cargo.toml` is the nested host crate. The scripts are also self-locating rather than path-configured — each one begins

```
set -uo pipefail
root="$(cd "$(dirname "$0")/.." && pwd)"
cd "$root" || exit
```

and then walks the tree below `$root` with `find`. That is what makes one file work unmodified in four repositories, and it is also what makes the same file work when the repository is checked out as a submodule of a larger tree: the scope is always “everything under the directory containing my parent”, never a hard-coded project name.

By subject, the twenty-one divide as: nine over `docs/*.html`, four over `README.md`, three over man pages, two over the CI workflow, two over the test scripts themselves, and one over `Cargo.toml`.

What they actually check

The checks are mostly floors — the presence of a thing whose absence is a silent failure downstream. `docs-has-h1.sh`, `docs-has-body-tag.sh`, and `docs-has-html-closing.sh` pin structural complete-

ness of generated pages. `readme-has-badges.sh`, `readme-has-h2-section.sh`, and `readme-has-https-link.sh` pin that the README has not been reduced to a stub. `man-page-synopsis-section.sh` pins `.SH SYNOPSIS`. Five of the twenty-one check for a trailing newline, which sounds trivial until you have concatenated two files without one.

Two of them are not floors but prohibitions, and both encode a specific failure the organization has already hit. `docs-no-inline-handlers.sh` scans every `docs/*.html` for DOM event-handler attributes and its header states the reason at length: Tauri v2's release Content-Security-Policy is `script-src 'self'`, an inline `onclick=` is equivalent to an inline `<script>` under that policy, and a handler that works in the dev server and silently does nothing in the release build is the canonical way to lose an afternoon. Several of the organization's Tauri applications compile their `docs/` into bundled assets, so an inline handler written into a documentation page becomes a defect in an application. The other is `docs-no-http-links.sh`, which is the same shape of rule applied to mixed content.

The prohibition scripts are also where the allowlist discipline appears, and it is worth reading because it is the correct way to write one. `docs-no-inline-handlers.sh` exempts exactly two things: a page whose `<meta name="generator">` says `rustdoc`, because rustdoc emits `onclick="history.back();" on its own generated pages and the fix belongs upstream; and a file beginning <!--PORT-REPORT-SCHEMA, because that page is regenerated wholesale by a report generator on every run. Both exemptions are keyed on a marker inside the file that identifies its generator, not on a filename, and both name a party responsible for the exemption. An allowlist keyed on paths would have rotted the first time a file moved.`

Running them

The scripts are ordinary programs and produce ordinary output. Running the full set inside one repository:

```
$ cd lsofrs 66 for f in tests/*.sh; do bash "$f" >/dev/null 2>&1 \
  || echo "FAIL $f"; done; echo done
done

$ bash tests/man-page-synopsis-section.sh | tail -2
---
Summary: 1 man pages checked, 0 without .SH SYNOPSIS

$ bash tests/docs-no-inline-handlers.sh | tail -2
---
Summary: 2 docs/*.html files checked, 0 allowlisted (rustdoc / port_report), 0
  ~ inline-handler tags across 0 file(s)

$ bash tests/readme-has-badges.sh | tail -1
Summary: README.md has 6 badge(s)
```

Twenty-one pass, zero fail. Every script prints a `Summary:` line with the count of things it examined, which is the property that makes a gate readable rather than merely binary: a gate that passes because it found nothing to check is indistinguishable from a gate that passes because everything was correct, unless it tells you how many files it looked at. `1 man pages checked` is a useful number; a bare `PASS` would not be.

The convention they reveal

Running the man-page gate exposes a naming convention the rest of the book only implies. Six of the tools ship two man pages, not one:

```
$ find lsofrs/man htoprs/man temprs/man -type f
lsofrs/man/man1/lsofrs.1
lsofrs/man/man1/lsofrsall.1
htoprs/man/man1/htoprsall.1
htoprs/man/man1/htoprs.1
temprs/man/man1/temprsall.1
temprs/man/man1/temprs.1
```

`<name>.1` is the conventional page — NAME, SYNOPSIS, DESCRIPTION, OPTIONS — and `<name>all.1` is a comprehensive cheatsheet with its own structure. The SYNOPSIS gate explicitly skips any page whose basename ends in `all`, and its header explains why in terms of the consumer: the tooling that reads SYNOPSIS is completion-generation machinery (`whatis`, `apropos`, fish's `create_manpage_completions`, zsh's `_man`), and that tooling targets the canonical page. A gate that demanded the same structure of both would have forced the cheatsheet into a shape no consumer wanted.

The combined man-page surface across the tools that have one:

Tool	<code><name>.1</code> + <code><name>all.1</code> , total lines
<code>nmaprs</code>	1,536
<code>lsofrs</code>	1,274
<code>temprs</code>	1,052
<code>iftoprs</code>	839
<code>htoprs</code>	776
<code>storageshower</code>	476
<code>grcrs</code>	273 (<code>grc.1</code> + <code>grcat.1</code> , one per binary)

`grcrs` is the exception in two ways: it ships one page per binary rather than a page plus a cheatsheet, and its pages live at `man/` rather than `man/man1/`. The consequence is mechanical and worth stating because it is the kind of thing a gate is supposed to catch and this one does not: the SYNOPSIS check finds man pages with `find ... -name '*.1' -path '*/man/man1/*'`, so `grcrs`'s pages are outside its search path entirely. They would not be checked even if the scripts were present in that repository — and they are not, because `grcrs`, `zcolorizer`, `zwire-host`, and `zpwrrchrome-host` carry no `tests/*.sh` at all.

That is the honest limit of this mechanism. Twenty-one byte-identical scripts enforcing a documentation floor is a real, working cross-repository convention, and it currently covers four of the ten Rust repositories in this book plus one repository that contains no Rust. It is a convention with a coverage gap, not a fleet-wide law, and the way to close the gap is to copy twenty-one files.

How they run in CI

The gates are not left to a developer's habit; the five repositories that have them wire every one into GitHub Actions, and the wiring is worth reading because it is a small piece of workflow design that gets something right. `lsofrs`'s `ci.yml` declares nine jobs — four about the code and five about the documentation:

```
docs-gates:
  name: Docs gates
  steps:
    - run: bash tests/docs-target-blank-rel-noopener.sh
    - run: bash tests/docs-no-http-links.sh
    - run: bash tests/docs-no-inline-handlers.sh
polish-gates:
  name: Polish gates
  steps:
    - run: bash tests/man-page-no-trailing-whitespace.sh
    - run: bash tests/man-page-final-newline.sh
    - run: bash tests/man-page-synopsis-section.sh
    - run: bash tests/tests-shell-executable.sh
    - run: bash tests/tests-shell-shebang.sh
```

with `semantic-gates`, `newline-gates`, and `structure-gates` covering the remaining thirteen. Twenty-one scripts, five jobs, grouped by *what a failure means* rather than by what the script scans: a docs-gates failure is a rendering or security problem, a polish-gates failure is a formatting problem, a newline-gates failure is a whitespace problem. Each `run` step is a separate line so the failing script is named in the job log without opening it, and each job is independent so one failing category does not hide the others behind a `fail-fast`.

The code jobs beside them are `fmt` (`cargo fmt --all --check`), `clippy` (`cargo clippy --all-targets -- -D warnings`), `doc` (`cargo doc --no-deps` with `RUSTDOCFLAGS: -D warnings`), and `test`, the last on a two-entry matrix of `ubuntu-latest` and `macos-latest`. All four are configured to fail on a warning rather than to report one — the pattern the whole fleet’s CI follows, and the reason the working trees are clean of the low-grade lint drift that otherwise accumulates in a project maintained by one party.

The flag surface, measured — what the ports advertise against what the classics do

The parser-compatibility chapter describes how each port accepts its upstream’s argument grammar. This chapter measures the result: how many options each program advertises, how the advertised sets compare, and — the more interesting question — how far the *accepted* surface extends past the advertised one. Every figure below comes from running `--help` on the installed binaries and on the classic tools present on the same machine, and extracting option tokens from the output.

The reference versions, read from the tools themselves:

```
$ lsof -v | head -2      revision: 4.91
$ nmap --version | head -1 Nmap version 7.99 ( https://nmap.org )
$ htop --version        htop 3.5.2
$ ccze --version        ccze 0.2.1
$ iftop -h | head -1    iftop: display bandwidth usage on an interface by host
```

Counting what is printed

Extracting `--long` and `-x` tokens from each program’s help output gives the following. Read these as *help-surface* counts — options the program advertises when asked — not as totals, because several of the classics document more in their man pages than in their usage text.

Program	help lines	long options	short options
<code>lsof</code> 4.91	33	0	25
<code>lsotrs</code> 4.9.3	87	22	23
<code>nmap</code> 7.99	115	52	—
<code>nmaprs</code> 0.1.10	146	108	—
<code>htop</code> 3.5.2	23	16	13
<code>htoprs</code> 0.5.12	57	16	13
<code>iftop</code>	24	0	14
<code>iftoprs</code> 2.22.10	49	16	14
<code>ccze</code> 0.2.1	18	11	11
<code>zcolorizer</code> 0.2.3	51	17	10
<code>temprs</code> 2.9.13	112	43	30

The short-option counts for `nmap` and `nmaprs` are omitted because both programs’ short options are two-character families (`-sS`, `-PE`, `-oX`) that a single-letter extractor mis-parses; they are counted separately below.

htoprs: the same sixteen, the same thirteen

The strongest single result in this chapter is a diff that prints nothing:

```
$ diff <(extract_long htop.help) <(extract_long htoprs.help); echo "rc=$?"
rc=0
$ diff <(extract_short htop.help) <(extract_short htoprs.help); echo "rc=$?"
rc=0
```

htoprs's advertised option set is identical to **htop** 3.5.2's — the same sixteen long options and the same thirteen short ones, no additions and no omissions. This is the flag-surface counterpart to the port-purity gate described in the methodology chapter: the gate constrains which functions may exist, and the option set shows the same discipline applied at the user-visible boundary. It is also the only tool in the fleet where the port adds nothing. **htoprs**'s help text is 57 lines against **htop**'s 23 — the difference is layout and chrome, not options.

lssofrs: a help surface smaller than the accepted surface

lssof advertises twenty-five single-letter options and no long ones; **lssofrs** advertises twenty-three short and twenty-two long. Comparing the short sets, five of **lssof**'s letters do not appear in **lssofrs --help** — **-b**, **-O**, **-S**, **-x**, **-X** — and three of **lssofrs**'s are new: **-h**, **-J**, **-W**.

The absence is not a gap. All five are accepted:

```
$ for opt in -b -O -X "-S 5" -x; do printf '%-5s => ' "$opt"; \
    lssofrs $opt -p $$ >/dev/null 2>&1; echo "rc=$?"; done
-b      => rc=0
-O      => rc=0
-X      => rc=0
-S 5    => rc=0
-x      => rc=0
$ lssofrs +M -p $$ >/dev/null 2>&1; echo "rc=$?"
rc=0
```

and an option that is genuinely unknown is not:

```
$ lssofrs -Q -p $$
error: unexpected argument '-Q' found
tip: to pass '-Q' as a value, use '-- -Q'
$ echo $?
2
$ lssofrs --nope -p $$
error: unexpected argument '--nope' found
tip: a similar argument exists: '--top'
```

The mechanism is in the argv normalizer, and it sorts **lssof**'s letters into categories by what they consume. **src/cli.rs:452** accepts and discards **b**, **O**, **C**, **M**, **X**, **E**, **L** and continues the cluster, because none takes an argument. **:456** handles **z**, **Z**, **S**, **f**, which may carry an attached value, by dropping the rest of the cluster. **:460** handles **A**, **k**, **m**, **D**, **e**, which require an argument, by consuming the following token as well. And three letters are not no-ops at all but rerouted: **:494** maps **-T** to **--tcp-info**, **:498** maps **-x** to **--cross-over**, **:502** maps **-K** to **--list-tasks**.

Three categories of accept-and-ignore, distinguished by argument arity, is the minimum structure this problem has. A single “ignore unknown letters” rule would have silently eaten the *argument* of **-k** as a target name, which is the compatibility bug that is hardest to notice because it produces plausible output.

The rule the fleet is following is stated plainly by the contrast between the two transcripts above: a flag from the upstream's grammar is accepted whether or not it does anything, and a flag from nobody's grammar is an error with a non-zero exit. Silence is reserved for compatibility; invention gets a diagnostic.

nmaprs: a superset of what nmap prints

Comparing long options between `nmap --help` and `nmaprs --help`:

```
$ comm -23 <(extract_long nmap.help) <(extract_long nmaprs.help) | wc -l
0
$ comm -12 <(extract_long nmap.help) <(extract_long nmaprs.help) | wc -l
52
```

Every long option `nmap 7.99` advertises appears in `nmaprs`'s help, and `nmaprs` advertises fifty-six more. The additions divide into three groups. Some are long spellings of `nmap` options that `nmap` itself only prints in short form — `--scan-type`, `--timing`, `--verbosity`, `--version-scan`, `--no-ping`, `--iL`, `--iR`, `--sL`, `--sN`, `--sO`, `--sI` — which exist because the port's normalizer expands the compact forms into these before `clap` sees them, and it is honest to document the expanded target. Some are fine-grained timing controls that `nmap` documents in its man page rather than its usage text: `--initial-rtt-timeout`, `--max-rtt-timeout`, `--max-scan-delay`, `--max-hostgroup`, `--max-parallelism`, `--defeat-rst-ratelimit`, `--defeat-icmp-ratelimit`. And the rest are the port's own: `--log-errors`, `--stats-every`, `--fuzzy`, `--unique`, `--allports`.

The scan-type and host-discovery families show the same pattern:

```
$ grep -oE '\-s[A-Z]' nmap.help | sort -u | tr '\n' ' '
-sC -sI -sL -sN -sO -sS -sU -sV -sY
$ grep -oE '\-s[A-Z]' nmaprs.help | sort -u | tr '\n' ' '
-sA -sC -sF -sI -sL -sM -sN -sO -sS -sT -sU -sV -sW -sX -sY -sZ
$ grep -oE '\-P[A-Zn0]' nmap.help | sort -u | tr '\n' ' '
-PE -Pn -PO -PS
$ grep -oE '\-P[A-Zn0]' nmaprs.help | sort -u | tr '\n' ' '
-PA -PE -PM -Pn -PO -PP -PS -PU -PY
```

Sixteen scan types advertised against nine, nine discovery probes against four. Both differences are documentation, not capability — `nmap` supports the ACK, FIN, Xmas, Maimon, and Window scans it does not list in its usage summary — and that is the right way to read the comparison. What the numbers show is a different philosophy about what `--help` is for: `nmap`'s usage text is a curated introduction with the man page behind it, and `nmaprs`'s is the complete surface. Neither is wrong, and a reader comparing the two counts without knowing this would conclude something false.

iftoprs and zcolorizer: the same letters, plus long spellings

`iftop` advertises fourteen short options and no long ones. `iftoprs` advertises fourteen short and sixteen long — but the sets are not the same fourteen. Two of `iftop`'s letters (`-G`, `-m`) are not in `iftoprs`'s help, and two of `iftoprs`'s are new: `-V` for version and `-Z` to hide the owning-process column, a column `iftop` does not have. Fourteen of `iftoprs`'s sixteen long options are paired with a short letter and two are long-only (`--json`, `--list-colors`), which is the `clap`-derived convention and the reason its long count is sixteen while `iftop`'s is zero.

`zcolorizer` against `ccze` is the loosest pairing in the book and its numbers say so: eleven long options against seventeen, eleven short against ten, and almost no overlap in what they name. `ccze`'s options are about plugins, output modes, and HTML generation; `zcolorizer`'s are about themes, modules, config merging, and two output filters (`--grep`, `--highlight`) with no upstream counterpart. The parity chapter already states that byte-parity is not the goal for most of `zcolorizer`'s surface, and the option sets are the cleanest evidence: these are two programs that colour logs, not one program and its port.

temprs: the largest surface, and no upstream at all

The widest flag surface in the fleet belongs to the tool with nothing to be compatible with. `temprs --help` advertises 43 long and 30 short options over 112 lines — every operation with both spellings, from `-i/--input` and `-o/--output` through `-D/--diff`, `-M/--mv`, `-x/--dup`, `-S/--swap`, and `-R/--rename`. That is more advertised options than `lsotrs` and `htotrs` combined.

The observation worth extracting is that surface size tracks *whether the program is a port*, and it tracks it in the opposite direction from the intuition. `htotrs` matches `htop` exactly and adds nothing; `lsotrs` and `iftotrs` add a long-option layer and a handful of modes; `nmaprs` documents more than `nmap` prints but invents little; and the tool with no upstream has the largest surface of all. A port's option set is a constraint it inherited. An original's option set is a decision it has to keep making, and the discipline that keeps a ported tool's surface honest — the upstream said no — does not exist for the originals.

From inventory to result — the fleet's suites, actually run

Every test count in this book so far has been an inventory: a count of `#[test]` attributes found by `grep`, which the methodology chapter is careful to describe as a count of intentions. It also names the number that would settle the matter — the `test result:` line of a `cargo test --locked` run — and states that the book does not have it. This chapter has it for seven of the ten Rust crates, and the gap between the two kinds of number turns out to be more interesting than either.

The results

Seven crates were built and run with `cargo test --locked` on macOS arm64. Every line below is the sum of the `test result:` lines from one such run:

Crate	executed	failed	ignored
<code>temprs</code>	5,650	0	0
<code>iftoprs</code>	3,991	0	0
<code>lsofrs</code>	3,325	0	0
<code>nmaprs</code>	1,467	0	0
<code>htoprs</code>	1,374	0	0
<code>zcolorizer</code>	112	0	0
<code>grcrs</code>	71	0	0
total	15,990	0	0

Two smaller results are worth printing in full because they are the two cases where the book's inventory figure and the executed figure agree exactly:

```

$ cd grcrs 66 cargo test --locked
  Running unittests src/grcrs.rs
test result: ok. 17 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in
  0.01s
  Running unittests src/grcatrs.rs
test result: ok. 27 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in
  0.01s
  Running tests/cli.rs
test result: ok. 27 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in
  1.06s

$ cd zcolorizer 66 cargo test --locked
  Running unittests src/lib.rs
test result: ok. 94 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in
  3.98s
  Running tests/integration.rs
test result: ok. 17 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in
  1.91s
  Doc-tests zcolorizer
test result: ok. 1 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in
  1.75s

```

`grcrs`'s "71 total: 17 launcher + 27 colouriser + 27 end-to-end" and `zcolorizer`'s "94 unit + 17 integration + 1 doc = 112" are confirmed exactly, and now confirmed as *passing* rather than merely present.

Where inventory and result diverge, and why

For the larger crates the two numbers do not match, and they diverge in both directions. Both divergences have mechanical causes worth knowing, because a reader who assumes a mismatch means a broken count will reach the wrong conclusion.

Executed can be lower than the inventory, because of `cfg`. `htoprs` has 1,394 `#[test]` attributes under `src/` and ran 1,356 unit tests. The missing thirty-eight are gated to platforms this machine is not: the port carries backend trees for Linux, FreeBSD, NetBSD, OpenBSD, DragonFly, Solaris, and PCP alongside Darwin, and their tests compile only on their platform. `nmaprs` shows the same effect at a scale of one — 1,199 attributes, 1,198 executed.

Executed can be much higher, because a module can be compiled into several test binaries. `lsofrs` has 1,029 `#[test]` attributes under `src/` and its run reports three unit-test binaries:

```

warning: file `src/main.rs` found to be present in multiple build targets:
* `bin` target `lsf`
* `bin` target `lsofrs`
  Running unittests src/lib.rs      test result: ok. 687 passed
  Running unittests src/main.rs    test result: ok. 964 passed
  Running unittests src/main.rs    test result: ok. 964 passed

```

2,615 unit-test executions from 1,029 attributes. The `lsf` alias is declared as a second `[[bin]]` over the same `src/main.rs`, so every test in the binary's module tree runs twice, and the modules shared between the library and the binary are compiled into both. `iftoprs` shows the same shape without the duplicate binary: 1,741 attributes, 1,715 executed in the lib target and 1,731 in the bin target, 3,446 executions total.

Integration tests do not have this problem, and the numbers show it. In all five crates with a `tests/` directory, the `grep` count and the executed count agree to the unit: `lsofrs` 710, `iftoprs` 545, `temprs` 585, `nmaprs` 269, `htoprs` 18. An integration test file is its own crate compiled once, so counting attributes in it counts executions.

The rule this yields is short. An attribute count over `tests/` is a reliable proxy for executions. An attribute count over `src/` is not a proxy for anything — it can be an overcount because of `cfg` or an undercount because of

multiple targets, and which one it is depends on the crate's `[[bin]]` layout.

What the suites do not skip

One number is worth calling out on its own because it contradicts the pessimistic reading the methodology chapter allows for. Across all ten Rust crates there is exactly one `#[ignore]`:

```
$ for d in lsofrs nmaprs temprs iftoprs htoprs storageshower zcolorizer grcrs \
    zwire-host zpwichrome/zpwichrome-host; do
    printf '%-17s %s\n' "${basename $d}" \
        "$(grep -rho '#\[ignore\]' $d/src $d/tests 2>/dev/null | wc -l)"
done
lsofrs          0
nmaprs          0
temprs          0
iftoprs         0
htoprs          1
storageshower   0
zcolorizer      0
grcrs           0
zwire-host      0
zpwichrome-host 0
```

and every `test result:` line above reports `0 ignored`. The concern that a grep-counted inventory silently includes disabled tests is, for this fleet, unfounded — there is one disabled test in roughly sixteen thousand. `#[should_panic]` is nearly as rare: three in `nmaprs`, eleven in `htoprs`, zero everywhere else.

The skip that a green line cannot show

There is a real version of the concern, and it is not `#[ignore]`. The parity chapter notes that the oracle suites skip gracefully when their reference is absent, and that a suite going green by skipping can silently stop testing anything. That is demonstrable:

```
$ cd htoprs 66 cargo test --locked --test parity
test result: ok. 18 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out

$ HTOP_REF=/nonexistent/htop cargo test --locked --test parity
test result: ok. 18 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out
```

The same eighteen tests, the same green line, with and without a reference `htop` to compare against. The harness returns the reference path “only if it exists AND is the ported 3.5 series; otherwise `None` so callers skip the comparison” — which is the right behaviour for CI, and which means the `test result:` line cannot distinguish “diffed against `htop` 3.5.2 and matched” from “found no `htop` and asserted nothing”. On this machine `htop` 3.5.2 is installed, so the first run did the real comparison; a reader with a different machine cannot tell from the output which run they got. An oracle suite needs a second signal — a printed count of comparisons performed, in the manner of the hygiene gates’ `Summary:` lines — and none of the parity suites emits one.

Re-reading the port gate’s own numbers

The methodology chapter describes `htoprs`’s build-time port-purity gate and the figures around it. Those figures are the ones most worth re-reading from source, because they are the fleet’s only *build-blocking* correctness claim and because an audit number that nobody re-derives is an audit number waiting to go stale. Reading them again:

```
$ wc -l tests/data/htop_c_fn_names.txt tests/data/fake_fn_allowlist.txt
4137 tests/data/htop_c_fn_names.txt
  10 tests/data/fake_fn_allowlist.txt
$ grep -rho '/// Port of' src | wc -l
1937
$ find src/ported -name '*.rs' | wc -l ; find src/extensions -name '*.rs' | wc -l
141
  22
```

4,137 C function names in the snapshot the gate checks against, 1,937 `/// Port of` citations in the Rust, 141 ported modules beside 22 extension modules. The allowlist file is ten lines and the ten lines are its own header:

```
# Architectural Rust-only helpers with no htop C counterpart.
...
# Empty for now: every ported fn maps 1:1 to an htop C name.
```

Zero exemptions, which is the claim the methodology chapter makes and which is worth confirming by reading the file rather than trusting the sentence, because an allowlist is precisely the artifact that grows quietly.

The stub count is the figure where a naive re-measurement disagrees with the book, and the disagreement is instructive rather than substantive. A plain grep finds fifty-seven mentions:

```
$ grep -rho 'todo!()\|unimplemented!()' src | wc -l
57
```

but filtering out lines whose text begins a comment leaves none of them — every one of the fifty-seven is inside a doc comment explaining the port's state, not inside a function body. The real bodies use the message form, and there are fifteen:

```
$ grep -rn 'todo!' src | grep -v ':\s*//' | wc -l
15
src/ported/processtable.rs:212 todo!("platform ProcessTable_goThroughEntries - /proc
  ~ scan, filled by linux/ layer")
src/ported/vector.rs:549      todo!("port of Vector.c:329 - commented-out (dead) code in
  ~ the C source")
src/ported/affinitypanel.rs:405 todo!("port of AffinityPanel.c:94 - hwloc-only (no
  ~ libhwloc in htoprs)")
...
```

By reason: five in `affinitypanel.rs` are `hwloc`-only entry points and the port does not link `libhwloc`; eight are in the `dragonflybsd`, `freebsd`, `netbsd`, and `openbsd` platform layers, none of which is the platform this reading was taken on; one is a `Vector.c` function that is commented out in the C source, so there is no behaviour to port; and one is the platform-dispatch hook in `processtable.rs` that each OS layer fills in.

Every one of the fifteen carries its C origin and a stated reason in the panic message, which is the property that makes the number usable. A `todo!()` with no message is a gap of unknown size; `todo!("port of AffinityPanel.c:94 - hwloc-only (no libhwloc in htoprs)")` is a scoping decision recorded at the site where it applies. The lesson for anyone re-deriving a coverage figure is that the grep has to match what the project's own generator matches — the report generator counts `todo!()` and `unimplemented!()` *bodies* and scores them as stubbed, and a grep over raw text counts documentation about them too. Both numbers are real; only one of them answers the question.

The one that would not finish

`storageshower` is missing from the table, and the reason is a measurement in its own right. Its integration tests are not a handful of files:

```
$ find storageshower/tests -name '*.rs' | wc -l
1336
```

1,336 separate integration test files, which is 1,336 separate crates, each compiled and *linked* into its own executable. The naming shows the generation pattern — 514 begin `cli_`, 296 `scan_`, 132 `ui_`, 129 `helpers_`, 106 `prefs_`, 95 `dedup_` — and the total is 49,990 lines carrying 3,101 test attributes. A `cargo test --locked` run on this crate spent 91 seconds of wall time, produced 2.2 GB in `target/`, and terminated during linking with `No space left on device`. The suite's contents are not in question; its packaging is. The same 3,101 tests distributed over a dozen files would compile into a dozen binaries instead of 1,336 and would link in a fraction of the time and disk.

That is the sharpest illustration available of a point the methodology chapter makes abstractly: a test count is not a measure of a suite. Two suites in this book have roughly comparable inventories — `storageshower` at 3,697 attributes and `iftoprs` at 2,286 — and one of them runs in four seconds while the other cannot be run on a machine without several gigabytes free.

What ships — release profiles, binary size, and the linkage the ports do not have

The packaging chapter covers how the fleet gets onto a machine. This chapter is about what arrives when it does: the size of each binary, the release profile that produced it, and the list of shared libraries it needs at run time. Those three are the whole of what a user receives, and the third one — linkage — is where the ports differ from the programs they replace by the largest margin.

Every figure here was read from the installed artifacts on one machine (macOS arm64, Homebrew prefix `/opt/homebrew`), with the source manifests read from the working trees.

The versions that are actually installed

Before sizes, a reconciliation. The book's ledger reads versions from `Cargo.toml`; a user's machine has whatever was last installed. Reading both:

```
$ brew list --versions lsofrs nmaprs temprs iftoprs htoprs storageshower \
  zcolorizer grcrs
grcrs 1.13.1      htoprs 0.5.12      iftoprs 2.22.10    lsofrs 4.9.3
nmaprs 0.1.10    storageshower 0.28.9 temprs 2.9.13      zcolorizer 0.2.3
```

Against the manifests in the working trees at the time of this reading — `lsofrs` 4.9.2, `nmaprs` 0.1.10, `temprs` 2.9.13, `iftoprs` 2.22.10, `htoprs` 0.5.12, `storageshower` 0.28.9, `zcolorizer` 0.2.3, `grcrs` 1.13.1, `zwire-host` 0.3.15, `zpwchrome-host` 0.10.3 — seven of the eight installed tools match their source exactly and one does not: the installed `lsofrs` reports 4.9.3 where the checked-out manifest reads 4.9.2. Both readings are correct measurements of different artifacts, which is the entire reason the book's ledger names its source for every number. A version is a property of a build, and there are always at least two builds in play: the one in the tree and the one on the `PATH`.

Every version in this chapter's tables also sits one patch release above the ledger in Appendix C — `lsofrs` 4.9.1 → 4.9.2, `nmaprs` 0.1.9 → 0.1.10, `temprs` 2.9.12 → 2.9.13, `iftoprs` 2.22.9 → 2.22.10, `htoprs` 0.5.7 → 0.5.12, `storageshower` 0.28.8 → 0.28.9, `zcolorizer` 0.2.2 → 0.2.3, `grcrs` 1.13.0 → 1.13.1, `zwire-host` 0.3.14 → 0.3.15, `zpwchrome-host` 0.10.2 → 0.10.3. Ten out of ten moved, most by a single patch and `htoprs` by five. That is the drift the appendix predicts, observed.

Ten release profiles, no two identical

The `[profile.release]` section is where a crate states what it wants from the compiler, and reading all ten shows a convention with real variation:

Crate	<code>opt-level</code>	<code>lto</code>	<code>codegen-units</code>	<code>panic</code>	<code>strip</code>
<code>lsofrs</code>	3	true	1	—	true
<code>nmaprs</code>	—	true	1	abort	true

Crate	opt-level	lto	codegen-units	panic	strip
temprs	3	true	1	abort	true
iftoprs	3	true	—	—	true
htoprs	—	—	—	—	—
storageshower	3	true	—	—	true
zcolorizer	—	true	—	—	true
grcrs	—	true	1	—	—
zwire-host	z	true	1	abort	true
zpwchrome-host	—	true	1	—	true

Nine of the ten set `lto = true`; eight set `strip = true`; six set `codegen-units = 1`. Those are the shared convention. The departures each mean something. `htoprs` has no `[profile]` section at all, so it ships at Cargo's defaults — the only tool in the book that does. `zwire-host` sets `opt-level = "z"`, optimising for size rather than speed, which is the correct choice for a process whose job is to relay messages and whose cost is dominated by being resident rather than by throughput. `grcrs` sets `lto` and `codegen-units` but not `strip`, so its binaries carry symbols the others do not. And three crates set `panic = "abort"` — `nmaprs`, `temprs`, and `zwire-host` — which the `unsafe-` surface chapter already identifies as the one profile setting with observable semantics.

`opt-level = 3` appears explicitly in four manifests, which is worth noting only because it is Cargo's release default: those four are stating an intent rather than changing a behaviour.

What the binaries weigh

The installed release binaries, and the classic tools they stand next to:

Port	bytes	Classic	bytes	ratio
lsofrs	3,020,264	lsf 4.91	307,600	9.8×
nmaprs	4,695,112	nmap 7.99	3,086,656	1.5×
htoprs	2,762,488	htop 3.5.2	268,032	10.3×
iftoprs	2,857,928	iftop	116,386	24.6×
zcolorizer	2,371,896	ccze 0.2.1	113,360	20.9×
storageshower	1,566,960	—	—	—
temprs	735,432	—	—	—
grc	1,678,048	—	—	—
grcat	1,777,208	—	—	—

The ports are larger, by between 1.5× and 25×, and the spread is the informative part. It is inversely related to how much of the classic's work lives in shared libraries: `nmap` is only 1.5× smaller than `nmaprs` because `nmap` is a large program in its own right, while `iftop` is 25× smaller than `iftoprs` because `iftop` is a thin program over `libpcap` and `ncurses` and `iftoprs` carries its TUI stack inside the executable.

`temprs` is the smallest at 735 KB and the reason is its dependency list: five direct crates, no TUI, no regex engine. It is the clearest demonstration in the fleet that Rust binary size is a function of what was linked in, not of the language.

The two-binary crates ship genuinely separate executables rather than links:

```
$ readlink $(command -v lsf); stat -L -f%z $(command -v lsf)
../Cellar/lsofrs/4.9.3/bin/lsf
3020256
$ stat -L -f%z $(command -v lsofrs)
3020264
```

Eight bytes apart — two compilations of the same source differing only in the embedded program name. `nms` against `nmaprs` and `tp` against `tempr`s show the same eight-byte difference. A hard link or a symlink with `argv[0]` dispatch would have saved a few megabytes across the fleet; the cost of not doing it is paid once at install and the benefit is that neither binary can be broken by someone moving the other.

Debug builds, for scale, from the two working trees that carried one:

```
$ stat -f'%z %N' nmaprs/target/debug/nmaprs htoprs/target/debug/htoprs
40260952 nmaprs/target/debug/nmaprs
11563368 htoprs/target/debug/htoprs
```

8.6× and 4.2× their release counterparts. The gap is debug info and the absence of LTO, and it is the practical reason the release profile matters at all for tools of this size.

The linkage, which is where the ports actually differ

Size is the visible difference and it is the less important one. Listing shared library dependencies of each binary:

```
$ otool -L $(command -v nmap) | tail -n +2 | awk '{print $1}'
/opt/homebrew/opt/pcr2/lib/libpcr2-8.0.dylib
/opt/homebrew/opt/libssh2/lib/libssh2.1.dylib
/opt/homebrew/opt/openssl@3/lib/libssl.3.dylib
/opt/homebrew/opt/openssl@3/lib/libcrypto.3.dylib
/usr/lib/libz.1.dylib
/opt/homebrew/opt/liblinear/lib/liblinear.6.dylib
/opt/homebrew/opt/lua/lib/liblua.5.5.dylib
/usr/lib/libSystem.B.dylib
/usr/lib/libc++.1.dylib

$ otool -L $(command -v nmaprs) | tail -n +2 | awk '{print $1}'
/usr/lib/libc++.1.dylib
/usr/lib/libiconv.2.dylib
/usr/lib/libSystem.B.dylib
```

`nmap` needs six libraries from a package manager’s prefix — a regex engine, an SSH library, two halves of OpenSSL, a machine-learning library, and a Lua interpreter. `nmaprs` needs three, all of them shipped with the operating system. Upgrading Lua or OpenSSL on this machine can break `nmap` and cannot break `nmaprs`, and that is a durability property rather than a performance one.

The full comparison:

Binary	shared libraries	any outside the OS?
lsuf	1	no
nmap	9	yes — 6 from the Homebrew prefix
htop	4	yes
iftop	3	yes
ccze	3	yes
lsufrs	3	no
nmaprs	3	no
htoprs	4	no
iftoprs	5	no
storageshower	5	no
zcolorizer	2	no
tempr	2	no
grc / grcat	2	no

Not one of the ten port binaries links anything outside `/usr/lib` and `/System/Library`. The counts are not smaller across the board — `iftoprs` and `storageshower` link five each against `iftop`'s three — but *what* they link is categorically different. `iftoprs`'s five are `Security.framework`, `CoreFoundation`, `libpcap`, `libiconv`, and `libSystem`; `htoprs`'s four are `IOKit`, `CoreFoundation`, `libiconv`, and `libSystem`. These are the operating system's own interfaces, present on every machine of that platform by definition, and the packaging chapter's claim that there is no `-dev` package to install is exactly this list.

`libpcap` deserves a note as the one genuine third-party dependency in the set, and it is genuine: `iftoprs` cannot capture packets without it. On macOS it ships with the system, which is why it appears as `/usr/lib/libpcap.A.dylib` rather than a prefix path; on a Linux build it is the one library that has to come from somewhere.

What the crates carry that is not code

Three of the ports vendor their reference material into the repository, and the sizes are small enough to be worth stating because “vendored upstream” can mean anything from a header to a kernel:

```
$ find grcrs/vendor -type f | wc -l ; du -sh grcrs/vendor
131
564K    grcrs/vendor
$ ls grcrs/vendor/grc/colourfiles | wc -l
83
$ find zcolorizer/vendor -type f | wc -l ; find zcolorizer/vendor -name '*.c' | wc -l
72
27
$ wc -l nmaprs/c/expr_match.cpp nmaprs/data/*.txt
138 nmaprs/c/expr_match.cpp
151 nmaprs/data/nmap_ip_protocols_fast.txt
1000 nmaprs/data/top_ports.txt
```

`grcrs` vendors the upstream `grc` distribution including its 83 colour-definition files, which are not reference material but the same kind of file the shipped program reads at run time — they are data, not a specification. `zcolorizer` vendors 27 C files of `ccze` as a read-only specification the Rust never calls. And `nmaprs` carries the one piece of foreign code that is compiled into the build — 138 lines of C++ — plus 1,151 lines of port and protocol tables. Three different relationships to an upstream, distinguishable by asking whether the vendored bytes are read at build time, at run time, or never.

The `grcrs` case has a consequence a user can observe. Because the colour files are data rather than code, the port and the upstream produce identical output from the same file, and the launcher adds nothing to what the colouriser does:

```
$ df -h / | head -2 | grcat vendor/grc/colourfiles/conf.df | cat -v | head -2
^[0mFilesystem      Size   Used  Avail Capacity iused ifree %iused  Mounted on^[0m
^[0m^[34m/dev^[0m^[1m^[34m/disk3s1s1^[0m^[34m ^^[0m ^^[0m^[31m 926Gi ^^[0m ...

$ grc --colour=on df -h / | cat -v | head -2
^[0mFilesystem      Size   Used  Avail Capacity iused ifree %iused  Mounted on^[0m
^[0m^[34m/dev^[0m^[1m^[34m/disk3s1s1^[0m^[34m ^^[0m ^^[0m ^^[0m^[31m 926Gi ^^[0m ...

$ diff <(df -h / | head -2 | grcat vendor/grc/colourfiles/conf.df) \
    <(grc --colour=on df -h / | head -2) && echo BYTE_IDENTICAL
BYTE_IDENTICAL
```

Byte-identical output from the two paths, which is what “the launcher pipes the command through the colouriser” means when it is true rather than asserted.

The build cost, for the one crate small enough to state honestly

One timing is worth publishing because it is the floor rather than an average. Building and running `grcrs`'s complete suite from an empty `target/`, with its nine-package dependency closure:

```
$ cd grcrs && time cargo test --locked
...
test result: ok. 27 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in
  ~ 1.71s
cargo test --locked 11.51s user 1.96s system 40% cpu 33.114 total
```

Thirty-three seconds from nothing to a green suite, of which under two seconds is the tests. That number is not representative of the fleet — it is the smallest crate in it — but it is the useful anchor for the ones that are not published here, because it establishes that the compile time in a Rust CLI project of this kind is a function of the dependency closure and very little else. Nine packages is thirty-three seconds; the crates at 242 and 273 packages are the ones where a cold build is worth planning around, and the crate at 1,336 integration binaries is the one where it is worth restructuring.

The repository shape – what a fleet repository contains besides its source

A tool is not a `src/` directory. It is a `src/` directory, a completion file, a man page or two, a rendered documentation page, a README, a licence, a CI workflow, and a commit history — and the ratio between those is a fair description of how a project is maintained. This chapter measures the non-source contents of every repository in the book, because the per-tool chapters cover what each program does and none of them covers what each repository *is*.

The inventory

Reading each repository for source size, completions, man pages, rendered docs, README length, workflow count, and commit count:

Repository	<code>src/</code> lines	files	comple- tions	man pages	<code>*.html</code> in <code>docs/</code>	README lines	workflows	commits
<code>htoprs</code>	106,964	165	1	2	3	271	2	270
<code>temprs</code>	55,655	11	1	2	2	457	2	306
<code>lsofrs</code>	25,296	29	1	2	2	521	2	285
<code>nmaprs</code>	24,074	33	1	2	2	313	2	150
<code>iftoprs</code>	22,913	21	1	2	2	456	2	257
<code>storageshower</code>	14,060	13	1	2	2	601	2	260
<code>zwire-host</code>	6,518	25	0	0	2	318	2	64
<code>zpwchrome-host</code>	6,343	30	0	0	2	212	2	309
<code>zcolorizer</code>	4,793	11	0	0	2	186	2	21
<code>grcrs</code>	1,686	2	0	2	2	254	2	22
<code>revolver</code>	318	1	1	0	0	69	1	37
<code>fasd-simple</code>	620	1	0	1	0	378	1	317
<code>gh_reveal</code>	85	1	1	0	0	125	1	119
<code>kubectl-aliases</code>	804	1	0	0	0	109	1	58

The commit counts are the count of commits reachable from the checked-out head (`git log --oneline |`

`wc -l`) and are the least comparable column, because four of these repositories began as forks of an existing project and carry its history. `head -1 LICENSE` returns `MIT License` in all ten Rust repositories; three of the shell-side four carry the upstream project's licence file and `gh_reveal` carries none at all, which is the one licensing gap the inventory turns up.

Two shapes stand out. `htoprs` at 106,964 lines across 165 files is four times the next-largest source tree and the reason is structural rather than featural: the port mirrors `htop`'s file layout one Rust module per C file, including seven per-OS backend trees the machine running it will never compile. And `temprs` is 55,655 lines across *eleven* files, an average of 5,060 lines per file, which is the signature of a generated test battery living inside a source module rather than of large hand-written files.

At the other end, `gh_reveal` is 85 lines and `revolver` is 318 — and both ship a completion, a README, a CI workflow, and a test suite. The overhead of being a maintained project rather than a script in a dotfile is roughly constant, and for the smallest tools it exceeds the tool.

Six completions, one convention

Six of the Rust tools ship a zsh completion; two of the shell tools do:

```
$ wc -l */completions/* revolver/src/_revolver gh_reveal/_reveal
118 lsofrs/completions/_lsofrs
168 nmaprs/completions/_nmaprs
 99 temprs/completions/_temprs
 62 iftoprs/completions/_iftoprs
 48 htoprs/completions/_htoprs
 56 storageshower/completions/_storageshower
 34 revolver/src/_revolver
 33 gh_reveal/_reveal
```

Every one begins with a `#compdef` line, and three of them name two commands:

```
$ head -1 lsofrs/completions/_lsofrs      #compdef lsofrs lsf
$ head -1 nmaprs/completions/_nmaprs     #compdef nmaprs nms
$ head -1 temprs/completions/_temprs     #compdef tp temprs
```

That is the correct handling of the alias-binary pattern described in the packaging chapter: two binaries, one completion, declared once. A separate completion file per binary would have been the obvious mistake and would have drifted the first time a flag was added.

The completions are `_arguments`-based rather than hand-rolled, and they lean on zsh's own completion library where a value has a known type. `lsofrs`'s completion passes `_sequence -s`, `_pids` for `-p`, `_sequence -s`, `_pgids` for `-g`, and `_users -S`, for `-u`, so completing a PID list offers the live process table with comma handling already correct. Where the value type is specific to the tool, it uses the `->state` mechanism and resolves it in a `case` below: `->filedes`, `->addresses`, `->fields`, `->states`.

`temprs`'s completion is the one that reads live state. It resolves the stack directory the same way the program does —

```
stack_dir="${TEMPRS_DIR:-${TMPDIR:-/tmp}/temprs}"
master_file="${stack_dir}/temprs-stack"
```

— and offers the indices and tags currently on the stack. That is the property the worked-sessions chapter mentions in passing, and it is worth naming as a design rule rather than a detail: a completion for a stateful tool should read the same state the tool reads, resolved the same way, or it will disagree with reality in exactly the situations where the user most needs it to agree.

The two shell-side completions are smaller and do one thing each. `_revolver` is 34 lines completing the four subcommands and the spinner-style list. `_reveal` is 33 lines that read `git remote -v` into a name-to-URL map and offer each remote plus an `All:` entry, and — the part worth copying — it declines rather than returning nothing when the question does not apply:

```
if ! command git rev-parse --git-dir 2> /dev/null 1>&2; then
  _message "$PWD is not a git dir"
  return 1
fi
```

An empty completion list and a completion list that cannot exist look identical at the prompt unless the completion says so.

Two workflows per Rust repository, one per shell repository

Every Rust repository carries exactly two workflow files, `ci.yml` and `release.yml`; every shell repository carries one, `ci.yml`. The job counts inside them vary considerably:

Repository	CI jobs	job names
<code>tempris</code>	11	check, test, fmt, clippy, doc, release-build, + 5 gate jobs
<code>lsofrs</code> , <code>nmaprs</code> , <code>iftoprs</code>	9	fmt, clippy, doc, test, + 5 gate jobs
<code>zpwchrome</code>	6	test, + 5 gate jobs
<code>storageshower</code>	5	check, test, fmt, clippy, doc
<code>grcrs</code>	4	fmt, clippy, doc, test
<code>htoprs</code>	3	clippy, doc, test
<code>zcolorizer</code>	2	lint, test
<code>zwire-host</code>	1	build-test

The five gate jobs are the documentation suite described in the previous chapter, and their presence tracks the presence of the `tests/*.sh` scripts exactly. The variation in the rest is real variation in rigour: `tempris` builds a release profile in CI and the others do not; `htoprs` has no `fmt` job; `zwire-host` collapses everything into one.

Two things are uniform. Every Rust repository runs `cargo clippy --all-targets -- -D warnings` — all nine, verified by reading the step out of each `ci.yml` — which means a new warning fails a build rather than accumulating. And every one of the nine runs its tests on a platform matrix with `fail-fast` disabled:

```
strategy:
  fail-fast: false
  matrix:
    os: [ubuntu-latest, macos-latest]
```

Eight use exactly that pair; `zwire-host` adds `windows-latest`, which is the manifest-level Windows dependency showing up again at the other end of the project.

`fail-fast: false` is the detail that matters for a fleet of platform-sensitive tools. With it, a Linux failure and a macOS failure in the same run are both reported; without it, the first one cancels the other and a two-platform bug looks like a one-platform bug for as many iterations as it takes to fix the first.

The shell repositories' single workflow is the same shape at a smaller scale: install `zsh`, clone and build the test runner from source, set `TERM`, `COLUMNS`, and `LINES` explicitly, and run `zunit --verbose tests/*.zsh`. The explicit terminal environment is not decoration — a runner that formats output to terminal width behaves

differently on a runner with no terminal, and pinning the three variables is what makes a local run and a CI run comparable.

`revolver`'s workflow has one additional step that is a genuine curiosity: the test runner requires the `revolver` binary as a hard dependency, and the repository being tested *is* `revolver`, so the workflow puts the repository's own `bin/` on `PATH` before running the suite. A project whose test runner depends on the project has to bootstrap itself, and doing it by `PATH` rather than by installing a released copy means the suite runs against the working tree rather than against the last release — which is the only version of that arrangement that tests anything.

What the shape says

Three observations fall out of the table that no per-tool chapter states.

The documentation surface is roughly constant across the Rust tools and independent of their size. `gicris` at 1,686 source lines ships 273 lines of man page, two rendered HTML pages, and a 254-line README; `htoprs` at 106,964 source lines ships 776 lines of man page, three HTML pages, and a 271-line README. The tool that is sixty times larger has a shorter README. Documentation here is sized to the *interface*, not to the implementation, which is the correct proportion and an unusual one.

The completion surface tracks flag count, as it should. `nmaprs` has the largest advertised flag surface and the longest completion at 168 lines; `htoprs` matches `htop`'s sixteen-and-thirteen exactly and has the shortest of the six at 48.

And the three Rust tools with no man page — `zcolorizer`, `zwire-host`, and `zpwchrome-host` — divide into two kinds. The two native hosts are not invoked by a human, so a man page would have no reader; `zcolorizer` is invoked by a human and does not have one, which is the single clearest documentation gap the measurements in this chapter expose. The `man-page-synopsis-section.sh` gate cannot catch it, because a repository with no man pages passes every man-page gate trivially: `0 man pages checked, 0 without .SH SYNOPSIS` is a pass. A floor that checks the quality of what exists cannot require that anything exist, and closing that gap needs a different kind of check — one that asserts a man page is present for every declared `[[bin]]`.

The TUI layer, compared — four full-screen programs and the widget library none of them use

Four of the tools in this book paint a full screen: `lsdifrs` in five of its modes, `iftoprs` always, `storageshower` always, and `htoprs` for its extension overlays. All four declare the same two dependencies, `ratatui 0.30` and `crossterm 0.29`, and every one of the four READMEs describes a “ratatui TUI.” Read across the grain, though, the four are not four instances of one pattern. They agree on the crate and disagree on almost everything the crate is usually brought in to provide. This chapter measures the disagreement, because “built on ratatui” turns out to say much less about a program in this fleet than it sounds like it should.

The dependency they share

The manifests line up exactly:

```
$ grep -h '^ratatui\|^crossterm' {lsdifrs,iftoprs,htoprs,storageshower}/Cargo.toml
lsdifrs      crossterm = "0.29"    ratatui = "0.30"
iftoprs      crossterm = "0.29.0" ratatui = "0.30.0"
htoprs       crossterm = "0.29.0" ratatui = { version = "0.30.0", default-features =
  ~ false, features = ["std"] }
storageshower ratatui = "0.30"    crossterm = "0.29"
```

`htoprs` is the one that reads differently, and its manifest explains itself in a comment: ratatui is “Extensions-only,” pulled with `default-features = false` so that it brings no terminal backend and therefore no second copy of `crossterm`, because `htoprs` drives the terminal through its own `crossterm` directly. That is already a signal — a tool whose main interface is a full-screen process viewer has arranged for the terminal-UI crate not to touch the terminal.

The widgets nobody imports

The measurement that reframes the whole comparison is a single `grep`:

```
$ grep -rc 'ratatui::widgets' --include='*.rs' {lsdifrs,iftoprs,storageshower,htoprs}/src
0 files in all four
```

Not one of the four references ratatui’s widget module. There is no `Block`, no `Paragraph`, no `Table`, no `Gauge`, no `List`, no `Tabs`, no `Sparkline` constructed from the library anywhere in the four source trees — and three of the four draw sparklines, tables, bordered panels and tab bars on screen. Those are all hand-drawn.

What the four actually import is a much narrower slice:

Tool	ratatui items imported
<code>lsotprs</code>	<code>Terminal</code> , <code>backend::CrosstermBackend</code> , <code>buffer::Buffer</code> , <code>layout::Rect</code> , <code>style::{Color, Modifier, Style}</code> the same five plus <code>Frame</code> and <code>layout::{Constraint, Direction, Layout}</code>
<code>storgeshower</code>	<code>DefaultTerminal</code> , <code>Frame</code> , <code>buffer::Buffer</code> , <code>style::{Color, Modifier, Style}</code> , <code>layout::Rect</code>
<code>htotprs</code>	<code>buffer::Buffer</code> , <code>layout::Rect</code> , <code>style::{Color, Modifier, Style}</code> — and nothing else

Two of them do not even import a layout solver. `lsotprs` and `storgeshower` never mention `Constraint` or `Layout`; they compute their own rectangles. `iftotprs` is the only one of the four that uses ratatui's constraint solver at all.

The drawing primitive, in all four, is direct cell indexing into the buffer:

```
$ grep -rhoE 'buf\[\' --include='*.rs' <tool>/src | wc -l
lsotprs 16    storgeshower 18    iftotprs 9    htotprs 3
```

`buf[(x, y)]` yields a mutable `Cell`; the code sets its symbol and its style and moves on. `iftotprs` supplements this with five `set_string` calls and four `set_char` calls; the other three use none. So the honest description of what these four programs use ratatui *for* is: a double-buffered grid of styled cells with a diffing flush, a `Rect` type, and a `Style` type. The widget toolkit, the part of ratatui that most of its users mean when they name it, is inert dependency weight in all four crates.

This is not a criticism dressed as an observation, and it is worth saying why. A widget library is a layout and composition policy, and three of these four tools are reproducing an *existing* program's exact screen — iftop's flow table, htop's meter header and process rows — where the layout policy is already fixed by the thing being ported. A `Table` widget that lays out columns its own way is an obstacle in that situation, not a help. What ratatui contributes to a port is the part underneath the widgets: the cell grid, the diff, and the flush that only writes the cells that changed. That is exactly the part all four use.

Four different ways to own the terminal

Below the buffer, the four diverge again, and this time the divergence is in how much of the terminal lifecycle each one writes itself.

`storgeshower` writes none of it. Its `main.rs` is two calls:

```
$ grep -n 'ratatui::init|ratatui::restore' storgeshower/src/main.rs
49:     let mut terminal = ratatui::init();
54:     ratatui::restore();
```

`ratatui::init()` in 0.30 sets up raw mode, the alternate screen, and a panic hook in one call, and `restore()` undoes it. The consequence is measurable: `storgeshower` contains zero calls to `enable_raw_mode` and zero to `disable_raw_mode` anywhere in its source. The terminal contract is entirely delegated.

`iftotprs` writes it twice — two `enable_raw_mode` sites and two matching `disable_raw_mode` sites — one for the main UI and one for a secondary path.

`htotprs` writes it once and tears it down twice, and is the only one of the four that installs its own panic hook:

```
$ grep -rn 'panic::set_hook' htoprs/src
src/ported/commandline.rs:624:         std::panic::set_hook(Box::new(move |info| {
```

That hook is the crash-log machinery — it persists the panic message and backtrace to `~/.cache/htoprs/crash.log` (`extensions/crashlog.rs:33-55`) — and it exists because the ported C had a SIGSEGV/SIGBUS reporter to reproduce.

`lsOfrs` writes it five times. This is the finding of the section, and it is plainly visible:

```
$ grep -rn 'enable_raw_mode' lsOfrs/src
src/tui_tabs.rs:3064      src/monitor.rs:61      src/follow.rs:30
src/watch.rs:33          src/tui_app.rs:118
```

Five independent full-screen paths, one per mode: the unified tabbed TUI (`--tui`), the top-like monitor (`-W`), the single-process follow view (`--follow`), the file-watch view (`--watch`), and the standalone TUI app. Each enters raw mode itself, each has its own event loop, and each polls on its own cadence:

Mode	Entry	Poll interval
<code>--tui</code> tabs	<code>tui_tabs.rs:3064</code>	variable <code>poll_ms</code> (<code>:3393</code>)
<code>-W</code> monitor	<code>monitor.rs:61</code>	caller-supplied <code>timeout</code> (<code>:187</code>)
<code>--follow</code>	<code>follow.rs:30</code>	<code>Duration::from_secs(interval)</code> (<code>:177</code>)
<code>--watch</code>	<code>watch.rs:33</code>	200 ms (<code>:119</code>)
tui app	<code>tui_app.rs:118</code>	100 ms (<code>:159</code>)

Every one of the five has a matching `disable_raw_mode`, so the balance is correct on the normal exit path; the count is five and five. What none of the five has is a panic hook, which is the difference between `lsOfrs` and the other three: `storagehower` gets one from `ratatui::init()`, `htoprs` installs its own, and `iftoprs` and `lsOfrs` install none. The signals-and-terminal-state chapter covers what that costs; the point here is structural rather than behavioural. Five copies of a lifecycle is five places to fix when the lifecycle needs a sixth step, and the absence of a shared enter/leave helper in a crate that needs one five times is the single clearest piece of internal duplication the TUI comparison turns up.

htoprs has no terminal at all

The strangest arrangement in the four belongs to `htoprs`, and it follows from the manifest comment quoted at the top. Because `ratatui` is pulled without a backend, `htoprs` has no `ratatui::Terminal` and cannot call `draw`. Its overlays — the themed help screen, the theme chooser, the saved-filter editor — render into a free-standing buffer:

```
$ grep -n 'Buffer::empty\|fn blit' htoprs/src/extensions/overlay.rs
647:         let mut b = Buffer::empty(area);
834:pub(crate) fn blit<W: Write>(out: &mut W, buf: &Buffer) {
838:         let cell = &buf[(x, y)];
```

`blit` is a hand-written flush: it walks the buffer's cells and emits crossterm commands for the ones that matter, skipping the blank backdrop (there is a test named for exactly that, `blit_skips_blank_backdrop_and_emits_modal` at `overlay.rs:1865`). So `htoprs` uses `ratatui` as a *layout scratchpad* — a typed grid it can compose a modal into — and then does the terminal write itself, because the terminal already belongs to the ported htop drawing code. That is a coherent design and an unusual one, and it is the clearest single answer to why the four tools' shared dependency tells you so little about them.

Sizes, and where the screen code actually lives

The last measurement is the crudest and the most useful for anyone opening these trees for the first time:

```
$ wc -l lsofrs/src/tui_tabs.rs iftoprs/src/ui/render.rs storageshower/src/ui.rs
~ htoprs/src/extensions/overlay.rs
5308 lsofrs/src/tui_tabs.rs
2754 iftoprs/src/ui/render.rs
4086 storageshower/src/ui.rs
1914 htoprs/src/extensions/overlay.rs
```

Plus the state that drives them: `iftoprs/src/ui/app.rs` at 4,066 lines, `storageshower/src/app.rs` at 1,503, `lsofrs/src/tui_app.rs` at 572. Between render and state, `lsofrs` and `storageshower` each carry on the order of five to six thousand lines of hand-written screen code, and `iftoprs` close to seven thousand. For comparison, `grcrs` — the whole tool, both binaries — is 1,686 lines. The screens are the expensive part of these programs, and they are expensive precisely because the widget layer that would have absorbed some of that cost is the layer none of the four uses.

What the tools remember – six persistence conventions and the files they leave behind

A command-line tool that keeps state has to answer four questions: where the state lives, what format it is in, when it is written, and what happens on a machine that has never run the tool before. Across this fleet the four answers are given six different ways, and no two tools agree on all four. This chapter collects them, because the divergence is invisible from any single chapter and because two of the six answers have consequences a user can trip over on first run.

Where

Tool	Path	Chosen at
<code>lsotrs</code>	<code>~/.lsotrs.conf</code>	<code>config.rs:85</code>
<code>iftotrs</code>	<code>~/.iftotrs.conf</code> , or <code>--config PATH</code>	<code>config/prefs.rs:113</code>
<code>storageshower</code>	<code>~/.storageshower.conf</code> , or <code>--config PATH</code>	<code>prefs.rs:83</code>
<code>htotrs</code>	<code>~/.config/htotrs/prefs.json</code> (+ <code>filters.json</code>)	<code>extensions/prefs.rs:63-66</code> , <code>panels.rs:1072</code>
<code>zcolorizer</code>	<code>~/.config/zcolorizer/config.toml</code>	<code>config.rs:53</code>
<code>temprs</code>	<code>\$TMPDIR/temprs/</code> , or <code>\$TEMPRS_DIR</code>	<code>model/app.rs:59-62</code>

Three of the six put a dotfile directly in `$HOME` and resolve it with `dirs::home_dir()`. Two use the XDG config directory — `htotrs` composing it by hand so it can honour `$XDG_CONFIG_HOME` explicitly (`prefs.rs:69-76`), `zcolorizer` taking it from `dirs::config_dir()`. And one, `temprs`, does not keep a config file at all: its persistent state is a *directory* under the system temporary directory, which is the right shape for what it stores and unlike anything else in the fleet.

`htotrs` is worth a note for what it deliberately does *not* touch. Its preferences live in its own file rather than in htop's `htoprc`, and the module doc says so outright (`extensions/prefs.rs:4`). A port that shares its upstream's config file inherits its upstream's config schema; keeping the extension preferences in a separate file is what lets the ported settings code stay a faithful port while the Rust-only additions accumulate somewhere else.

What format

TOML in four (`lsotrs`, `iftotrs`, `storageshower`, `zcolorizer`), JSON in one (`htotrs`), and in `temprs` a set of plain files with no serialisation format at all. The `temprs` directory, created live and inspected:

```
$ TMPRS_DIR=/private/tmp/clifleet2/tpsr tpsr -l
$ ls -la /private/tmp/clifleet2/tpsr
-rw-r--r--  0  tpsr-inflight
-rw-r--r--  0  tpsr-stack
-rw-r--r--  0  tpsr-stack.lock
```

Three files, all empty on a fresh stack, named by constants in `util/consts.rs:6,8,10,130` — `TEMP_DIR = "tpsr"`, `MASTER_RECORD_FILENAME = "tpsrs-stack"`, `INFLIGHT_RECORD_FILENAME = "tpsrs-inflight"`, and `TEMPFILE_PREFIX = "tempfile"`. The payload files are `tempfile<milliseconds>`; the `.lock` is the `fs2` advisory lock the dependency ledger accounts for. This is the only state in the fleet designed to be read by another process while the writer is still running, and it is the only state with a lock file, which is not a coincidence.

The serialised structures are of comparable size and completely different vocabulary. `lsofrs`'s `Prefs` carries 10 fields (`config.rs:26-57`), `iftoprs`'s 18, `storageshower`'s 24 (`prefs.rs:8-42`), `htoprs`'s 11 (`extensions/prefs.rs`). Three of them store a saved custom palette; all three name its six numbers differently, which the theme chapter takes up.

When it is written — and the two tools that write before you ask

Here the six split into three behaviours, and the split matters.

Write on change, never otherwise. `lsofrs` saves only from the TUI paths, and its `save` is compiled out under test:

```
$ grep -n 'cfg(not(test))' lsofrs/src/config.rs iftoprs/src/config/prefs.rs
lsofrs/src/config.rs:  #[cfg(not(test))]
iftoprs/src/config/prefs.rs:  #[cfg(not(test))]
```

Both `lsofrs` and `iftoprs` gate the body of `save_prefs` behind `#[cfg(not(test))]`, so the write path does not execute under `cargo test` at all. That is a deliberate protection — a test suite that serialises preferences into the developer's real `$HOME` is a hazard — and it is also an honest gap: the tests exercise `toml::to_string_pretty` and the round-trip, never the file write.

Write the shipped template on first read. `storageshower` does this, and it is the behaviour most likely to surprise. `load_prefs_from` checks for the file and, if it is absent and no `--config` was given, writes the crate's own default configuration into `$HOME` before reading it (`prefs.rs`, the `if !path.exists() && custom_path.is_none()` branch), where the default is compiled in with `include_str!("../storageshower.default.conf")`. Verified against an isolated home directory:

```
$ HOME=/private/tmp/clifleet2/home storageshower --export-theme >/dev/null
$ ls -la /private/tmp/clifleet2/home
-rw-r--r--  7626  .storageshower.conf
$ diff -q $HOME/.storageshower.conf storageshower/storageshower.default.conf
(identical)
```

`--export-theme` prints a palette and exits. It is not a command that changes anything, and running it created a 7,626-byte file in the home directory, byte-for-byte the shipped template — comment banner, box-drawing header and all. Whether that is good or bad depends on taste, and the case for it is real: the file a user then edits is the annotated one, with every option documented in place, rather than a bare serialisation. But it is a write to `$HOME` performed by a read-only invocation, and nothing in `--help` says so.

`iftoprs` has the same first-run instinct and a different implementation: when the config is unreadable it constructs `Prefs::default()` and calls `save_prefs` (`config/prefs.rs:124-129`), which writes `toml::to_string_pretty` of the struct — the values without the commentary. The repository also ships a 1,624-byte `iftoprs.default.conf` whose header says “let iftoprs create it on first run,” and that file is *not* what gets

created; it is documentation of the format, not the seed of it. The same test-time gate applies, so the write only happens outside `cargo test`. In the isolated-home check above, `iftoprs --list-colors` created nothing, because that flag exits before preferences are loaded.

Write nothing, ever, unless asked. `zcolorizer` ships `config.example.toml` (3,794 bytes) and never installs it; `Config::load_default()` returns an empty configuration when the file is absent (`config.rs`), and the example's first line is an instruction to the reader to copy it. `htoprs` writes `prefs.json` only when a preference changes. `lsofrs` the same. `temprs` creates its directory on first use, which it must, since the directory *is* the feature.

The guard that exists because of an accident

`temprs` carries the one piece of persistence code in the fleet whose comment records a specific incident, and it belongs in this chapter because it is the sharpest illustration of what “the state is a directory” costs:

```
$ awk 'NR>=748,NR<=760' temprs/src/model/app.rs
// Pre-fix, `tp -c` did `remove_dir_all` on the parent of the master
// record file. With `TEMPRS_DIR=$HOME` set, that wiped $HOME. The
// safe contract: TEMPRS_DIR's basename must contain "temprs" (the
// default `~/.temprs` and any sane override). Anything else is
// refused — the user can manually delete if they really meant it.
```

`clear_all` now refuses unless the directory's basename lower-cases to something containing `temprs`. A tool whose “clear” operation is `remove_dir_all` on a user-supplied path has exactly one line of defence, and this is it. The environment-contract chapter lists `TEMPRS_DIR` among the variables that change where a tool writes; this is the reason that particular variable is the most dangerous one in the fleet.

What a reader should take from the spread

Six tools, six conventions, and no shared configuration crate — the manifests carry no `path = "..."` dependency anywhere among the six, so nothing is shared by construction. The costs are small individually and cumulative in aggregate: a user who runs four of these tools ends up with `~/.lsofrs.conf`, `~/.iftoprs.conf`, `~/.storageshower.conf`, `~/.config/htoprs/prefs.json`, `~/.config/zcolorizer/config.toml` and `$TMPDIR/temprs/`, in two formats, created under three different policies. The fleet's own house style — XDG for the newer tools, a home dotfile for the older ones — is visible in the split, and the direction of travel is clear enough from the dates, but the older half has not moved and nothing forces it to.

The completion sets, measured — six zsh files against six binaries

Every Rust tool in this book that a human types at a prompt ships a zsh completion, and nothing else. There is no bash completion, no fish completion, no PowerShell generator anywhere in the six repositories:

```
$ find {lsofrs,nmaprs,tempers,iftoprs,htoprs,storageshower} -name '*.bash' -o -name
~ '*.fish'
(no output)
```

That is a defensible position for this stack — the author’s shell is the product — but it is a fleet-wide fact worth stating once, because the packaging chapter’s description of “a completion” reads as shell-agnostic and is not.

The six files

```
$ wc -l */completions/_*
118 lsofrs/completions/_lsofrs
168 nmaprs/completions/_nmaprs
99 tempers/completions/_tempers
62 iftoprs/completions/_iftoprs
48 htoprs/completions/_htoprs
56 storageshower/completions/_storageshower
```

The first line of each is the interesting one, because three of the six complete more than one command name:

```
$ head -1 */completions/_*
#compdef lsofrs lsf
#compdef nmaprs nms
#compdef tp tempers
#compdef iftoprs
#compdef htoprs
#compdef storageshower
```

Those aliases are not conveniences invented by the completion; they are declared binaries. `lsofrs/Cargo.toml` declares `[[bin]]` targets named `lsf` and `lsofrs`, `nmaprs` declares `nmaprs` and `nms`, `tempers` declares `tempers` and `tp`, and all six of those names resolve on this machine’s `PATH`. `iftoprs` and `storageshower` declare no `[[bin]]` stanza at all and take the default binary name from the crate. `tempers` is the only one of the three whose `#compdef` line puts the short name first, which has no functional effect and is a small reminder that these six files were written independently.

The machinery

All six are hand-written `_arguments` specs, not generated:

Tool	<code>_arguments</code>	<code>_describe</code>	<code>state</code> references
<code>lsofrs</code>	1	1	6
<code>nmaprs</code>	1	0	2

Tool	<code>_arguments</code>	<code>_describe</code>	<code>state</code> references
<code>temprs</code>	1	1	0
<code>iftoprs</code>	4	1	1
<code>htoprs</code>	1	1	2
<code>storageshower</code>	1	0	0

`lsotprs` is the most elaborate — six references to `state`, driving the four dedicated handlers the packaging chapter names (`->filesdes`, `->addresses`, `->states`, `->fields`) — and `iftoprs` is the outlier in the other direction, calling `_arguments` four separate times rather than once with a computed argument array. `storageshower` and `nmaprs` have no state machine at all: `storageshower` because its grammar is flat, `nmaprs` because its 96 long options are almost all simple flags or plain value-takers even though the tool behind them is the most complex in the fleet.

Coverage, measured against `--help`

For each tool I extracted the set of long options mentioned in the completion and the set printed by the binary's own help, and compared them:

Tool	in completion	in <code>--help</code>	in help but not completion
<code>lsotprs</code>	24	22	none
<code>nmaprs</code>	96	96	none
<code>temprs</code>	43	43	none
<code>iftoprs</code>	17	16	none
<code>htoprs</code>	21*	16	none
<code>storageshower</code>	36	25	none

Six for six, the completion is a superset of the help. Nothing a user can read about on screen is missing from what the shell will offer, which is the property that actually matters and is not the property most projects achieve.

The asterisk on `htoprs` is an artefact of extraction rather than a real count, and it is worth spelling out because it is the kind of thing that turns into a false claim if it is not chased down. Two of that file's entries use zsh brace expansion:

```
$ grep -n 'no-colo\|no-function' htoprs/completions/_htoprs
17: '(-[ --no-color --no-colour]'{-[,--no-colo{,u}r}]{use a monochrome color scheme}
20: '(--no-function-bar --no-functionbar)'--no-function{-,}bar'[hide the function bar]'
```

A naive `grep` pulls `--no-colo` out of that first line as if it were an option, and it is not; the shell expands the brace to `--no-color` `--no-colour`. The same `grep` pulls `--drop-capabilities` out of a *comment* on line 10 that says the opposite of what the extraction implies — “`htoprs` does NOT implement `htop`'s Linux-only `--drop-capabilities`” — and the binary confirms the comment:

```
$ htoprs --drop-capabilities --max-iterations 1
htoprs: unrecognized option '--drop-capabilities' (exit 1)
```

The comment is accurate, the completion is right not to offer the flag, and the count of 21 should be read as “17 real long options, four of them alternate spellings.”

Where the completion knows more than the help

Two tools have a completion that documents interface the help screen omits, and in both cases the completion is the more accurate document.

`htoprs` is the clean case. Its `getopt` table is a direct port of `htop`'s, and it has 18 entries:

```
$ awk 'NR>=222,NR<=240' htoprs/src/ported/commandline.rs
    let spec: [(&str, c_int, c_int); 18] = [
        ("help", 0, ...), ("version", 0, ...), ("delay", 1, ...), ("max-iterations", 1, ...),
        ("sort-key", 1, ...), ("user", 2, ...), ("no-color", 0, ...), ("no-colour", 0, ...),
        ("no-mouse", 0, ...), ("no-unicode", 0, ...), ("no-meters", 0, 129),
        ("tree", 2, ...), ("pid", 1, ...), ("filter", 1, ...), ("no-functionbar", 0, 130),
        ("no-function-bar", 0, 130), ("highlight-changes", 2, ...), ("readonly", 0, 128),
    ];
```

Eighteen entries, sixteen distinct options, because `no-color/no-colour` and `no-functionbar/no-function-bar` are pairs of spellings sharing a value. The help screen prints one spelling of each pair, so it lists 16; the completion offers both spellings of both pairs, so it covers all 18 entries. The binary accepts every one of them:

```
$ for f in --no-colour --no-functionbar --no-function-bar --no-meters; do
    htoprs "$f" --max-iterations 1; echo "$f rc=$?"; done
--no-colour      rc=0
--no-functionbar rc=0
--no-function-bar rc=0
--no-meters      rc=0
```

(The `--max-iterations 1` is load-bearing in that check. `htoprs --version` short-circuits before argument validation, so a nonsense flag paired with `--version` also exits 0 — `htoprs --drop-nothing --version` prints the banner — and any acceptance test that uses `--version` as its harness measures nothing.)

`lsifrs` is the less clean case. Its completion offers two long options its help never prints, `--help` and `--theme`, and the second one comes with a menu:

```
$ grep -n 'theme' lsifrs/completions/_lsifrs
53: '--theme[color theme for TUI modes]:theme:(neon-sprawl classic solar-flare
   ~ ice-breaker matrix
     blade-runner synthwave acid-rain ghost-wire red-sector deep-net overlock sakura-den
   ~ data-stream
     neon-noir chrome-heart void-walker toxic-waste cyber-frost plasma-core steel-nerve
   ~ dark-signal
     glitch-pop holo-shift night-city laser-grid quantum-flux bio-hazard darkwave
   ~ megacorp zaibatsu)'
```

Thirty-one names, offered by the shell, for a flag the help screen does not mention. The declaration behind it accepts anything at all:

```
$ grep -n -A2 'Color theme for TUI modes' lsifrs/src/cli.rs
157:     /// Color theme for TUI modes (neon-sprawl, classic, solar-flare, ice-breaker,
   ~ matrix)
158:     #[arg(long = "theme", default_value = "neon-sprawl")]
159:     pub theme_name: String,
$ lsifrs --theme not-a-theme; echo rc=$?
rc=0
```

A `String` with a default and no `value_parser`, so an unrecognised palette name is accepted silently and the tool falls back to whatever its lookup does with an unknown key. Three documents describe this one flag and all three say something different: the doc comment lists five palettes, the completion lists thirty-one, and the parser accepts any string. The completion is the closest of the three to the truth and still not equal to it.

The one flag the completions miss

`storageshower`'s completion mentions 36 long options; its parser declares 37. The difference:

```
$ comm -23 <(declared long options) <(completion long options)
--reclaim
```

`--reclaim` is the entry point to the reclaimable-space overlay, which is behind the crate's optional `reclaim` cargo feature (`Cargo.toml, reclaim = ["dep:rayon", "dep:lz4_flex"]`). The completion therefore describes the default build rather than every possible build, which is the right choice — offering a flag that the installed binary will reject is worse than omitting one it would accept — but it does mean that a user who installs with `--features reclaim` has a completion that is one option short, and nothing regenerates it.

The man pages against the binaries

Twelve man pages ship across six repositories, two per tool, and they are the only documentation in the fleet that a user reads without the program running. That makes them the easiest documentation to let drift and the easiest to check mechanically. This chapter checks them: version stamps against manifests, flag lists against the binaries, prose claims against measured behaviour, and roff against a linter.

Two pages per tool, and one command that does not exist

```
$ wc -l */man/man1/*.1
 373 lsofrs/man/man1/lsofrs.1          901 lsofrs/man/man1/lsofrsall.1
 499 nmaprs/man/man1/nmaprs.1        1037 nmaprs/man/man1/nmaprsall.1
 366 temprs/man/man1/temprs.1         686 temprs/man/man1/temprsall.1
 231 iftoprs/man/man1/iftoprs.1       608 iftoprs/man/man1/iftoprsall.1
  99 htoprs/man/man1/htoprs.1         677 htoprs/man/man1/htoprsall.1
 224 storageshower/man/man1/storageshower.1 280
  storageshower/man/man1/storageshowerall.1
```

The `Xall` pages document a command name that is not installed and never will be — there is no `lsofrsall` binary — and the convention is stated in the file's own header comment: “modeled after `zshall(1)`: every man page in one document.” `zshall(1)` is likewise not a command. The intent is a single grep-able document, and `lsofrsall.1` says so directly, listing thirteen numbered sections from `1. INVOCATION` through `13. EXAMPLES` so that a reader can jump with `/`. The ratio is consistent: the `all` page is between two and seven times the short page, and the two extremes are `storageshower` (224 → 280, a 1.25× expansion) and `htoprs` (99 → 677, a 6.8× one).

Version stamps: twelve for twelve

Every page carries a `.TH` line with a version string, and every one of the twelve matches its crate's manifest version exactly:

Tool	Cargo.toml	X.1 .TH	Xall.1 .TH
<code>lsofrs</code>	4.9.2	<code>lsofrs 4.9.2</code>	<code>lsofrs 4.9.2</code>
<code>nmaprs</code>	0.1.10	<code>nmaprs 0.1.10</code>	<code>nmaprs 0.1.10</code>
<code>temprs</code>	2.9.13	<code>temprs 2.9.13</code>	<code>temprs 2.9.13</code>
<code>iftoprs</code>	2.22.10	<code>iftoprs 2.22.10</code>	<code>iftoprs 2.22.10</code>
<code>htoprs</code>	0.5.12	<code>htoprs 0.5.12</code>	<code>htoprs 0.5.12</code>
<code>storageshower</code>	0.28.9	<code>storageshower 0.28.9</code>	<code>storageshower 0.28.9</code>

Twelve stamps, twelve matches, in six repositories with no shared release tooling. That is the strongest single piece of documentation discipline the fleet exposes to measurement, and it is worth recording as such, because every other measurement in this chapter is a gap.

The dates are less disciplined but harmlessly so. Seven pages use an ISO date (2026-07-02, 2026-07-10, 2026-07-05, 2026-06-05) and five use May 2026, which mandoc cannot parse and passes through verbatim with a warning. Within temprs the two pages disagree with each other: temprs.1 is dated 2026-06-05 and temprsall.1 May 2026.

The one page whose stamp has moved since it was first quoted here is lsofrs.1. The lsofrs chapter records it as “versioned lsofrs 4.9.1,” the drift ledger in the packaging chapter already records 4.9.1 → 4.9.2, and the page now reads 4.9.2 — so the stamp tracked the manifest through the bump, which is the property being tested. The installed binary on this machine reports 4.9.3, one patch ahead of the checked-out tree, which is a property of the machine rather than of the software and is the ordinary situation for anyone who installs from a release and reads from a working copy.

While the versions agree, their formatting does not. Five of the six binaries print a bare name semver; storageshower prints a v:

```
$ for b in lsofrs nmaprs temprs iftoprs htoprs storageshower; do $b --version; done
lsofrs 4.9.3          nmaprs 0.1.10      temprs 2.9.13
iftoprs 2.22.10      htoprs 0.5.12      storageshower v0.28.9
```

htoprs’s bare form is not a style choice — it is parity, since the htop it ports prints htop 3.5.x and the parity harness canonicalises exactly that shape (tests/parity/harness.rs:86-97). The other five are free, and four of the five agree.

What the pages document that the binaries do not, and the reverse

Comparing the long options named in each short man page against the long options the binary prints in --help:

Tool	in man	in --help	only in man	only in --help
lsofrs	19	22	--help	--color, --dir, --dir-recurse, --tui
iftoprs	16	16	--completions	--config
storageshower	26	25	(prose, see below)	—
temprs	43	43	—	—
nmaprs	50	96	—	46 options
htoprs	17	16	--no-colour	—

temprs is exact: forty-three in the page, forty-three on the screen, the same forty-three. iftoprs trades one for one — its page documents a --completions flag the help does not print, and the help prints a --config the page does not document.

lsofrs’s gap is the interesting one, because the four options missing from the page are not obscure. --tui is the flag that starts the unified tabbed interface; --color selects auto/always/never; --dir and --dir-

`recurse` are the long spellings of Isof's `+d` and `+D`, which the page documents in their `+` form. A reader of `lsufrs.1` can find `+d` and cannot find `--dir`, and cannot find the TUI at all.

`nmpr's` gap is the largest in absolute terms and the least alarming in practice. Its short page documents 50 long options; the binary advertises 96. The 46 that appear only on screen are, almost without exception, the Nmap compatibility surface the parser chapter describes as accepted-for-parity and logged rather than acted on — `--defeat-rst-ratelimit`, `--nsock-engine`, `--release-memory`, `--script-*`, `--send-eth`, `--servicedb`, and the rest. A man page that documented all 96 would have to explain, forty-six times, that the option parses and does nothing. Leaving them to the help screen and the capability matrix is a legitimate editorial choice; it is only worth flagging because the arithmetic looks alarming until you know what the difference is made of.

A prose claim, tested

`storageshower.1` makes a blanket statement in its description:

```
$ awk "NR>=24,NR<=28" storageshower/man/man1/storageshower.1
All CLI flags have a matching
.B --no-flag
inverse so a runtime config in
.RB \{(aq "~/storageshower.conf" \{(aq
can be overridden per-invocation.
```

Measured, the claim is true of the booleans and false as written. The parser declares 37 long options and hides 10 of them:

```
$ grep -c 'hide = true' storageshower/src/cli.rs
10
$ comm -23 <{(declared long options)} <{(options printed by print_help)}
--bars --border --header --no-compact --no-full-mount --no-local
--no-reverse --tooltips --used --virtual
```

The ten hidden options are exactly the ten the hand-written `print_help()` at `cli.rs:177` does not print, so the visible surface is 27 and the accepted surface is 37. Every hidden one is the other half of a `overrides_with` pair — `--compact/--no-compact`, `--reverse/--no-reverse`, and so on — which is the mechanism the parser chapter describes.

Where the claim breaks is on the options that take a value. Prefixing `--no-` to any of the seven value-takers is rejected:

```
$ for f in --no-sort --no-theme --no-refresh --no-color --no-units --no-warn --no-crit; do
    storageshower "$f" --export-theme; echo "$f rc=$?"; done
all seven: rc=2
$ storageshower --no-bogusflag --export-theme; echo rc=$?
rc=2
```

and one boolean's inverse is not spelled the way the rule predicts:

```
$ storageshower --no-local-only --export-theme; echo rc=$?      # rc=2
$ storageshower --no-local --export-theme; echo rc=$?          # rc=0
```

The flag is `--local-only`; the inverse is `--no-local` (`cli.rs:32`). A user following the man page's rule literally will type the wrong flag and get exit 2. The rule is a good rule and the page should say "every boolean flag," and should name the one pair whose spellings do not correspond.

The flags the page documents and the binary throws away

The `lsotrs` compatibility chapter records the accept-and-ignore set from the source. Checked against the installed binary, with a rejection control:

```
$ for f in -b -O -X -S +M -Q --nope; do lsotrs "$f" >/dev/null; echo "$f rc=$?"; done
-b rc=0      -O rc=0      -X rc=0      -S rc=0      +M rc=0
-Q rc=2      --nope rc=2
```

Five upstream options parse, produce the normal listing, and change nothing; two unknown ones are rejected by clap with exit 2. That is the contract working exactly as the source describes it (`cli.rs:452` for the no-argument set, `:456` for the optional-argument set, `:568` for the `+` toggles), and it is worth having the empirical confirmation next to the source citation, because a silently-accepted flag is precisely the kind of behaviour that a refactor can break without any test noticing. One correction to how this set is sometimes described: `-x` is *not* in it. Lower-case `-x` is rewritten to `--cross-over` (`cli.rs:499`) and is wired to real behaviour; it is upper-case `-X` that is dropped. The two differ by a shift key and by everything else.

Roff, checked

`mandoc -T lint` over all twelve pages produces 55 diagnostics, and their distribution is the useful part:

Page	total	.B in table	unparsable date	empty UR	unterminated quote
<code>lsotrs.1</code>	0				
<code>lsotrsall.1</code>	8	6			
<code>nmaprs.1</code>	2		1	1	
<code>nmaprsall.1</code>	5	2	1	1	
<code>temprs.1</code>	0				
<code>temprsall.1</code>	6	2	1	1	
<code>iftoprs.1</code>	2		1		1
<code>iftoprsall.1</code>	18	12	1	1	3
<code>htoprs.1</code>	0				
<code>htoprsall.1</code>	13	11		1	
<code>storageshower.1</code>	1				
<code>storageshowerall.1</code>	0				

Fifty of the fifty-five land on the six `all` pages; three of the six short pages are perfectly clean, and the one spotless `all` page is `storageshowerall.1`. The dominant class, 33 of 55, is a single mistake repeated: `.B` inside a `tbl` table cell, which `mandoc` reports as `ignoring macro in table: B Key|.B Action`. The practical effect is that the bold header row of every affected table renders unbolded — the tables are still correct, still aligned, and slightly less legible than their author intended. The five `empty block: UR` diagnostics are a `.UR/.UE` URL pair with nothing between them, and the four `unterminated quoted argument` reports in the

two `iftoprs` pages are the only diagnostics in the set that can change what a line says rather than how it looks.

None of this is severe and all of it is mechanical. The umbrella hygiene gate described earlier checks that every man page has a `.SH SYNOPSIS`; a gate that also ran `mandoc -T lint` and failed on a non-empty result would have caught all fifty-five, and would keep catching them, at the cost of one line in the same script.

The theme module's real API — one file, three copies, and the fields nobody reads

The shape-of-the-fleet chapter records that the 31-palette set is “a genuine shared component, not merely a shared aesthetic,” and the theme-engine chapter traces its lineage from `storageshower` through `iftoprs` into `lsofrs`, `htoprs`, and `zcolorizer`. Both statements are true and neither is the whole picture, because “shared component” in this fleet does not mean what the phrase normally means. There is no theme crate. There is a file, and there are copies of it. This chapter reads the actual API and measures the copies.

There is no crate

The first thing to establish is negative and easy to check. None of the six Rust manifests declares a path dependency:

```
$ grep -n 'path = "' {lsofrs,iftoprs,htoprs,storageshower,zcolorizer,grcrs}/Cargo.toml
lsofrs:19,23  [[bin]] paths
htoprs:17,21,29 [[lib]], [[bin]], [[test]] paths
zcolorizer:29 [[bin]] path
grcrs:27,31  [[bin]] paths
```

Every hit is a target path inside its own crate. There is no `menketotechnologies-theme` on crates.io in any of these lockfiles and no `../theme` on disk. The sharing is by copy.

The API, as it stands in `iftoprs`

The canonical copy — the one the others were taken from — is `iftoprs/src/config/theme.rs`, 1,263 lines. Its public surface is small:

Item	Line	What it is
<code>enum ThemeName</code>	31 variants	one per palette, <code>NeonSprawl ... Iftopcolor</code>
<code>ThemeName::ALL</code>	:75	<code>&'static [ThemeName]</code> , declaration order, drives <code>--list-colors</code>
<code>ThemeName::display_name()</code>	:109	variant → “ <code>Neon Sprawl</code> ”, the string the UI prints
<code>fn palette()</code>	:151	private; variant → <code>(u8, u8, u8, u8, u8, u8)</code>
<code>pub struct Theme</code>	:185	22 <code>Color</code> fields, the resolved theme

Item	Line	What it is
<code>pub struct CustomThemeColors</code>	:234	the six numbers, as serialised into the config
<code>Theme::from_name()</code>	:251	variant → resolved <code>Theme</code>
<code>Theme::from_palette_raw()</code>	:256	six raw indices → resolved <code>Theme</code>
<code>Theme::palette_values()</code>	:308	variant → <code>[u8; 6]</code> , for the editor
<code>Theme::swatch()</code>	:314	variant → <code>Vec<(Color, &'static str)></code> , for the preview strip

The design is a two-stage lookup: a name resolves to six 256-colour cube indices, and the six indices fan out into twenty-two semantic slots. The fan-out is where `shift_color_lighter` earns its place — several slots are not one of the six but a computed brightening of one of them, which is why a six-number palette can drive a twenty-two-field theme without looking flat. `from_palette_raw` is the entry point that makes user-defined palettes work: the config stores six numbers and the same fan-out runs over them, so a hand-written palette gets the same twenty-two slots as a built-in.

The palette table itself is the data:

```
$ awk 'NR>=152,NR<=156' iftoprs/src/config/theme.rs
ThemeName::NeonSprawl => (27, 48, 135, 141, 63, 99),
ThemeName::AcidRain => (28, 46, 34, 40, 22, 35),
ThemeName::IceBreaker => (19, 39, 25, 33, 21, 32),
ThemeName::SynthWave => (91, 177, 128, 134, 93, 97),
ThemeName::RustBelt => (172, 214, 178, 220, 166, 130),
```

Thirty-one such lines, the last of which is `Iftopcolor => (21, 46, 28, 48, 33, 19)`.

The copy in `htoprs`, measured

`htoprs/src/extensions/theme.rs` is also 1,263 lines. The two files are not symlinks and their checksums differ. What differs inside them is one line:

```
$ diff iftoprs/src/config/theme.rs htoprs/src/extensions/theme.rs | grep -c '^<'
166
$ diff iftoprs/src/config/theme.rs htoprs/src/extensions/theme.rs \
| grep '^[<>]' | grep -v 'Indexed\|AnsiValue'
< use ratatui::style::Color;
> use crossterm::style::Color;
```

One hundred and sixty-six differing lines, of which one hundred and sixty-five are the mechanical substitution `Color::Indexed(n) → Color::AnsiValue(n)`, and the one hundred and sixty-sixth is the import that forces it. `iftoprs` types its theme in ratatui's colour enum; `htoprs`, which uses ratatui without a backend and drives the terminal through crossterm, types the identical theme in crossterm's. Two libraries' names for the same 256-colour index, and that single type difference is the entire distance between the two files.

That is the precise reason a shared crate has not been extracted, and it is worth stating as a design problem rather than as a lapse. A common theme crate would have to be generic over the colour type or return raw `u8` indices and let each consumer construct its own `Color`. Neither is difficult. Both are more work than copying the file, and the copy has stayed correct — 1,097 of the 1,263 lines are byte-identical, which means every palette number, every display name, every **ALL** entry and every doc comment has stayed in sync across two repositories with no mechanism enforcing it.

What the copy carried with it

The copy is faithful, and faithfulness has a cost that shows up as soon as you ask what `htoprs` actually uses. The twenty-two fields of `Theme` are named for `iftoprs`'s screen — a bandwidth monitor's screen. Counting the lines that name each field outside the theme module itself, in `htoprs` — `grep -c '\.<field>\b'` over `htoprs/src` with `src/extensions/theme.rs` excluded, so a line naming a field twice counts once:

Field	lines naming it in <code>htoprs</code> outside <code>theme.rs</code>
<code>help_bg</code> , <code>help_border</code> , <code>help_title</code> , <code>help_key</code>	4 each
<code>help_val</code>	2
<code>bar_color</code> , <code>help_section</code> , <code>scale_line</code> , <code>select_bg</code>	1 each
<code>bar_color_mid</code> , <code>bar_text</code> , <code>host_src</code> , <code>host_dst</code> , <code>arrow</code> , <code>rate_2s</code> , <code>rate_10s</code> , <code>rate_40s</code> , <code>scale_</code> <code>label</code> , <code>total_label</code> , <code>cum_label</code> , <code>peak_label</code> , <code>proc_name</code>	0

Thirteen of the twenty-two slots are never read. `host_src` and `host_dst` are the source and destination host colours of a flow table; `rate_2s`, `rate_10s` and `rate_40s` are the three rolling-average columns iftop is famous for; `arrow` is the `=>/<=` direction glyph between two hosts. A process viewer has no flows, no hosts, and no rolling-rate columns, and it computes and stores colours for all of them on every theme resolution.

The carry-over is visible at the level of names as well. `htoprs` ships a palette called `Iftopcolor`, whose display name is the lower-case string `"iftopcolor"`:

```
$ grep -n 'Iftopcolor\|iftopcolor' htoprs/src/extensions/theme.rs
69:     Iftopcolor,
106:         ThemeName::Iftopcolor,
141:         ThemeName::Iftopcolor => "iftopcolor",
179:         ThemeName::Iftopcolor => (21, 46, 28, 48, 33, 19),
451:     fn iftopcolor_theme_renders() {
709:     fn iftopcolor_theme_help_section_is_indexed() {
931:     fn iftopcolor_theme_peak_label_is_indexed() {
```

`Iftopcolor` exists in `iftoprs` for a good reason: it is the parity palette, the one that makes `iftoprs` look like iftop. In `htoprs` it is a palette named after a different program, offered in the theme chooser of a process viewer, and pinned by three tests whose names say `iftopcolor`. Nothing is broken — it is a perfectly usable set of six colours — and the enum-length assertion at `theme.rs:335` (`assert_eq!(ThemeName::ALL.len(), 31)`) means removing it would require touching the test that keeps the count honest. It is simply a fact about what a verbatim copy does: it brings the neighbour's vocabulary with it.

Three names for the same six numbers

The third copy is not a copy of the code at all. `storageshower` keeps the palettes as data files:

```
$ cat storageshower/themes/neon_sprawl.toml
# Neon Sprawl (Default)
# Electric blues and purples - the original cyberdeck palette
[custom_themes.neon_sprawl]
blue           = 27
green          = 48
purple         = 135
light_purple   = 141
royal          = 63
dark_purple    = 99
```

The six numbers are 27, 48, 135, 141, 63, 99 — identical, in order, to `theme.rs:152`'s `NeonSprawl` tuple. Ten such files ship in `storageshower/themes/`.

So the same six values are addressed by three different sets of names across the fleet. `storageshower` calls them `blue`, `green`, `purple`, `light_purple`, `royal`, `dark_purple` — colour words, describing the default palette and describing nothing accurate about any other. `lsofrs` calls them `c1` ... `c6` (`config.rs:9-22`). `iftoprs` and `htoprs` take them positionally as `from_palette_raw(c1, c2, c3, c4, c5, c6)` and document the roles in a comment: primary, brightest accent, secondary, mid, dim, darkest. A user who exports a palette from `storageshower` and wants it in `lsofrs` has to know that `royal` means `c5` means “dim,” and nothing in either tool says so.

That is the concrete shape of what a shared crate would have to settle, and it is a larger job than the type mismatch: not just one `Color`, but one vocabulary for the six roles, one serialisation key set, and one story for the fourteen fields `htoprs` does not use. Until that is done, “shared theme component” should be read the way the measurements support it — as a well-maintained set of copies, kept in sync by care rather than by the compiler.

Process attribution — the backends that turn a descriptor into a name

Three of the tools in this book answer a question of the form “which program is responsible for this?” `lsOfrs` answers it for open descriptors, `iftoprs` for network flows, `htoprs` for everything a process owns. The identity chapter covers the keys those answers are recorded under and how they decay. This chapter covers the layer below: the mechanism by which a number obtained from the kernel becomes a program name in a column, what that mechanism costs, and what it cannot see.

Two bindings to the same three syscalls

On macOS, the whole of process introspection outside `sysctl` runs through three calls in `libproc`: enumerate the PIDs, ask for a process's info, ask for one descriptor's info. Two tools in this fleet reach them two different ways.

`lsOfrs` declares them itself:

```
$ awk 'NR>=372,NR<=386' lsOfrs/src/darwin.rs
unsafe extern "C" {
    fn proc_listpids(type: u32, typeinfo: u32, buffer: *mut c_void, buffersize: c_int) ->
        c_int;
    fn proc_pidinfo(...) -> c_int;
    fn proc_pidfdinfo(...) -> c_int;
}
```

Seventeen lines of `extern "C"` and no crate, followed by ten call sites (:485, :491, :511, :530, :548, :555, :575, :595, :797, :835) and the struct definitions the calls fill in — `vnode_info_path`, socket info, and the rest, hand-transcribed with `libc::fsid_t`, `libc::in_addr` and `libc::in6_addr` for the members that already have libc equivalents (`darwin.rs`:160, :222–223).

`iftoprs` uses the `libproc` crate:

```
$ awk 'NR>=61,NR<=65' iftoprs/src/util/procinfo.rs
use libproc::libproc::bsd_info::BSDInfo;
use libproc::libproc::file_info::{ListFDs, ProcFDType, pidfdinfo};
use libproc::libproc::net_info::{SocketFDInfo, SocketInfoKind};
use libproc::libproc::proc_pid::{listpidinfo, name, pidinfo, pidpath};
use libproc::processes::{ProcFilter, pids_by_type};
```

Same three kernel entry points, typed wrappers instead of hand-written ones. The tradeoff is exactly the one the unsafe-surface chapter frames in general terms, and here it is concrete: `lsOfrs` owns the buffer arithmetic and has to reason about it in comments —

```
$ grep -n 'oversized|undersized' lsOfrs/src/darwin.rs
998:    // proc_pidfdinfo will reject the undersized buffer.
1038:    // Oversized is safe — proc_pidfdinfo only writes 168 bytes into our 176-byte
    // buffer.
```

— while `iftoprs` inherits whatever the crate's authors decided about the same 168 bytes. Neither is wrong. `lsOfrs` needs socket and vnode detail at a granularity the crate does not expose, which is a real reason to write the bindings; `iftoprs` needs only “which PID owns this port,” which the crate covers completely. The

fleet-level observation is that a dependency floor tuned for durability (“prefer foundational crates”) produced both answers in the same codebase, because `libc` and a thin FFI block are as foundational as it gets, and so is a crate that does nothing but wrap the same calls.

The direction of the walk

The mechanisms differ more than the bindings do, and the difference follows from the question each tool is answering.

`lsOfirs` walks *forwards*: enumerate every PID with `proc_listpids`, and for each PID enumerate its descriptors with `proc_pidinfo(PROC_PIDLISTFDS)`, then ask `proc_pidfdinfo` about each one. The output is one row per descriptor and the attribution is free, because the PID was the loop variable.

`iftoprs` cannot do that. A captured packet carries addresses and ports, not a PID, so the walk has to be inverted into an index built ahead of time:

```
$ awk "NR>=13,NR<=20" iftoprs/src/util/procinfo.rs
struct ProcTable {
    /// Maps (local_port, protocol) → (pid, name)
    by_port: HashMap<u16, u8>, (u32, String)>,
    /// Maps pid → on-disk executable path, for socket-owning processes.
    exe_by_pid: HashMap<u32, PathBuf>,
}
```

and refreshed wholesale rather than per lookup — the doc comment on `refresh_proc_table` says so plainly: “Call this periodically from the proc-lookup thread (e.g. every 2s). One full enumeration per call, cheaper than a per-flow lookup on every packet” (`procinfo.rs:37-40`). That is a sampling decision with a visible consequence: a flow whose owning process started and exited between two refreshes is attributed to whatever the last enumeration believed, or to nothing. The sampling chapter’s general argument has its sharpest instance here, because the packet capture is continuous and the attribution behind it is not.

On Linux the same index is assembled from two pseudo-files instead of a syscall: `/proc/net/tcp`, `/proc/net/tcp6`, `/proc/net/udp`, `/proc/net/udp6` give inode → port, and a walk of `/proc/[pid]/fd/` gives inode → PID (`procinfo.rs:150-160`). Two platforms, two mechanisms, one index shape — which is the right way to split a backend, and is why the `#[cfg(target_os = ...)]` in this file wraps the table *builders* and not the table.

`iftoprs` then adds a layer neither of the others has. `exe_by_pid` exists to feed a provenance resolver that identifies the *binary* rather than the process, keyed on the executable’s (`device`, `inode`, `mtime`) triple and cached on it (`util/provenance.rs:51-60`). The name in the column comes from the process; the identity behind the name comes from the file on disk, and the cache invalidates when the file changes underneath. That is a stronger notion of “which program” than a command string, and it is the only place in the fleet where the question is asked of the executable rather than of the process table.

What an unprivileged answer actually contains

All of this is bounded by what the kernel will tell a user about processes that are not theirs, and the bound is large. Measured on this machine as an ordinary user, with no elevation:

```
$ ps -A -o pid= | wc -l
781
$ lsOfirs | tail -n +2 | awk '{print $2}' | sort -u | wc -l
494
$ lsOf | tail -n +2 | awk '{print $2}' | sort -u | wc -l
508
```

Seven hundred and eighty-one processes exist; descriptors can be listed for four hundred and ninety-four of them. Roughly thirty-seven per cent of the process table is opaque, and the two tools disagree about the boundary by fourteen processes — a small enough gap that it is more likely to be timing (processes starting and exiting between two runs) than a difference in privilege handling, and I did not chase it further.

The row counts diverge much more than the PID counts do:

```
$ lsof | wc -l
5982
$ lsof | wc -l
14522
```

That looks like a coverage gap of nearly two to one and is almost entirely one categorical difference. `lsof` emits rows for descriptor classes that are not descriptors — the mapped executable and libraries (`txt`) and the working directory (`cwd`):

```
$ lsof | tail -n +2 | awk '{print $4}' | sort | uniq -c | sort -rn | head -4
8159 txt
2998 <n>u
2557 <n>r
489 cwd
```

Eight thousand one hundred and fifty-nine `txt` rows plus four hundred and eighty-nine `cwd` rows account for 8,648 of `lsof`'s 14,522, leaving 5,874 — within one per cent of `lsof`'s 5,982. `lsof` lists numbered descriptors and does not list mapped images or working directories, and the two tools agree closely on everything else.

The filter grammar, though, accepts the names of the classes the gatherer never produces:

```
$ lsof -d txt | tail -n +2 | wc -l      0
$ lsof -d txt | tail -n +2 | wc -l    7939
$ lsof -d cwd | tail -n +2 | wc -l      0
$ lsof -d cwd | tail -n +2 | wc -l    509
$ lsof -d 0 | tail -n +2 | wc -l     516
```

`-d cwd` and `-d txt` parse — the parser has explicit cases for them (`filter.rs:831, :838`) — and return nothing, while `-d 0` returns 516 rows. This is the accept-and-ignore contract in a place where it is less obviously harmless than a kernel-tuning flag: a script that filters `lsof -d txt` to find which processes have a given library mapped gets an empty result under `lsof` rather than an error, and an empty result reads as “nothing matched.” Of the compatibility decisions in the fleet this is the one most likely to be mistaken for an answer.

The same shape of failure appears one flag over, and there it diverges from the upstream outright. `lsof` validates the argument to `-p`; `lsof` does not:

```
$ lsof | wc -l      6180
$ lsof -p CHR | wc -l      6180      (exit 0)
$ lsof -p 858 | wc -l      10
$ lsof -p CHR
lsof: illegal process ID: CHR
```

A non-numeric process ID is rejected by `lsof` with a message naming the offending value. Under `lsof` the same argument produces the *unfiltered* listing — every descriptor on the machine — with exit 0 and nothing on stderr. The failure mode is the more dangerous of the two available: a filter that silently matches nothing produces an empty result a caller may notice, while a filter that silently matches everything produces six thousand rows that look exactly like a successful narrow query returning a lot of hits. Of everything the attribution layer does, this is the one behaviour worth a test the next time the selection parser is touched.

Headless — NDJSON, JSON, and the fleet without a terminal

Half of these tools exist to paint a screen and half exist to be piped into something. The dividing line is not where the tool names suggest it is: the two programs in the fleet that never meet a terminal have the most carefully specified machine interface, and the four that own the terminal have the least. This chapter collects the headless surface — what each tool emits when nothing is watching — and reads the protocols where there are protocols.

What each tool offers

Tool	Headless output
<code>lsotfrs</code>	<code>-J/--json</code> , <code>--csv</code> , plus <code>lsotfrs</code> 's <code>-F</code> field mode and <code>-t</code> terse mode
<code>iftoprs</code>	<code>--json</code> — “Stream NDJSON to stdout (no TUI)”
<code>storageshower</code>	none for data; <code>--list-colors</code> and <code>--export-theme</code> only
<code>htoprs</code>	none; <code>--version</code> and <code>--sort-key=help</code> are the only non-interactive outputs
<code>zcolorizer</code> , <code>grcrs</code>	the whole tool is a filter; there is no other mode
<code>zwire-host</code> , <code>zpwichrome-host</code>	the whole tool is a protocol; there is no terminal mode

`storageshower`'s absence is the notable one, because it is the tool whose data most obviously wants to be piped. It walks a filesystem and computes per-subtree sizes, and the only ways to get those numbers out are the screen and the clipboard/export keys inside the TUI. Its twenty-seven visible options contain no `--json`, no `--csv`, and no one-shot print mode; `--export-theme` exports a palette, not a scan. A disk-usage tool with no machine-readable output is a design choice with a real cost, and it is the clearest single feature gap the headless survey turns up.

`iftoprs` sits at the other end: `--json` suppresses the TUI entirely and streams newline-delimited JSON, which is the correct shape for a continuously-updating flow table — one object per emission, no enclosing array, so a consumer can read it incrementally and a truncated stream is still valid up to its last newline.

One writer, two framings

`zwire-host` is where the protocol thinking actually lives, and its central abstraction is small enough to quote whole. There is one output type, `Peer`, and it carries a framing discriminant:

```
$ awk "NR>=59,NR<=75" zwire-host/src/proto.rs
pub fn send(&self, v: &Value) -> io::Result<()> {
    let data = serde_json::to_vec(v).unwrap_or_default();
    let mut o = self.w.lock().unwrap();
    match self.framing {
        Framing::Native => {
            o.write_all(&(data.len() as u32).to_le_bytes())?;
            o.write_all(&data)?;
        }
        Framing::Ndjson => {
            o.write_all(&data)?;
            o.write_all(b"\n")?;
        }
    }
    o.flush()
}
```

`Framing::Native` is Chrome’s native-messaging framing — a little-endian `u32` length prefix followed by the JSON — and `Framing::Ndjson` is a newline. Every reply in the program goes through this one function, so the browser transport and the Unix-socket transport cannot drift apart: they are the same serialiser with a two-line difference in how the bytes are delimited. The readers mirror it exactly, `read_native` at `proto.rs:104` and `read_ndjson` at `:113`, and both are bounded by one constant, `MAX_MSG = 128 * 1024 * 1024` (`proto.rs:19`), which is the only backpressure in the design.

Two details in that file are worth pulling out because they are the kind of decision that only shows up under load. First, `respond` copies the request’s `id` onto the reply when one is present (`proto.rs:84–88`), which is what makes a single socket safely multiplexable — several callers can have requests in flight and match replies without ordering assumptions. Second, `respond` records to the host log, and the streaming paths deliberately do not go through it: “streaming frames (sysinfo/pty/job) use `out.send` directly and are intentionally not logged” (`proto.rs:89–91`). A host that logged every frame of a live system-info stream would spend its life writing its own log.

The command surface behind that transport is 70 distinct verbs, distributed across six modules:

```
$ grep -rhoE '^\\s*" "[a-z_0-9]+' =>' zwire-host/src/*.rs | sort -u | wc -l
70
session.rs 45  fsops.rs 8   osops.rs 5   zbus.rs 5   jobs.rs 4   procs.rs 3
```

The protocol, exercised

Run offline against a local socket in a scratch directory, the behaviour is exactly what the source predicts:

```
$ zwire-host serve --socket /private/tmp/clifleet2/zw.sock 6
zwire-host: listening on /private/tmp/clifleet2/zw.sock

$ zwire-host call --socket ... '{"cmd":"ping"}'
{"arch":"aarch64","caps":["hello","kv","fs","exec","jobs","bus","peer","watch","open",
"clipboard","notify","hostinfo","scheme","ui","palette","sysinfo","procs","pty"],
"host":"zwire-host","ok":true,"os":"macos","pid":21272,"version":"0.3.15"}

$ zwire-host call --socket ... '{"cmd":"nope"}'
{"cmd":"nope","err":"unknown_cmd","ok":false}

$ zwire-host call --socket ... 'not json'
{"err":"empty","ok":false}
```

Three things are visible in those four lines. The `ping` reply is a capability announcement rather than a pong — eighteen named capabilities, the host name, the version, the OS, the architecture and the PID — so a client’s

first round trip tells it everything it needs to decide what to ask for next. That is the right design for a host that is spoken to by a browser extension, a tmux binding and an editor, all of which may be older or newer than the host.

The unknown-command reply echoes the command back alongside `err: "unknown_cmd"`, which makes a log of failures self-describing.

And the third line is the small rough edge. A syntactically invalid request produces `{"err":"empty","ok":false}` — the same reply an actually empty request would get. The cause is in the reader: `read_ndjson` parses the line and, on failure, yields `Value::Null` rather than an error, “so the caller can reply with an error rather than dropping the connection” (`proto.rs:110-112`). Not dropping the connection is the right call; the consequence is that the error the caller then replies with cannot distinguish malformed JSON from a blank request. For a protocol whose clients are all written by the same author that is a minor thing. For one whose whole purpose is to be spoken to by other people’s code, `bad_json` and `empty` are different problems and a client debugging the first will be told the second.

`zpwirchrome-host` is the smaller of the two hosts — 6,343 lines against `zwire-host`’s 6,518 — and speaks only the native-messaging framing, because its only peer is Chrome. Its structure is the port-plus-extensions split the anatomy chapter describes: `src/ported/` holding the browserpass-native protocol and `src/extensions/` holding the eight added actions (`dl`, `gpg_unlock`, `otp`, `run_command`, `search`, `zcite`, and the host/mod plumbing).

The asymmetry, and why it is the wrong way round

Set the two halves side by side. `zwire-host` and `zpwirchrome-host` have a versioned handshake, a capability list, correlation IDs, a message size bound, two framings behind one writer, and conformance tests that drive `serve_conn` with batches of NDJSON lines (`tests/wire_conformance.rs:62`). The four TUI tools have, between them, one NDJSON stream (`iftoprs --json`), one JSON and one CSV formatter (`lsotfrs`), and nothing at all in the other two.

The asymmetry is easy to explain and still worth naming. A native-messaging host has no other interface: if the protocol is bad, the tool does not work, so the protocol got the attention. A TUI has a screen that is the interface, and the machine-readable path is the thing added afterwards if at all. But the tools whose output is most worth capturing — a descriptor census, a disk scan, a flow table — are the four with the screens, and two of the four cannot be captured at all. The lesson the two hosts hold for the other four is not their protocol; it is that they were designed with the consumer in mind from the first line, and a `--json` added later is never quite the same shape as one designed in.

Two shapes of the same scan

`lsotfrs` is the only tool in the fleet that offers two machine formats at once, and they are not two encodings of one structure — they are two structures. `-J` groups by process:

```
$ lsotfrs -J | head -9
[
  {
    "command": "loginwindow",
    "pid": 858,
    "uid": 501,
    "pgid": 858,
    "ppid": 1,
    "files": [
      {
```

one object per process carrying `command`, `pid`, `uid`, `pgid`, `ppid` and a nested `files` array. `--csv` flattens the same data to one row per descriptor with the process columns repeated:

```
$ lsofirs --csv | head -2  
COMMAND,PID,USER,FD,TYPE,DEVICE,SIZE/OFF,NODE,NAME  
loginwindow,858,wizard,0r,CHR,"214,15226924",0t0,336,/dev/null
```

with the embedded comma in the device field correctly quoted. A full unfiltered scan on this machine produced about 6.1 thousand rows in each of the table and CSV forms and a 1,556,599-byte JSON document over 58,927 lines — because `-J` is pretty-printed, and because it is a single enclosing array. That last property is the one that matters to a consumer: unlike `iftopirs --json`, the `lsofirs` JSON cannot be read incrementally, and a scan interrupted halfway leaves a document that will not parse. For a one-shot census that is a reasonable trade, and it is worth knowing which of the two headless formats in this fleet streams and which does not.

The suite that would not finish, finished

An earlier chapter reported a measurement it could not complete. `storageshower`'s integration suite was described as 1,336 separate crates whose `cargo test` run “spent 91 seconds of wall time, produced 2.2 GB in `target/`, and terminated during linking with `No space left on device`,” and the chapter drew the conclusion that the suite “cannot be run on a machine without several gigabytes free.” That was an honest report of what happened and an incomplete measurement, because it left the actual ceiling unknown — “several gigabytes” could mean four or forty. This chapter finishes it, on a volume with room, in an isolated target directory outside the repository, and then revisits the other unfinished measurement of that chapter: the parity oracle whose green line means two different things.

The inventory, re-counted

```
$ find storageshower/tests -name '*.rs' | wc -l
1337
$ cat storageshower/tests/*.rs | wc -l
50242
$ grep -h '#\[test\]' storageshower/tests/*.rs | wc -l
3114
$ grep -rh '#\[test\]' storageshower/src/*.rs | wc -l
622
```

One more file than the earlier count, 252 more lines, thirteen more test attributes in `tests/` and twenty-six more in `src/`. The generation pattern is unchanged — 514 files beginning `cli_`, 296 `scan_`, 132 `ui_`, 129 `helpers_`, 106 `prefs_`, 95 `dedup_`, 32 `app_`, 9 `types_`, 8 `system_`, 7 `columns_`, 2 `keys_`, 2 `is_`, and one each of `toml_`, `theme_`, `serde_`, `copy_` and the un-prefixed `integration.rs`, which is the full 1,337 — and the manifest still declares no `[[test]]` stanza, so Cargo's auto-discovery makes each of those files its own integration crate.

The build, measured to the end

Built with `CARGO_TARGET_DIR` pointed at a scratch directory so nothing lands in the repository:

```
$ cargo test --no-run --test app_bookmarks_integration
real 20.96
$ du -sm $CARGO_TARGET_DIR
498

... and then the remaining targets ...
real 168.80
$ du -sm $CARGO_TARGET_DIR
5680

$ cargo test --no-run          # nothing left to do
real 2.84
$ du -sm $CARGO_TARGET_DIR
5678
```

The whole suite compiles and links in 189.76 seconds of wall time across those two invocations, and the confirmation that nothing was left out is the third command: an unfiltered `cargo test --no-run` afterwards finished in 2.84 seconds with every target already built.

The ceiling is therefore **about 5.6 GB and about three minutes**, on this machine, from a cold target directory. It is a ceiling, not a constant: a later cold rebuild of the same tree in a fresh scratch directory — `/usr/bin/time -p cargo test --locked --no-run` in one invocation — finished in **300.78 s** and left **5,713 MB**, so read 5,678 and 5,713 as two samples of the same quantity rather than as a figure that drifted. Where it goes:

```
$ find $CARGO_TARGET_DIR/debug/deps -type f -perm -u+x ! -name '*.d' | wc -l
1351
$ ... total size
2645 MB          mean 1.96 MB per executable
```

One thousand three hundred and fifty-one linked executables averaging just under two megabytes each — 2.6 GB of binaries — with the remaining 3 GB in dependency rlibs, incremental artefacts and metadata. The earlier report's 2.2 GB was not the suite's size; it was how far the build got before the volume filled. The correct statement is the one this measurement supports: the suite needs about 5.6 GB free, which is a great deal for a test suite and is not the impossibility the truncated run implied.

And it passes

```
$ /usr/bin/time -p cargo test --locked      # warm target: everything already linked
real 293.79
user  8.60
sys 28.64

$ grep -c '^      Running' run.log
1339
$ grep -h '^test result:' run.log | awk '{p+=$4; f+=$6; i+=$8} END {...}'
passed=3727 failed=0 ignored=0   (1340 result lines)
```

Three thousand seven hundred and twenty-seven tests pass, none fail, none are ignored. The suite is correct. The three counts — 3,727 passing, 1,339 `Running` lines, 1,340 result lines — reproduce exactly on re-measurement. The wall clock does not, and the condition is worth stating because it is the whole difference: both runs above are against a **warm** target directory with every one of the 1,337 test binaries already linked, so neither figure includes compilation, and the same command still took **662.63 s** on the first reading and **293.79 s** on a later re-run of the identical tree. Wall clock here is a measurement of how many processes the machine can start per second under whatever else it is doing, and it moves by a factor of two between runs.

What does not move is the user CPU time: **8.41 s** on the first reading, **8.60 s** on the second. That ratio is the whole point, and it is worth staring at. Under nine seconds of either run was spent executing test code. The rest was 1,339 process spawns, 1,339 dynamic-loader runs, 1,339 test-harness start-ups and 1,339 tear-downs, plus the ~29 seconds of system time those spawns cost the kernel. The tests are not slow; the packaging is, and the packaging is also the only part of the measurement that is unstable. The earlier chapter's proposed remedy — the same 3,100 attributes distributed over a dozen files rather than 1,337 — would have cut the build from 5.6 GB to a fraction of it and the run to something dominated by the nine seconds of work.

One loose end, stated rather than explained: the grep inventory counts 3,736 `#[test]` attributes (3,114 in `tests/`, 622 in `src/`) and the run reports 3,727 passed. Nine attributes did not run, and **0 ignored** in every result line rules out `#[ignore]` as the cause. The crate has feature-gated and platform-gated code (the optional `reclaim` feature accounts for at least one), which is the likely explanation and is not one I verified case by case. It is a nine-in-3,736 discrepancy between two ways of counting, and the run is the count that means something.

The oracle that cannot fail, in detail

The same earlier chapter demonstrated that `htoprs`'s parity suite reports an identical green line with and without a reference `htop` to compare against. Reproduced here, in a scratch target directory on a machine where `htop 3.5` is installed:

```
$ cargo test --test parity
test result: ok. 18 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in
  0.85s

$ HTOP_REF=/nonexistent/htop cargo test --test parity
test result: ok. 18 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in
  0.90s
```

The finding stands, and two things can now be added to it.

The first is that timing offers no signal either. The real comparison spawns the reference binary several times and the skipped run spawns nothing, and the difference between the two — 0.85 s against 0.90 s — is noise, and it points the wrong way. A reader hoping to infer “did this actually compare anything?” from the duration has nothing to work with.

The second is the anatomy of the eighteen, which the summary line flattens. The skip is implemented in one place:

```
$ awk 'NR>=102,NR<=106' htoprs/tests/parity/harness.rs
pub fn assert_stdout_parity(args: &[&str]) {
    let Some(refbin) = ref_available() else {
        return;
    };
};
```

and `ref_available` returns `None` whenever `HTOP_REF` names a path that does not exist — note that setting the variable *suppresses* the fallback search of `/opt/homebrew/bin`, `/usr/local/bin`, `/usr/bin` and `/bin` entirely, since the `if let Ok(p)` branch returns unconditionally (`harness.rs:45-48`). So pointing `HTOP_REF` at a nonexistent file is not “prefer this `htop`”; it is “assert nothing.” Tracing the eighteen tests through that:

Group	Count	Behaviour with no reference
<code>cli_parity::version / gaps</code> calling <code>assert_stdout_parity</code>	5	become complete no-ops (:71, :76, :90, :115, :122)
<code>cli_parity::help</code> calling <code>assert_branded_help</code>	2	keep the <code>USAGE: /MONITOR</code> assertions, drop only the divergence check (:36-44)
self-comparisons of two spellings	2	unaffected; never needed a reference (:59, :80)
<code>xutils_parity</code>	9	all gated on the compiled C harness; <code>eq()</code> returns early when <code>cref()</code> is <code>None</code> (<code>xutils_parity.rs:70</code>)

Fourteen of the eighteen stop asserting anything about the upstream when the reference is missing, two keep a weaker assertion, and two were never about the upstream in the first place. The line still says `18 passed`.

None of this is a flaw in the harness's *logic*; skipping when the oracle is absent is the correct behaviour for a CI runner that has no `htop`, and the module doc says so at `harness.rs:6-10`. It is a flaw in the harness's *reporting*, and the remedy is the one the earlier chapter proposed and is worth restating now that the shape is precise: a comparison counter. A harness that finished with `parity: 14 comparisons performed, 0 divergences` on one machine and `parity: 0 comparisons performed (no reference htop 3.5`

`found)` on the other would make the two runs distinguishable at a glance, would cost a static counter and one `println!`, and would turn the suite's green line from a claim about the tests into a claim about the port.

Appendix A – the complete command & flag reference

The per-tool chapters describe each tool's flags in prose, where the surrounding sentences explain *why* a flag reads the way it does. This appendix is the other view: every command and flag the book documents, gathered into per-tool tables so a reader who already knows what a tool does can find the one invocation they need without re-reading the chapter. Nothing here is new — every entry traces to the chapter of the same name — and where a flag has a subtlety the prose already stated (an accept-and-ignore no-op, a TTY gate, a divergence from the upstream), the table carries a one-line note rather than repeating the paragraph. Flags accepted for parity but not wired to behavior are marked (*no-op*); where a tool's short binary alias exists it is named in the heading.

lsOfrs / lsf

Selection and the lsof-compatible core:

Flag	Effect
<code>-p PID[,PID]</code>	files held by the given PIDs (^ negates)
<code>-c CMD / -c /regex/</code>	command by prefix or /regex/
<code>-u USER[,USER]</code>	files for the given users (^ negates)
<code>-d SPEC</code>	descriptor set, e.g. <code>0-10</code>
<code>-i [46][proto][@host][:port]</code>	network selector (<code>-i 4TCP:22</code> is one token)
<code>-a</code>	flip selection from OR to AND
<code>-s [proto:state]</code>	show size always, or filter sockets by state
<code>-o[N]</code>	offset always, optional N-digit precision (<code>-o9</code>)
<code>+L[n]</code>	list NLINK column; <code>+L1</code> selects unlinked-but-open files
<code>-t</code>	terse — bare PIDs, one per line
<code>-F [pcfnu...]</code>	lsof field output (<code>p f n</code> default set)
<code>-0</code>	NUL-terminate fields instead of newline
<code>-J/--json, --csv</code>	JSON array / RFC 4180 CSV
<code>-V</code>	report search items that matched nothing
<code>-v</code>	version — prints <code>lsOfrs 4.9.1</code>
<code>--color always never auto</code>	force colour policy

Flag	Effect
<code>-K</code> → <code>--list-tasks</code> , <code>-T</code> → <code>--tcp-info</code> , <code>-x</code> → <code>--cross-over</code>	Isf-only flags mapped to real behavior
<code>-b -O -C -M -X -E -D ...</code>	(no-op) accepted-and-ignored for script compatibility

Derived views (single-shot when piped, interactive on a TTY; all accept the selection filters above):

```
lsf --tree          lsf --summary      lsf --ports
lsf --net-map       lsf --pipe-chain  lsf --stale
lsf --top 10        lsf --watch PATH  lsf --follow PID
lsf --leak-detect=10,5 lsf --stall      lsf --monitor
lsf --delta -r 2    lsf --tui
```

nmaprs / nms

Scan-type selectors (glued Nmap tokens accepted; expanded before clap):

Flag	Technique
<code>-sT</code>	TCP connect (default)
<code>-sS</code>	SYN half-open (needs privilege; falls back to <code>-sT</code> unprivileged)
<code>-sU</code>	UDP
<code>-sN</code> / <code>-sF</code> / <code>-sX</code>	NULL / FIN / Xmas
<code>-sA</code> / <code>-sW</code> / <code>-sM</code>	ACK / Window / Maimon
<code>--s0</code>	IP-protocol scan
<code>--sI ZOMBIE</code>	idle (zombie) scan
<code>-sY</code> / <code>-sZ</code>	SCTP INIT / COOKIE-ECHO
<code>-b</code>	FTP bounce
<code>-sn</code>	ping scan only, no ports
<code>-sV</code>	version detection (skips Perl-only regex constructs)
<code>-O</code>	OS detection (IPv4 partial; IPv6 TTL heuristic)
<code>-A</code>	<code>-O</code> + <code>-sV</code> + default <code>-sC</code> + <code>--traceroute</code>

Ports, timing, discovery, output:

Flag	Effect
<code>-p SPEC</code> / <code>-p -</code>	port list / all 65536
<code>-F</code> , <code>-r</code> , <code>--top-ports N</code>	fast list / sequential / top-N by frequency

Flag	Effect
<code>-T0...-T5</code>	timing template (default <code>-T3</code> → 256 workers, 1500 ms)
<code>--min-rate/--max-rate,</code> <code>--min/--max-parallelism</code>	rate and concurrency controls
<code>-Pn</code>	skip host discovery
<code>-PS/-PA/-PU/-PY/-PO/-PE/-PP/-PM</code>	the <code>-P*</code> discovery probe family
<code>-oN/-oG/-oX/-oA/-oS</code>	normal / grepable / XML / all / script-kiddie
<code>-oM, -oH</code>	partial: grepable-style / placeholder
<code>-iL FILE, -iR N, --resume STATE</code>	input list / random hosts / checkpoint
<code>--proxies, --dns-servers, --system-dns</code>	proxy chain / custom DNS
<code>-f/--mtu, -D, -S, -g, --data*, --ttl, --badsum,</code> <code>--spoof-mac</code>	evasion / raw-packet controls
<code>--privileged / --unprivileged</code>	force raw / force connect mode
<code>--resolve-all, --nsock-engine,</code> <code>--release-memory, ...</code>	(<i>no-op</i>) accepted for parity, logged not acted on

temprs / tp

Thirty-eight operations; index arguments accept a signed position (`-1` = top) or a tag name interchangeably.

Group	Flags
I/O	<code>-i IDX</code> (write into entry), <code>-o IDX</code> (read to stdout), <code>-A IDX</code> (append), <code>-v, -q</code> (<i>records intent; runtime does not yet suppress</i>)
Stack algebra	<code>-p/--pop, -s/--shift, -u/--unshift, -a IDX, -r IDX, -M FROM TO, -S A B, -x IDX, --rev, --sort name\ size\ mtime</code>
Inspection	<code>-l, -n, -L, -N, -k/--count, -I IDX, -d/--dir, -m/--master, --path IDX</code>
Coreutils-analogue	<code>--head IDX N, --tail IDX N, --wc IDX</code> (line count only), <code>--size IDX, -g PATTERN</code> (literal substring; exit 0/1), <code>-D A B</code> (shells <code>diff -u</code>), <code>-C IDX...</code> (<code>cat</code>)
Editing / lifecycle	<code>-e IDX (\$EDITOR), --replace IDX PAT REPL, -c/--clear, --expire HOURS</code>
Naming	<code>-w/--name NAME, -R/--rename OLD NEW</code>

Hardening worth remembering: `--clear` refuses unless the directory basename contains `temprs`; `--expire` rejects negative/NaN/inf before the cast; `--replace` rejects an empty pattern.

```

ps aux | tp -w procs      # push, tagged "procs"
tp -g python procs && echo hit
tp --path procs          # print backing file path
tp -D procs -l           # diff two entries by name/index
tp --expire 1            # drop entries older than 1h

```

iftoprs

Single binary; raw capture needs root. Non-TUI modes are handled before the terminal is touched.

Flag	Effect
<code>-i NAME</code>	interface to monitor
<code>-f EXPR</code>	BPF filter, e.g. " <code>tcp port 80</code> "
<code>-F CIDR</code>	IPv4 local network scope
<code>-p</code>	promiscuous mode
<code>-n / -N</code>	disable reverse DNS / port → service names
<code>-b / -B</code>	disable bars / show bytes not bits
<code>-P / -Z</code>	hide ports / hide process column
<code>--json</code>	stream NDJSON, no TUI
<code>-c FILE</code>	config path (default <code>~/.iftoprs.conf</code>)
<code>-l / --list-colors</code>	list interfaces / preview all 31 themes
<code>--completions SHELL</code>	zsh bash fish elvish powershell

In-TUI keys (selection): `1/2/3` sort by 2s/10s/40s, `</>` sort by name, `r` reverse, `o` freeze order, `t` cycle line display, `b` cycle bar style, `B` bits/bytes, `Tab` flow ↔ process view, `/` filter, `c/C` theme chooser/editor, `i` interface chooser, `F` pin, `y` copy, `e` export, `P` pause.

htoprs

CLI mirrors htop's `getopt_long` table exactly (driven through the real `libc::getopt_long`); `-h/-V` route to branded screens.

Flag	Effect
<code>-d DELAY</code>	update delay, tenths of a second (1..100)
<code>-n NUMBER</code>	exit after N refresh iterations
<code>-s COLUMN</code>	sort by column (<code>--sort-key help</code> lists)
<code>-u [USER]</code>	only this user's processes
<code>-C/--no-color</code>	monochrome
<code>-M/--no-mouse</code>	disable mouse
<code>-U/--no-unicode</code>	ASCII graph meters
<code>-t/--tree</code>	tree view
<code>-p PID,...</code>	show only the given PIDs
<code>-F FILTER</code>	fixed-string filter (<code> </code> multi-term)

Flag	Effect
<code>-H[=DELAY]</code>	highlight new/old processes
<code>--no-meters, --no-function-bar, --readonly</code>	hide meters / bar; disable state-changing features

Extension keys (bound to keys htop leaves free): `f` fuzzy finder, `r` regex filter (saved), `d` snapshot/diff, `o` export JSON+CSV, `A` threshold alerts, `E` memory-exhaustion forecast, `G` braille CPU history, `y` aggregate/pivot, `:` command palette, `v` per-PID sparkline, `z/~` theme chooser/editor.

storageshower

Single invocation; CLI flags override the config, and most booleans have a hidden `--no-` inverse.

Group	Flags
Sorting	<code>-s name\ pct\ size, -R/--reverse, -l/--local-only, --virtual/--no-virtual</code>
Display	<code>-b gradient\ solid\ thin\ ascii, --color PALETTE (30), --theme NAME, -u human\ gib\ mib\ bytes, -k/--compact, -f/--full-mount, --bars/--border/--header/--used/--tooltips</code>
Thresholds	<code>-w PCT (warn), -C PCT (crit)</code>
Columns	<code>--col-mount, --col-bar-end, --col-pct (u16 width, 0 = auto)</code>
System	<code>-r SECS (refresh), -c PATH (config), -h, -V</code>
Introspection	<code>--list-colors, --export-theme</code>

In-TUI keys reach the same knobs; note the layering — `-f` on the command line is `--full-mount`, while the interactive `f` key cycles the refresh rate. Thresholds cycle in fixed steps (`t` walks warn 50 → 60 → 70 → 80, `z` walks crit 80 → 85 → 90 → 95).

zcolorizer

Streaming colouriser; reads stdin line-buffered with no file arguments.

Flag	Effect
<code>--theme NAME</code>	select one of 32 themes (<code>cyberpunk/default</code> alias <code>neon-sprawl</code>)
<code>-m MODULE</code>	enable a format module (repeatable; <code>-m all, -m auto</code>)
<code>-C/--force-color, --no-color</code>	force colour on / force passthrough
<code>-g PATTERN</code>	filter to matching lines, keeping colour

Flag	Effect
<code>-H PATTERN</code>	keep all lines, dim the non-matches
<code>--novelty</code>	statistical-novelty intensity; noise self-dims
<code>--novelty-decay D</code>	age counts ($0 < D \leq 1$); implies <code>--novelty</code>
<code>-w/--watch</code>	live-reload config on mtime change
<code>--list-themes/--list-modules/--list-rules</code>	enumerate the resolved sets
<code>--dump-config, --themes-json</code>	re-emit config / emit theme structs for a GUI picker

grcrs — grc / grcat

`grc <command>` (launcher) options: `-s/--stdout`, `-e/--stderr`, `-c/--config FILE`, `--colour=on|off|auto`, `--pty` (experimental), `--` (end option scanning). Abbreviated long options resolve by unique prefix (`--conf` → `--config`), and `--version` is not a flag — it falls through `resolve_long` to `help()`, whose banner leads `Generic Colouriser 1.13`.

`grcat <colourfile>` (colouriser) is driven from a pipeline (`cmd | grcat conf.log`). Its colourfile keywords: `regexp`, `colours` (`color/colors/colour` aliased), `count` (`more/once/stop/block/unblock/previous`), `command`, `skip`, `replace`, `concat`, plus the grcrs-only additive extension `trend/key/metric` for colouring a number by its motion over the stream.

zwire-host / zpwrchrome-host

`zwire-host` selects one of three run modes by first positional argument: bare (native-messaging stdio, what Chrome spawns), `serve` (NDJSON socket daemon, `--token/--peer` for TCP federation), and `call [--stream] JSON` (one-shot client). Its capability arms — advertised in the `hello` reply and gated by Cargo features — include `kv_*`, `fs_*`, `exec`, `job_*`, `bus` (`sub/pub`), `pty_*` (feature `pty`), `sysinfo_*`/`ps/kill/which` (feature `sysinfo-caps`), and `open/clipboard/notify/hostinfo`.

`zpwrchrome-host` speaks browserpass PROTOCOL.md v3.1.2 (`configure`, `list`, `tree`, `fetch`, `save`, `delete`, `echo`, error codes 10–32) and adds `otp`, `search`, `run.spawn`, `host.crawl/host.exec` (which call into the embedded `zwire_host` library), `zcite.save`, and the `dl.*` segmented download family (`dl.add/pause/resume/cancel/remove/clear/list`, worker as `--dl-worker GID`). CLI surface: `--install <ext-id>` (writes the native-messaging manifests), `--help`, `-version` (prints the browserpass protocol version 3.1.2).

Appendix B — the palette and theme catalog

The theme-engine chapter traced the shared palette set as a lineage; this appendix is the roster itself, gathered so a reader configuring a tool can see the whole set at once. The names below are the ones the per-tool chapters enumerate; the count differs by tool for the reasons the theme chapter gave (a parity palette added here, a hand-written classic there), and the differences are listed rather than averaged away.

The 31 generated palettes (zcolorizer’s kebab ids)

zcolorizer enumerates the shared set in full and in declaration order, which fixes the canonical sequence — `neon-sprawl` is first and is the default (aliased `cyberpunk/default`):

```
neon-sprawl  acid-rain    ice-breaker  synth-wave   rust-belt
ghost-wire   red-sector   sakura-den   data-stream  solar-flare
neon-noir    chrome-heart blade-runner void-walker   toxic-waste
cyber-frost  plasma-core steel-nerve  dark-signal  glitch-pop
holo-shift   night-city  deep-net    laser-grid   quantum-flux
bio-hazard   darkwave    overlock    megacorp     zaibatsu
iftopcolor
```

The thirty-second zcolorizer theme, `ccze-classic`, is not from this set — it is a hand-written 16-colour map approximating the original ccze look (cyan dates, bold-red errors, bold-yellow warnings) for parity. Name resolution is forgiving: `normalize` lowercases and collapses spaces/underscores to hyphens, so “Neon Sprawl”, `neon_sprawl`, and `NEON-SPRAWL` all match one palette.

The roster across the fleet

Tool	Count	Set
<code>storageshower</code>	30	the origin; <code>ColorMode</code> display names “Neon Sprawl” (<code>Default</code>) ... “Acid Rain” (<code>Green</code>) ... “Zaibatsu”
<code>iftoprs</code>	31	the 30 plus the parity palette <code>Iftopcolor</code> ; <code>ThemeName::ALL.len() == 31</code> asserted
<code>lsofrs</code>	31	NeonSprawl ... Zaibatsu; <code>ThemeName::ALL.len() == 31</code> asserted
<code>htoprs</code>	31	31 six-colour palettes, applied at <code>Ncurses::to_color</code>
<code>zcolorizer</code>	31 + 1	31 generated (above) + hand-written <code>ccze-classic</code> = 32; confirmed at runtime

`iftoprs`, `lsofrs`, `htoprs`, and `storageshower` all check in a `*.default.conf` (or `themes/` copies) that enumerates the theme names for hand-editing.

The six-colour custom-theme model

Every tool that carries the set also lets a user define a custom theme as six 256-colour indices, edited either by hand in the config or through a live in-app six-slot editor (bound to `C` in `lsofrs/iftoprs/storageshower`, `~` in `htoprs`). The six slots carry a fixed meaning — `zcolorizer` names them *primary*, *brightest accent*, *secondary*, *mid*, *dim*, *darkest*; `storageshower` names the channels `blue`, `green`, `purple`, `light_purple`, `royal`, `dark_purple`. A `storageshower` custom theme in TOML:

```
[custom_themes.neonpink]
blue      = 199
green     = 46
purple    = 201
light_purple = 213
royal     = 196
dark_purple = 161

active_theme = "neonpink"
```

Two collision-avoidance and generation details from the theme chapter belong here as the catalog's fine print: `shift_lighter` (an index-cube arithmetic helper ported from `iftoprs` into `zcolorizer`) steps a 256-colour index one notch brighter, clamping at the cube and grayscale-ramp edges, and is used to derive a distinguishable extra shade when two tokens would otherwise land on the same slot (`process` vs `host`); and `zcolorizer`'s 31 non-classic themes are *generated* from the six palette colours by `theme_from_palette` rather than authored token-by-token, layering bold/italic/underline over the six roles so a six-colour palette styles roughly sixty tokens.

zcolorizer token vocabulary and modules

`zcolorizer`'s themes key on token names, and the well-known ones the builtin rules emit are worth listing because a config can restyle any of them: the generic vocabulary (`date`, `time`, `host`, `ip`, `mac`, `pid`, `process`, `error`, `warning`, `debug`, `info`, `good`, `bad`, `system`, `keyword`, `email`, `uri`, `dir`, `file`, `size`, `version`, `number`, `address`, `percentage`, `signal`, `http_method`, `http_code`, `string`), the ccze-plugin tokens (`gettime`, `getsize`, `ftp_code`, `ident`, the eleven `proxy_*` squid actions, and the rest), and the modern-format tokens (`thread`, `level`, `facility`, `duration`, `latency`, `hash`, `json_key`). The 47 format modules split 20 classic ccze ports (`syslog`, `httpd`, `squid`, `dpkg`, `postfix`, `exim`, `procmail`, `proftpd`, `vsftpd`, `ftpstats`, `xferlog`, `php`, `oops`, `icecast`, `fetchmail`, `apm`, `distcc`, `sulog`, `super`, `ulogd`) and 27 modern ones (`json`, `logfmt`, `journald`, `authlog`, `kernel`, `cron`, `auditd`, `nginx`, `haproxy`, `caddy`, `traefik`, `postgres`, `mysql`, `redis`, `mongodb`, `klog`, `docker`, `spring`, `python`, `rust-log`, `rails`, `named`, `dnsmasq`, `dhcpcd`, `fail2ban`, `ufw`, `dovecot`).

Appendix C – the provisional-numbers ledger

The book marks every version, test count, and benchmark figure as provisional and states, at each site, which tool produced it. This appendix consolidates those numbers into one place so a reader can see the whole ledger at a glance — and so the provisional caveat is impossible to miss. **Read every cell in this appendix under the methodology chapter’s rule: a test count is a count of `#[test]` attributes found by `grep` (or, where noted, a runtime enumeration), not a `test result` line; a benchmark is the author’s own measurement on one machine, not reproduced for this book; a version is read from a `Cargo.toml/manifest`, not confirmed against a compiled artifact.** The numbers are the counts the sources reported at the time of writing, and they will drift.

Versions and editions

Tool	Version	Edition	Licence
<code>lsofrs</code>	4.9.1	2024	MIT
<code>nmaprs</code>	0.1.9	2021	MIT
<code>temprs</code>	2.9.12	2024	MIT
<code>iftoprs</code>	2.22.9	2024 (Rust $\geq$ 1.85)	MIT
<code>htoprs</code>	0.5.7	2021	MIT
<code>storageshower</code>	0.28.8	2024	MIT
<code>zcolorizer</code>	0.2.2	2021	MIT
<code>grcrs</code>	1.13.0	—	MIT
<code>zwire-host</code>	0.3.14	—	MIT
<code>zpwrchrome-host</code>	0.10.2	—	MIT

Test-attribute counts (provisional; `grep` inventory unless noted)

Tool	Reported test surface
<code>lsofrs</code>	1,029 <code>#[test]</code> in <code>src/</code> + 710 in <code>tests/</code> + 21 shell gate scripts
<code>nmaprs</code>	~1,468 test fns total; <code>tests/cli.rs</code> alone 204
<code>temprs</code>	5,065 unit (4,449 <i>generated</i> flag tests in <code>opts.rs</code>) + 585 integration + 21 shell
<code>iftoprs</code>	2,286 annotations (1,741 in <code>src/</code> , 545 in <code>tests/</code>)
<code>htoprs</code>	1,412 <code>#[test]</code> ; plus the <code>tests/parity/</code> byte-diff suite

Tool	Reported test surface
<code>storageshower</code>	3,114 in <code>tests/</code> + 622 in <code>src/</code> $\approx$ 3,736 attributes; suite run in-session: 3,727 passed, 0 failed, 0 ignored ; warm-target wall clock 662.63 s and 293.79 s on two runs of the same tree, user CPU 8.41 s and 8.60 s
<code>zcolorizer</code>	94 unit + 17 integration + 1 doc = 112 (reported passing)
<code>grcrs</code>	71 total: 17 launcher + 27 colouriser + 27 end-to-end
<code>zpwchrome-host</code>	121 (from README, not built here)

Port-coverage and parity figures

Tool	Figure
<code>htoprs</code>	1,069 / 1,093 C functions ported (97.8%), 130/131 files, 1 stub; 1,937 <code>/// Port of</code> citations; 4,137-entry C-name index; empty allowlist
<code>ztmux</code> (Part I)	1,188 / 1,188 parity cases vs vendored tmux; <code>PROTOCOL_VERSION = 8</code>
<code>powerliners</code> (Part I)	134/137 upstream <code>.py</code> at DONE (97.8%); 2,436 lib tests; 462 parity tests
<code>nmaprs</code>	parity documented as a capability matrix, contract-tested
<code>grcrs</code>	end-to-end bytes cross-checked vs reference Python <code>grcat</code>
<code>zcolorizer</code>	47 modules, 32 themes (runtime-enumerated); precedence + sanitization pinned

Benchmark claims (author's own, one machine, not re-produced here)

Tool	Claim	Conditions
<code>lsofrs</code>	5–21× vs lsof 4.91 / lsofng	macOS, ~470 procs / ~5,000 open files, hyperfine
<code>nmaprs</code>	1.5–5.1× vs nmap 7.99	macOS arm64, <code>127.0.0.1, -n -Pn --min-rtt-timeout 50ms --max-retries 0</code> ; ~74–82 μ s Criterion connect microbench
<code>storageshower</code>	format ops tens of ns; 200-mount render ~15.7 μ s	Apple Silicon
<code>awkr</code> s (Part I)	14–110× fusevm offload vs bytecode interp; first awk pairing a bytecode VM with a persistent on-disk bytecode cache	pure-numeric accumulator loops
<code>zshrs</code> (Part I)	717 ms cold vs 7 ms warm	same script, rkyv image reuse

Tool	Claim	Conditions
<code>vimlrs</code> (Part I)	~15–100× JIT vs interpreter	20M-iteration loop

Source size and module counts (provisional; per-chapter figures)

The per-tool chapters cite source sizes in passing, usually to justify which module carries the load-bearing logic. Gathered here, they sketch the relative weight of the eight ports — with the same caveat as everything else in this appendix: these are `wc -l/module-count` figures the chapters reported at the time of writing.

Tool	Source size	Notable modules
<code>lssofrs</code>	29 modules under <code>src/</code>	<code>main.rs</code> dispatcher; <code>types.rs</code> data model; <code>cli.rs</code> normalizer; per-mode view modules
<code>nmaprs</code>	~24,000 lines / 35 files	<code>config.rs</code> 2,314; <code>os_scan.rs</code> 1,860; <code>syn.rs</code> 1,760; <code>lib.rs</code> 1,552; <code>vscan.rs</code> 1,499; <code>discovery.rs</code> 1,437
<code>temprs</code>	<code>opts.rs</code> ~47,000 (mostly generated tests) 22,913 lines / 21 files	<code>utils.rs</code> ~2,200; <code>model/state</code> , <code>model/app.rs</code> , <code>consts.rs</code>
<code>iftoprs</code>		<code>app.rs</code> 4,066; <code>parser.rs</code> 3,074; <code>render.rs</code> 2,754; <code>cli.rs</code> 2,344; <code>resolver.rs</code> 1,733; <code>flow.rs</code> 1,685
<code>htoprs</code>	141 ported modules + 22 extension modules	per-OS backends (<code>darwin/...solaris/</code>); the full meter set; 1,937 <code>/// Port of citations</code>
<code>storageshower</code>	~13,700 lines / ~12 modules	<code>ui.rs</code> ~3,860; <code>keys.rs</code> ~1,930; <code>cli.rs</code> ~1,550; <code>system.rs</code> ~1,510; <code>app.rs</code> ~1,320
<code>zcolorizer</code>	47 format modules over a 3-layer engine	<code>rules.rs</code> / <code>theme.rs</code> / <code>engine.rs</code> ; <code>novelty.rs</code> ; <code>modules.rs</code> (20) + <code>modules_modern.rs</code> (27)
<code>grcrs</code>	2 binaries, 2 dependencies	<code>grcrs.rs</code> launcher; <code>grcatrs.rs</code> colouriser; each carries its own <code>translate_regex</code>

Two shapes stand out in the ledger and both are documented in their chapters. `temprs`'s `opts.rs` is an outlier at ~47,000 lines because it is dominated by the 4,449 machine-generated flag-recognition tests, not by hand-written logic — the same reason its test count leads the fleet. And `htoprs`'s module count (141 ported + 22 extension) is high because the port mirrors htop's own file layout one module per C file, including the seven per-OS backend trees htop itself carries; the count is a property of faithfulness to the upstream's structure, not of feature sprawl.

The single instruction

The single instruction that governs this entire appendix is the one the methodology chapter states and the preface states before it: where a number is provisional the text marks it rather than rounding it into a claim, because versions move, suites grow, and a book that pretends otherwise goes stale the day it prints. These are the counts the sources reported at the time of writing — no more, and marked as no more.

Colophon

This book is generated from one source. `docs/book.md` is the manuscript; `scripts/book_pdf.sh` renders it to PDF through pandoc and LuaLaTeX with the project's own theme, and `scripts/book_epub.sh` renders the same manuscript — plus this apparatus — to a reflowable EPUB 3. The cover is not hand-placed either: `scripts/book_cover.sh` draws it at 1600×2560 from the same palette the PDF theme uses, so the cover on a storefront and the chapter headings inside the book cannot drift apart.

The display faces are Orbitron and Share Tech Mono; body text is set in Arimo. All three are licensed under the SIL Open Font License 1.1.

There is no generated reference manual for this book. Its subject is a fleet of separate programs rather than one crate, and each of them already ships its own man pages and `--help` surface; Appendix A consolidates the command and flag reference instead.

Every figure in these pages was read from a program's own source — a `Cargo.toml`, a `#[test]` count, a module listing — and is cited at the site where it appears. The book is deliberate about what kind of number each one is: a test count is a count of `#[test]` attributes found by `grep`, not a `test result:` line; a benchmark is the author's own measurement on one machine and was not reproduced for this book; a version is read from a manifest, not confirmed against a compiled artifact. Appendix C consolidates every provisional figure into one ledger so the caveat is impossible to miss, and the methodology chapter states the rule the ledger is read under.

Reproducing the Book's Claims

Each tool in this fleet is its own repository under github.com/MenkeTechnologies, so these commands are run inside a tool's own checkout rather than from one tree. They are the commands the book's figures came from.

The test-attribute inventory quoted for every tool — an inventory, not a suite result:

```
grep -rc '#\[test\]' src/ tests/
```

Turning an inventory into a result requires the suite itself, which is what the methodology chapter asks a reader to do before treating any of those counts as green:

```
cargo test
```

The version, edition and licence rows in Appendix C:

```
grep -E '^(version|edition|license)' Cargo.toml
```

For [htoprs](#), the port-coverage figure is the ratio of cited C functions to the functions present in the vendored htop source, and the citations are countable directly:

```
grep -rc '/// Port of' src/
```

For [grcrs](#), the parity claim is the end-to-end suite whose expectations were cross-checked byte for byte against the reference Python [grcat](#):

```
cargo test --test cli
```

Appendix C is the full ledger; these are the commands behind its load-bearing rows.

Also from MenkeTechnologies

The CLI Fleet is one volume in a stack that is documented the same way — from source, with the numbers cited to the tools that produce them, and with the upstream named whenever a program is a port. Its companion covers the graphical fleet; other volumes cover the shell, the terminal emulator, the bytecode virtual machine the language frontends share, and the language ports built on it.

Titles and availability change; the current catalog is published at github.com/MenkeTechnologies.